BCS502 COMPUTER NETWORKS

Module-1

Chapter 1: Introduction (Pg. 1 to 19): Data Communications, Networks, Network Types
Chapter 2: Networks Models (Pg. 1 to 19): Protocol Layering, TCP/IP Protocol suite, The OSI model
Chapter 7 (Pg. 1 to 19): Introduction to Physical Layer: Transmission media, Guided Media, Unguided Media: Wireless.
Chapter 8: Switching(Pg. 1 to 19): Packet Switching and its types.
Textbook: Ch. 1.1 - 1.3, 2.1 - 2.3, 7.1 – 7.3, 8.3.

Chapter 1: Introduction: Data Communications, Networks, Network Types

---

Data Communications - *Components - Data Representation - Data Flow*

- Data communications are the exchange of data between two devices via some form of transmission medium such as a wire cable.
- For data communications to occur, the communicating devices must be part of a communication system made up of a combination of hardware (physical equipment) and software (programs).
- The effectiveness of a data communications system depends on **four fundamental characteristics**:
    1. Delivery
    2. Accuracy
    3. Timeliness, and
    4. Jitter.

1. Delivery: The system must deliver data to the correct destination. Data must be received by the intended device or user and only by that device or user.
2. Accuracy: The system must deliver the data accurately. Data that have been altered in transmission and left uncorrected are unusable.
3. Timeliness: The system must deliver data in a timely manner. Data delivered late are useless. In the case of video and audio, timely delivery means delivering data as they are produced, in the same order that they are produced, and without significant delay. This kind of delivery is called real-time transmission.
4. Jitter: Jitter refers to the variation in the packet arrival time. It is the uneven delay in the delivery of audio or video packets. For example, let us assume that video packets are sent every 30 ms. If some of the packets arrive with 30-ms delay and others with 40-ms delay, an uneven quality in the video is the result.

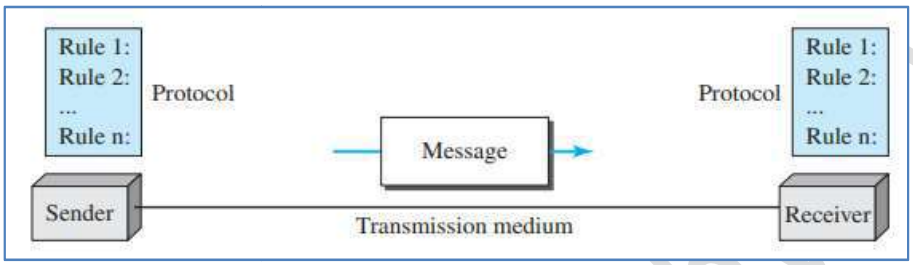
Components

- A data communications system has **five components.**
    1. Message
    2. Sender
    3. Receiver
    4. Transmission medium
    5. Protocol

1. **Message.** The message is the information (data) to be communicated. Popular forms of information include text, numbers, pictures, audio, and video.
2. **Sender.** The sender is the device that sends the data message. It can be a computer, workstation, telephone handset, video camera, and so on.

Prof. Parthasarathy P V

3. **Receiver.** The receiver is the device that receives the message. It can be a computer, workstation, telephone handset, television, and so on.
4. **Transmission medium**. The transmission medium is the physical path by which a message travels from sender to receiver. Some examples of transmission media include twisted-pair wire, coaxial cable, fiber-optic cable, and radio waves.
5. **Protocol.** A protocol is a set of rules that govern data communications. It represents an agreement between the communicating devices. Without a protocol, two devices may be connected but not communicating, just as a person speaking French cannot be understood by a person who speaks only Japanese.



***

Data Representation

- Information today comes in different forms such as text, numbers, images, audio, and video.

Text

- In data communications, text is represented as a bit pattern, a sequence of bits (0s or 1s). Different sets of bit patterns have been designed to represent text symbols.
- Each set is called a code, and the process of representing symbols is called coding.
- Today, the prevalent coding system is called Unicode, which uses 32 bits to represent a symbol or character used in any language in the world.
- The American Standard Code for Information Interchange (ASCII), developed some decades ago in the United States, now constitutes the first 127 characters in Unicode and is also referred to as Basic Latin.

Numbers

- Numbers are also represented by bit patterns. However, a code such as ASCII is not used to represent numbers; the number is directly converted to a binary number to simplify mathematical operations.

Images

- Images are also represented by bit patterns. In its simplest form, an image is composed of a matrix of pixels (picture elements), where each pixel is a small dot. The size of the pixel depends on the ***resolution***. For example, an image can be divided into 1000 pixels or 10,000 pixels. In the second case, there is a better representation of the image (better resolution), but more memory is needed to store the image.
- After an image is divided into pixels, each pixel is assigned a bit pattern. The size and the value of the pattern depend on the image. For an image made of only blackand-white dots (e.g., a chessboard), a 1-bit pattern is enough to represent a pixel.
- If an image is not made of pure white and pure black pixels, we can increase the size of the bit pattern to include gray scale. For example, to show four levels of gray scale, we can use 2-bit patterns. A black pixel can be represented by 00, a dark gray pixel by 01, a light gray pixel by 10, and a white pixel by 11.

Prof. Parthasarathy V

- There are several methods to represent color images. One method is called RGB, so called because each color is made of a combination of three primary colors: red, green, and blue. The intensity of each color is measured, and a bit pattern is assigned to it. Another method is called YCM, in which a color is made of a combination of three other primary colors: yellow, cyan, and magenta.

Audio

- Audio refers to the recording or broadcasting of sound or music. Audio is by nature different from text, numbers, or images. It is continuous, not discrete. Even when we use a microphone to change voice or music to an electric signal, we create a continuous signal.

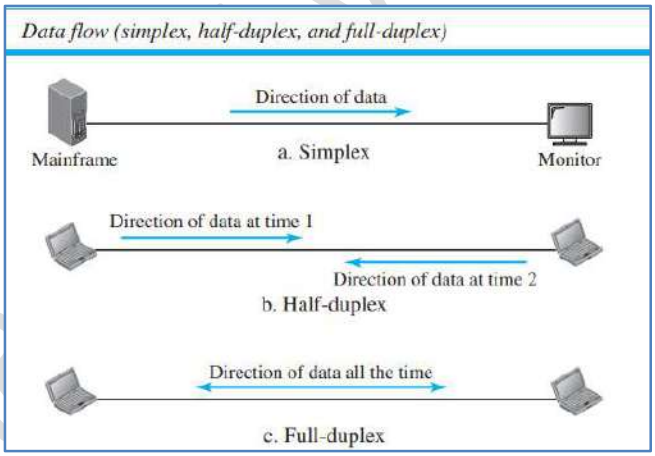
Video

- Video refers to the recording or broadcasting of a picture or movie. Video can either be produced as a continuous entity (e.g., by a TV camera), or it can be a combination of images, each a discrete entity, arranged to convey the idea of motion.

***

Data Flow

- Communication between two devices can be simplex, half-duplex, or full-duplex.



Simplex

- In ***simplex mode, the communication is unidirectional***, as on a one-way street. Only one of the two devices on a link can transmit; the other can only receive.
- Keyboards and traditional monitors are examples of simplex devices.
- The keyboard can only introduce input; the monitor can only accept output. The simplex mode can use the entire capacity of the channel to send data in one direction.

Half-Duplex

- In ***half-duplex mode, each station can both transmit and receive, but not at the same time***. When one device is sending, the other can only receive, and vice versa. The half-duplex mode is like a one-lane road with traffic allowed in both directions.
- In a half-duplex transmission, the entire capacity of a channel is taken over by whichever of the two devices is transmitting at the time. ***Walkie-talkies and CB (citizens band) radios are both half-duplex systems.*** The half-duplex mode is used in cases where there is no need for communication in both directions at the same time; the entire capacity of the channel can be utilized for each direction.

Full-Duplex

- In ***full-duplex mode (also called duplex), both stations can transmit and receive simultaneously***. The full-duplex mode is like a two-way street with traffic flowing in both directions at the same time.
- In full-duplex mode, signals going in one direction share the capacity of the link with signals going in the other direction. This ***sharing can occur in two ways***: ***Either the link must contain two physically separate transmission paths, one for sending and the other for receiving; or the capacity of the channel is divided between signals traveling in both directions***. One common example of full-duplex communication is the telephone network.

Prof. Pathan PV

- When two people are communicating by a telephone line, both can talk and listen at the same time. The full-duplex mode is used when communication in both directions is required all the time. The capacity of the channel, however, must be divided between the two directions.

Networks: *Network Criteria - Physical Structure*

- A ***network*** is the ***interconnection of a set of devices capable of communication***. In this definition, ***a device can be a host*** (or an end system as it is sometimes called) ***such as a large computer, desktop, laptop, workstation, cellular phone, or security system.***
- A device in this definition can also be a ***connecting device such as a router***, which connects the network to other networks, a switch, which connects devices together, a modem (modulator-demodulator), which changes the form of data, and so on.
- These devices in a network are connected using wired or wireless transmission media such as cable or air. When we connect two computers at home using a plug-and-play router, we have created a network, although very small.

Network Criteria

- A network must be able to meet a certain number of criteria. The most important of these are ***performance***, ***reliability***, and ***security***.

Performance

- Performance can be measured in many ways, including ***transit time*** and ***response time***.
  - ***Transit time*** is the amount of time required for a message to travel from one device to another.
  - ***Response time*** is the elapsed time between an inquiry and a response.
- The performance of a network depends on a ***number of factors***, including
  - the number of users
  - the type of transmission medium
  - the capabilities of the connected hardware, and
  - The efficiency of the software.
- Performance is often evaluated by ***two networking metrics***: ***throughput*** and ***delay***. We often need more throughput and less delay. However, these two criteria are often contradictory. If we try to send more data to the network, we may increase throughput but we increase the delay because of traffic congestion in the network.

Reliability

- In addition to accuracy of delivery, network reliability is measured by the frequency of failure, the time it takes a link to recover from a failure, and the network's robustness in a catastrophe.

Security

- Network security issues include protecting data from unauthorized access, protecting data from damage and development, and implementing policies and procedures for recovery from breaches and data losses.

****

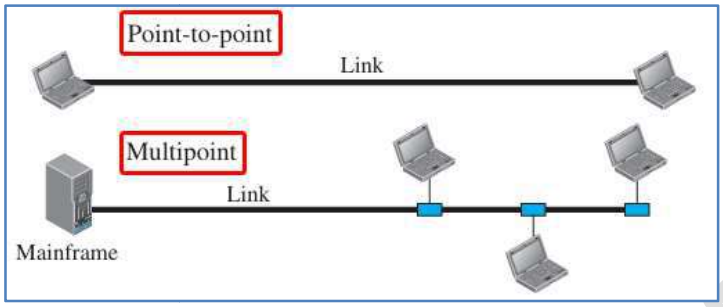
Physical Structures - Type of Connection - Physical Topology

Type of Connection

- A network is two or more devices connected through links.
- A ***link*** is a communications pathway that transfers data from one device to another. For visualization purposes, it is simplest to imagine any link as a line drawn between two points.

Prof. Parthasarathy P V

- For communication to occur, ***two devices must be connected*** in some way to the same link at the same time. There are ***two possible types of connections:***
  - ***point-to-point*** and
  - ***multipoint***



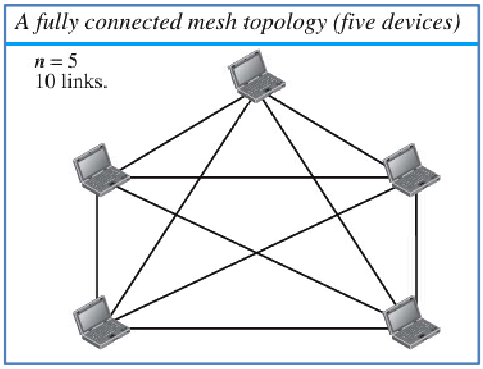
Point-to-Point

- A point-to-point connection provides a dedicated link between two devices. The entire capacity of the link is reserved for transmission between those two devices. Most point-to-point connections use an actual length of wire or cable to connect the two ends, but other options, such as microwave or satellite links, are also possible. When we change television channels by infrared remote control, we are establishing a point-to-point connection between the remote control and the television's control system.

Multipoint

- A multipoint (also called multidrop) connection is one in which more than two specific devices share a single link.
- In a multipoint environment, the capacity of the channel is shared, either spatially or temporally.
- If several devices can use the link simultaneously, it is a spatially shared connection. If users must take turns, it is a timeshared connection.
- The term physical topology refers to the way in which a network is laid out physically. Two or more devices connect to a link; two or more links form a topology. The topology of a network is the geometric representation of the relationship of all the links and linking devices (usually called nodes) to one another.
- There are ***four basic topologies*** possible: ***mesh***, ***star***, ***bus***, and ***ring***.

1. Mesh Topology



- In a mesh topology, every device has a dedicated point-to-point link to every other device.
- The term ***dedicated*** means that the link carries traffic only between the two devices it connects. To find the number of physical links in a fully connected mesh net work with n nodes, we first consider that each node must be connected to every other node.
- Node 1 must be connected to n – 1 nodes, node 2 must be connected to n – 1 nodes, and finally node n must be connected to n – 1 nodes. We need n (n – 1) physical links.
- If each physical link allows communication in both directions (duplex mode), we can divide the number of links by 2 [we need n (n – 1) / 2 duplex-mode links].
- To accommodate that many links, every device on the network must have n – 1 input/output (I/O) ports (see Figure 1.4) to be connected to the other n – 1 stations.

Prof Parthasarathy P V

- ***One practical example*** of a mesh topology is the connection of telephone regional offices in which each regional office needs to be connected to every other regional office.

Advantages

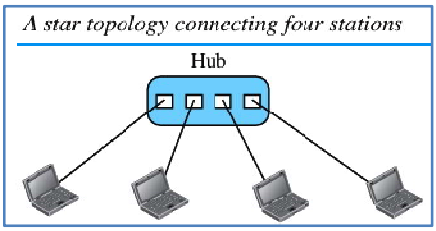
1. The use of dedicated links guarantees that each connection can carry its own data load, thus eliminating the traffic problems that can occur when links must be shared by multiple devices.
2. A mesh topology is robust. If one link becomes unusable, it does not incapacitate the entire system.
3. There is the advantage of privacy or security. When every message travels along a dedicated line, only the intended recipient sees it. Physical boundaries prevent other users from gaining access to messages.
4. Point-to-point links make fault identification and fault isolation easy. Traffic can be routed to avoid links with suspected problems. This facility enables the network manager to discover the precise location of the fault and aids in finding its cause and solution.

Disadvantages

The main disadvantages of a mesh are related to the amount of cabling and the number of I/O ports required.

1. Because every device must be connected to every other device, installation and reconnection are difficult.
2. The sheer bulk of the wiring can be greater than the available space (in walls, ceilings, or floors) can accommodate.
3. The hardware required to connect each link (I/O ports and cable) can be prohibitively expensive. For these reasons a mesh topology is usually implemented in a limited fashion.

2. Star Topology



- In a star topology, each device has a dedicated point-to-point link only to a central controller, usually called a hub. The devices are not directly linked to one another.
- Unlike a mesh topology, a star topology does not allow direct traffic between devices. The controller acts as an exchange: If one device wants to send data to another, it sends the data to the controller, which then relays the data to the other connected device.
- The star topology is **used in local-area networks (LANs).**
- High-speed LANs often use a star topology with a central hub.

Advantages

- A **star topology is less expensive than a mesh topology**. In a star, each device needs only one link and one I/O port to connect it to any number of others. This factor also makes it easy to install and reconfigure. Far less cabling needs to be housed, and additions, moves, and deletions involve only one connection: between that device and the hub.
- **Robustness**. If one link fails, only that link is affected. All other links remain active. This factor also lends itself to easy fault identification and fault isolation. As long as the hub is working, it can be used to monitor link problems and bypass defective links.

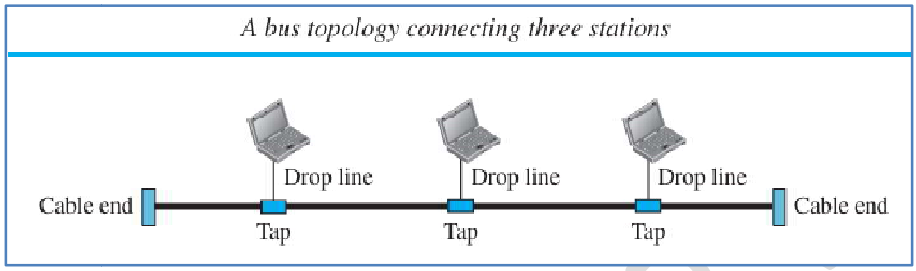
Disadvantages

- The **dependency of the whole topology on one single point, the hub**. **If the hub goes down, the whole system is dead**.
- Although a star requires far less cable than a mesh, each node must be linked to a central hub. For this reason, **often more cabling is required in a star** than in some other topologies (such as ring or bus).

Prof Parthasarathy P V

3. Bus Topology

- A bus topology is multipoint. One long cable acts as a backbone to link all the devices in a network.
- Nodes are connected to the bus cable by drop lines and taps. A **drop line** is a connection running between the **device and the main cable**.
- A **tap** is a connector that either splices into the main cable or punctures the sheathing of a cable to create a contact with the metallic core. As a signal travels along the backbone, some of its energy is transformed into heat. Therefore, it becomes weaker and weaker as it travels farther and farther. For this reason there is a limit on the number of taps a bus can support and on the distance between those taps.



Advantages

- **Ease of installation** - Backbone cable can be laid along the most efficient path, and then connected to the nodes by drop lines of various lengths. In this way, a bus uses less cabling than mesh or star topologies.
- In a star, for example, four network devices in the same room require four lengths of cable reaching all the way to the hub. **In a bus, this redundancy is eliminated**. Only the backbone cable stretches through the entire facility. Each drop line has to reach only as far as the nearest point on the backbone.

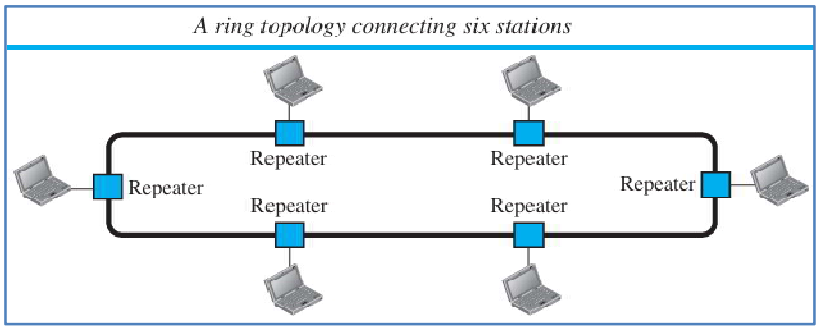
Disadvantages

- **Difficult in reconnection and fault isolation.** A bus is usually designed to be optimally efficient at installation. It can therefore be difficult to add new devices. Signal reflection at the taps can cause degradation in quality. This degradation can be controlled by limiting the number and spacing of devices connected to a given length of cable.
- Adding new devices may therefore require modification or replacement of the backbone.
- In addition, a fault or break in the bus cable stops all transmission, even between devices on the same side of the problem. The damaged area reflects signals back in the direction of origin, creating noise in both directions. Bus topology was the one of the first topologies used in the design of early local area networks. Traditional Ethernet LANs can use a bus topology.

---

4. Ring Topology

- In a ring topology, each device has a dedicated point-to-point connection with only the two devices on either side of it.
- A signal is passed along the ring in one direction, from device to device, until it reaches its destination.
- Each device in the ring incorporates a repeater. When a device receives a signal intended for another device, its repeater regenerates the bits and passes them along.



Prof. Parthasarathy P

Advantages

- A ring is relatively easy to install and reconfigure.
- Each device is linked to only its immediate neighbors (either physically or logically).
- To add or delete a device requires changing only two connections. The only constraints are media and traffic considerations (maximum ring length and number of devices).
- Fault isolation is simplified. Generally, in a ring a signal is circulating at all times. If one device does not receive a signal within a specified period, it can issue an alarm. The alarm alerts the network operator to the problem and its location.

Disadvantages

- Unidirectional traffic can be a disadvantage. In a simple ring, a break in the ring (such as a disabled station) can disable the entire network. This weakness can be solved by using a dual ring or a switch capable of closing off the break.

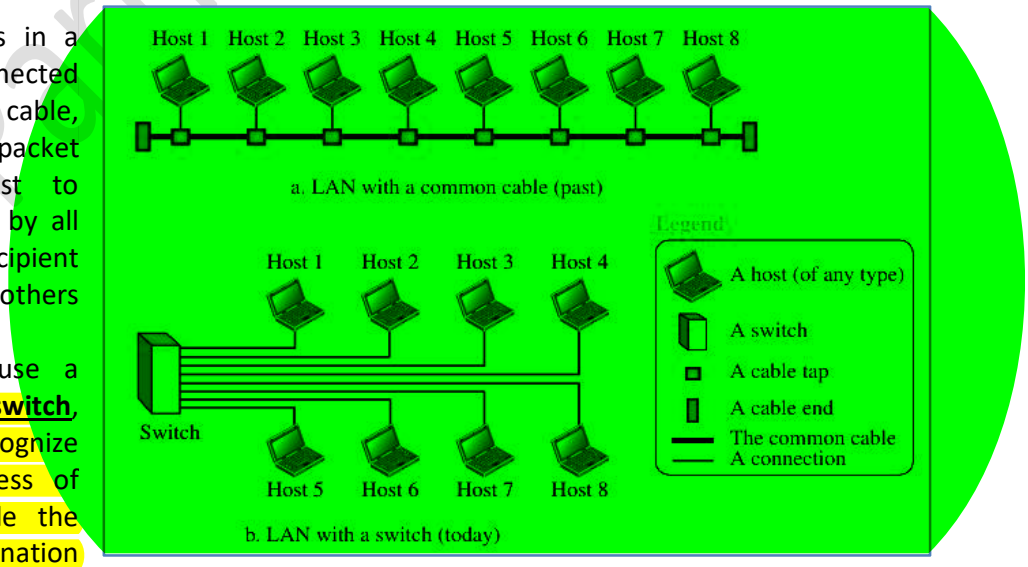
Network Types (Geographical coverage)

1. **Local Area Network**
2. **Wide Area Network**
3. **Switching**

---

1. Local Area Network

- A local area network (LAN) is usually privately owned and connects some hosts in a single office, building, or campus.
- Depending on the needs of an organization, a LAN can be as simple as two PCs and a printer in someone's home office, or it can extend throughout a company and include audio and video devices.
- Each host in a LAN has an identifier, an address that uniquely defines the host in the LAN.
- A packet sent by a host to another host carries both the source host's and the destination host's addresses.
- In the past, all hosts in a network were connected through a common cable, which meant that a packet sent from one host to another was received by all hosts. The intended recipient kept the packet; the others dropped the packet.
- Today, most LANs use a **smart connecting switch**, which is able to recognize the destination address of the packet and guide the packet to its destination without sending it to all other hosts.



- The switch improve the traffic in the LAN and allows more than one pair to communicate with each other at the same time if there is no common source and destination among them.
- Note that the above definition of a LAN does not define the minimum or maximum number of hosts in a LAN. Figure shows a LAN using either a common cable or a switch.

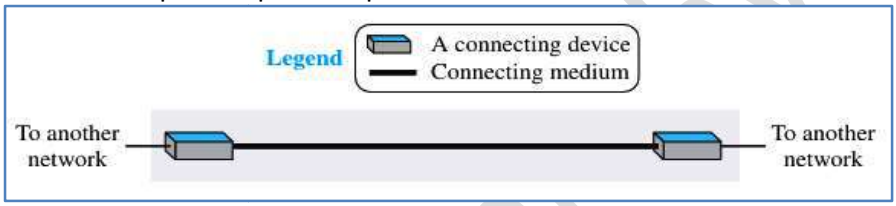
Prof. Parthasarathy P V

2. Wide Area Network

- A wide area network (WAN) is also an interconnection of devices capable of communication.
- However, there are some differences between a LAN and a WAN. A LAN is normally limited in size, spanning an office, a building, or a campus; a WAN has a wider geographical span, spanning a town, a state, a country, or even the world.
- A **LAN interconnects hosts**; a **WAN interconnects connecting devices such as switches, routers, or modems**.
- A LAN is normally privately owned by the organization that uses it; a WAN is normally created and run by communication companies and leased by an organization that uses it.
- Two distinct examples of WANs today:
  1. Point-to-point WANs
  2. Switched WANs.

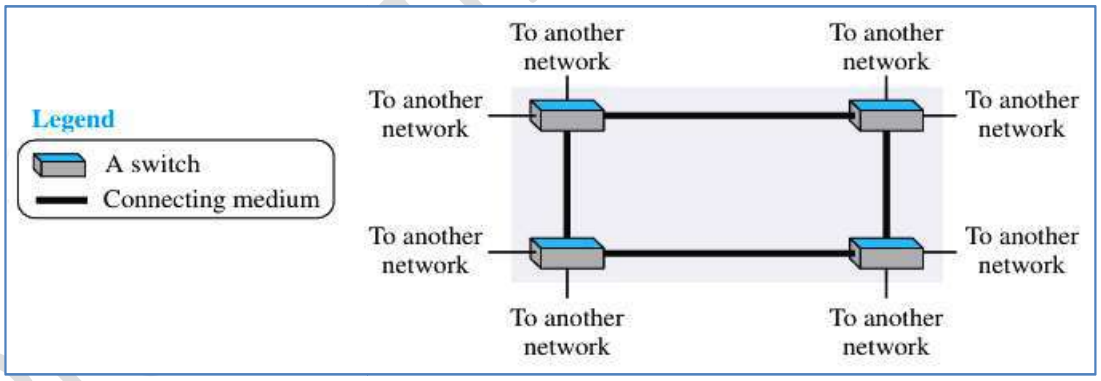
Point-to-Point WAN

- A point-to-point WAN is a network that **connects two communicating devices through a transmission media (cable or air)**.
- Figure shows an example of a point-to-point WAN.



Switched WANs

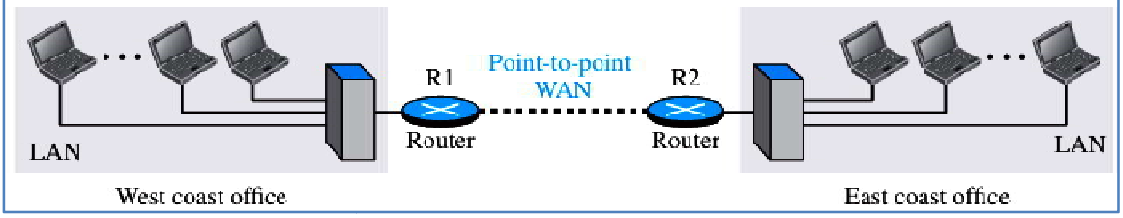
- A switched WAN is a network with more than two ends. A switched WAN is used in the backbone of global communication today. We can say that a switched WAN is a combination of several point-to-point WANs that are connected by switches. Figure shows an example of a switched WAN.



Internetwork

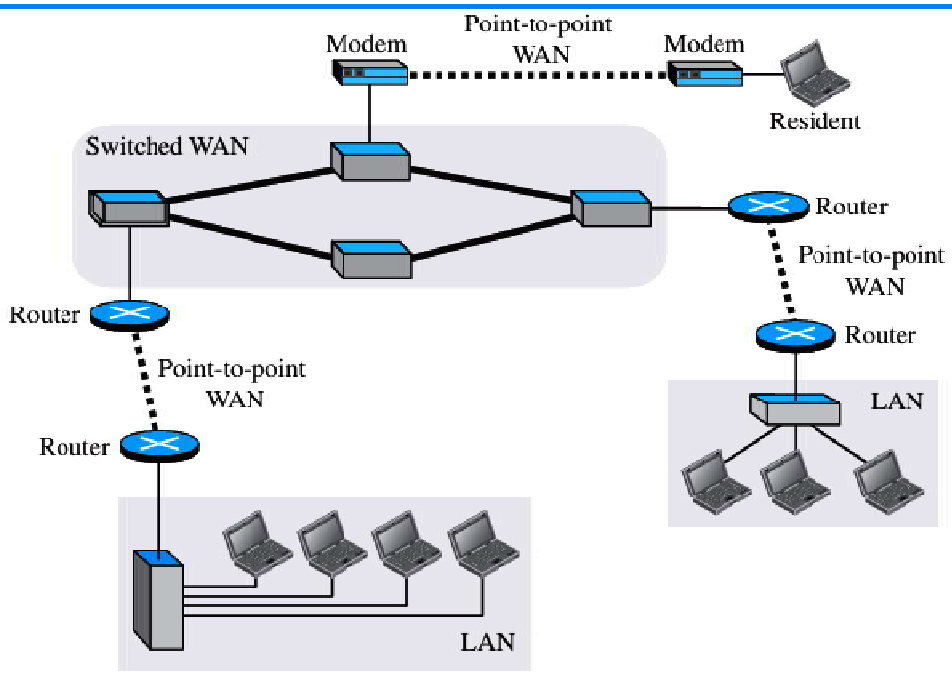
- When two or more networks (a LAN or a WAN) are connected, they make an internetwork, or internet.
- As an example, assume that an organization has two offices, one on the east coast and the other on the west coast. Each office has a LAN that allows all employees in the office to communicate with each other. To make the communication between employ ees at different offices possible, the management leases a point-to-point dedicated WAN from a service provider, such as a telephone company, and connects the two LANs.
- Now the company has an internetwork, or a private internet. Communication between offices is now possible. Figure shows this internet.

Figure *An internetwork made of two LANs and one point-to-point WAN*



- When a host in the west coast office sends a message to another host in the same office, the **router blocks the message, but the switch directs the message to the destination**. On the other hand, when a host on the west coast sends a message to a host on the east coast, router R1 routes the packet to router R2, and the packet reaches the destination. The following figure shows another internet with several LANs and WANs connected. One of the WANs is a switched WAN with four switches.

Figure *A heterogeneous network made of four WANs and three LANs*

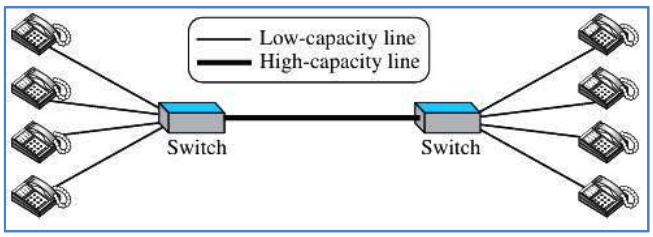


3. Switching

- An **internet is a switched network in which a switch connects at least two links together**.
- A **switch needs to forward data from a network to another network when required**.
- The **two most common types of switched networks** are
  1. circuit-switched networks
  2. packet-switched networks

Circuit-switched networks

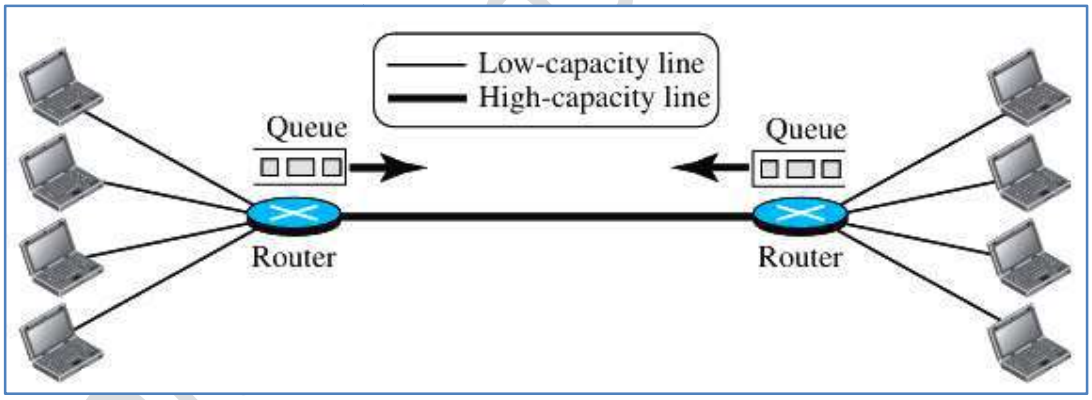
- In a circuit-switched network, a dedicated connection, called a circuit, is always available between the two end systems; the switch can only make it active or inactive. The following figure shows a very simple switched network that connects four telephones to each end.
- We have used telephone sets instead of computers as an end system because circuit switching was very common in telephone networks in the past.



- In Figure, the four telephones at each side are connected to a switch. The switch connects a telephone set at one side to a telephone set at the other side.
- The thick line connecting two switches is a high-capacity communication line that can handle four voice communications at the same time; the capacity can be shared between all pairs of telephone sets. The switches used in this example have forwarding tasks but no storing capability.
- Let us look at two cases.
  - In the first case, all telephone sets are busy; four people at one site are talking with four people at the other site; the capacity of the thick line is fully used.
  - In the second case, only one telephone set at one side is connected to a tele phone set at the other side; only one-fourth of the capacity of the thick line is used.
- This means that a circuit-switched network is efficient only when it is working at its full capacity; most of the time, it is inefficient because it is working at partial capacity.
- The reason that we need to make the capacity of the thick line four times the capacity of each voice line is that we do not want communication to fail when all telephone sets at one side want to be connected with all telephone sets at the other side.

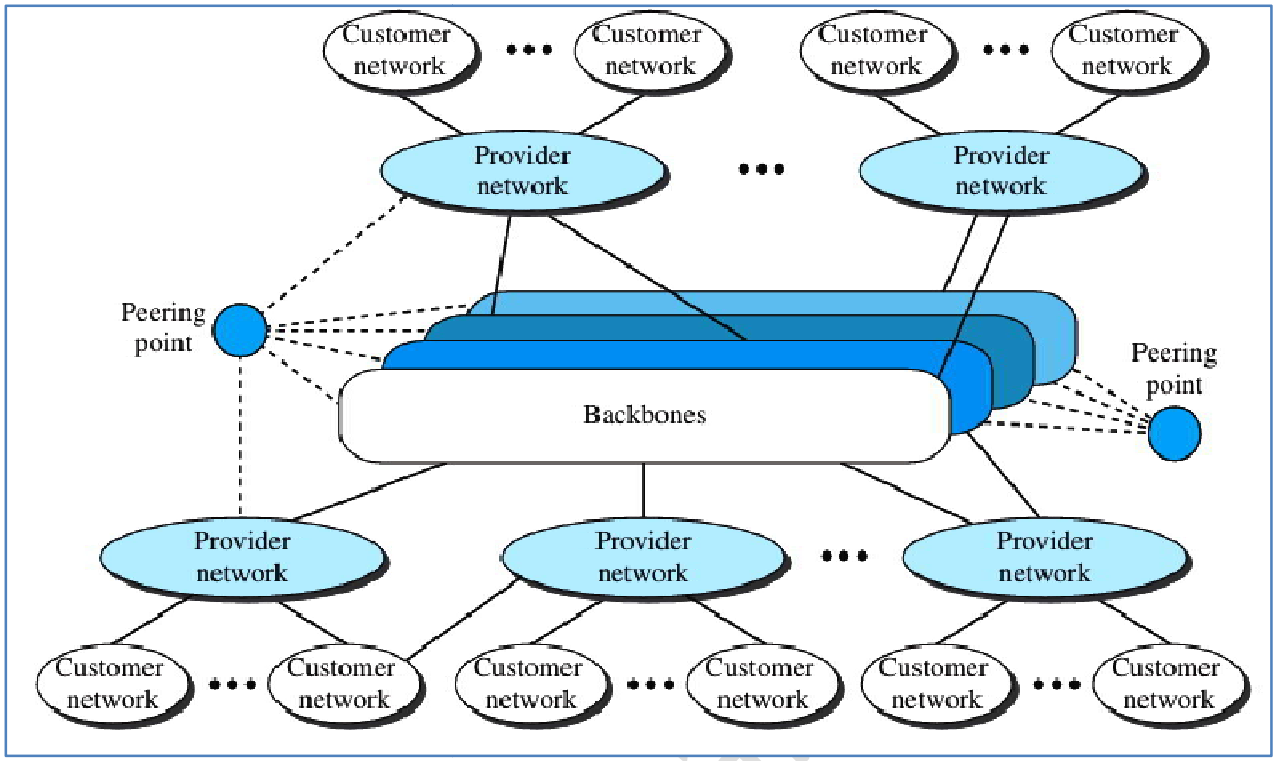
Packet-switched networks

- In a computer network, the communication between the two ends is done in blocks of data called packets. In other words, instead of the continuous communication we see between two telephone sets when they are being used, we see the exchange of individual data packets between the two computers. This allows us to make the switches function for both storing and forwarding because a packet is an independent entity that can be stored and sent later.
- The following figure shows a small packet-switched network that con nects four computers at one site to four computers at the other site.



- A router in a packet-switched network has a queue that can store and forward the packet.
- Now assume that the capacity of the thick line is only twice the capacity of the data line connecting the computers to the routers. If only two computers (one at each site) need to communicate with each other, there is no waiting for the packets.
- However, if packets arrive at one router when the thick line is already working at its full capacity, the packets should be stored and forwarded in the order they arrived.
- The two simple examples show that a packet-switched network is more efficient than a circuit switched network, but the packets may encounter some delays.

The Internet



- Backbones and provider networks are also called **Internet Service Providers (ISPs).**
- The backbones are often referred to as international ISPs; the **provider networks are often referred to as national or regional ISPs.**

Accessing the Internet

- The Internet today is an internetwork that allows any user to become part of it. The user, however, needs to be physically connected to an ISP. The physical connection is normally done through a point-to-point WAN. They are
  1. ***Using Telephone Networks***
  2. ***Using Cable Networks***
  3. ***Using Wireless Networks***
  4. ***Direct Connection to the Internet***

Using Telephone Networks

- Today most residences and small businesses have telephone service, which means they are connected to a telephone network.
- Since most telephone networks have already connected themselves to the Internet, one option for residences and small businesses to connect to the Internet is to change the voice line between the residence or business and the telephone center to a point-to-point WAN. This can be done in two ways.
  1. Dial-up service
  2. DSL Service

Using Cable Networks

- More and more residents over the last two decades have begun using cable TV services instead of antennas to receive TV broadcasting. The cable companies have been upgrading their cable networks and connecting to the Internet. A residence or a small business can be connected to

Prof. Pathasa

the Internet by using this service. It provides a higher speed connection, but the speed varies depending on the number of neighbors that use the same cable.

Using Wireless Networks

- Wireless connectivity has recently become increasingly popular. A household or a small business can use a combination of wireless and wired connections to access the Internet. With the growing wireless WAN access, a household or a small business can be connected to the Internet through a wireless WAN.

Direct Connection to the Internet

- A large organization or a large corporation can itself become a local ISP and be con nected to the Internet. This can be done if the organization or the corporation leases a high-speed WAN from a carrier provider and connects itself to a regional ISP. For example, a large university with several campuses can create an internetwork and then connect the internetwork to the Internet.

Prof. Parthasarathy P V

Chapter 2: Networks Models: Protocol Layering, TCP/IP Protocol suite, The OSI model

- **Two models** have been devised to define computer network operations:
  1. the **TCP/IP protocol suite** and
  2. the **OSI model**.
- In this chapter, we first discuss a general subject, **protocol layering**, which is used in both models.
- We then concentrate on the TCP/IP protocol suite, on which the book is based.

PROTOCOL LAYERING (Scenarios - Principles of Protocol Layering - Logical Connections)

- In data communication and networking, **a protocol defines the rules that both the sender and receiver and all intermediate devices need to follow to be able to communicate effectively**.
- When communication is simple, we may need only one simple protocol; **when the communication is complex**, we may need to divide the task between different layers, in which case we need a protocol at each layer, or **protocol layering**.

Scenarios

- Let us develop two simple scenarios to better understand the need for protocol layering.

First Scenario

- In the first scenario, communication is so simple that it can occur in only one layer. Assume Maria and Ann are neighbors with a lot of common ideas. Communication between Maria and Ann takes place in one layer, face to face, in the same language, as shown in Figure.

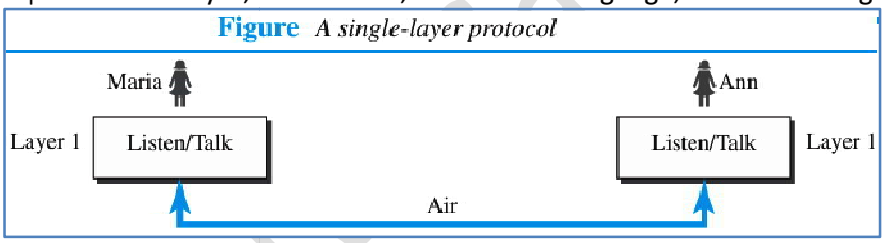


Figure *A single-layer protocol*

- Even in this simple scenario, we can see that a set of rules needs to be followed.
  - First, Maria and Ann know that they should greet each other when they meet.
  - Second, they know that they should confine their vocabulary to the level of their friendship.
  - Third, each party knows that she should refrain from speaking when the other party is speaking.
  - Fourth, each party knows that the conversation should be a dialog, not a monolog: both should have the opportunity to talk about the issue.
  - Fifth, they should exchange some nice words when they leave.

Second Scenario

- In the second scenario, we assume that Ann is offered a higher-level position in her company, but needs to move to another branch located in a city very far from Maria.
- The two friends still want to continue their communication and exchange ideas because they have come up with an innovative project to start a new business when they both retire.
- They decide to continue their conversation using regular mail through the post office.

Prof. Partha Pratim Ray

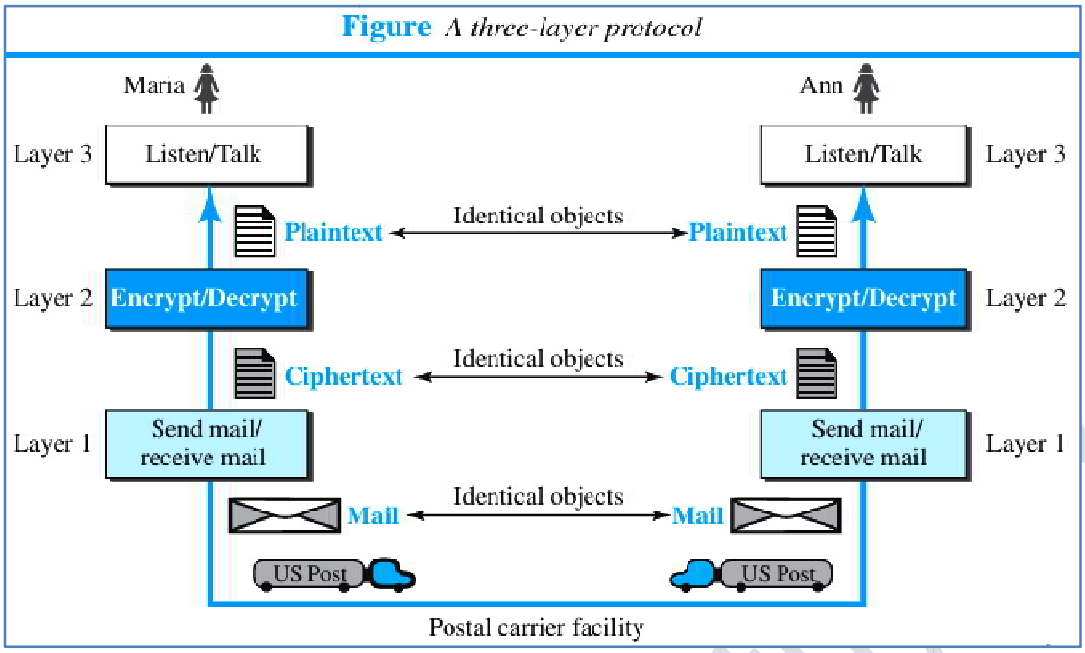


Figure *A three-layer protocol*

- **Protocol layering enables us to divide a complex task into several smaller and simpler tasks**.
- For example, in Figure, we could have used only one machine to do the job of all three machines. However, if Maria and Ann decide that the encryption/ decryption done by the machine are not enough to protect their secrecy, they would have to change the whole machine. **In the present situation, they need to change only the second layer machine**; the other two can remain the same. This is referred to as modularity. Modularity in this case means independent layers. A layer (module) can be defined as a black box with inputs and outputs, without concern about how inputs are changed to outputs. If two machines provide the same outputs when given the same inputs, they can replace each other. For example, Ann and Maria can buy the second layer machine from two different manufacturers. As long as the two machines create the same cipher text from the same plaintext and vice versa, they do the job.

Advantages of protocol layering

- Protocol layering allows us to separate the services from the implementation. A layer needs to be able to receive a set of services from the lower layer and to give the services to the upper layer; we don't care about how the layer is implemented.
- Protocol layering in the Internet, is that communication does not always use only two end systems; there are intermediate systems that need only some layers, but not all layers. If we did **not use protocol layering, we would have to make each intermediate system as complex as the end systems, which makes the whole system more expensive**.

***

Principles of Protocol Layering

First Principle

- **If we want bidirectional communication, we need to make each layer so that it is able to perform two opposite tasks, one in each direction**. For example, the third layer task is to listen (in one direction) and talk (in the other direction). The second layer needs to be able to encrypt and decrypt. The first layer needs to send and receive mail.

Second Principle

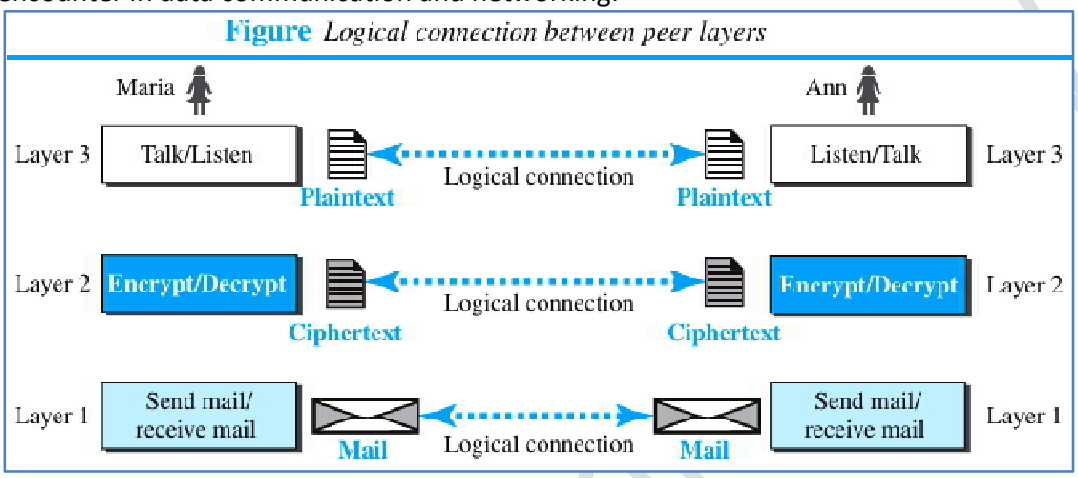
Prof. Parthasarathi

- The two objects under each layer at **both sites should be identical**. For example, the object under layer 3 at both sites should be a plaintext letter. The object under layer 2 at both sites should be a ciphertext letter. The object under layer 1 at both sites should be a piece of mail.

Logical Connections

- After following the above two principles, we can think about logical connection between each layer as shown in Figure below.
- This means that we have layer-to-layer communication.
- The concept of logical connection will help us better understand the task of layering we encounter in data communication and networking.

Figure *Logical connection between peer layers*



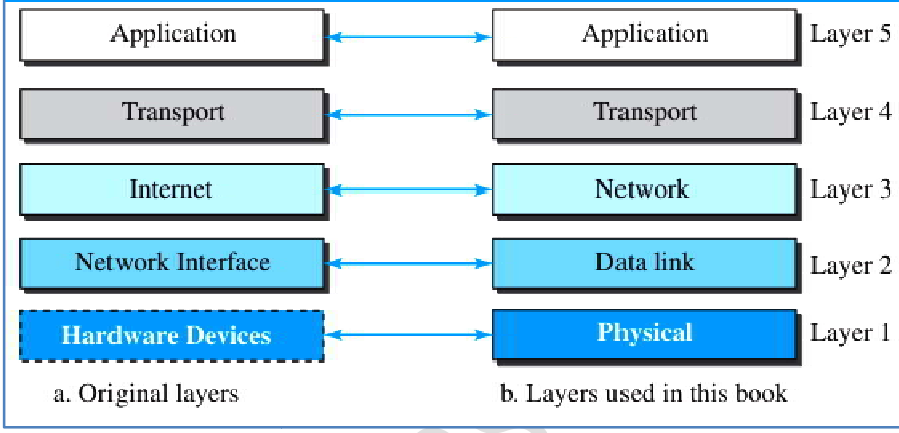
Prof. Parthasa

TCP/IP Protocol suite (P-35)

***Topics Covered*: Layered Architecture - Layers in the TCP/IP Protocol Suite - Description of Each Layer - Encapsulation and Decapsulation – Addressing - Multiplexing and Demultiplexing**

- **TCP/IP (Transmission Control Protocol/Internet Protocol)** is a protocol suite (a set of protocols organized in different layers) used in the Internet today. It is a hierarchical protocol made up of interactive modules, each of which provides a specific functionality.
- The term hierarchical means that each upper level protocol is supported by the services provided by one or more lower level protocols.
- The original TCP/IP protocol suite was defined as four software layers built upon the hardware. Today, however, TCP/IP is thought of as a five-layer model.

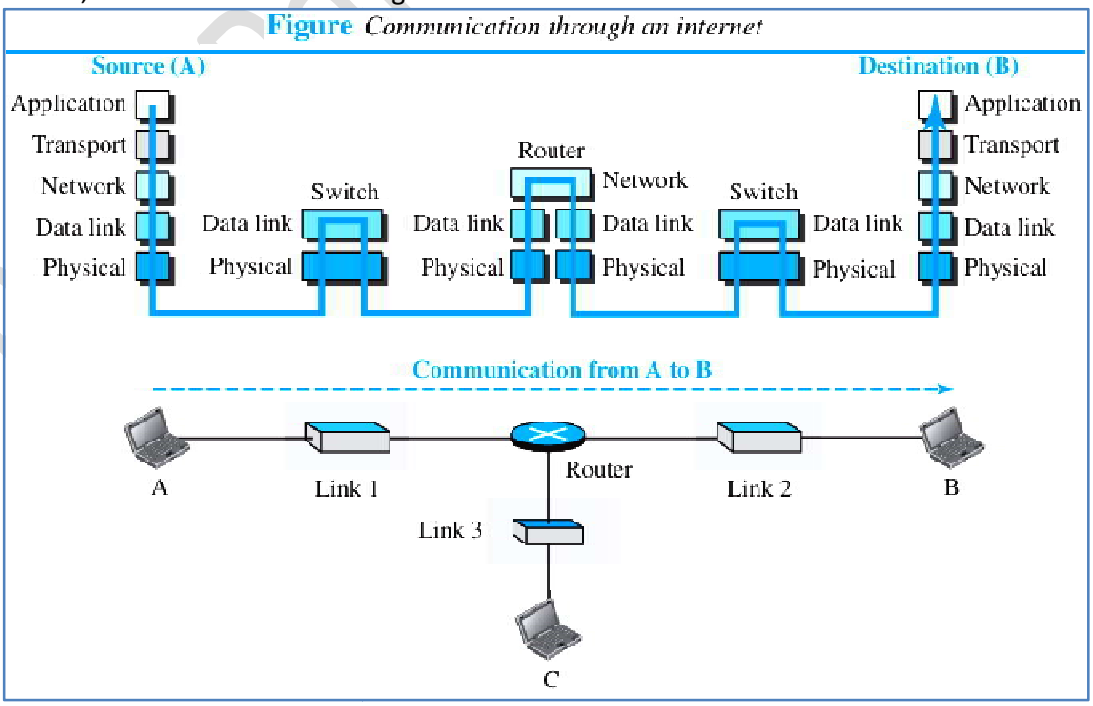
Figure *Layers in the TCP/IP protocol suite*



Layered Architecture (Functions and responsibilities)

- To show how the layers in the TCP/IP protocol suite are involved in communication between two hosts, we assume that we want to use the suite in a small internet made up of three LANs (links), each with a link-layer switch. We also assume that the links are connected by one router, as shown in the below figure.

Figure *Communication through an internet*

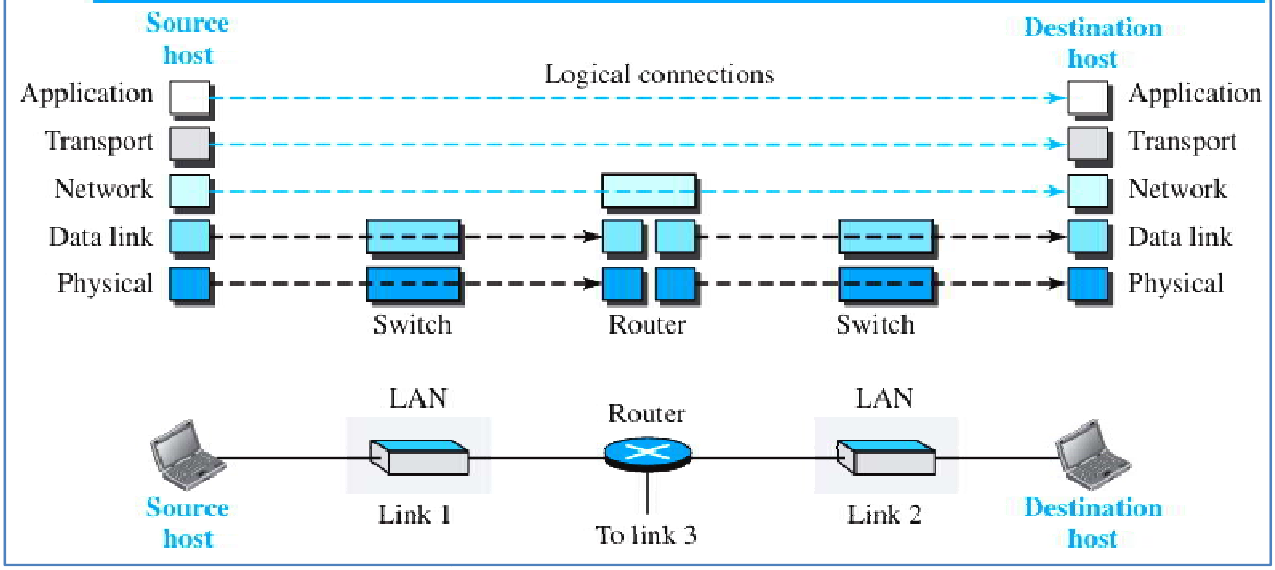


- Let us assume that computer A communicates with computer B.
- As the figure shows, we have five communicating devices in this communication:
    1. source host (computer A)
    2. the link-layer switch in link 1
    3. the router
    4. the link-layer switch in link 2, and
    5. the destination host (computer B).
- Each device is involved with a set of layers depending on the role of the device in the internet.
- The two hosts are involved in all five layers; the source host needs to create a message in the application layer and send it down the layers so that it is physically sent to the destination host.
- The destination host needs to receive the communication at the physical layer and then deliver it through the other layers to the application layer.
- ***The router is involved in only three layers***; there is no transport or application layer in a router as long as the router is used only for routing. Although a router is always involved in one network layer, it is involved in n combinations of link and physical layers in which **"n"** is the number of links the router is connected to.
- The reason is that each link may use its own data-link or physical protocol. For example, in the above figure, the router is involved in three links, but the message sent from source A to destination B is involved in two links.
- Each link may be using different link-layer and physical-layer protocols; the router needs to receive a packet from link 1 based on one pair of proto cols and deliver it to link 2 based on another pair of protocols.
- A ***link-layer switch in a link, however, is involved only in two layers, data-link and physical***. Although each switch in the above figure has two different connections, the connections are in the same link, which uses only one set of protocols. This means that, unlike a router, a link-layer switch is involved only in one data-link and one physical layer.

Layers in the TCP/IP Protocol Suite

- To better understand the duties of each layer, we need to think about the logical connections between layers. The following figure shows logical connections in our simple internet.

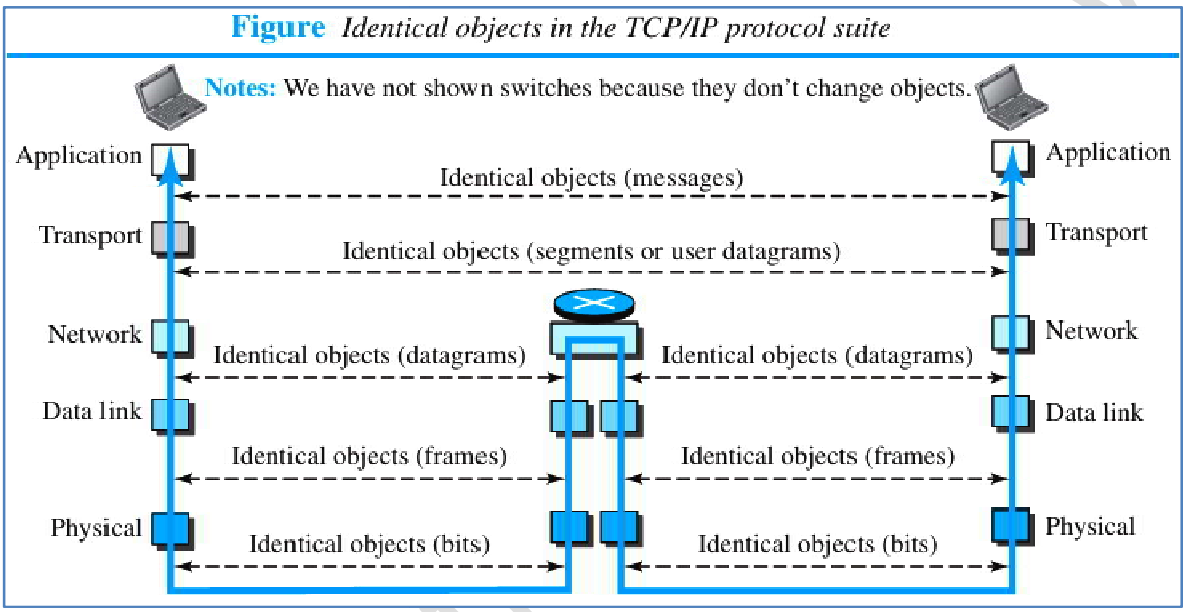
Figure *Logical connections between layers of the TCP/IP protocol suite*



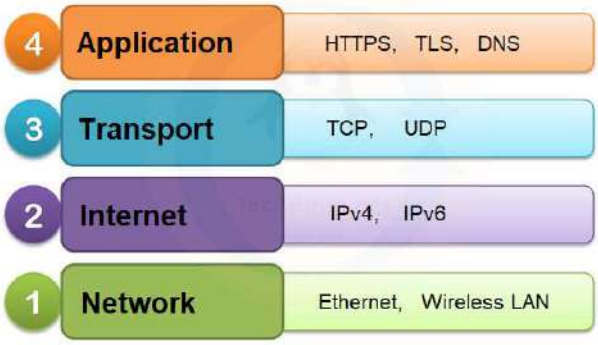
- Using logical connections makes it easier for us to think about the duty of each layer. As the figure shows, the ***duty of the application, transport, and network layers is end-to-end***.

- However, the duty of the data-link and physical layers is hop-to-hop, in which a hop is a host or router. In other words, the ***domain of duty of the top three layers is the internet***, and the ***domain of duty of the two lower layers is the link***.
- Another way of thinking of the logical connections is to **think about the data unit created from each layer**.
- In the ***top three layers, the data unit (packets) should not be changed by any router or link-layer switch***.
- In the ***bottom two layers***, the ***packet created by the host is changed only by the routers***, not by the link-layer switches.
- The following figure shows the second principle discussed previously for protocol layering. We show the identical objects below each layer related to each device.

Figure *Identical objects in the TCP/IP protocol suite*



- Note that, although the logical connection at the network layer is between the two hosts, we can only say that identical objects exist between two hops in this case because a ***router may fragment the packet at the network layer*** and send more packets than received. Note that the link between two hops does not change the object.
- **TCP/IP MODEL PROTOCOLS OF EACH LAYER**



Description of Each Layer

1. Physical Layer

- Physical layer is responsible for carrying individual bits in a frame across the link.
- Although the physical layer is the lowest level in the TCP/IP protocol suite, the communication between two devices at the physical layer is still a logical communication because there is another, hidden layer, the transmission media, under the physical layer.
- Two devices are connected by a ***transmission medium (cable or air).***
- We need to know that the ***transmission medium does not carry bits; it carries electrical or optical signals***. So the ***bits received in a frame from the data-link layer are transformed and sent through the transmission media***, but we can think that the logical unit between two physical layers in two devices is a bit.
- There are several protocols that transform a bit to a signal.

2. Data-link Layer

- An internet is made up of several ***links (LANs and WANs) connected by routers***. There may be several overlapping sets of links that a ***datagram*** can travel from the host to the destination.
- The ***routers are responsible for choosing the best links***. However, when the next link to travel is determined by the router, the data-link layer is responsible for taking the datagram and moving it across the link.
- The link can be a ***wired LAN with a link-layer switch, a wireless LAN, a wired WAN, or a wireless WAN***.
- We can also have different protocols used with any link type. In each case, ***the data-link layer is responsible for moving the packet through the link***.
- ***TCP/IP*** does not define any specific protocol for the data-link layer. It supports all the standard and proprietary protocols.
- Any protocol that can take the ***datagram*** and carry it through the link suffices for the network layer. The data-link layer takes ***a datagram and encapsulates it in a packet called a frame***.
- Each link-layer protocol may provide a different service. ***Some link-layer protocols provide complete error detection and correction, some provide only error correction***.

3. Network Layer

- The ***network layer is responsible for creating a connection between the source computer and the destination computer***. The ***communication at the network layer is host-to-host***.
- The network layer is responsible for host-to-host communication and routing the packet through possible routes.
- Why we need the network layer?
  - One reason is the separation of different tasks between different layers.
  - The second reason is that the routers do not need the application and transport layers. Separating the tasks allows us to use fewer protocols on the routers.
- The ***network layer in the Internet includes the main protocol, Internet Protocol (IP), that defines the format of the packet, called a datagram at the network layer***. IP also defines the format and the structure of addresses used in this layer. ***IP is also responsible for routing a packet from its source to its destination***, which is achieved by each router forwarding the datagram to the next router in its path.
- **IP is a connectionless protocol that provides no flow control, no error control, and no congestion control services.** The net work layer also includes ***unicast (one-to-one) and multicast (one-to-many) routing protocols***.

Prof. Parthasarathy P V

- ***A routing protocol does not take part in routing (it is the responsibility of IP), but it creates forwarding tables for routers to help them in the routing process.***
- The network layer also has some auxiliary protocols that help IP in its delivery and routing tasks.
- The ***Internet Control Message Protocol (ICMP)*** helps IP to ***report some problems when routing a packet***.
- The ***Internet Group Management Protocol (IGMP)*** is another protocol that helps IP in ***multitasking***.
- The ***Dynamic Host Configuration Protocol (DHCP)*** helps IP to get the network-layer address for a host.
- The ***Address Resolution Protocol (ARP)*** is a protocol that ***helps IP to find the link-layer address of a host or a router when its network-layer address is given***.

4. Transport Layer

- The logical connection at the transport layer is also end-to-end. The ***transport layer at the source host gets the message from the application layer, encapsulates it in a transport layer packet (called a segment or a user datagram in different protocols) and sends it, through the logical (imaginary) connection, to the transport layer at the destination host***.
- In other words, the transport layer is responsible for giving services to the application layer: to get a message from an application program running on the source host and deliver it to the corresponding application program on the destination host.
- The transport layer should be independent of the application layer. In addition, we will see that we have more than one protocol in the transport layer, which means that each application program can use the protocol that best matches its requirement.
- The main protocol, ***Transmission Control Protocol (TCP), is a connection-oriented protocol*** that first establishes a logical connection between transport layers at two hosts before transferring data. It ***creates a logical pipe between two TCPs for transferring a stream of bytes***. TCP provides flow control (matching the sending data rate of the source host with the receiving data rate of the destination host to prevent overwhelming the destination), error control (to guarantee that the segments arrive at the destination without error and resending the corrupted ones), and congestion control to reduce the loss of segments due to congestion in the network.
- The other common protocol, ***User Datagram Protocol (UDP), is a connectionless protocol that transmits user datagrams without first creating a logical connection***. In UDP, each user datagram is an independent entity without being related to the previous or the next one (the meaning of the term connectionless). ***UDP is a simple protocol that does not provide flow, error, or congestion control***. Its simplicity, which means small overhead, is attractive to an application program that needs to send short messages and cannot afford the retransmission of the packets involved in TCP, when a packet is corrupted or lost.
- A new protocol, ***Stream Control Transmission Protocol (SCTP) is designed to respond to new applications that are emerging in the multimedia.***

5. Application Layer

- The logical connection between the two application layers is end to-end. The two application layers exchange messages between each other as though there were a bridge between the two layers. However, we should know that the communication is done through all the layers.
- Communication at the application layer is between two processes (two programs running at this layer). To communicate, a process sends a request to the other process and receives a response. Process-to-process communication is the duty of the application layer. The

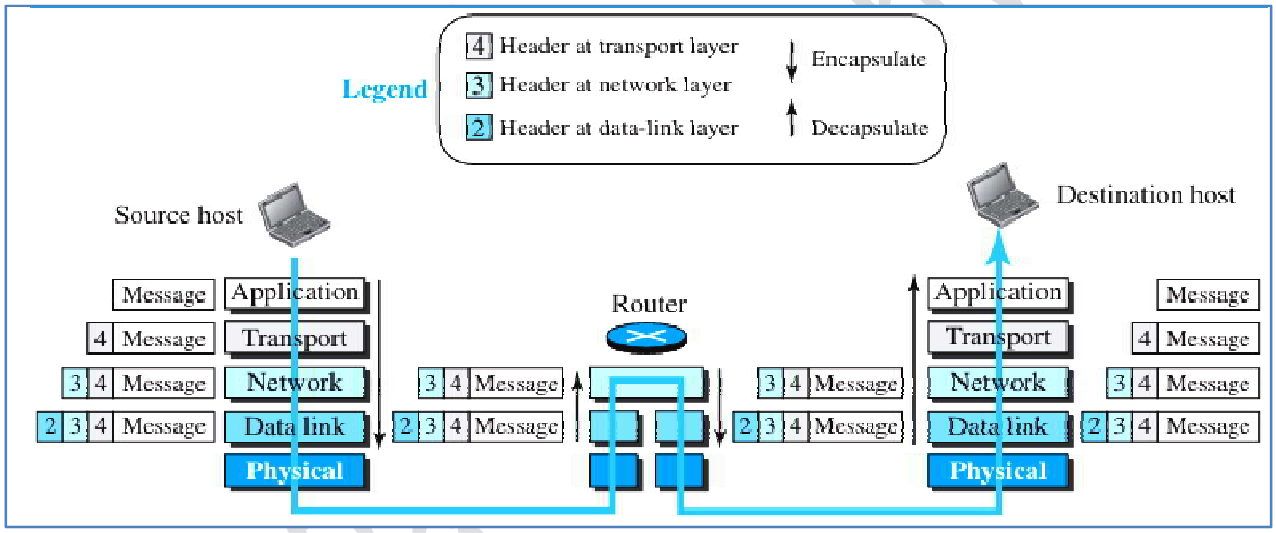
Prof. Parthasarathy P V

application layer in the Internet includes many predefined protocols, but a user can also create a pair of processes to be run at the two hosts.

- The ***Hypertext Transfer Protocol (HTTP)*** is a vehicle for accessing the ***World Wide Web (WWW).*** The ***Simple Mail Transfer Protocol (SMTP)*** is the main protocol used in ***electronic mail (e-mail) service***. The ***File Transfer Protocol (FTP)*** is used for transferring files from one host to another. The ***Terminal Network (TELNET) and Secure Shell (SSH)*** are used for ***accessing a site remotely***.
- The ***Simple Network Management Protocol (SNMP) is used by an administrator to manage the Internet at global and local levels***.
- The **Domain Name System (DNS) is used by other protocols to find the network-layer address of a computer**.
- The ***Internet Group Management Protocol (IGMP)*** is used to ***collect membership in a group***.

Encapsulation and Decapsulation

- One of the important concepts in protocol layering in the Internet is encapsulation/ decapsulation.



- We have not shown the layers for the link-layer switches because ***no encapsulation/ decapsulation occurs in this device***. In Figure 2.8, we show the encapsulation in the source host, decapsulation in the destination host, and encapsulation and decapsulation in the router.

Encapsulation at the Source Host

- At the source, we have only encapsulation.

1. ***At the application layer, the data to be exchanged is referred to as a message***. A message normally does not contain any header or trailer, but if it does, we refer to the whole as the message. The ***message is passed to the transport layer***.
2. The ***transport layer takes the message as the payload***, the load that the transport layer should take care of. It adds the transport layer header to the payload, which contains the identifiers of the source and destination application programs that want to communicate plus some more information that is needed for the end-to end delivery of the message, such as information needed for flow, error control, or congestion control. ***The result is the transport-layer packet, which is called the segment (in TCP) and the user datagram (in UDP).*** The transport layer then passes the packet to the network layer.
3. The network layer takes the transport-layer packet as data or payload and adds its own header to the payload. The header contains the addresses of the source and destination

hosts and some more information used for error checking of the header, fragmentation information, and so on. The result is the network-layer packet, called a datagram. The network layer then passes the packet to the data-link layer.

4. The data-link layer takes the network-layer packet as data or payload and adds its own header, which contains the link-layer addresses of the host or the next hop (the router). The result is the link-layer packet, which is called a frame. The frame is passed to the physical layer for transmission.

Decapsulation and Encapsulation at the Router

- At the router, we have both decapsulation and encapsulation because the router is connected to two or more links.
  1. After the set of bits are delivered to the data-link layer, this layer decapsulates the datagram from the frame and passes it to the network layer.
  2. The network layer only inspects the source and destination addresses in the datagram header and consults its forwarding table to find the next hop to which the datagram is to be delivered. The contents of the datagram should not be changed by the network layer in the router unless there is a need to fragment the datagram if it is too big to be passed through the next link. The datagram is then passed to the data-link layer of the next link.
  3. The data-link layer of the next link encapsulates the datagram in a frame and passes it to the physical layer for transmission.

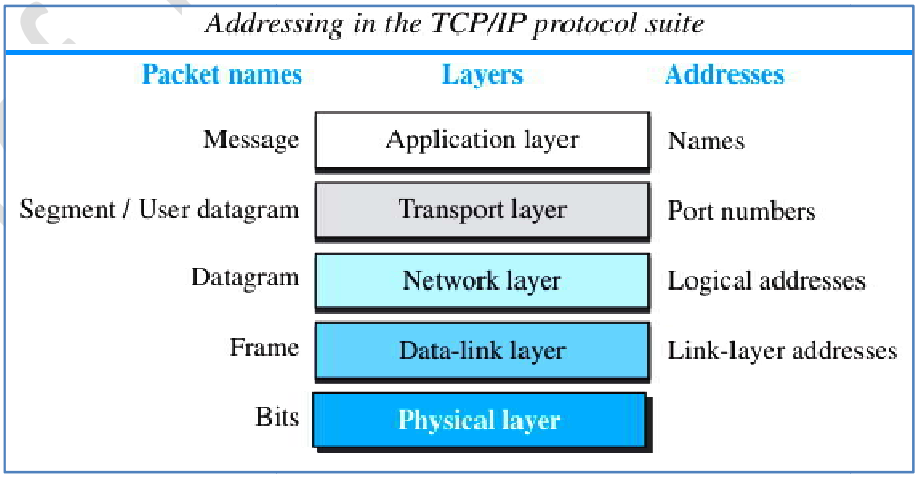
Decapsulation at the Destination Host

- At the destination host, each layer only decapsulates the packet received, removes the payload, and delivers the payload to the next-higher layer protocol until the message reaches the application layer. It is necessary to say that decapsulation in the host involves error checking.

Addressing

- Any communication that involves ***two parties needs two addresses***: ***source address*** and ***destination address***.
- Although it looks as if we need ***five pairs of addresses, one pair per layer, we normally have only four because the physical layer does not need addresses;*** the unit of data exchange at the physical layer is a bit, which definitely cannot have an address.
- The following Figure shows the addressing at each layer.

Addressing in the TCP/IP protocol suite



- As the figure shows, there is a relationship between the layer, the address used in that layer, and the packet name at that layer.

Prof. Parthasarathy P V

- At the application layer, we normally use names to define the site that provides services, such as ***someorg.com***, or the ***e-mail address, such as somebody@coldmail.com***.
- At the ***transport layer, addresses are called port numbers***, and these define the application-layer programs at the source and destination. ***Port numbers are local addresses*** that distinguish between several programs running at the same time.
- At the ***network-layer, the addresses are global, with the whole Internet as the scope***. A network-layer address uniquely defines the connection of a device to the Internet.
- The link-layer addresses, sometimes called ***MAC addresses***, are locally defined addresses, each of which defines a specific host or router in a network (***LAN or WAN***).

Multiplexing and Demultiplexing

- Since the TCP/IP protocol suite uses several protocols at some layers, we can say that we have ***multiplexing at the source*** and ***demultiplexing at the destination***.
- Multiplexing in this case means that a protocol at a layer can encapsulate a packet from several next-higher layer protocols (one at a time); demultiplexing means that a protocol can decapsulate and deliver a packet to several next-higher layer protocols (one at a time).
- The following Figure shows the concept of multiplexing and demultiplexing at 3 upper layers.

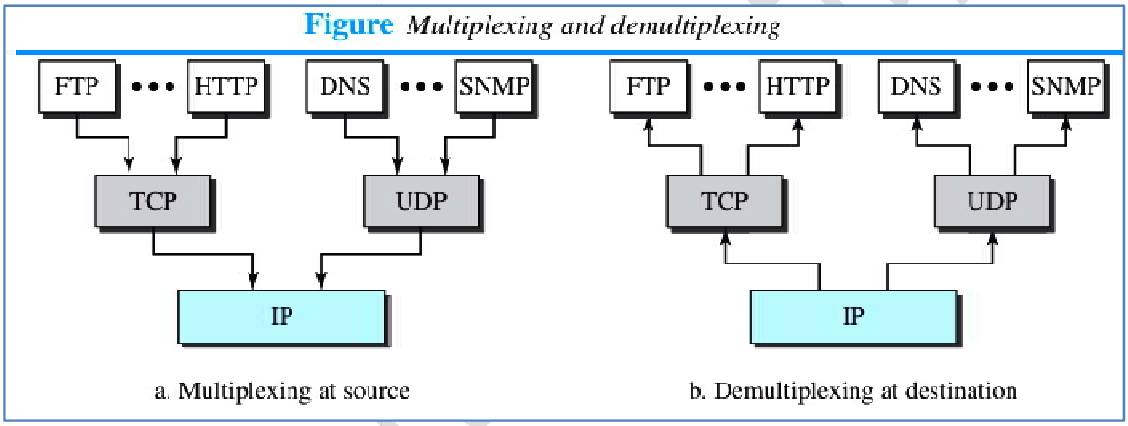


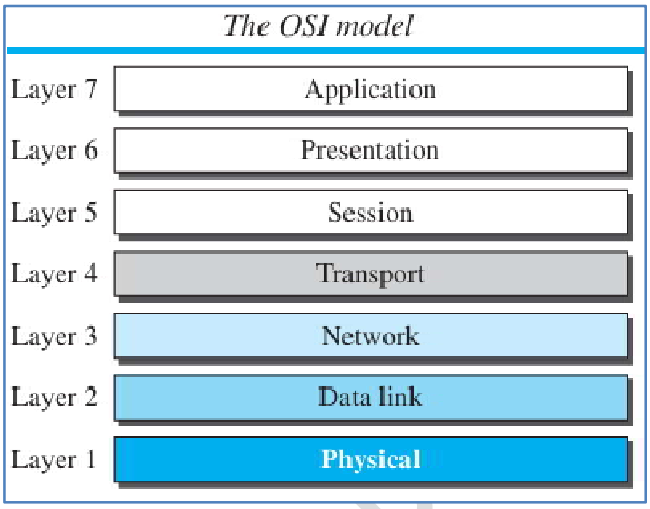
Figure *Multiplexing and demultiplexing*

- ***To be able to multiplex and demultiplex, a protocol needs to have a field in its header to identify to which protocol the encapsulated packets belong***.
- At the transport layer, either UDP or TCP can accept a message from several application-layer protocols.
- At the network layer, IP can accept a segment from TCP or a user datagram from UDP. IP can also accept a packet from other protocols such as ICMP, IGMP, and so on.
- At the data-link layer, a frame may carry the payload coming from IP or other protocols such as ARP.

The OSI model

- Although, when speaking of the Internet, everyone talks about the TCP/IP protocol suite, this suite is not the only suite of protocols defined.
- Established in 1947, the International Organization for Standardization (ISO) is a multinational body dedicated to worldwide agreement on international standards. Almost three-fourths of the countries in the world are represented in the ISO. An ISO standard that covers all aspects of network communications is the Open Systems Interconnection (OSI) model. It was first introduced in the late 1970s.
- An open system is a set of protocols that allows any two different systems to communicate regardless of their underlying architecture.

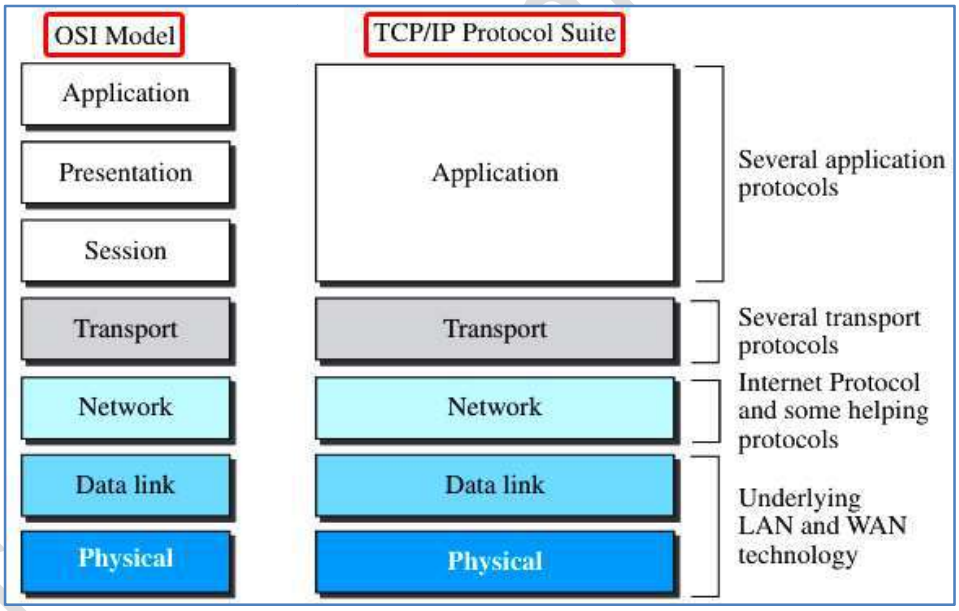
Prof. Parameshwara M V



- The purpose of the OSI model is to show how to facilitate communication between different systems without requiring changes to the logic of the underlying hardware and software.
- The OSI model is not a protocol; it is a model for understanding and designing a network architecture that is flexible, robust, and interoperable.
- The OSI model was intended to be the basis for the creation of the protocols in the OSI stack. The OSI model is a layered framework for the design of network systems that allows communication between all types of computer systems. It consists of seven separate but related layers, each of which defines a part of the process of moving information across a network (see Figure).

OSI versus TCP/IP

- When we compare the two models, we find that two layers, session and presentation, are missing from the TCP/IP protocol suite. These two layers were not added to the TCP/IP protocol suite after the publication of the OSI model. The application layer in the suite is usually considered to be the combination of three layers in the OSI model.



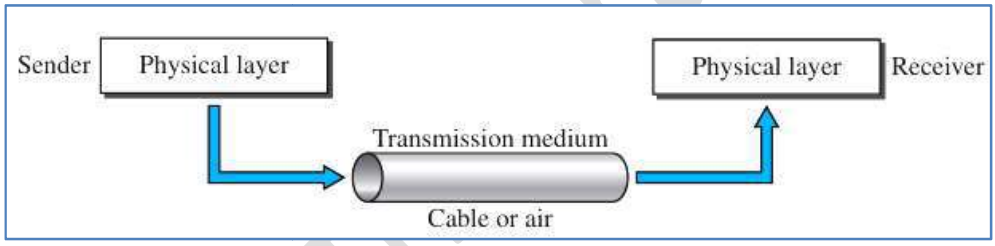
- Two reasons were mentioned for this decision.
  - First, TCP/IP has more than one transport-layer protocol. Some of the functionalities of the session layer are available in some of the transport-layer protocols.
  - Second, the application layer is not only one piece of software. Many applications can be developed at this layer. If some of the functionalities mentioned in the session and presentation layers are needed for a particular application, they can be included in the development of that piece of software.

Chapter 7: Introduction to Physical Layer: Transmission media, Guided Media, Unguided Media: Wireless

- We definitely need transmission media to conduct signals from the source to the destination. However, the media can be wired or wireless. This chapter is divided into three sections:
  - The first section introduces the transmission media and defines its position in the Internet model. It shows that we can ***classify transmission media into two broad categories: guided and unguided media.***
  - The second section discusses ***guided media***. The first part describes twisted-pair cables and their characteristics and applications. The second part describes coaxial cables and their characteristics and applications. Finally, the third part describes fiber-optic cables and their characteristics and applications.
  - The third section discusses ***unguided media***. The first part describes radio waves and their characteristics and applications. The second part describes microwaves and their characteristics and applications. Finally, the third part describes infrared waves and their characteristics and applications.

INTRODUCTION

- Transmission media are actually located below the physical layer and are directly controlled by the physical layer. We could say that transmission media belong to layer zero. Figure shows the position of transmission media in relation to the physical layer.

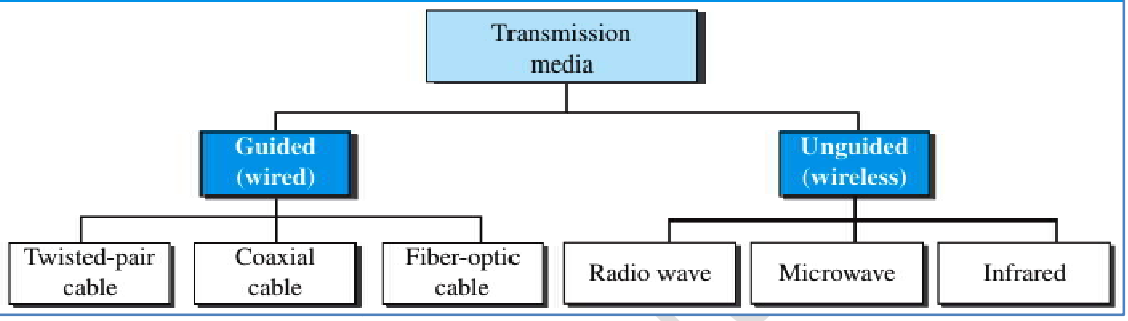


- A ***transmission medium*** can be broadly defined as anything that can carry information from a source to a destination.
- The transmission medium is usually free space, metallic cable, or fiber-optic cable. The information is usually a signal that is the result of a conversion of data from another form.
- The use of long-distance communication using electric signals started with the invention of the telegraph by Morse in the 19th century. Communication by telegraph was slow and dependent on a metallic medium.
- Extending the range of the human voice became possible when the telephone was invented in 1869. Telephone communication at that time also needed a metallic medium to carry the electric signals that were the result of a conversion from the human voice. The communication was, however, unreliable due to the poor quality of the wires. The lines were often noisy and the technology was unsophisticated.
- Wireless communication started in 1895 when Hertz was able to send high frequency signals. Later, Marconi devised a method to send telegraph-type messages over the Atlantic Ocean. We have come a long way.
- ***Better metallic media have been invented (twisted-pair and coaxial cables, for example).***
- The use of optical fibers has increased the data rate incredibly. Free space (air, vacuum, and water) is used more efficiently, in part due to the technologies (such as modulation and multiplexing).
- Computers and other telecommunication devices use signals to represent data.

Prof. Parushavathy P V

- These signals are transmitted from one device to another in the form of electromagnetic energy, which is propagated through transmission media.
- Electromagnetic energy, a combination of electric and magnetic fields vibrating in relation to each other, includes power, radio waves, infrared light, visible light, ultraviolet light, and X, gamma, and cosmic rays. Each of these constitutes a portion of the electromagnetic spectrum.
- Not all portions of the spectrum are currently usable for telecommunications, however. The media to harness those that are usable are also limited to a few types.
- In telecommunications, transmission media can be divided into two broad categories: ***guided and unguided***.
- **Guided media include twisted-pair cable, coaxial cable, and fiber-optic cable**.
- Unguided medium is free space. Figure shows this taxonomy.

Figure *Classes of transmission media*

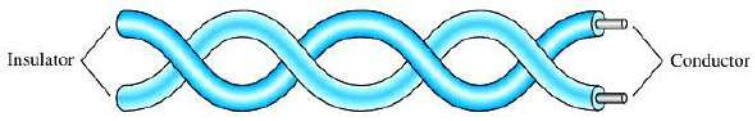


Guided Media

- Guided media, which are those that provide a conduit from one device to another, include
  - twisted-pair cable
  - coaxial cable, and
  - fiber-optic cable.
- A signal traveling along any of these media is directed and contained by the physical limits of the medium.
- **Twisted-pair and coaxial cable use metallic (copper) conductors that accept and transport signals in the form of electric current**.
- **Optical fiber is a cable that accepts and transports signals in the form of light**.

Twisted-Pair Cable

- A twisted pair consists of two conductors (normally copper), each with its own plastic insulation, twisted together, as shown in Figure.



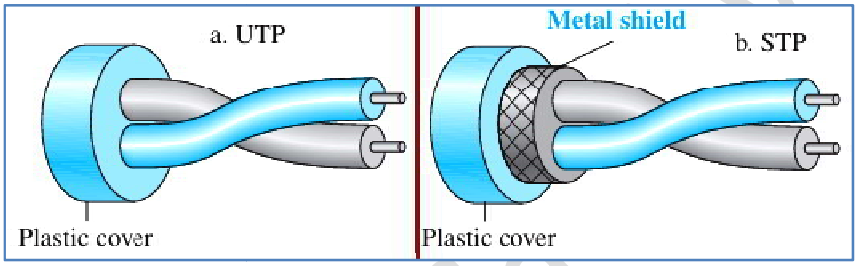
- One of the wires is used to carry signals to the receiver, and the other is used only as a ground reference. The receiver uses the difference between the two. In addition to the signal sent by the sender on one of the wires, interference (noise) and crosstalk may affect both wires and create unwanted signals. If the two wires are parallel, the effect of these unwanted signals is not the same in both wires because they are at different locations relative to the noise or crosstalk sources (e.g., one is closer and the other is farther). This results in a difference at the receiver.

Prof. Parthasarathy V

- ***By twisting the pairs, a balance is maintained***. For example, suppose in one twist, one wire is closer to the noise source and the other is farther; in the next twist, the reverse is true. Twisting makes it probable that both wires are equally affected by external influences (noise or crosstalk). This means that the receiver, which calculates the difference between the two, receives no unwanted signals. The unwanted signals are mostly canceled out. From the above discussion, it is clear that the number of twists per unit of length (e.g., inch) has some effect on the quality of the cable.

Unshielded Versus Shielded Twisted-Pair Cable

- The most common twisted-pair cable used in communications is referred to as ***unshielded twisted-pair (UTP)***. ***IBM*** has also produced a version of twisted-pair cable for its use, called ***shielded twisted-pair (STP).*** STP cable has a metal foil or braided mesh covering that encases each pair of insulated conductors. Although metal casing improves the quality of cable by preventing the penetration of noise or crosstalk, it is bulkier and more expensive. The following Figure shows the difference between UTP and STP. Our discussion focuses primarily on UTP because STP is seldom used outside of IBM.

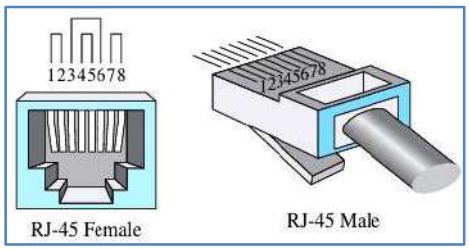


Categories

- The Electronic Industries Association (EIA) has developed standards to classify unshielded twisted-pair cable into seven categories. Categories are determined by cable quality, with 1 as the lowest and 7 as the highest. Each EIA category is suitable for specific uses. The following table shows these categories.

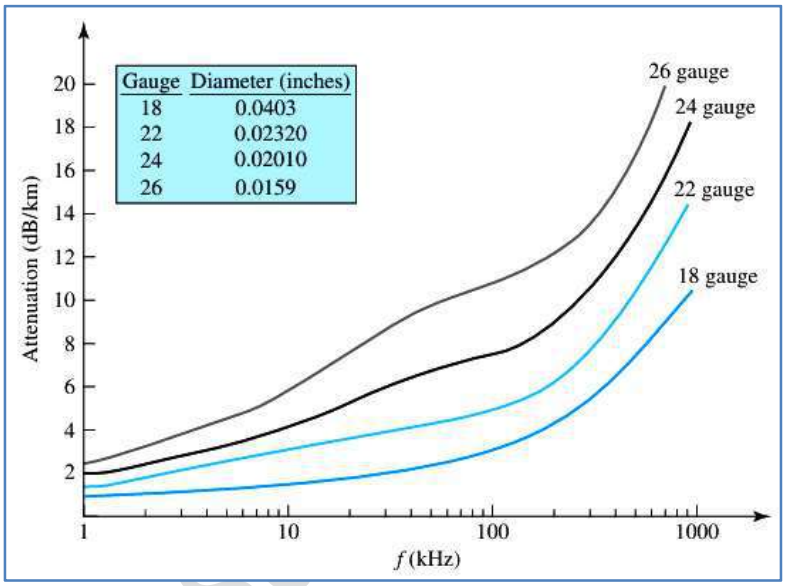
Category	*Specification*	*Data Rate (Mbps)*	*Use*
1	Unshielded twisted-pair used in telephone	< 0.1	Telephone
2	Unshielded twisted-pair originally used in T lines	2	T-1 lines
3	Improved CAT 2 used in LANs	10	LANs
4	Improved CAT 3 used in Token Ring networks	20	LANs
5	Cable wire is normally 24 AWG with a jacket and outside sheath	100	LANs
5E	An extension to category 5 that includes extra features to minimize the crosstalk and electromagnetic interference	125	LANs
6	A new category with matched components coming from the same manufacturer. The cable must be tested at a 200-Mbps data rate.	200	LANs
7	Sometimes called *SSTP (shielded screen twisted-pair)*. Each pair is individually wrapped in a helical metallic foil followed by a metallic foil shield in addition to the outside	600	LANs

Connectors



- The most common UTP connector is RJ45 (RJ stands for registered jack), as shown in the following figure. The RJ45 is a keyed connector, meaning the connector can be inserted in only one way.

Performance



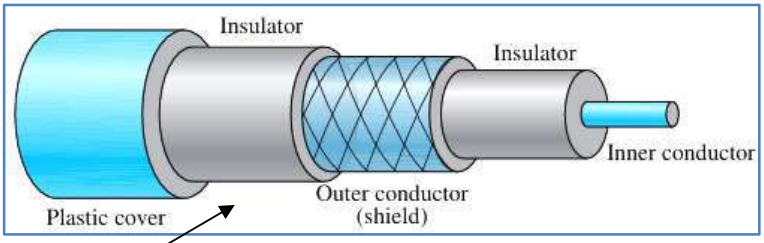
- One way to measure the performance of twisted-pair cable is to compare attenuation versus frequency and distance.
- A twisted-pair cable can pass a wide range of frequencies. However, the following figure shows that with increasing frequency, the attenuation, measured in decibels per kilometer (dB/km), sharply increases with frequencies above 100 kHz.
- Note that gauge is a measure of the thickness of the wire.

Applications

- Twisted-pair cables are used in telephone lines to provide voice and data channels. The local loop—the line that connects subscribers to the central telephone office— commonly consists of unshielded twisted-pair cables.
- The DSL lines that are used by the telephone companies to provide high-data-rate connections also use the high-bandwidth capability of unshielded twisted-pair cables.
- Local-area networks, such as 10Base-T and 100Base-T, also use twisted-pair cables.

Coaxial Cable

- Coaxial cable (or coax) carries signals of higher frequency ranges than those in twisted pair cable, in part because the two media are constructed quite differently.
- Instead of having two wires, coax has a central core conductor of solid or stranded wire (usually copper) enclosed in an insulating sheath, which is, in turn, encased in an outer conductor of metal foil, braid, or a combination of the two.
- The outer metallic wrapping serves both as a shield against noise and as the second conductor, which completes the circuit. This outer conductor is also enclosed in an insulating sheath, and the whole cable is protected by a plastic cover (see the Figure).



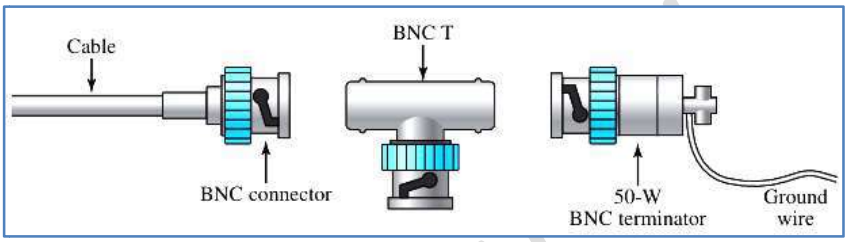
Prof. Parthasa

- Coaxial Cable Standards Coaxial cables are categorized by their ***Radio Government (RG)*** ratings.

Category	*Impedance*	*Use*
RG-59	75 Ω	Cable TV
RG-58	50 Ω	Thin Ethernet
RG-11	50 Ω	Thick Ethernet

- Each ***RG*** number denotes a unique set of physical specifications, including the wire gauge of the inner conductor, the thickness and type of the inner insulator, the construction of the shield, and the size and type of the outer casing. Each cable defined by an RG rating is adapted for a specialized function, as shown in Table.

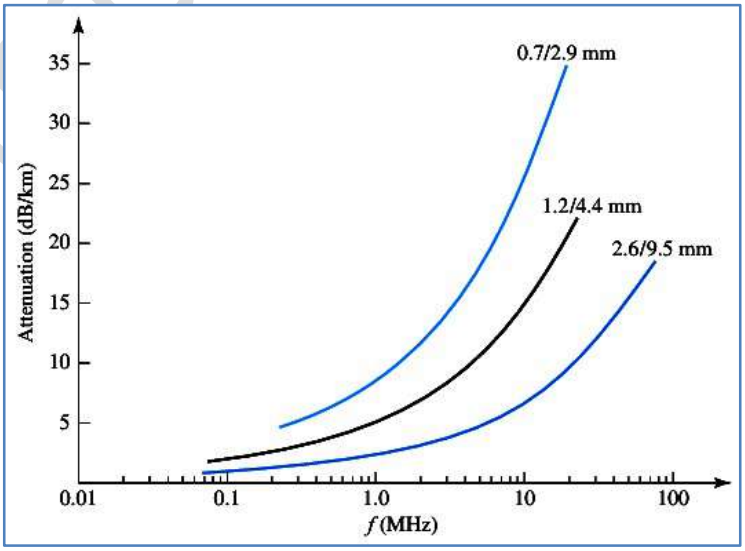
Coaxial Cable Connectors



- To connect coaxial cable to devices, we need coaxial connectors. The most common type of connector used today is the **Bayonet Neill-Concelman (BNC) connector**. The following figure shows three popular types of these connectors: ***the BNC connector, the BNC T connector, and the BNC terminator***.
- The BNC connector is used to connect the end of the cable to a device, such as a TV set. The BNC T connector is used in Ethernet networks to branch out to a connection to a computer or other device. The BNC terminator is used at the end of the cable to prevent the reflection of the signal.

Performance

- The attenuation is much higher in coaxial cable than in twisted-pair cable. In other words, although coaxial cable has a much higher bandwidth, the signal weakens rapidly and requires the frequent use of repeaters.



Applications

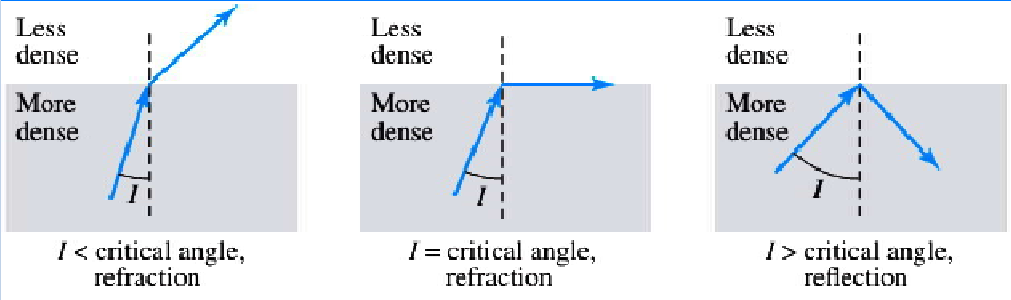
- ***Coaxial cable was widely used in analog telephone networks where a single coaxial network could carry 10,000 voice signals.*** Later it was used in digital telephone networks where a single coaxial cable could carry digital data up to 600 Mbps. However, coaxial cable in telephone networks has largely been replaced today with fiber-optic cable.
- ***Cable TV networks also use coaxial cables***. In the traditional cable TV network, the entire network used coaxial cable. Hybrid networks use coaxial cable only at the network boundaries, near the consumer premises. ***Cable TV uses RG-59 coaxial cable.***
- **Traditional Ethernet LANs -** Because of its high bandwidth, and consequently high data rate, coaxial cable was chosen for digital transmission in early Ethernet LANs.
- The **10Base-2**, or **Thin Ethernet**, **uses RG-58 coaxial cable** with BNC connectors to transmit data at 10 Mbps with a range of 185 m. The 10Base5, or Thick Ethernet, uses RG-11 (thick coaxial cable) to transmit 10 Mbps with a range of 5000 m. Thick Ethernet has specialized connectors.

Prof Parthasarathy

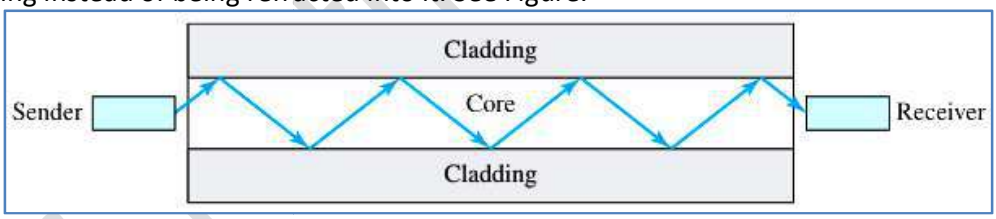
Fiber-Optic Cable

- A fiber-optic cable is made of glass or plastic and transmits signals in the form of light. To understand optical fiber, we first need to explore several aspects of the nature of light.
- Light travels in a straight line as long as it is moving through a single uniform sub stance.
- If a ray of light traveling through one substance suddenly enters another substance (of a different density), the ray changes direction. Figure shows how a ray of light changes direction when going from a more dense to a less dense substance.

Figure *Bending of light ray*



- As the figure shows, if the angle of incidence I (the angle the ray makes with the line perpendicular to the interface between the two substances) is less than the critical angle, the ray refracts and moves closer to the surface. If the angle of incidence is equal to the critical angle, the light bends along the interface. If the angle is greater than the critical angle, the ray reflects (makes a turn) and travels again in the denser substance. Note that the critical angle is a property of the substance, and its value differs from one substance to another.
- ***Optical fibers use reflection to guide light through a channel. A glass or plastic core is surrounded by a cladding of less dense glass or plastic.*** The difference in density of the two materials must be such that a beam of light moving through the core is reflected off the cladding instead of being refracted into it. See Figure.



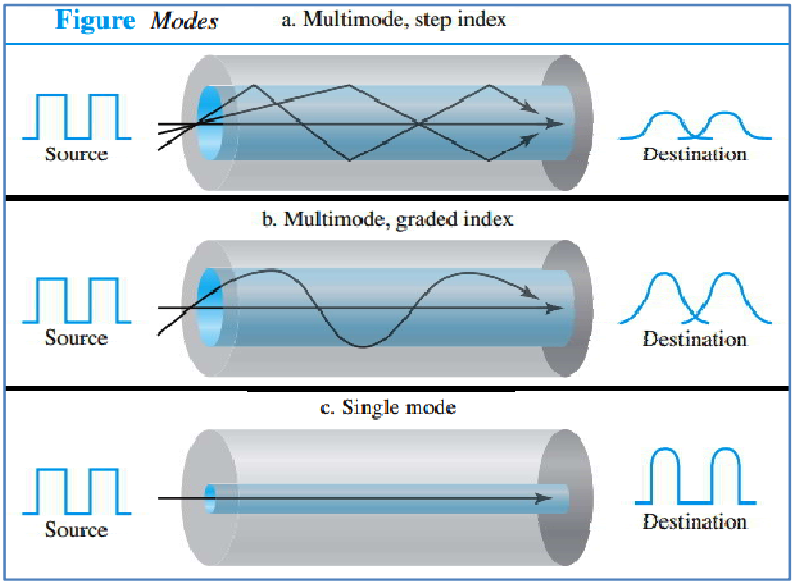
Propagation Modes

- Current technology supports **two modes (multimode and single mode)** for propagating light along optical channels, each requiring fiber with different physical characteristics. **Multimode can be implemented in two forms: step-index or graded-index.**

Multimode

- Multimode is so named because multiple beams from a light source move through the core in different paths.

Figure *Modes*



How these **beams move within the cable depends on the structure of the core**.

- In **multimode step-index fiber**, the density of the core remains constant from the center to the edges. A beam of light moves through this constant density in a straight line until it reaches the interface of the core and the cladding. At the interface, there is an abrupt change due to a lower density; this alters the angle of the beam's motion. The term ***step-index refers to the suddenness of this change***, which contributes to the distortion of the signal as it passes through the fiber.
- A second type of fiber, called **multimode graded-index fiber**, **decreases this distortion of the signal through the cable**. The word index here refers to the index of refraction. As we saw above***, the index of refraction is related to density***. A graded index fiber is one with varying densities. Density is highest at the center of the core and decreases gradually to its lowest at the edge. Figure shows the impact of this variable density on the propagation of light beams.

Single-Mode

- Single-mode uses step-index fiber and a highly focused source of light that limits beams to a small range of angles, all close to the horizontal.
- The single-mode fiber itself is manufactured with a much smaller diameter than that of multimode fiber, and with substantially lower density (index of refraction).
- The ***decrease in density results in a critical angle that is close enough to 90° to make the propagation of beams almost horizontal***. In this case, propagation of different beams is almost identical, and delays are negligible. All the beams arrive at the destination "together" and can be recombined with little distortion to the signal.

Fiber Sizes

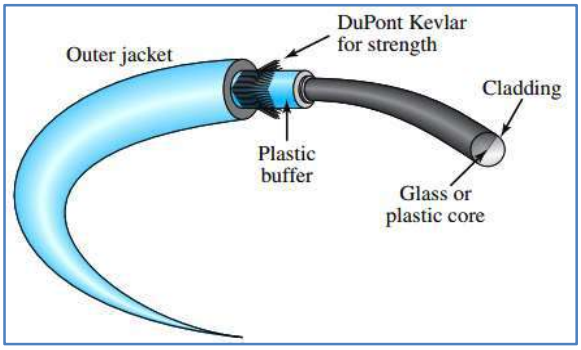
- Optical fibers are defined by the ratio of the diameter of their core to the diameter of their cladding, both expressed in micrometers. The common sizes are shown in Table. Note that the last size listed is for single-mode only.

Table *Fiber types*

Type	*Core (μm)*	*Cladding (μm)*	*Mode*
50/125	50.0	125	Multimode, graded index
62.5/125	62.5	125	Multimode, graded index
100/125	100.0	125	Multimode, graded index
7/125	7.0	125	Single mode

Cable Composition

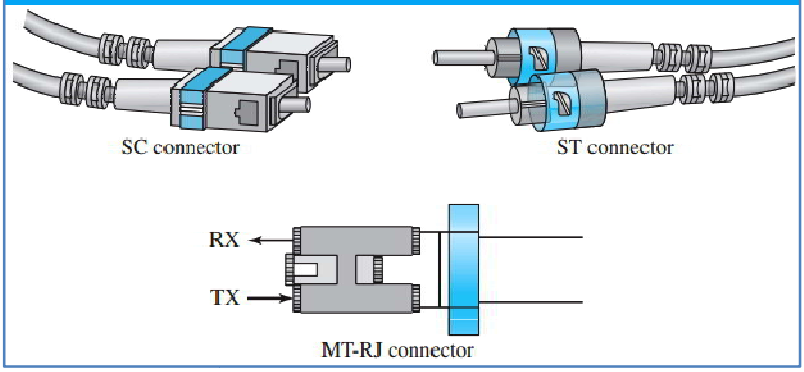
- Following Figure shows the composition of a typical fiber-optic cable. The outer jacket is made of either PVC or Teflon.
- Inside the jacket are Kevlar strands to strengthen the cable. Kevlar is a strong material used in the fabrication of bulletproof vests.
- Below the Kevlar is another plastic coating to cushion the fiber. The fiber is at the center of the cable, and it consists of cladding and core.



Fiber-Optic Cable Connectors

- There are three types of connectors for fiber-optic cables, as shown in Figure. The subscriber channel (SC) connector is used for cable TV. It uses a push/pull locking system. The straight-tip (ST) connector is used for connecting cable to networking devices. It uses a bayonet locking system and is more reliable than SC. MT-RJ is a connector that is the same size as RJ45.

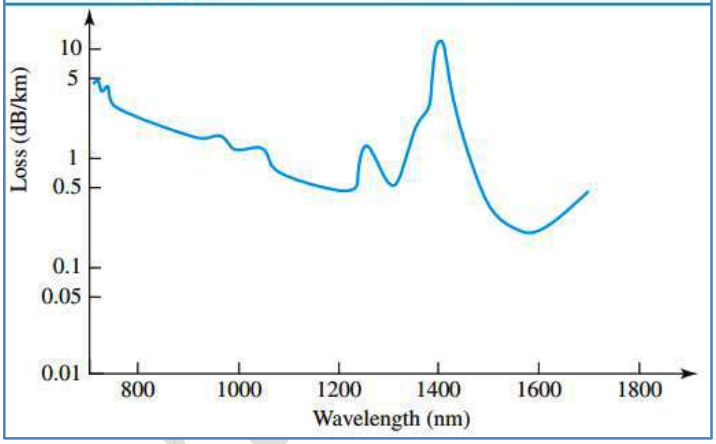
Figure *Fiber-optic cable connectors*



Performance

- The plot of attenuation versus wavelength in Figure 7.16 shows a very interesting phenomenon in fiber-optic cable. Attenuation is flatter than in the case of twisted-pair cable and coaxial cable. The performance is such that we need fewer (actually one tenth as many) repeaters when we use fiber-optic cable.

Figure *Optical fiber performance*



Applications

- Fiber-optic cable is often found in backbone networks because its wide bandwidth is cost-effective. Today, with wavelength-division multiplexing (WDM), we can transfer data at a rate of 1600 Gbps. The SONET network provides such a backbone.
- Some cable TV companies use a combination of optical fiber and coaxial cable, thus creating a hybrid network. Optical fiber provides the backbone structure while coaxial cable provides the connection to the user premises. This is a cost-effective configuration since the narrow bandwidth requirement at the user end does not justify the use of optical fiber.
- Local-area networks such as 100Base-FX network (Fast Ethernet) and 1000Base-X also use fiber-optic cable.

Advantages and Disadvantages of Optical Fiber

Advantages - Fiber-optic cable has several advantages over metallic cable (twisted-pair or coaxial))

1. ***Higher bandwidth -*** Fiber-optic cable can support dramatically higher bandwidths (and hence data rates) than either twisted-pair or coaxial cable. Currently, data rates and bandwidth utilization over fiber-optic cable are limited not by the medium but by the signal generation and reception technology available.
2. ***Less signal attenuation -*** Fiber-optic transmission distance is significantly greater than that of other guided media. A signal can run for 50 km without requiring regeneration. We need repeaters every 5 km for coaxial or twisted-pair cable.

3. ***Immunity to electromagnetic interference -*** Electromagnetic noise cannot affect fiber-optic cables.
4. ***Resistance to corrosive materials -*** Glass is more resistant to corrosive materials than copper.
5. ***Light weight -*** Fiber-optic cables are much lighter than copper cables.
6. ***Greater immunity to tapping -*** Fiber-optic cables are more immune to tapping than copper cables. Copper cables create antenna effects that can easily be tapped.

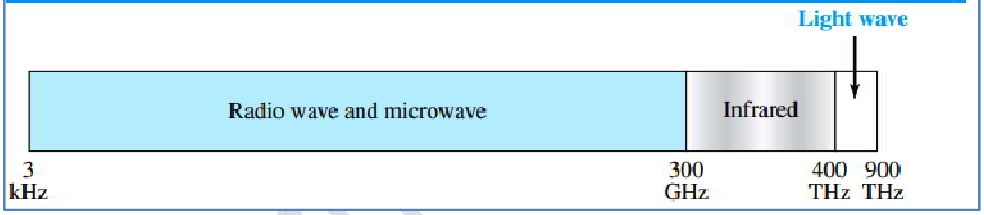
Disadvantages

1. ***Installation and maintenance***. Fiber-optic cable is a relatively new technology. Its installation and maintenance require expertise that is not yet available everywhere.
2. ***Unidirectional light propagation.*** Propagation of light is unidirectional. If we need bidirectional communication, two fibers are needed.
3. ***Cost.*** The cable and the interfaces are relatively more expensive than those of other guided media. If the demand for bandwidth is not high, often the use of optical fiber cannot be justified.

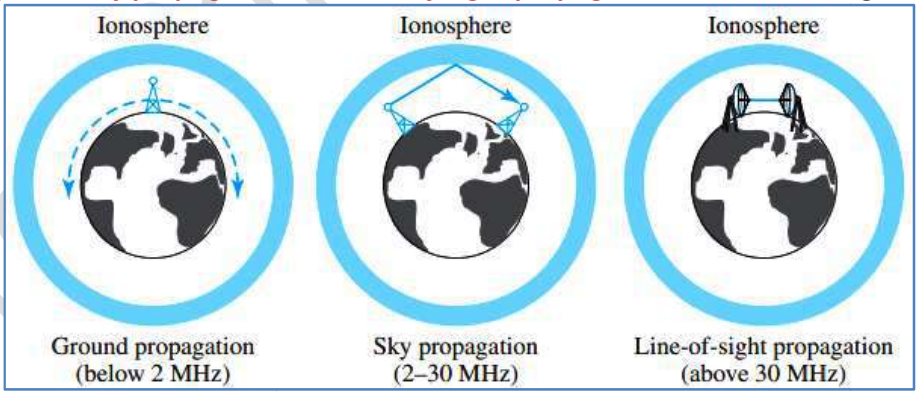
Unguided Media: Wireless

- ***Unguided medium*** transport electromagnetic waves without using a physical conductor. This type of communication is often referred to as ***wireless communication***. Signals are normally broadcast through free space and thus are available to anyone who has a device capable of receiving them. Figure shows the part of the electromagnetic spectrum, ranging from 3 kHz to 900 THz, used for wireless communication.

Figure *Electromagnetic spectrum for wireless communication*



- ***Unguided signals can travel from the source to the destination in several ways: ground propagation, sky propagation, and line-of-sight propagation, as shown in Figure.***



- ***In ground propagation***, radio waves travel through the lowest portion of the atmosphere, hugging the earth. These **low-frequency signals** emanate in all directions from the transmitting antenna and follow the curvature of the planet. Distance depends on the amount of power in the signal: ***The greater the power, the greater the distance***.
- In ***sky propagation***, ***higher-frequency radio waves radiate upward into the ionosphere*** (the layer of atmosphere where particles exist as ions) where they are reflected back to earth. This type of transmission allows for greater distances with lower output power.

- In ***line-of-sight propagation***, very high-frequency signals are transmitted in straight lines directly from antenna to antenna. Antennas must be directional, facing each other, and either tall enough or close enough together not to be affected by the curvature of the earth. Line-ofsight propagation is tricky because radio transmissions cannot be completely focused.
- The section of the electromagnetic spectrum defined as radio waves and microwaves is divided into eight ranges, called bands, each regulated by government authorities. These bands are rated from very low frequency (VLF) to extremely high frequency (EHF). Table lists these bands, their ranges, propagation methods, and some applications.

Prof. Parthasarathy P V

Chapter 8: Switching: Packet Switching and its types. (P-252)

- Switching is a topic that can be discussed at several layers. We have switching at the physical layer, at the data-link layer, at the network layer, and even logically at the application layer (message switching).
- In data communications, we need to send messages from one end system to another. If the message is going to pass through a packet-switched network, it needs to be divided into packets of fixed or variable size. The size of the packet is determined by the network and the governing protocol.
- In packet switching, there is **no resource allocation for a packet**. This means that there is **no reserved bandwidth on the links**, and there is **no scheduled processing time for each packet**.
- **Resources are allocated on demand**. The allocation is done on a first-come, first-served basis. When a switch receives a packet, no matter what the source or destination is, the packet must wait if there are other packets being processed.

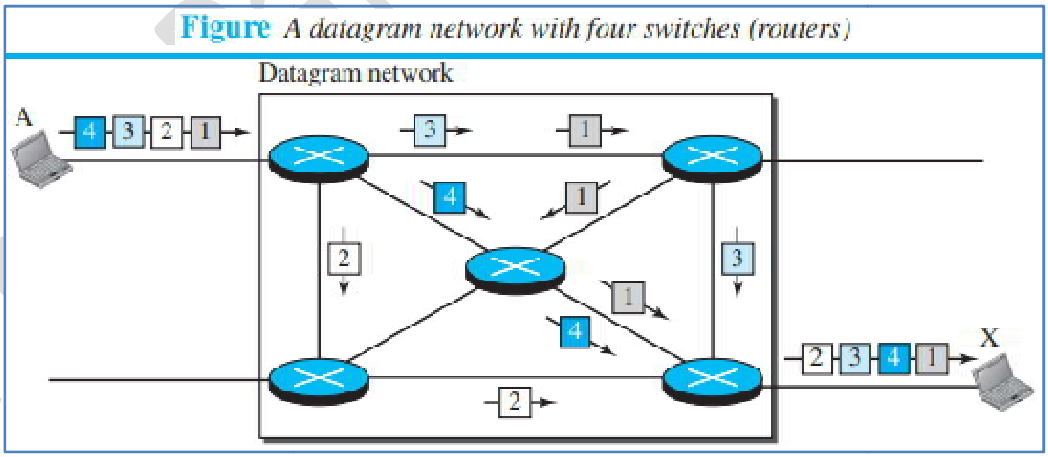
> **In a packet-switched network, there is no resource reservation; resources are allocated on demand.**

- As with other systems in our daily life, this lack of reservation may create delay. For example, if we do not have a reservation at a restaurant, we might have to wait.
- We can have **two types of packet-switched networks**:
  1. datagram networks and
  2. virtual circuit networks.

1. Datagram Networks

- In a datagram network, **each packet is treated independently of all others**.
- Even if a packet is part of a multipacket transmission, the network treats it as though it existed alone. **Packets in this approach are referred to as datagrams**.
- **Datagram** switching is normally done at the **network layer**.
- The following figure shows how the datagram approach is used to deliver four packets from station A to station X. The switches in a datagram network are traditionally referred to as routers. That is why we use a different symbol for the switches in the figure.

Figure *A datagram network with four switches (routers)*



- In this example, all four packets (or datagrams) belong to the same message, but may travel different paths to reach their destination. This is so because the links may be involved in carrying packets from other sources and do not have the necessary bandwidth available to carry all the packets from A to X. This approach can cause the datagrams of a transmission to arrive at their destination out of order with different delays between the packets.
- Packets may also be lost or dropped because of a lack of resources.

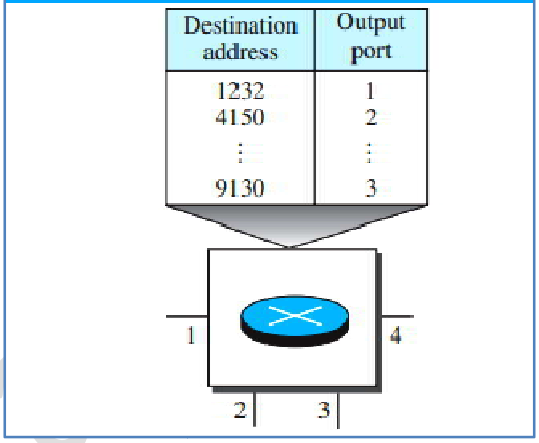
- In most protocols, it is the **responsibility of an upper-layer protocol to re-order the datagrams** or **ask for lost datagrams before passing them on to the application**.
- The **datagram networks are sometimes referred to as connectionless networks**. The term **connectionless here means that the switch (packet switch) does not keep information about the connection state**. There are no setup or teardown phases. Each packet is treated the same by a switch regardless of its source or destination.

Routing Table

If there are no setup or teardown phases, how are the packets routed to their destinations in a datagram network?

- In this type of network, **each switch (or packet switch) has a routing table which is based on the destination address**.
- The **routing tables are dynamic and are updated periodically**. The destination addresses and the corresponding forwarding output ports are recorded in the tables. This is different from the table of a circuit-switched network in which each entry is created when the setup phase is completed and deleted when the teardown phase is over. The figure shows the routing table for a switch.

Figure *Routing table in a datagram network*



A switch in a datagram network uses a routing table that is based on the destination address.

Destination Address

- **Every packet in a datagram network carries a header that contains, among other information, the destination address of the packet**.
- When the switch receives the packet, this **destination address is examined; the routing table is consulted to find the corresponding port through which the packet should be forwarded**.
- **This address, unlike the address in a virtual-circuit network, remains the same during the entire journey of the packet**.

The destination address in the header of a packet in a datagram network remains the same during the entire journey of the packet.

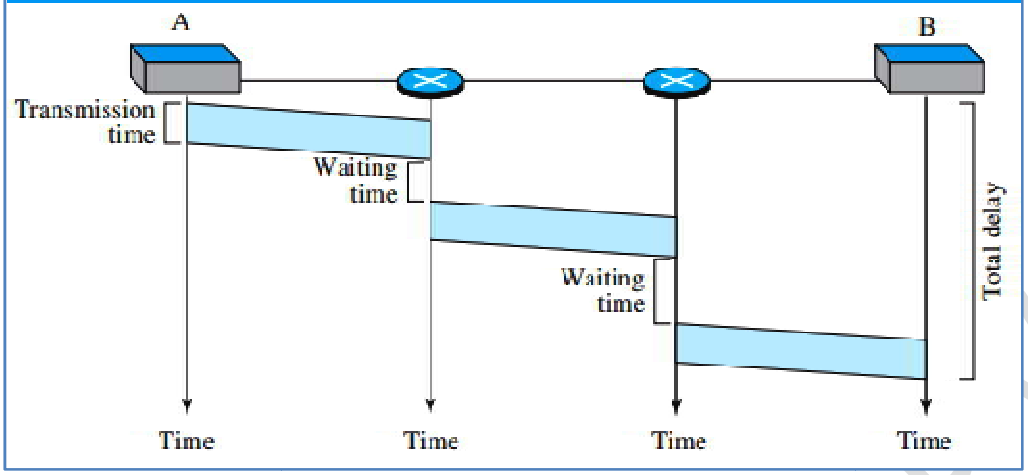
Efficiency

- The **efficiency of a datagram network is better than that of a circuit-switched network**; resources are allocated only when there are packets to be transferred.
- If a source sends a packet and there is a delay of a few minutes before another packet can be sent, the resources can be reallocated during these minutes for other packets from other sources.

Delay

- There may be **greater delay in a datagram network than in a virtual-circuit network**. Although there are no setup and teardown phases, each packet may experience a wait at a switch before it is forwarded. In addition, since not all packets in a message necessarily travel through the same switches, the delay is not uniform for the packets of a message. Figure gives an example of delay in a datagram network for one packet.

Figure *Delay in a datagram network*



- The packet travels through two switches. There are three transmission times (3T), three propagation delays (slopes 3τ of the lines), and two waiting times (w1 + w2). We ignore the processing time in each switch.

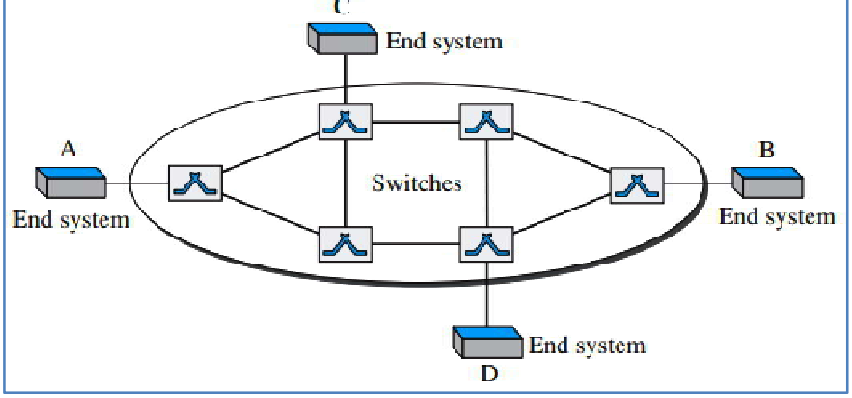
$$\text{Total delay} = 3T + 3\tau + w_1 + w_2$$

---

2. Virtual-Circuit Networks

- A **virtual-circuit network** is a **cross between a circuit-switched network** and a **datagram network**. It has some characteristics of both.
  1. As in a circuit-switched network, there are **setup and pull down phases in addition to the data transfer phase**.
  2. **Resources can be allocated during the setup phase**, as in a circuit-switched network, or on demand, as in a datagram network.
  3. As in a **datagram network, data are packetized and each packet carries an address in the header**. However, the address in the header has local jurisdiction (it defines what the next switch should be and the channel on which the packet is being carried), not end-to-end jurisdiction. The reader may ask how the intermediate switches know where to send the packet if there is no final destination address carried by a packet. The answer will be clear when we discuss virtual-circuit identifiers in the next section.
  4. As in a **circuit-switched network, all packets follow the same path established during the connection**.
  5. A **virtual-circuit network is normally implemented in the data-link layer**, while a **circuit-switched network is implemented in the physical layer** and a **datagram network in the network layer**.
- Figure is an example of a virtual-circuit network. The **network has switches that allow traffic from sources to destinations**. A **source or destination can be a computer, packet switch, bridge, or any other device that connects other networks**.

Figure *Virtual-circuit network*



Addressing

- In a virtual-circuit network, **two types of addressing** are involved:
    1. Global
    2. Local (virtual-circuit identifier).

1. Global Addressing

- A source or a destination needs to have a global address—an address that can be unique in the scope of the network or internationally if the network is part of an international network. However, we will see that a global address in virtual-circuit networks is used only to create a virtual-circuit identifier.

2. Local (Virtual-Circuit Identifier) Addressing

- The identifier that is actually used for data transfer is called the **virtual-circuit identifier (VCI)** or the label. A VCI, **unlike a global address, is a small number that has only switch scope**; it is **used by a frame between two switches**.
- **When a frame arrives at a switch, it has a VCI; when it leaves, it has a different VCI**.
- Figure shows how the VCI in a data frame changes from one switch to another. Note that a VCI does not need to be a large number since each switch can use its own unique set of VCIs.

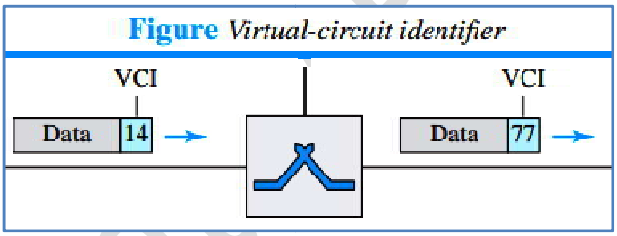


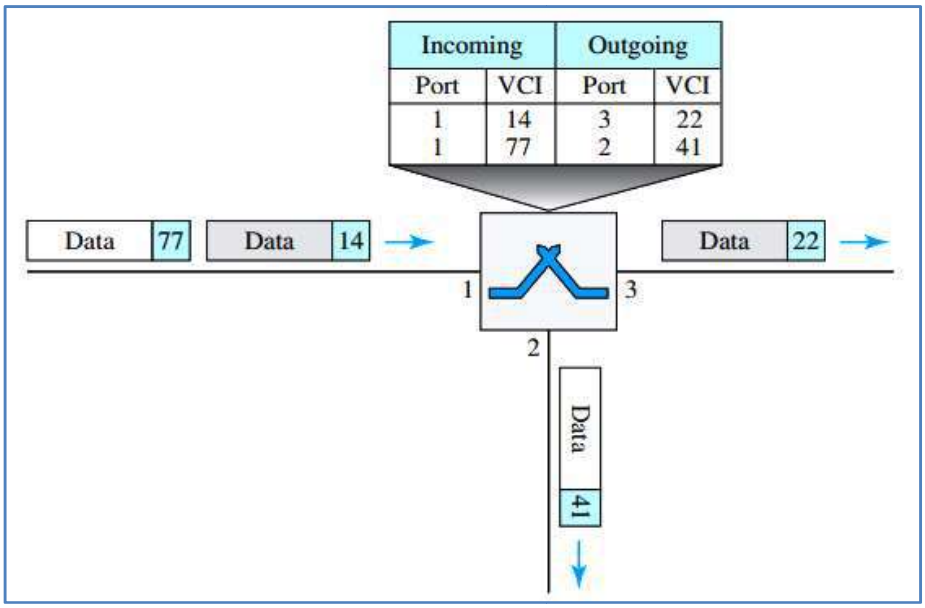
Figure *Virtual-circuit identifier*

Three Phases

- As in a circuit-switched network, a source and destination need to go through **three phases in a virtual-circuit network**:
    1. Setup
    2. data transfer, and
    3. teardown.
- In the **setup phase**, the **source and destination use their global addresses to help switches make table entries for the connection**.
- **Data transfer** occurs between these **two phases**. We **first discuss the data-transfer phase**, which is more straightforward; we then **talk about the setup and teardown phases**.
- In the **teardown** phase, **the source and destination inform the switches to delete the corresponding entry**.

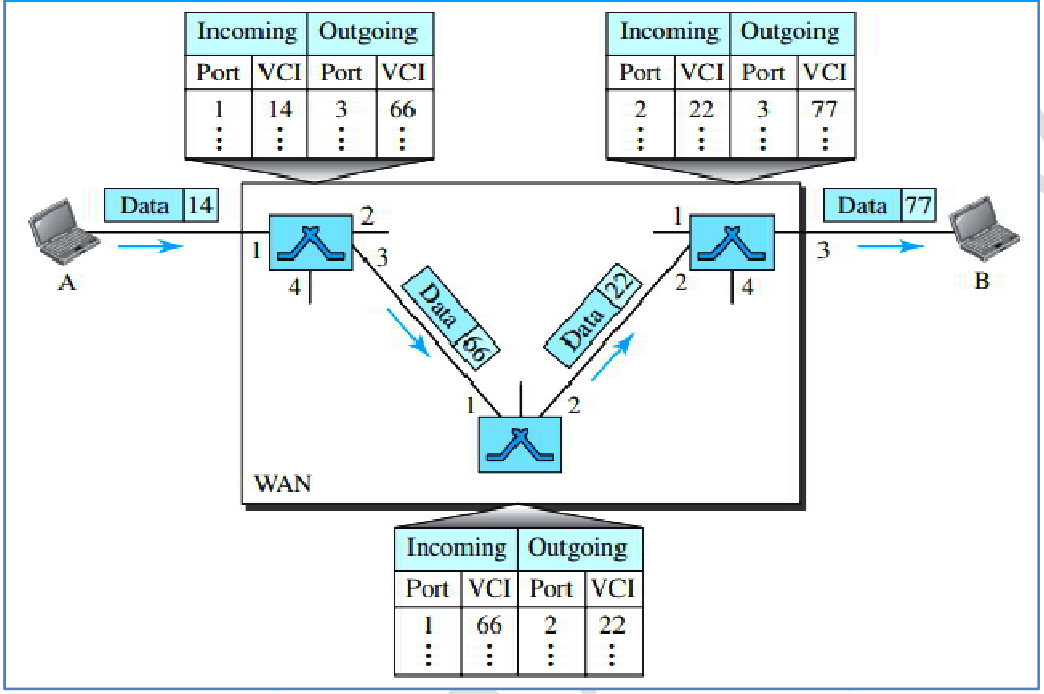
1. Data-Transfer Phase

- To **transfer a frame from a source to its destination, all switches need to have a table entry for this virtual circuit**. The **table has four column**s. This means that the switch holds four pieces of information for each virtual circuit that is already set up. **Each switch has a table with entries for all active virtual circuits**. Figure shows such a switch and its corresponding table.



- The following figure shows how a frame from source A reaches destination B and how its VCI changes during the trip. Each switch changes the VCI and routes the frame. The data-transfer phase is active until the source sends all its frames to the destination. The procedure at the switch is the same for each frame of a message. The process creates a virtual circuit, not a real circuit, between the source and destination.

Figure *Source-to-destination data transfer in a virtual-circuit network*



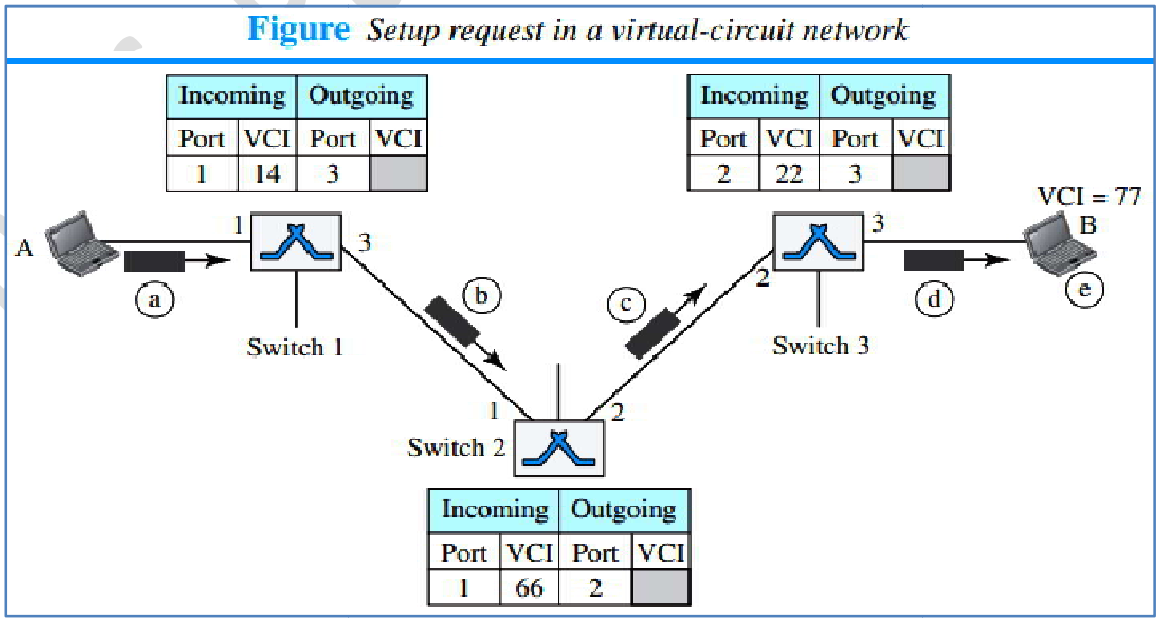
2. Setup Phase

- In the setup phase, a **switch creates an entry for a virtual circuit**. For example, suppose **source A needs** to create a virtual circuit to **B**.
- **Two steps** are required: **the setup request and the acknowledgment**.

Setup Request

- A setup request frame is sent from the source to the destination. Figure shows the process.

Figure *Setup request in a virtual-circuit network*



a) Source A sends a setup frame to switch 1.

b) Switch 1 receives the setup request frame. It knows that a frame going from A to B goes out through port 3. The switch, in the setup phase, acts as a packet switch; it has a routing table which is different from the switching table. For the moment, assume that it knows the output port. The switch creates an entry in its table for this virtual circuit, but it is only able to fill three of the four columns. The switch assigns the incoming port (1) and chooses an available incoming VCI (14) and the outgoing port (3). It does not yet know the outgoing VCI, which will be found during the acknowledgment step. The switch then forwards the frame through port 3 to switch 2.

Switching	Routing
Layer 2 Switches perform Switching	Layer 3 devices like Router perform Routing
Switching will be faster as switch uses ASIC technology	Routing will be slower as it is software based.
Switching is done at layer 2 of OSI Model	Routing is done at layer 3 of OSI Model
If the destination is not known to switch it will broadcast the frame.	If the destination is not known to router it will drop the packet.
Switching is done in same broadcast domain.	Routing is done in different networks.
Switching is done by using MAC address.	Routing is done by using IP address.
Protocol data unit at layer 2 is frame	Protocol data unit at layer 3 is packet

c) Switch 2 receives the setup request frame. The same events happen here as at switch 1; three columns of the table are completed: in this case, incoming port (1), incoming VCI (66), and outgoing port (2).

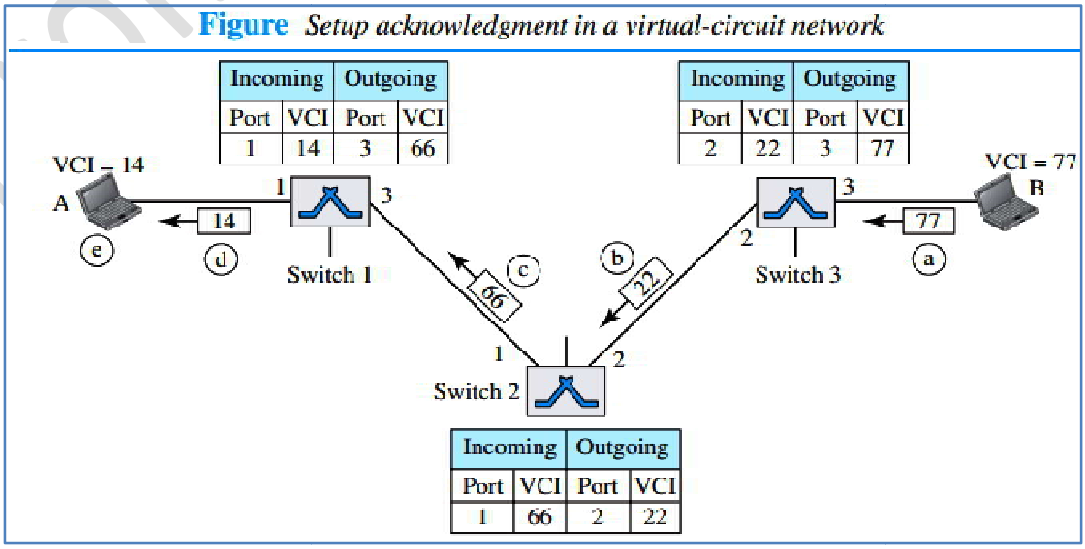
d) Switch 3 receives the setup request frame. Again, three columns are completed: incoming port (2), incoming VCI (22), and outgoing port (3).

e) Destination B receives the setup frame, and if it is ready to receive frames from A, it assigns a VCI to the incoming frames that come from A, in this case 77. This VCI lets the destination know that the frames come from A, and not other sources.

Acknowledgment

- A special frame, called the acknowledgment frame, completes the entries in the switching tables. Figure shows the process.

Figure *Setup acknowledgment in a virtual-circuit network*



a) The destination sends an acknowledgment to switch 3. The acknowledgment carries the global source and destination addresses so the switch knows which entry in the table is to be completed. The frame also carries VCI 77, chosen by the destination as the incoming VCI for frames from A. Switch 3 uses this VCI to complete the outgoing VCI column for this entry. Note that 77 is the incoming VCI for destination B, but the outgoing VCI for switch 3.

b) Switch 3 sends an acknowledgment to switch 2 that contains its incoming VCI in the table, chosen in the previous step. Switch 2 uses this as the outgoing VCI in the table.

c) Switch 2 sends an acknowledgment to switch 1 that contains its incoming VCI in the table, chosen in the previous step. Switch 1 uses this as the outgoing VCI in the table.

d) Finally switch 1 sends an acknowledgment to source A that contains its incoming VCI in the table, chosen in the previous step.

e) The source uses this as the outgoing VCI for the data frames to be sent to destination B.

Teardown Phase

- In this phase, source A, after sending all frames to B, sends a special frame called a teardown request. Destination B responds with a **teardown confirmation frame**. **All switches delete the corresponding entry from their tables.**

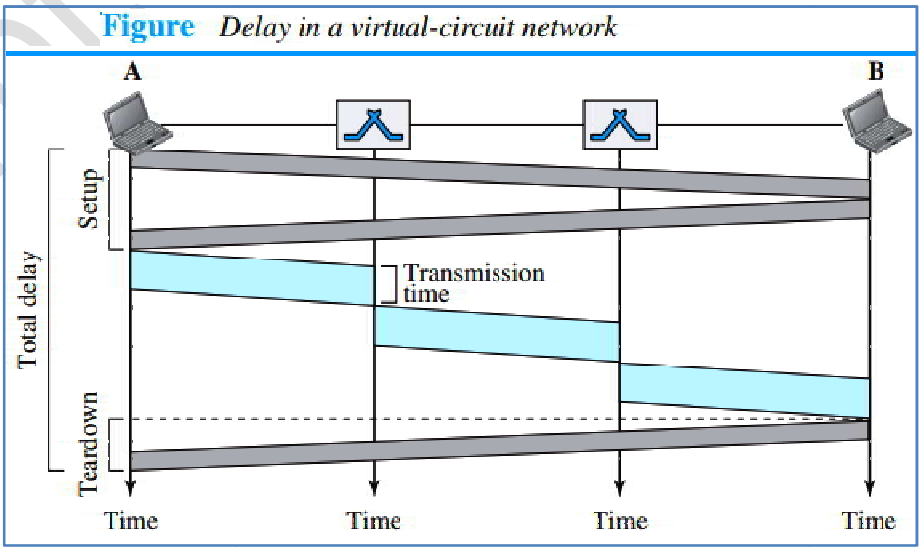
Efficiency

- Resource reservation in a virtual-circuit network can be made
  - during the setup or
  - can be on demand during the data-transfer phase.
- In the first case, the delay for each packet is the same; in the second case, each packet may encounter different delays. There is one big advantage in a virtual-circuit network even if resource allocation is on demand. The source can check the availability of the resources, without actually reserving it.
- Consider a family that wants to dine at a restaurant. Although the restaurant may not accept reservations (allocation of the tables is on demand), the family can call and find out the waiting time. This can save the family time and effort.

Delay in Virtual-Circuit Networks

- In a virtual-circuit network, there is a one-time delay for setup and a one-time delay for teardown. If resources are allocated during the setup phase, there is no wait time for individual packets. Figure shows the delay for a packet traveling through two switches in a virtual-circuit network.

Figure *Delay in a virtual-circuit network*



Prof Parthasarathy P V

- The packet is traveling through two switches (routers). There are three transmission times (**3*T***), three propagation times (***3τ***), data transfer depicted by the sloping lines, a setup delay (which includes transmission and propagation in two directions), and a teardown delay (which includes transmission and propagation in one direction). We ignore the processing time in each switch. The total delay time is

$$\text{Total delay} + 3T + 3\tau + \text{setup delay} + \text{teardown delay}$$

- Circuit-Switched Technology in WANs, virtual-circuit networks are used in switched WANs such as ATM networks. The data-link layer of these technologies is well suited to the virtual circuit technology.

Prof. Parthasarathy P V

BCS502 COMPUTER NETWORKS

Module-2

Chapter 10: Data Link Layer (Pg. 257[296] to 282): Error Detection and Correction: Introduction, Block Coding, Cyclic Codes.

Chapter 11: Data link control (Pg. 293[342] to 319): DLC Services: Framing, Flow Control, Error Control, Connectionless and Connection Oriented, Data link layer protocols, High Level Data Link Control.

Chapter 12: Media Access Control (Pg. 325[364] to 344): Random Access, Controlled Access. Check Sum and Point to Point Protocol.

Textbook: Ch. 10.1-10.4, 11.1 -11.4, 12.1 - 12.2

Chapter 10: Data Link Layer (Pg. 257[296])

Error Detection and Correction: Introduction, Block Coding, Cyclic Codes

---

- Networks must be able to transfer data from one device to another with acceptable accuracy. For most applications, a system must guarantee that the data received are identical to the data transmitted.
- Any time data are transmitted from one node to the next, they can become corrupted in passage. Many factors can alter one or more bits of a message.
- Some applications require a mechanism for detecting and correcting errors. Some applications can tolerate a small level of error. For example, random errors in audio or video transmissions may be tolerable, but when we transfer text, we expect a very high level of accuracy.
- At the ***data-link layer, if a frame is corrupted between the two nodes, it needs to be corrected before it continues its journey to other nodes***. However, most link-layer protocols simply discard the frame and let the upper-layer protocols handle the retransmission of the frame. Some multimedia applications, however, try to correct the corrupted frame.

Topics Covered: Error Detection and Correction: Introduction, Block Coding, Cyclic Codes

---

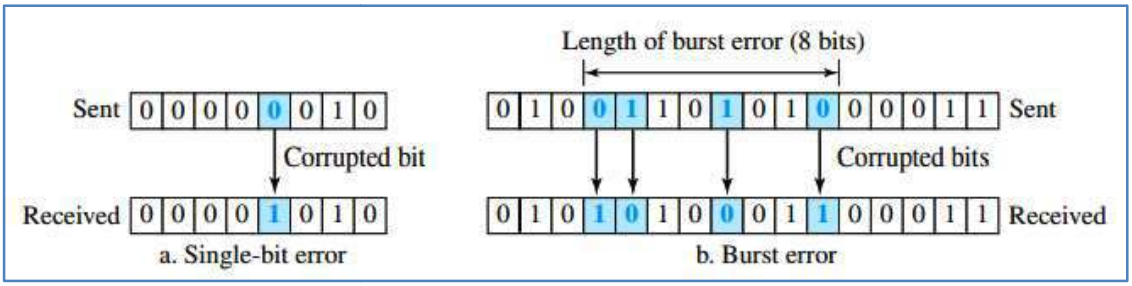
INTRODUCTION

Types of Errors – Redundancy - Detection versus Correction – Coding

---

Types of Errors

- Whenever ***bits flow from one point to another***, they are ***subject to unpredictable changes because of interference***. This ***interference can change the shape of the signal***.
- The term ***single-bit error*** means that only 1 bit of a given data unit (such as a byte, character, or packet) is changed from 1 to 0 or from 0 to 1.
- The term ***burst error*** means that 2 or more bits in the data unit have changed from 1 to 0 or from 0 to 1. Figure shows the effect of a single-bit and a burst error on a data unit.



Prof. Parthasarathy P V

- A ***burst error is more likely to occur than a single-bit error*** because the ***duration of the noise signal is normally longer than the duration of 1 bit***, which means that when noise affects data, it affects a set of bits. The number of bits affected depends on the data rate and duration of noise.
- For example, if we are ***sending data at 1 kbps, a noise of 1/100 second can affect 10 bits***; if we are sending data at ***1 Mbps, the same noise can affect 10,000 bits***.

Redundancy

- The central concept in detecting or correcting errors is ***redundancy***. To be able to detect or correct errors, we need to send some extra bits with our data. These redundant bits are added by the sender and removed by the receiver. Their presence allows the receiver to detect or correct corrupted bits.

Detection versus Correction

- The ***correction of errors is more difficult than the detection***.
- In ***error detection***, we are only looking to see if any error has occurred. The answer is a simple yes or no. We are not even interested in the number of corrupted bits. A single-bit error is the same for us as a burst error.
- In ***error correction***, we need to know the exact number of bits that are corrupted and, more importantly, their location in the message. The ***number of errors and the size of the message are important factors***. If we ***need to correct a single error in an 8-bit data unit, we need to consider eight possible error locations***; if ***we need to correct two errors*** in a data unit of the same size, we need to ***consider 28 (permutation of 8 by 2) possibilities***. You can imagine the receiver's difficulty in finding 10 errors in a data unit of 1000 bits.

Coding

- Redundancy is achieved through various coding schemes.
- The ***sender adds redundant bits through a process*** that creates a relationship between the redundant bits and the actual data bits.
- The ***receiver checks the relationships between the two sets of bits to detect errors***.
- The ratio of redundant bits to data bits and the robustness of the process are important factors in any coding scheme.
- We can divide coding schemes into two broad categories: ***block coding*** and ***convolution coding***. In ***this book, we concentrate on block coding***; ***convolution coding is more complex*** and beyond the scope of this book.

BLOCK CODING

- In ***block coding, we divide our message into blocks***, ***each of k bits, called datawords***.
- We add ***"r"*** redundant bits to each block to make the length ***"n = k + r".*** The resulting ***"n-bit"*** blocks are called ***codewords***.
- For the moment, it is important to know that we have a set of ***datawords***, each of size ***"k"***, and a set of ***codewords***, each of size of ***"n"***.
- With ***"k"*** bits, we can create a combination of ***"2k" datawords***; with ***"n"*** bits, we can create a combination of ***"2n"*** codewords. Since ***"n > k",*** the number of possible ***codewords*** is larger than the number of possible ***datawords***.

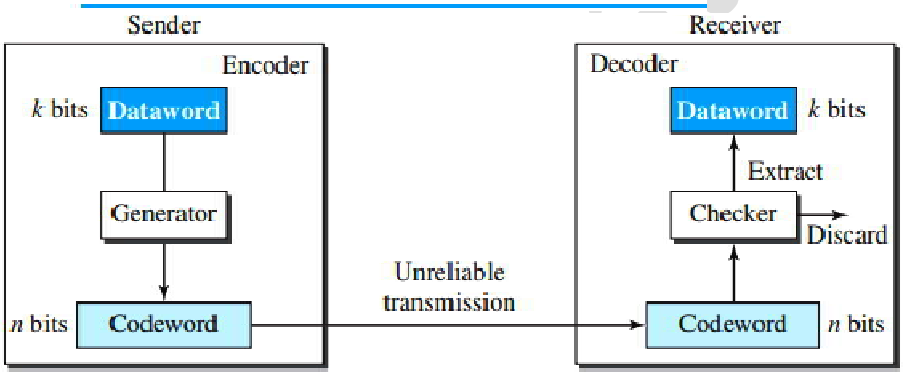
Prof Parthasarathy P V

- The ***block coding*** process is ***one-to-one***; the same ***dataword*** is always ***encoded as the same codeword***. This means that we have ***"2n – 2k" codewords*** that are not used. We call these ***codewords*** invalid or illegal. The trick in error detection is the existence of these invalid codes.
- ***If the receiver receives an invalid codeword, this indicates that the data was corrupted during transmission.***

Error Detection

- How can errors be detected by using block coding? If the following two conditions are met, the receiver can detect a change in the original codeword.
  1. The receiver has (or can find) a list of valid codewords.
  2. The original codeword has changed to an invalid one.
- ***Figure shows the role of block coding in error detection***. The sender creates codewords out of datawords by using a generator that applies the rules and procedures of encoding. Each codeword sent to the receiver may change during transmission. ***If the received codeword is the same as one of the valid codewords, the word is accepted; the corresponding dataword is extracted for use.*** If the ***received codeword is not valid, it is discarded***. However, ***if the codeword is corrupted during transmission but the received word still matches a valid codeword, the error remains undetected***.

Figure *Process of error detection in block coding*



Example

Let us assume that $k = 2$ and $n = 3$. Table shows the list of datawords and codewords.

Table *A code for error detection in Example*

Dataword	*Codeword*	*Dataword*	*Codeword*
00	000	10	101
01	011	11	110

Assume the sender encodes the dataword 01 as 011 and sends it to the receiver. Consider the following cases:

1. The receiver receives 011. It is a valid codeword. The receiver extracts the dataword 01 from it.
2. The codeword is corrupted during transmission, and 111 is received (the leftmost bit is corrupted). This is not a valid codeword and is discarded.
3. The codeword is corrupted during transmission, and 000 is received (the right two bits are corrupted). This is a valid codeword. The receiver incorrectly extracts the dataword 00. Two corrupted bits have made the error undetectable.

An error-detecting code can detect only the types of errors for which it is designed; other types of errors may remain undetected.

Hamming Distance

- One of the central concepts in coding for error control is the idea of the ***Hamming distance***. The ***Hamming distance between two words (of the same size) is the number of differences between the corresponding bits***.
- We show the Hamming distance between two words x and y as d(x, y). We may wonder why Hamming distance is important for error detection. The reason is that the Hamming distance between the received codeword and the sent codeword is the number of bits that are corrupted during transmission.
- For example, if the codeword 00000 is sent and 01101 is received, 3 bits are in error and the Hamming distance between the two is d(00000, 01101) = 3. In other words, if the Hamming distance between the sent and the received codeword is not zero, the codeword has been corrupted during transmission.
- The Hamming distance can easily be found if we apply the XOR operation **(⊕)** on the two words and count the number of 1s in the result. Note that the Hamming distance is a value greater than or equal to zero.

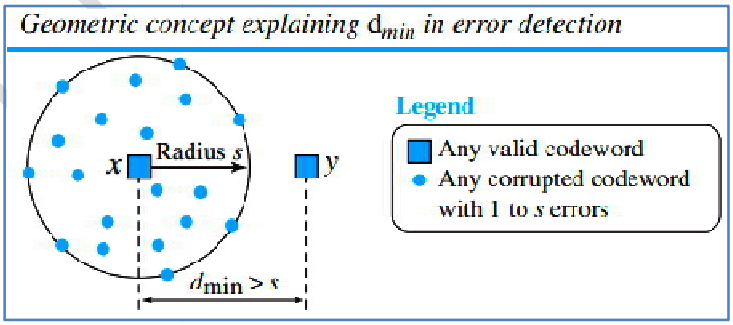
Example

Let us find the Hamming distance between two pairs of words.

1. The Hamming distance d(000, 011) is 2 because (000 ⊕ 011) is 011 (two 1s).
2. The Hamming distance d(10101, 11110) is 3 because (10101 ⊕ 11110) is 01011 (three 1s).

Minimum Hamming Distance for Error Detection

- ***In a set of codewords, the minimum Hamming distance is the smallest Hamming distance between all possible pairs of codewords.***
- Now let us find the minimum Hamming distance in a code if we want to be able to detect up to **"s"** errors. If **"s"** errors occur during transmission, the Hamming distance between the sent codeword and received codeword is **"s"**.
- If our system is to detect up to **"s"** errors, the minimum distance between the valid codes must be **"(s + 1)"**, so that the received codeword does not match a valid codeword.
- In other words, if the minimum distance between all valid codewords is **"(s + 1)"**, the received codeword cannot be erroneously mistaken for another codeword. The error will be detected. We need to clarify a point here: Although a code with $d_{min} = s + 1$ may be able to detect more than **"s"** errors in some special cases, only **"s"** or fewer errors are guaranteed to be detected.



- We can look at this criteria geometrically. Let us assume that the sent codeword **"x"** is at the center of a circle with radius **"s"**. All received codewords that are created by **0** to **"s"** errors are points inside the circle or on the perimeter of the circle. All other valid codewords must be outside the circle, as shown in Figure. This means that d_{min} must be an integer greater than s or $d_{min} = s + 1$.

Example

- The minimum Hamming distance for our first code scheme (Table) is 2.

Prof Parthasarathy P V

- This code guarantees detection of only a single error. For example, if the third codeword (101) is sent and one error occurs, the received codeword does not match any valid codeword. **If two errors occur, however, the received codeword may match a valid codeword and the errors are not detected**.

Example

- A code scheme has a Hamming distance $d_{min} = 4$. This code guarantees the **detection of up to three errors** ($d = s + 1$ or $s = 3$).

Linear Block Codes

- Almost all block codes used today belong to **a subset of block codes called linear block codes**.
- The **use of nonlinear block codes for error detection and correction is not as widespread** because **their structure makes theoretical analysis and implementation difficult**.
- The **formal definition of linear block codes requires the knowledge of abstract algebra.** We therefore give an informal definition. For our purposes, ***a linear block code is a code in which the exclusive OR (addition modulo-2) of two valid codewords creates another valid codeword***.

Example

- The code in Table is a linear block code because the result of XORing any codeword with any other codeword is a valid codeword. For example, the XORing of the second and third codewords creates the fourth one.

Minimum Distance for Linear Block Codes

- It is simple to find the minimum Hamming distance for a linear block code. The minimum Hamming distance is the number of 1s in the nonzero valid codeword with the smallest number of 1s.

Example

- In our first code (Table), the numbers of 1s in the nonzero codewords are 2, 2, and 2. So the minimum Hamming distance is $d_{min} = 2$.

Parity-Check Code

- Perhaps the **most familiar error-detecting code is the parity-check code**. This code is a linear block code. In this code, a k-bit dataword is changed to an n-bit codeword where $n = k + 1$. The extra bit, called the parity bit, is selected to make the total number of 1s in the codeword even. Although some implementations specify an odd number of 1s, we discuss the even case. The minimum Hamming distance for this category is $d_{min} = 2$, which means that the code is a single-bit error-detecting code. Our first code (Table) is a parity-check code ($k = 2$ and $n = 3$). The code in the following table is also a parity-check code with $k = 4$ and $n = 5$.

Table *Simple parity-check code C(5, 4)*

Dataword	*Codeword*	*Dataword*	*Codeword*
0000	**00000**	1000	**10001**
0001	**00011**	1001	**10010**
0010	**00101**	1010	**10100**
0011	**00110**	1011	**10111**
0100	**01001**	1100	**11000**
0101	**01010**	1101	**11011**
0110	**01100**	1110	**11101**
0111	**01111**	1111	**11110**

- The following Figure shows a possible structure of an encoder (at the sender) and a decoder (at the receiver).

Prof. Parthasarathy P V

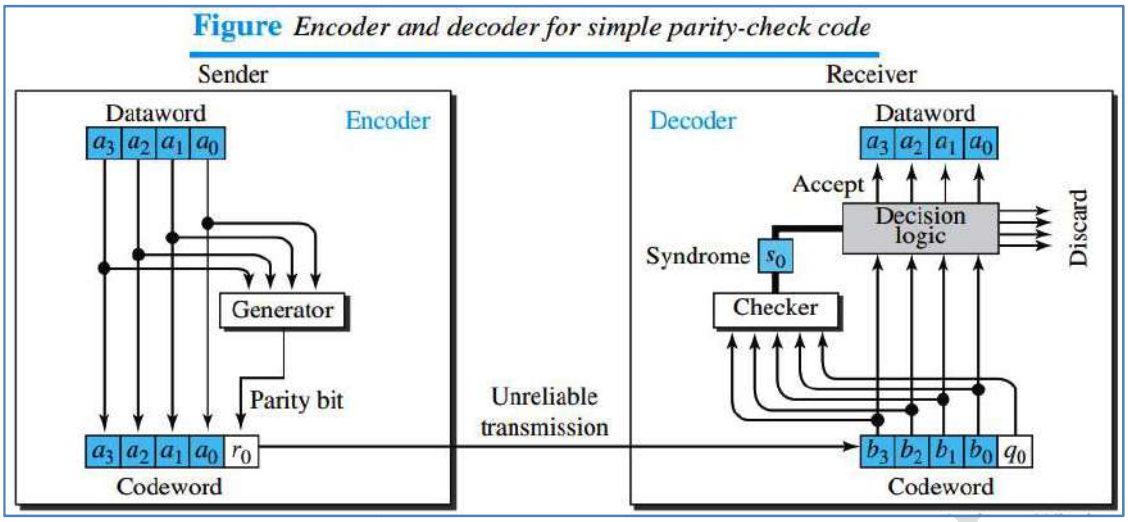


Figure *Encoder and decoder for simple parity-check code*

- The calculation is done in modular arithmetic. The encoder uses a generator that takes a copy of a 4-bit dataword (a_0, a_1, a_2, and a_3) and generates a parity bit r_0. The dataword bits and the parity bit create the ***5-bit codeword***. The parity bit that is added makes the number of 1s in the codeword even. This is normally done by adding the 4 bits of the dataword (modulo-2); the result is the parity bit. In other words, $r_0 = a_3 + a_2 + a_1 + a_0$ **(modulo-2)**
- If the number of 1s is even, the result is 0; if the number of 1s is odd, the result is 1. In both cases, the total number of 1s in the codeword is even.
- The sender sends the codeword, which may be corrupted during transmission. The receiver receives a 5-bit word. The checker at the receiver does the same thing as the generator in the sender with one exception: The addition is done over all 5 bits. The result, which is called the ***syndrome***, is just ***1 bit***. The syndrome is 0 when the number of 1s in the received codeword is even; otherwise, it is 1. $s_0 = b_3 + b_2 + b_1 + b_0 + q_0$ **(modulo-2)**
- The syndrome is passed to the decision logic analyzer.
- If the ***syndrome is 0, there is no detectable error in the received codeword; the data portion of the received codeword is accepted as the dataword***.
- If the ***syndrome is 1, the data portion of the received codeword is discarded. The dataword is not created***.

Example

Let us look at some transmission scenarios. Assume the sender sends the dataword 1011. The codeword created from this dataword is 10111, which is sent to the receiver. We examine five cases:

1. **No error occurs**; the received codeword is **10111**. The syndrome is **0**. The dataword **1011** is created.
2. **One single-bit error changes a_1**. The received codeword is **10011**. The syndrome is **1**. No dataword is created.
3. **One single-bit error changes r_0**. The received codeword is **10110**. The syndrome is **1**. No dataword is created. ***Note that although none of the dataword bits are corrupted, no dataword is created because the code is not sophisticated enough to show the position of the corrupted bit.***
4. An error changes r_0 and a second error changes a_3. The received codeword is **00110**. The syndrome is **0**. The dataword **0011** is created at the receiver. ***Note that here the dataword is wrongly created due to the syndrome value.*** The simple parity-check decoder cannot detect an even number of errors. The errors cancel each other out and give the syndrome a value of **0**.
5. Three bits—a_3, a_2, and a_1—are changed by errors. The received codeword is **01011**. The syndrome is **1**. The dataword is not created. ***This shows that the simple parity check, guaranteed to detect one single error, can also find any odd number of errors.***

Prof. Parthasarathy

Cyclic Codes

- Cyclic codes are special linear block codes with one extra property. In a cyclic code, if a codeword is cyclically shifted (rotated), the result is another codeword.
- For example, if 1011000 is a codeword and we cyclically left-shift, then 0110001 is also a codeword. In this case, if we call the bits in the first word a_0 to a_6, and the bits in the second word b_0 to b_6, we can shift the bits by using the following:

$$b_1 = a_0 \quad b_2 = a_1 \quad b_3 = a_2 \quad b_4 = a_3 \quad b_5 = a_4 \quad b_6 = a_5 \quad b_0 = a_6$$

- In the rightmost equation, the last bit of the first word is wrapped around and becomes the first bit of the second word.

Topics Covered (7 topics): **Cyclic Redundancy Check - Polynomials - Cyclic Code Encoder Using Polynomials - Cyclic Code Analysis - Advantages of Cyclic Codes - Other Cyclic Codes - Hardware Implementation.**

---

1. Cyclic Redundancy Check

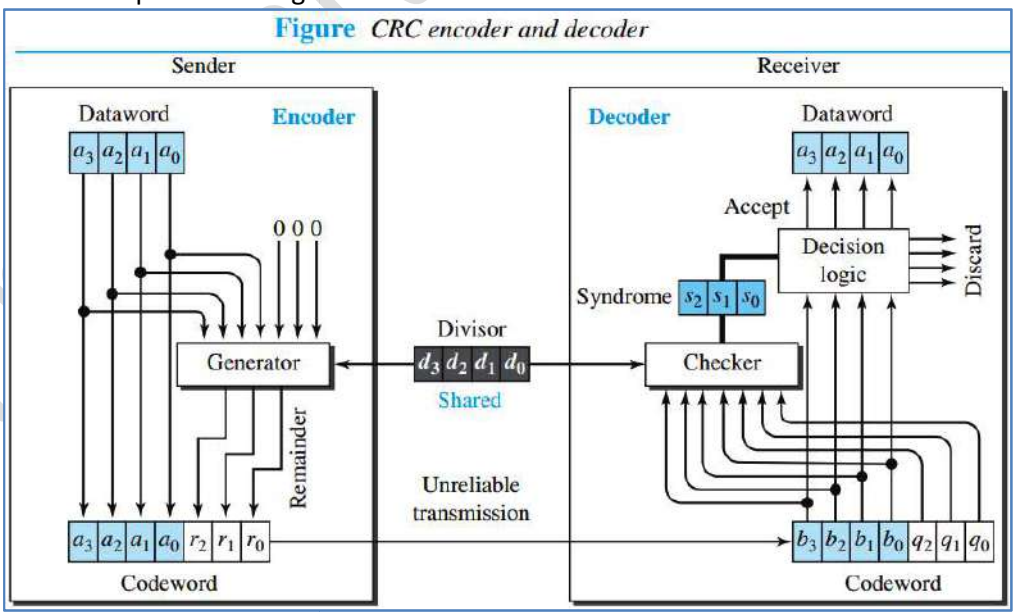
- We can create cyclic codes to correct errors. A subset of cyclic codes called the ***cyclic redundancy check (CRC),*** which is ***used in networks such as LANs and WANs***.
- Table shows an example of a CRC code. We can see both the linear and cyclic properties of this code.

Table *A CRC code with C(7, 4)*

Dataword	*Codeword*	*Dataword*	*Codeword*
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

- Figure shows one possible design for the encoder and decoder.

Figure *CRC encoder and decoder*



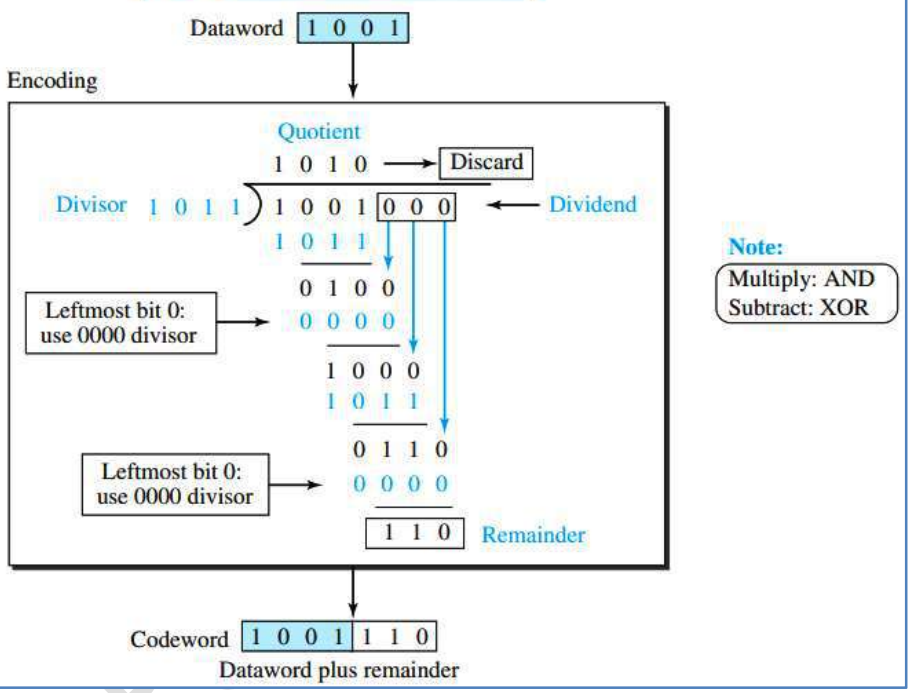
- In the encoder, the dataword has k bits (4 here); the codeword has n bits (7 here). The size of the dataword is augmented by adding n − k (3 here) 0s to the right-hand side of the word. The n-bit result is fed into the generator.

- The generator uses a divisor of size n − k + 1 (4 here), predefined and agreed upon. The generator divides the augmented dataword by the divisor (modulo-2 division). The quotient of the division is discarded; the remainder ($r_2r_1r_0$) is appended to the dataword to create the codeword.
- The decoder receives the codeword (possibly corrupted in transition). A copy of all n bits is fed to the checker, which is a replica of the generator. The remainder produced by the checker is a syndrome of n − k (3 here) bits, which is fed to the decision logic analyzer.
- The analyzer has a simple function. If the syndrome bits are all 0s, the 4 leftmost bits of the codeword are accepted as the dataword (interpreted as no error); otherwise, the 4 bits are discarded (error).

Encoder

- Let us take a closer look at the encoder. The encoder takes a dataword and augments it with n − k number of 0s. It then divides the augmented dataword by the divisor, as shown in Figure.

Figure *Division in CRC encoder*



- The process of modulo-2 binary division is the same as the familiar division process we use for decimal numbers. However, addition and subtraction in this case are the same; we use the XOR operation to do both.
- As in decimal division, the process is done step by step. In each step, a copy of the divisor is XORed with the 4 bits of the dividend. The result of the XOR operation (remainder) is 3 bits (in this case), which is used for the next step after 1 extra bit is pulled down to make it 4 bits long. There is one important point we need to remember in this type of division. If the leftmost bit of the dividend (or the part used in each step) is 0, the step cannot use the regular divisor; we need to use an all-0s divisor.
- When there are no bits left to pull down, we have a result. The 3-bit remainder forms the check bits (r_2, r_1, and r_0). They are appended to the dataword to create the codeword.

Decoder

- The codeword can change during transmission. The decoder does the same division process as the encoder.
- ***The remainder of the division is the syndrome***. If the ***syndrome is all 0s, there is no error*** with a high probability; the dataword is separated from the received codeword and accepted. ***Otherwise,***

everything is discarded. Figure shows two cases: The left-hand figure shows the value of the syndrome when no error has occurred; the syndrome is 000. The right-hand part of the figure shows the case in which there is a single error. The syndrome is not all 0s (it is 011).

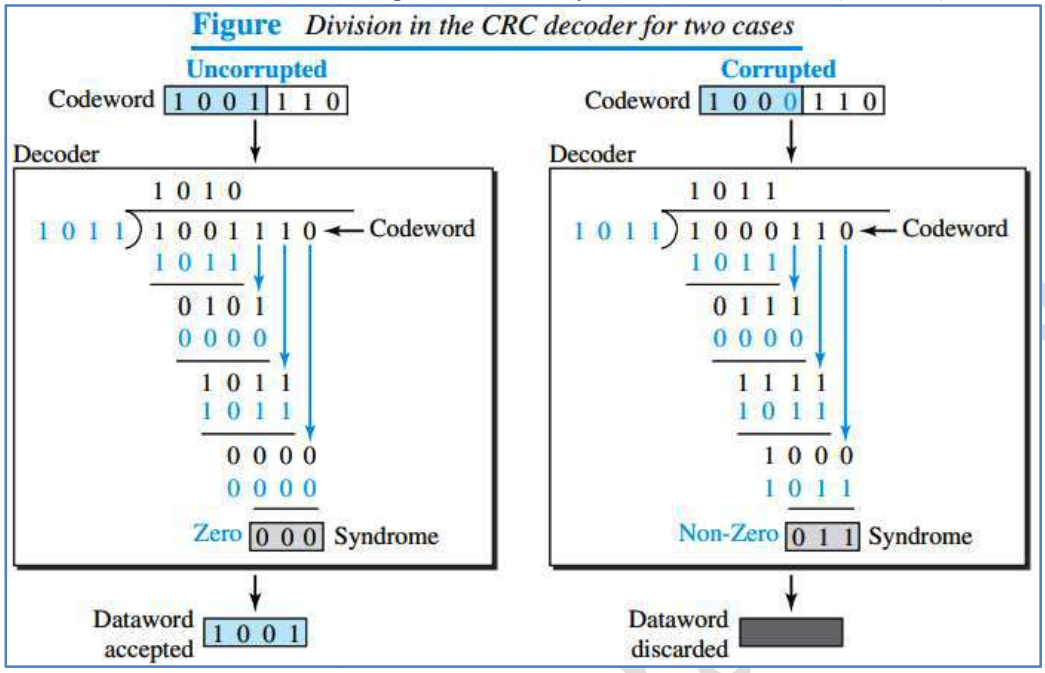


Figure *Division in the CRC decoder for two cases*

Divisor

- We may be wondering how the divisor 1011 is chosen. This depends on the expectation we have from the code.

Table *Standard polynomials*

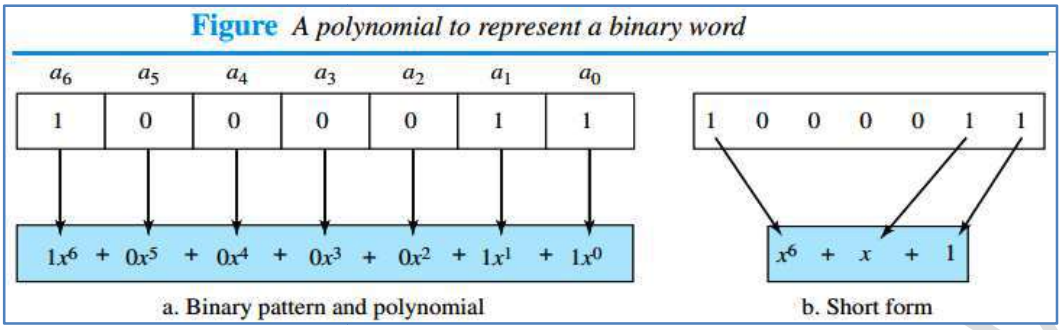
Name	*Polynomial*	*Used in*
CRC-8	$x^8 + x^2 + x + 1$ **100000111**	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$ **11000110101**	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$ **10001000000100001**	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ **10000010011000001000111011011 0111**	LANs

---

2. Polynomials

- A better way to understand cyclic codes and how they can be analyzed is to represent them as polynomials.
- A pattern of 0s and 1s can be represented as a polynomial with coefficients of 0 and 1. The power of each term shows the position of the bit; the coefficient shows the value of the bit.
- ***Figure*** shows a binary pattern and its polynomial representation. In ***Figure (a)*** we show how to translate a binary pattern into a polynomial; in ***Figure (b)*** we show how the polynomial can be shortened by removing all terms with zero coefficients and replacing x^1 by x and x^0 by 1.
- Figure shows in the next page is one immediate benefit; a 7-bit pattern can be replaced by three terms. The benefit is even more conspicuous when we have a polynomial such as $x^{23} + x^3 + 1$. ***Here***

the bit pattern is 24 bits in length (three 1s and twenty-one 0s) while the polynomial is just three terms.



Degree of a Polynomial

- The degree of a polynomial is the highest power in the polynomial. For example, the degree of the polynomial x6 + x + 1 is 6. Note that the degree of a polynomial is 1 less than the number of bits in the pattern. The bit pattern in this case has 7 bits.

Adding and Subtracting Polynomials

- Adding and subtracting polynomials in mathematics are done by adding or subtracting the coefficients of terms with the same power.
- In our case, the coefficients are only 0 and 1, and adding is in modulo-2. This has ***two consequences.***
- **First**, addition and subtraction are the same.
- **Second**, adding or subtracting is done by combining terms and deleting pairs of identical terms.
- For example, adding $x^5 + x^4 + x^2$ and $x^6 + x^4 + x^2$ gives just $x^6 + x^5$. The terms x^4 and x^2 are deleted. However, note that if we add, for example, three polynomials and we get x^2 three times, we delete a pair of them and keep the third.

Multiplying or Dividing Terms

- In this arithmetic, multiplying a term by another term is very simple; we just add the powers. For example, $x^3 \times x^4$ is x^7. For dividing, we just subtract the power of the second term from the power of the first. For example, x^5/x^2 is x^3.

Multiplying Two Polynomials

- Multiplying a polynomial by another is done term by term. Each term of the first polynomial must be multiplied by all terms of the second. The result, of course, is then simplified, and pairs of equal terms are deleted. The following is an example:

$$(x^5 + x^3 + x^2 + x)(x^2 + x + 1) = x^7 + x^6 + x^5 + x^5 + x^4 + x^3 + x^4 + x^3 + x^2 + x^3 + x^2 + x$$
$$= x^7 + x^6 + x^3 + x$$

Dividing One Polynomial by Another

- Division of polynomials is conceptually the same as the binary division we discussed for an encoder. We divide the first term of the dividend by the first term of the divisor to get the first term of the quotient. We multiply the term in the quotient by the divisor and subtract the result from the dividend. We repeat the process until the dividend degree is less than the divisor degree.

Shifting

- A binary pattern is often shifted a number of bits to the right or left. Shifting to the left means adding extra 0s as rightmost bits; shifting to the right means deleting some rightmost bits. Shifting to the left is accomplished by multiplying each term of the polynomial by x^m, where m is the number of shifted bits; shifting to the right is accomplished by dividing each term of the polynomial by x^m. The following shows shifting to the left and to the right. Note that we do not have negative powers in the polynomial representation.

Prof. Parthasarathy P V

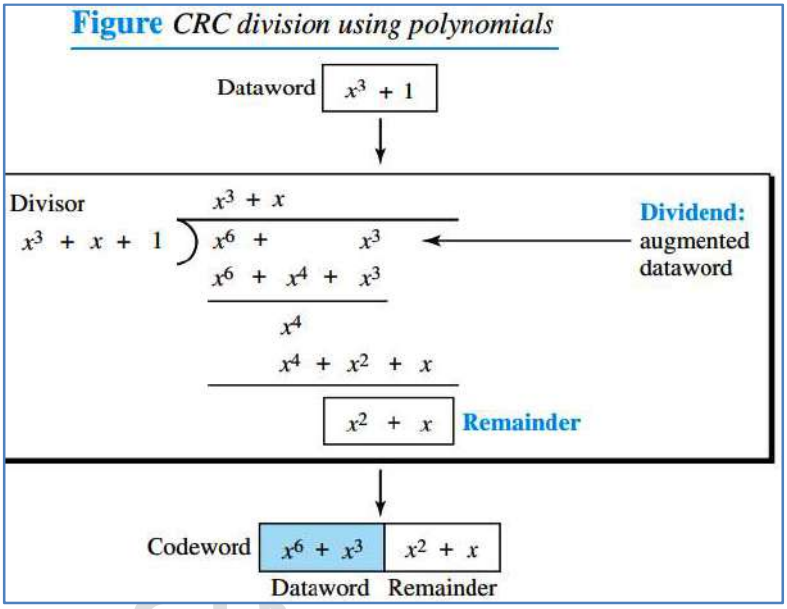
Shifting left 3 bits: 10011 becomes 10011000	$x^4 + x + 1$ becomes $x^7 + x^4 + x^3$
Shifting right 3 bits: 10011 becomes 10	$x^4 + x + 1$ becomes x

---

3. Cyclic Code Encoder Using Polynomials

- Now we see the creation of a codeword from a dataword. **(Figure).** The dataword 1001 is represented as x^3+1. The divisor 1011 is represented as x^3+x+1.
- To find the augmented dataword, we have left-shifted the dataword 3 bits (multiplying by x^3). The result is x^6+x^3. Division is straightforward. We divide the first term of the dividend, x^6, by the first term of the divisor, x^3. The first term of the quotient is then x^6/x^3, or x^3. Then we multiply x3 by the divisor and subtract (according to our previous definition of subtraction) the result from the dividend. The result is x^4, with a degree greater than the divisor's degree; we continue to divide until the degree of the remainder is less than the degree of the divisor.

Figure *CRC division using polynomials*



- It can be seen that the polynomial representation can easily simplify the operation of division in this case, because the two steps involving all-0s divisors are not needed here.
- In a polynomial representation, the ***divisor in a cyclic code is normally referred to as the generator polynomial t(x)***.

---

4. Cyclic Code Analysis

- We can analyze a cyclic code to find its capabilities by using polynomials. We define the following, where f(x) is a polynomial with binary coefficients.

Dataword: $d(x)$ **Codeword: $c(x)$** **Generator: $g(x)$** **Syndrome: $s(x)$** **Error: $e(x)$**

- ***If s(x) is not zero, then one or more bits is corrupted***. However, if ***s(x) is zero***, ***either no bit is corrupted or the decoder failed to detect any errors.*** (Note that ¦ means divide).

In a cyclic code,

1. If $s(x) \nmid 0$, one or more bits is corrupted.
2. If $s(x) = 0$, either
   a. No bit is corrupted, or
   b. Some bits are corrupted, but the decoder failed to detect them.

- In our analysis we want to find the criteria that must be imposed on the generator, g(x) to detect the type of error we especially want to be detected.
- Let us first find the ***relationship among the sent codeword, error, received codeword***, and the ***generator***. We can say **Received codeword = c(x) + e(x).**
- In other words, ***the received codeword is the sum of the sent codeword and the error***. The receiver divides the received codeword by ***g(x) to get the syndrome***. We can write this as

$$\frac{\text{Received codeword}}{g(x)} = \frac{c(x)}{g(x)} + \frac{e(x)}{g(x)}$$

Prof. Parthasa

- The ***first term at the right-hand side of the equality has a remainder of zero*** (according to the definition of codeword). So the ***syndrome is actually the remainder of the second term on the right-hand side***. If this term does not have a remainder (syndrome = 0), either e(x) is 0 or e(x) is divisible by g(x). ***We do not have to worry about the first case (there is no error); the second case is very important***. Those errors that are divisible by g(x) are not caught.
- Let us show ***some specific errors and see how they can be caught by a welldesigned g(x).***
  - ***Single-Bit Error***
  - ***Two Isolated Single-Bit Errors***
  - ***Odd Numbers of Errors***
  - ***Burst Errors***

Single-Bit Error

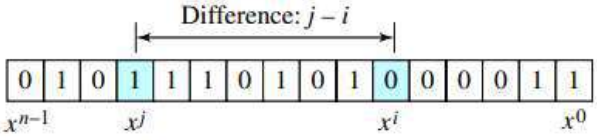
What should the structure of g(x) be to guarantee the detection of a single-bit error?

- A single-bit error is **$e(x) = x^i$**, where **"i"** is the position of the bit. If a single-bit error is caught, then **x^i** is not divisible by **g(x)**. (Note that when we say not divisible, we mean that there is a remainder.) If g(x) has at least two terms (which is normally the case) and the coefficient of x^0 is not zero (the rightmost bit is 1), then e(x) cannot be divided by g(x).
- ***If the generator has more than one term and the coefficient of x^0 is 1, all single-bit errors can be caught.***

Two Isolated Single-Bit Errors

- Now imagine there are two single-bit isolated errors. Under what conditions can this type of error be caught? We can show this type of error as **$e(x) = x^j + x^i$**. The values of **"i"** and **"j"** define the positions of the errors, and the difference ***j − i*** defines the distance between the two errors, as shown in Figure.

Representation of two isolated single-bit errors using polynomials



We can write $e(x) = x^j(x^{j-i} + 1)$. If $g(x)$ has more than one term and one term is x^0, it cannot divide x^j, as we saw in the previous section. So if $g(x)$ is to divide $e(x)$, it must divide $x^{j-i} + 1$. In other words, $g(x)$ must not divide $x^t + 1$, where t is between 0 and $n - 1$. However, $t = 0$ is meaningless and $t = 1$ is needed. This means t should be between 2 and $n - 1$

- If a generator cannot divide x^t+1 (t between 0 and n-1), then all isolated double errors can be detected.

Odd Numbers of Errors

A generator with a factor of $x + 1$ can catch all odd numbers of errors. This means that we need to make $x + 1$ a factor of any generator. Note that we are not saying that the generator itself should be $x + 1$; we are saying that it should have a factor of $x + 1$. If it is only $x + 1$, it cannot catch the two adjacent isolated errors.

For example, $x^4 + x^2 + x + 1$ can catch all odd-numbered errors since it can be written as a product of the two polynomials $x + 1$ and $x^3 + x^2 + 1$.

A generator that contains a factor of $x + 1$ can detect all odd-numbered errors.

Burst Errors

- Now let us extend our analysis to the burst error, which is the most important of all. A burst error is of the form **$e(x) = (x^j + \ldots + x^i)$**. Note the difference between a burst error and two isolated single-bit errors. ***The first can have two terms or more***; the ***second can only have two terms***. We can factor out **x^i** and write the error as **$x^i(x^{j-i} + \ldots + 1)$**.

- If our generator can detect a single error (minimum condition for a generator), then it cannot divide x^i. What we should worry about are those generators that divide $x^{j-i} + \ldots + 1$. In other words, the remainder of $(x^{j-i} + \ldots + 1)/(x^r + \ldots + 1)$ must not be zero. Note that the denominator is the generator polynomial.
- We can have ***three cases***: **j – i < r, j – i = r, j – i > r.**
  - ***All burst errors with L ≤ r will be detected.***
  - ***All burst errors with L 5 r 1 1 will be detected with probability 1 – (1/2)$^{r-1}$.***
  - ***All burst errors with L > r 1 1 will be detected with probability 1 – (1/2)^r.***

1. If $j - i < r$, the remainder can never be zero. We can write $j - i = L - 1$, where L is the length of the error. So $L - 1 < r$ or $L < r + 1$ or L ð r. This means all burst errors with length smaller than or equal to the number of check bits r will be detected.
2. In some rare cases, if $j - i = r$, or $L = r + 1$, the syndrome is 0 and the error is undetected. It can be proved that in these cases, the probability of undetected burst error of length $r + 1$ is $(1/2)^{r-1}$. For example, if our generator is $x^{14} + x^3 + 1$, in which $r = 14$, a burst error of length $L = 15$ can slip by undetected with the probability of $(1/2)^{14-1}$ or almost 1 in 10,000.
3. In some rare cases, if $j - i > r$, or $L > r + 1$, the syndrome is 0 and the error is undetected. It can be proved that in these cases, the probability of undetected burst error of length greater than $r + 1$ is $(1/2)^r$. For example, if our generator is $x^{14} + x^3 + 1$, in which $r = 14$, a burst error of length greater than 15 can slip by undetected with the probability of $(1/2)^{14}$ or almost 1 in 16,000 cases.

---

5. Advantages of Cyclic Codes

- We have seen that cyclic codes have a very good performance in detecting single-bit errors, double errors, an odd number of errors, and burst errors.
- They can easily be implemented in hardware and software.
- They are especially fast when implemented in hardware. This has made cyclic codes a good candidate for many networks.

---

6. Other Cyclic Codes

- The cyclic codes we have discussed in this section are very simple. The check bits and syndromes can be calculated by simple algebra. There are, however, more powerful polynomials that are based on abstract algebra involving Galois fields. These are beyond the scope of this book. One of the most interesting of these codes is the ***ReedSolomon code*** used today for both detection and correction.

---

7. Hardware Implementation *(Divisor - Augmented Dataword – Remainder - General Design)*

- One of the advantages of a cyclic code is that the encoder and decoder can ***easily and cheaply be implemented in hardware by using a handful of electronic devices***. Also, a ***hardware implementation increases the rate of check bit and syndrome bit calculation***.

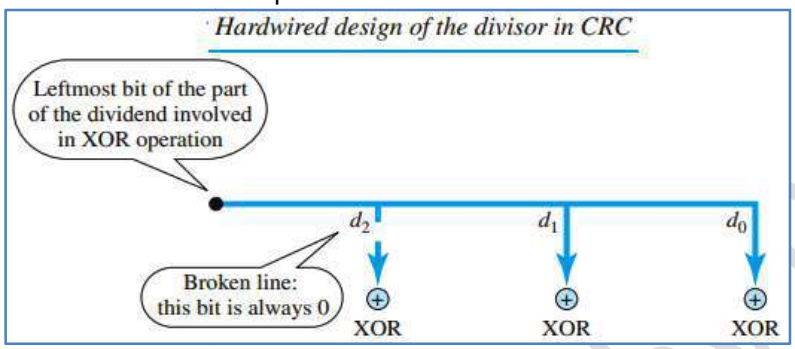
Divisor

- ***We need to note the following points:***
  1. The divisor is repeatedly XORed with part of the dividend.
  2. The divisor has n − k + 1 bits which either are predefined or are all 0s. In other words, the bits do not change from one dataword to another. In our previous example, the divisor bits were either 1011 or 0000. The choice was based on the leftmost bit of the part of the augmented data bits that are active in the XOR operation.

Prof. Parthasarathy V

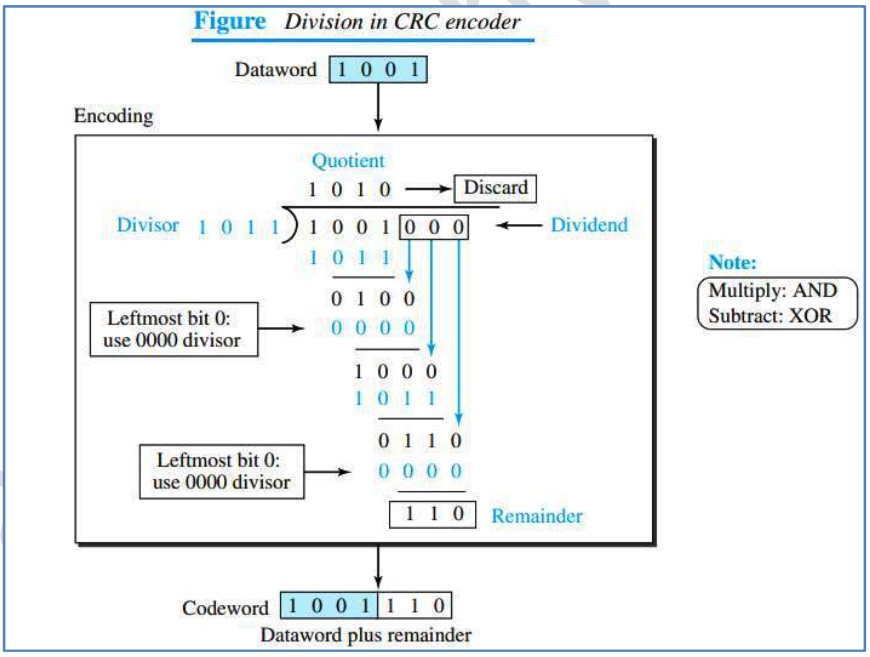
3. A close look shows that only n − k bits of the divisor are needed in the XOR operation. The leftmost bit is not needed because the result of the operation is always 0, no matter what the value of this bit. The reason is that the inputs to this XOR operation are either both 0s or both 1s. In our previous example, only 3 bits, not 4, are actually used in the XOR operation.

- Using these points, we can make a fixed (hardwired) divisor that can be used for a cyclic code if we know the divisor pattern. Figure shows such a design for our previous example. We have also shown the XOR devices used for the operation.



Augmented Dataword

- in Figure, we show the augmented dataword as fixed in position with the divisor bits shifting to the right, 1 bit in each step. ***The divisor bits are aligned with the appropriate part of the augmented dataword***.
- Now that our divisor is fixed, we need instead to shift the bits of the augmented dataword to the left (opposite direction) to align the divisor bits with the appropriate part.
- There is ***no need to store the augmented dataword bits***.

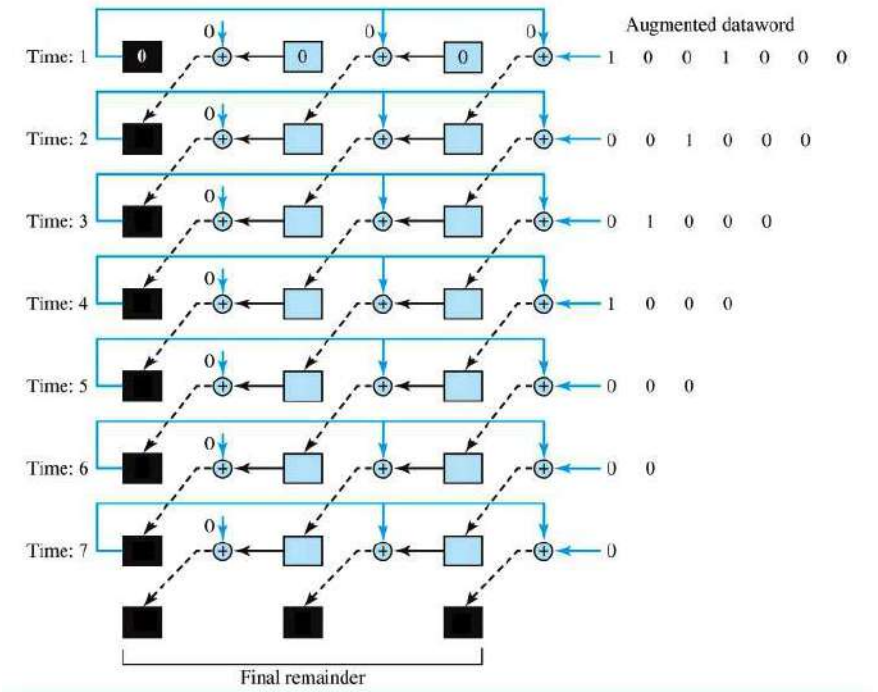


Remainder

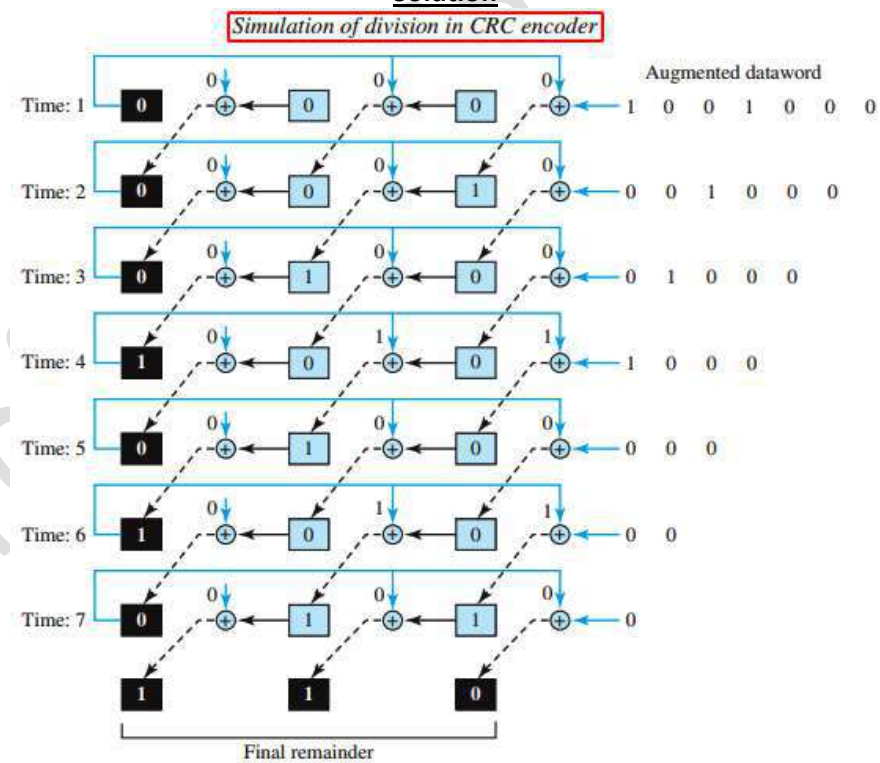
- In our previous example, the remainder is 3 bits (n − k bits in general) in length. We can use three registers (single-bit storage devices) to hold these bits. To find the final remainder of the division, we need to modify our division process.
- The following is the step-by-step process that can be used to simulate the division process in hardware (or even in software).
  1. We assume that the remainder is originally all 0s (000 in our example).
  2. At each time click (arrival of 1 bit from an augmented dataword), we repeat the following two actions:
     i. We use the leftmost bit to make a decision about the divisor (011 or 000).
     ii. The other 2 bits of the remainder and the next bit from the augmented dataword (total of 3 bits) are XORed with the 3-bit divisor to create the next remainder.

- The ***following Figure shows this simulator***, but note that this is not the final design; there will be more improvements.

Figure 10.12 *Simulation of division in CRC encoder*

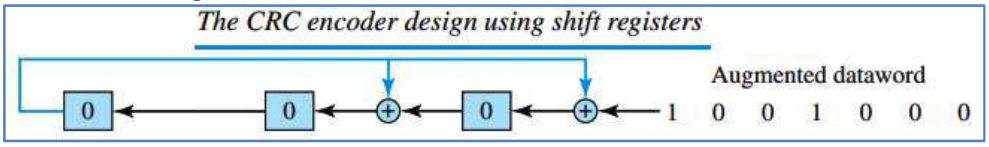


Solution



- The above design is for demonstration purposes only. It needs simplification to be practical. First, we ***do not need to keep the intermediate values of the remainder bits; we need only the final bits***. We therefore need only 3 registers instead of 24. After the XOR operations, we do not need the bit values of the previous remainder.

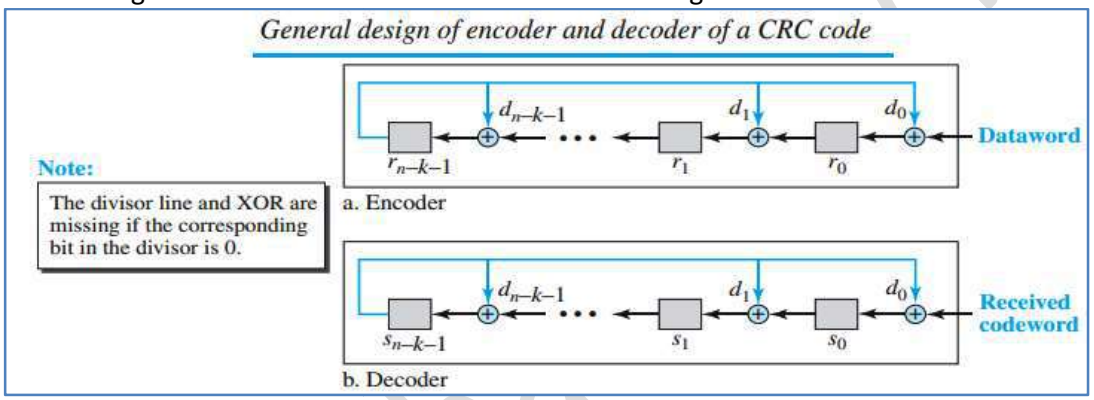
- Also, we do not need 21 XOR devices; two are enough because the output of an XOR operation in which one of the bits is 0 is simply the value of the other bit. This other bit can be used as the output. With these two modifications, the design becomes tremendously simpler and less expensive, as shown in Figure.



- We need, however, ***to make the registers shift registers***. A ***1-bit shift register holds a bit for a duration of one clock time***. At a time click, the ***shift register accepts the bit at its input port, stores the new bit, and displays it on the output port***. The content and the output remain the same until the next input arrives. When we connect several 1-bit shift registers together, it looks as if the contents of the register are shifting.

General Design

- A general design for the encoder and decoder is shown in Figure.



- Note that we have n − k 1-bit shift registers in both the encoder and decoder.
- We have up to n **− k XOR devices, but the divisors normally have several 0s in their pattern, which** reduces the number of devices.
- Also note that, instead of augmented datawords, we show the dataword itself as the input because after the bits in the dataword are all fed into the encoder, the extra bits, which all are 0s, do not have any effect on the rightmost XOR.
- Of course, the process needs to be continued for another n − k steps before the check bits are ready.
- This fact is one of the criticisms of this design. Better schemes have been designed to eliminate this waiting time (the check bits are ready after k steps), but we leave this as a research topic for the reader. In the decoder, however, the entire codeword must be fed to the decoder before the syndrome is ready.

==========

Prof. Partha P V

Chapter 11: Data Link Control (Pg. 293[342] to 319)

DLC Services: Framing, Flow Control, Error Control, Connectionless and Connection Oriented, Data link layer protocols, High Level Data Link Control.

Introduction

- The data-link layer is divided into two sublayers.
  - The upper sublayer of the **data-link layer (DLC)**.
  - The lower sublayer, **multiple access control (MAC)**
- This chapter is divided into four sections.
  - The first section discusses the ***general services provided by the DLC sublayer***. It first describes framing and two types of frames used in this sublayer. The section then discusses flow and error control. Finally, the section explains that a DLC protocol can be either connectionless or connection-oriented.
  - The second section discusses some ***simple and common data-link protocols that are implemented at the DLC sublayer.*** The section first describes the Simple Protocol. It then explains the Stop-and-Wait Protocol.
  - The third section introduces ***HDLC, a protocol that is the basis of all common data-link protocols in use today such as PPP and Ethernet***. The section first talks about configurations and transfer modes. It then describes framing and three different frame formats used in this protocol.
  - The fourth section discusses ***PPP, a very common protocol for point-to-point access***. It first introduces the services provided by the protocol. The section also describes the format of the frame in this protocol. It then describes the transition mode in the protocol using an FSM. The section finally explains multiplexing in PPP.

DLC Services: Framing, Flow Control, Error Control, Connectionless and Connection Oriented

- The ***data link control (DLC) deals with procedures for communication between two adjacent nodes—node-to-node communication—no matter whether the link is dedicated or broadcast***.
- ***Data link control functions include framing and flow and error control***.

Framing

- Data transmission in the physical layer means moving bits in the form of a signal from the source to the destination. The physical layer provides bit synchronization to ensure that the sender and receiver use the same bit durations and timing.
- The data-link layer, on the other hand, ***needs to pack bits into frames***, ***so that each frame is distinguishable from another***. Our postal system practices a type of framing. The simple act of inserting a letter into an envelope separates one piece of information from another; the envelope serves as the delimiter. In addition, each envelope defines the sender and receiver addresses, which is necessary since the postal system is a many-to-many carrier facility.
- ***Framing*** in the data-link layer ***separates a message from one source to a destination by adding a sender address and a destination address***. The destination address defines where the packet is to go; the sender address helps the recipient acknowledge the receipt.
- Although the whole message could be packed in one frame, that is not normally done. One reason is that a frame can be very large, making flow and error control very inefficient. When a message is carried in one very large frame, even a single-bit error would require the retransmission of the whole frame. When a message is divided into smaller frames, a single-bit error affects only that small frame.

Frame Size

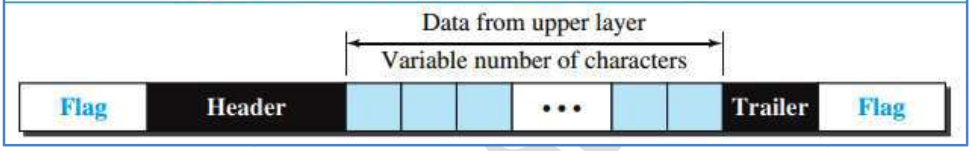
Prof. Parthasarathy P V

- Frames can be of ***fixed*** or ***variable*** size.
- In ***fixed-size framing***, there is ***no need for defining the boundaries of the frames***; the size itself can be used as a delimiter. An example of this type of framing is the ATM WAN, which uses frames of ***fixed size called cells***.
- Our main discussion in this chapter concerns ***variable-size framing***, ***prevalent in local-area networks***. In ***variable-size framing, we need a way to define the end of one frame and the beginning of the next***.
- Historically, ***two approaches*** were used for this purpose:
  1. Character-oriented approach and
  2. Bit-oriented approach.

1. Character-Oriented Framing

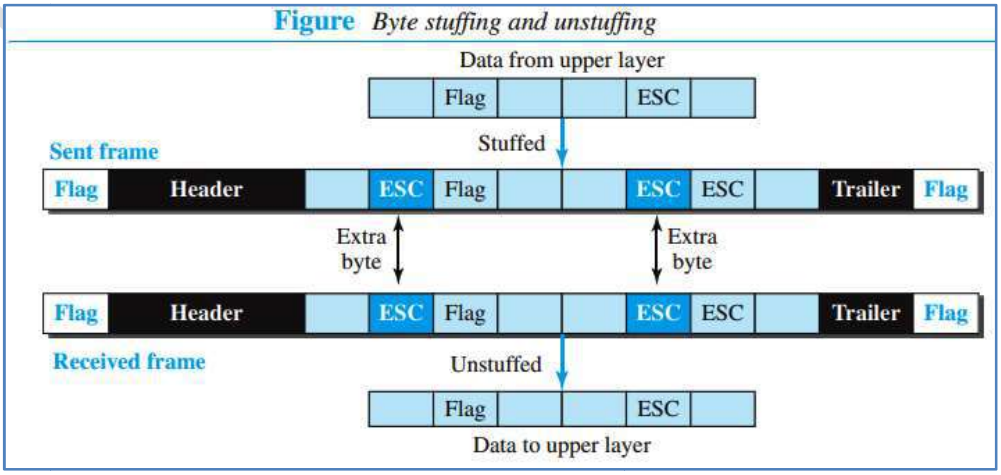
- In character-oriented (or byte-oriented) framing, ***data to be carried are 8-bit characters*** from a coding system such as ASCII. The ***header, which normally carries the source and destination addresses and other control information, and the trailer, which carries error detection redundant bits, are also multiples of 8 bits***.
- To separate one frame from the next, an 8-bit (1-byte) flag is added at the beginning and the end of a frame. The flag, composed of protocol-dependent special characters, signals the start or end of a frame. Figure shows the format of a frame in a character-oriented protocol.

Figure *A frame in a character-oriented protocol*



- ***Character-oriented framing was popular when only text was exchanged by the data-link layers***. The flag could be selected to be any character not used for text communication.
- Now, however, we send other types of information such as graphs, audio, and video; any character used for the flag could also be part of the information. If this happens, the receiver, when it encounters this pattern in the middle of the data, thinks it has reached the end of the frame. ***To fix this problem, a byte-stuffing strategy was added to character-oriented framing.***
- ***In byte stuffing*** (or character stuffing), a special byte is added to the data section of the frame when there is a character with the same pattern as the flag. The data section is stuffed with an extra byte. This byte is usually called the ***escape character (ESC)*** and has a predefined bit pattern. Whenever the receiver encounters the ESC character, it removes it from the data section and treats the next character as data, not as a delimiting flag. Figure shows the situation.

Figure *Byte stuffing and unstuffing*



- Byte stuffing by the escape character allows the presence of the flag in the data section of the frame, ***but it creates another problem***. What happens if the text contains one or more escape characters followed by a byte with the same pattern as the flag? ***The receiver removes the escape***

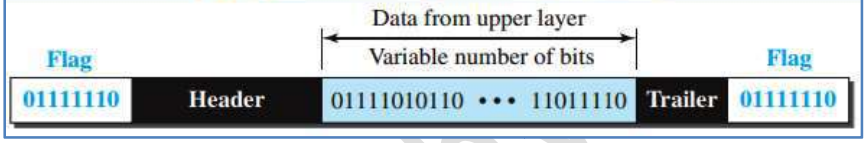
character, but keeps the next byte, which is incorrectly interpreted as the end of the frame. To solve this problem, ***the escape characters that are part of the text must also be marked by another escape character***. In other words, if the escape character is part of the text, an extra one is added to show that the second one is part of the text.

- Character-oriented protocols present another problem in data communications. The ***universal coding systems*** in use today, such as ***Unicode, have 16-bit and 32-bit characters*** that ***conflict with 8-bit characters***.

2. Bit-Oriented Framing

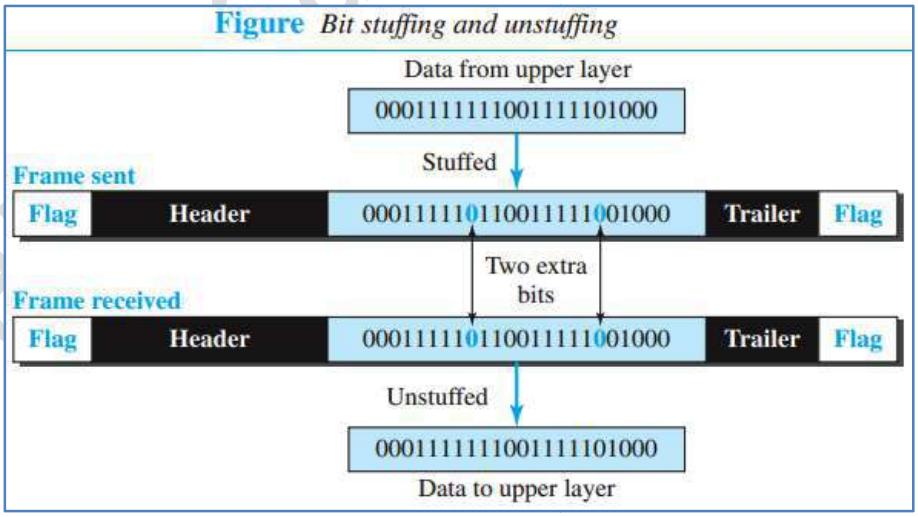
- In ***bit-oriented framing***, the ***data section of a frame is a sequence of bits to be interpreted by the upper layer as text, graphic, audio, video, and so on***. However, in addition to headers (and possible trailers), we still need a delimiter to separate one frame from the other. ***Most protocols use a special 8-bit pattern flag, 01111110, as the delimiter to define the beginning and the end of the frame, as shown in Figure.***

Figure *A frame in a bit-oriented protocol*



- This flag can create the same type of problem we saw in the character-oriented protocols. That is, if the flag pattern appears in the data, we need to somehow inform the receiver that this is not the end of the frame. We do this by stuffing ***1 single bit (instead of 1 byte)*** to prevent the pattern from looking like a flag. The strategy is called ***bit stuffing***.
- ***Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 0111110 for a flag.***
- ***This extra stuffed bit is eventually removed from the data by the receiver***. Note that the extra bit is added after one 0 followed by five 1s regardless of the value of the next bit. This guarantees that the flag field sequence does not inadvertently appear in the frame.
- Figure shows bit stuffing at the sender and bit removal at the receiver. Note that even if we have a 0 after five 1s, we still stuff a 0. The 0 will be removed by the receiver. This means that if the flaglike pattern 01111110 appears in the data, it will change to 011111010 (stuffed) and is not mistaken for a flag by the receiver. The real flag 01111110 is not stuffed by the sender and is recognized by the receiver.

Figure *Bit stuffing and unstuffing*



Flow Control, Error Control

- ***One of the responsibilities of the data-link control sublayer*** is ***flow and error control at the data-link layer***.

Flow Control

- Whenever an entity produces items and another entity consumes them, there should be a balance between production and consumption rates. If the items are produced faster than they can be consumed, the consumer can be overwhelmed and may need to discard some items. If the items

are produced more slowly than they can be consumed, the consumer must wait, and the system becomes less efficient. Flow control is related to the first issue. We need to prevent losing the data items at the consumer site.

- In communication at the data-link layer, we are dealing with four entities: network and data-link layers at the sending node and network and data-link layers at the receiving node. Although we can have a complex relationship with more than one producer and consumer, we ignore the relationships between networks and data-link layers and concentrate on the relationship between two data-link layers, as shown in Figure.

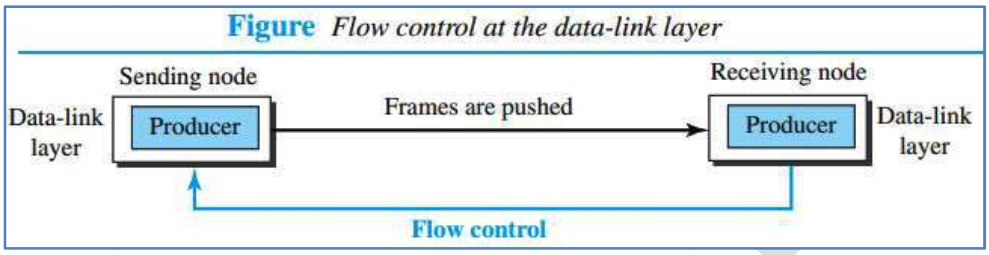


Figure *Flow control at the data-link layer*

- The figure shows that the data-link layer at the sending node tries to push frames toward the data-link layer at the receiving node. If the receiving node cannot process and deliver the packet to its network at the same rate that the frames arrive, it becomes overwhelmed with frames. Flow control in this case can be feedback from the receiving node to the sending node to stop or slow down pushing frames.

Buffers

- Although flow control can be implemented in several ways, one of the ***solutions is normally to use two buffers***; ***one at the sending data-link layer and the other at the receiving data-link layer***. A buffer is a set of memory locations that can hold packets at the sender and receiver. The flow control communication can occur by sending signals from the consumer to the producer. When the buffer of the receiving data-link layer is full, it informs the sending data-link layer to stop pushing frames.

Error Control

- Since ***the underlying technology at the physical layer is not fully reliable, we need to implement error control at the data-link layer to prevent the receiving node from delivering corrupted packets to its network layer***.
- Error control at the data-link layer is normally very simple and implemented using one of the ***following two methods***. In both methods, a CRC is added to the frame header by the sender and checked by the receiver.
  1. In the ***first method, if the frame is corrupted, it is silently discarded***; if it is ***not corrupted, the packet is delivered to the network layer***. This ***method is used mostly in wired LANs such as Ethernet***.
  2. In the ***second method***, if ***the frame is corrupted, it is silently discarded***; if ***it is not corrupted, an acknowledgment is sent (for the purpose of both flow and error control) to the sender***.

Combination of Flow and Error Control

- Flow and error control can be combined. In a ***simple situation, the acknowledgment that is sent for flow control can also be used for error control to tell the sender the packet has arrived uncorrupted. The lack of acknowledgment means that there is a problem in the sent frame***. A frame that carries an acknowledgment is normally called an ACK to distinguish it from the data frame.

Connectionless and Connection Oriented

- A DLC protocol can be either connectionless or connection-oriented.

Connectionless Protocol

- In a connectionless protocol, frames are sent from one node to the next without any relationship between the frames; ***each frame is independent***. Note that the term connectionless here ***does not mean that there is no physical connection (transmission medium) between the nodes; it means that there is no connection between frames.*** The frames are not numbered and there is no sense of ordering.
- **Most of the data-link protocols for LANs are connectionless protocols.**

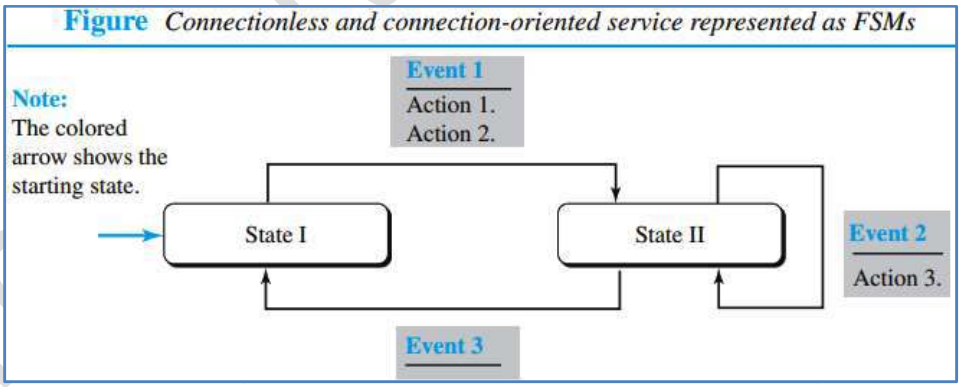
Connection-Oriented Protocol

- In a connection-oriented protocol, a ***logical connection should first be established between the two nodes (setup phase)***. After all frames that are somehow related to each other are transmitted (transfer phase), the logical connection is terminated (teardown phase). In this type of communication, the frames are numbered and sent in order. If they are not received in order, the receiver needs to wait until all frames belonging to the same set are received and then deliver them in order to the network layer.
- ***Connection-oriented protocols are rare in wired LANs, but we can see them in some point-to-point protocols, some wireless LANs, and some WANs.***

Data link layer protocols (P338)

- Traditionally ***four protocols*** have been defined for the data-link layer to deal with flow and error control: (i) ***Simple***, (ii) ***Stop-and-Wait***, (iii) ***Go-Back-N***, and (iv) ***Selective-Repeat***. Although the first two protocols still are used at the data-link layer.
- The behavior of a data-link-layer protocol can be better shown as a ***finite state machine (FSM).*** An FSM is thought of as a machine with a finite number of states. The machine is always in one of the states until an event occurs.
- Each event is associated with ***two reactions***:
  - defining the list (possibly empty) of actions to be performed and
  - determining the next state (which can be the same as the current state).
- One of the states must be defined as the initial state, the state in which the machine starts when it turns on.

Figure *Connectionless and connection-oriented service represented as FSMs*



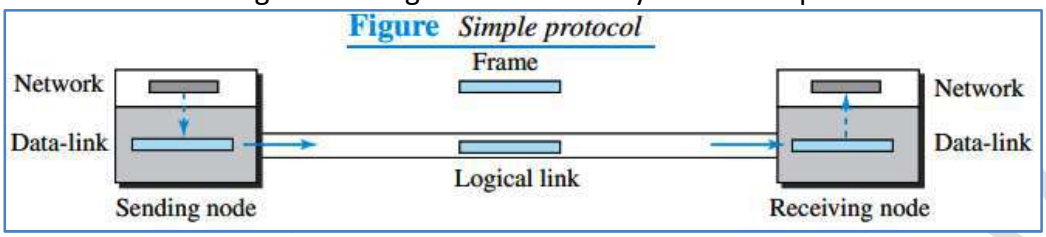
- In Figure, we show an example of a machine using FSM. We have used ***rounded-corner rectangles to show states, colored text to show events, and regular black text to show actions***. A horizontal line is used to separate the event from the actions, although later we replace the horizontal line with a slash. The arrow shows the movement to the next state.
- The figure shows a machine with three states. There are only three possible events and three possible actions. The machine starts in state I. If event 1 occurs, the machine performs actions 1 and 2 and moves to state II. When the machine is in state II, two events may occur. If event 1 occurs, the machine performs action 3 and remains in the same state, state II. If event 3 occurs, the machine performs no action, but move to state I.

Topics covered: ***Simple Protocol - Stop-and-Wait Protocol - Piggybacking***

Simple Protocol

- Our first protocol is a ***simple protocol with neither flow nor error control***. We assume that the receiver can immediately handle any frame it receives. In other words, the receiver can never be overwhelmed with incoming frames. Figure shows the layout for this protocol.

Figure *Simple protocol*

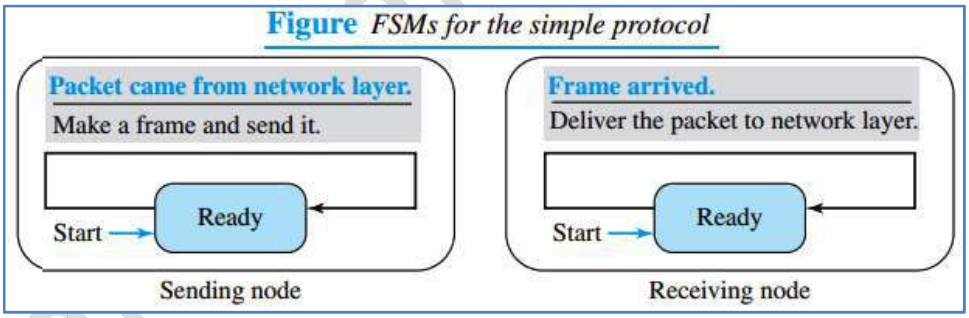


- The data-link layer at the sender gets a packet from its network layer, makes a frame out of it, and sends the frame. The data-link layer at the receiver receives a frame from the link, extracts the packet from the frame, and delivers the packet to its network layer. The data-link layers of the sender and receiver provide transmission services for their network layers.

FSMs (Finite State Machine)

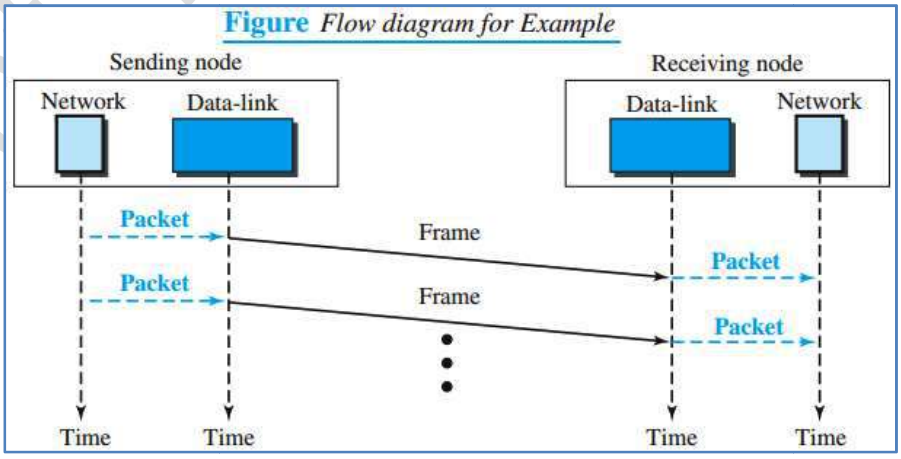
- The sender site should not send a frame until its network layer has a message to send. The receiver site cannot deliver a message to its network layer until a frame arrives. We can show these requirements using two FSMs. Each FSM has only one state, the ready state. The sending machine remains in the ready state until a request comes from the process in the network layer. When this event occurs, the sending machine encapsulates the message in a frame and sends it to the receiving machine. The receiving machine remains in the ready state until a frame arrives from the sending machine. When this event occurs, the receiving machine decapsulates the message out of the frame and delivers it to the process at the network layer. Figure shows the FSMs for the simple protocol.

Figure *FSMs for the simple protocol*



- ***Example***: The following Figure shows an example of communication using this protocol. It is very simple. The sender sends frames one after another without even thinking about the receiver.

Figure *Flow diagram for Example*

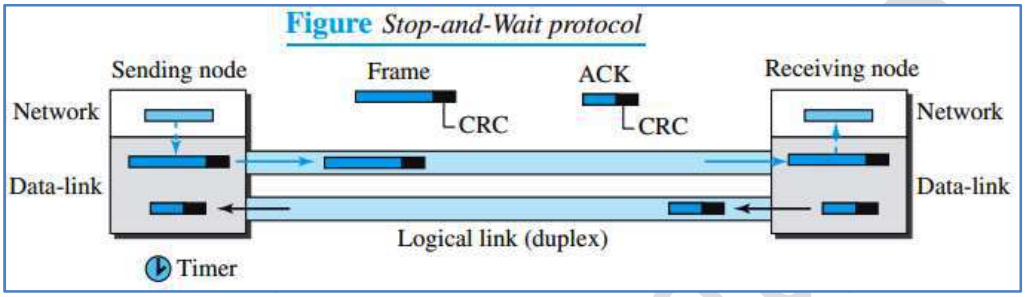


Stop-and-Wait Protocol

- Our second protocol is called the ***Stop-and-Wait protocol***, which uses ***both flow and error control***.

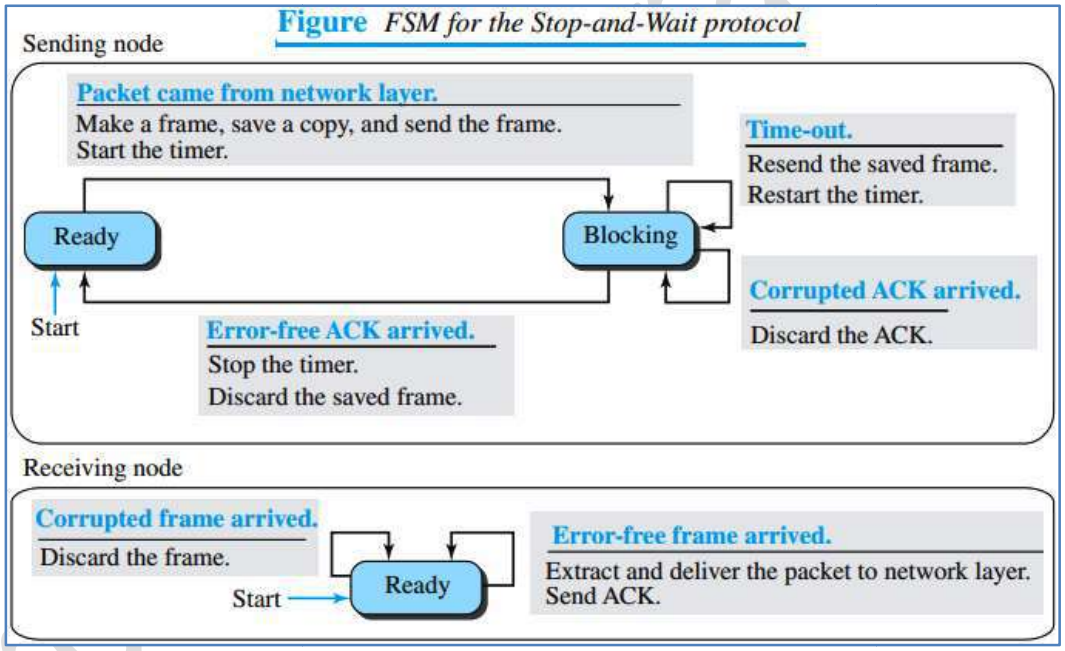
- ***In this protocol, the sender sends one frame at a time and waits for an acknowledgment before sending the next one***.
- To ***detect corrupted frames, we need to add a CRC to each data frame***. When a frame arrives at the receiver site, it is checked. If its CRC is incorrect, the frame is corrupted and silently discarded. The silence of the receiver is a signal for the sender that a frame was either corrupted or lost.
- Every time the sender sends a frame, it starts a timer. If an acknowledgment arrives before the timer expires, the timer is stopped and the sender sends the next frame (if it has one to send). If the timer expires, the sender resends the previous frame, assuming that the frame was either lost or corrupted. ***This means that the sender needs to keep a copy of the frame until its acknowledgment arrives***. When the corresponding acknowledgment arrives, the sender discards the copy and sends the next frame if it is ready. Figure shows the outline for the Stop-and-Wait protocol. Note that only one frame and one acknowledgment can be in the channels at any time.

Figure *Stop-and-Wait protocol*



FSMs (Finite State Machine)

Figure *FSM for the Stop-and-Wait protocol*



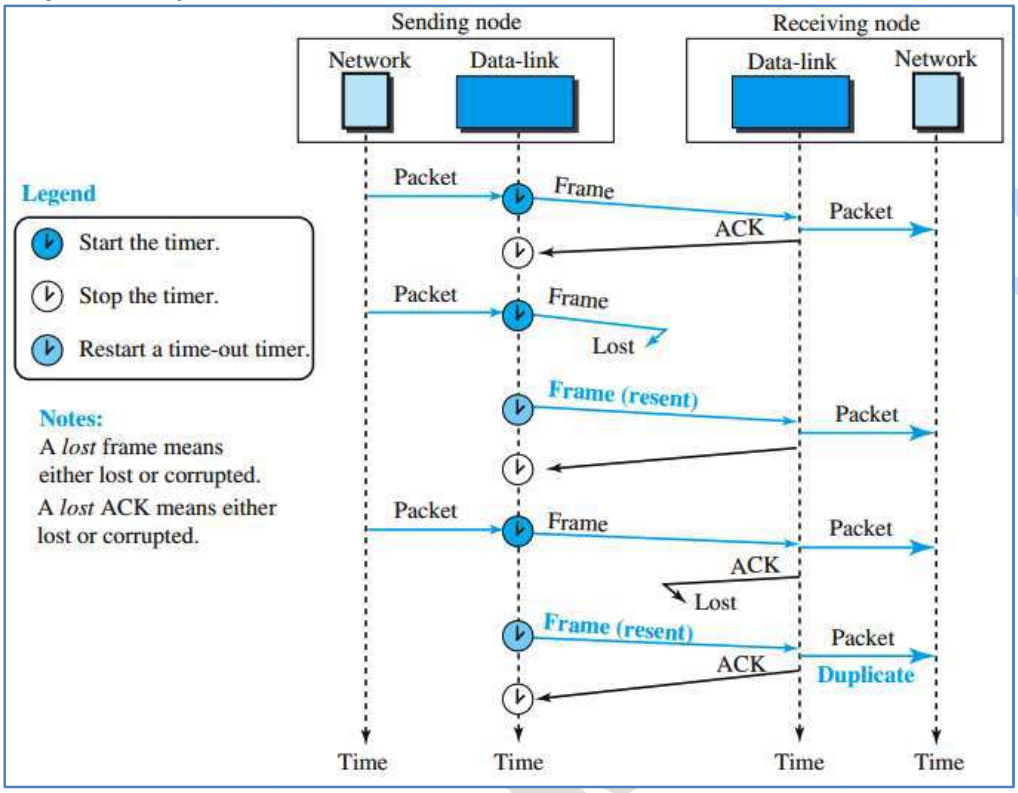
- **Problem Example:** The first frame is sent and acknowledged. The second frame is sent, but lost. After time-out, it is resent. The third frame is sent and acknowledged, but the acknowledgment is lost. The frame is resent. However, there is a problem with this scheme. The network layer at the receiver site receives two copies of the third packet, which is not right. In the next section, we will see how we can correct this problem using sequence numbers and acknowledgment numbers.

FSMs with Sequence and Acknowledgment Numbers

- We saw a problem in Example that needs to be addressed and corrected. Duplicate packets, as much as corrupted packets, need to be avoided. As an example, assume we are ordering some item online. If each packet defines the specification of an item to be ordered, duplicate packets mean ordering an item more than once.
- To correct the problem, we need to add sequence numbers to the data frames and acknowledgment numbers to the ACK frames. However, numbering in this case is very simple.

Sequence numbers are 0, 1, 0, 1, 0, 1, . . . ; the acknowledgment numbers can also be 1, 0, 1, 0, 1, 0, … In other words, the sequence numbers start with 0, the acknowledgment numbers start with 1. An acknowledgment number always defines the sequence number of the next frame to receive.

Flow diagram for Example



Piggybacking

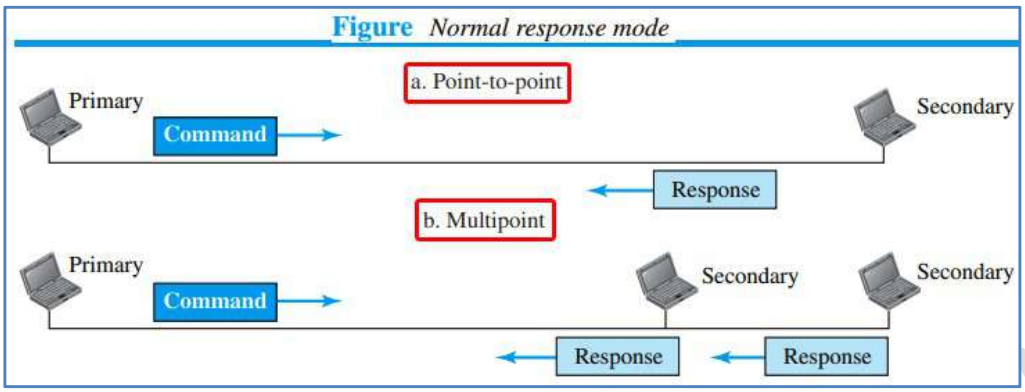
- ***The two protocols we discussed in this section are designed for unidirectional communication, in which data is flowing only in one direction although the acknowledgment may travel in the other direction***.
- ***Protocols have been designed in the past to allow data to flow in both directions***. However, to make the communication more efficient, the data in ***one direction is piggybacked with the acknowledgment in the other direction***.
- In other words, when node A is sending data to node B, Node A also acknowledges the data received from node B. Because ***piggybacking makes communication at the datalink layer more complicated, it is not a common practice***.

High Level Data Link Control (HDLC)

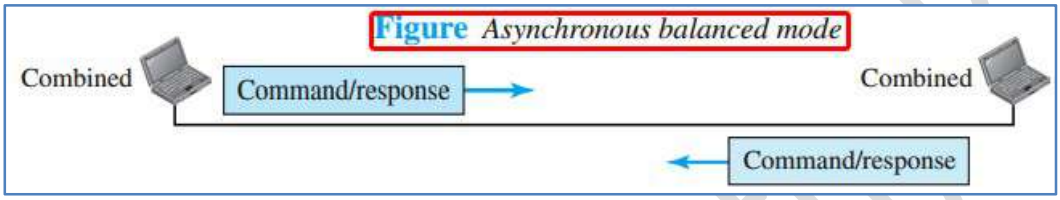
- ***High-level Data Link Control (HDLC) is a bit-oriented protocol for communication over point-to-point and multipoint links***. It ***implements the Stop-and-Wait protocol.*** Although this ***protocol is more a theoretical issue than practical***, most of the concept defined in this ***protocol is the basis*** for other practical protocols such as ***PPP*** or the ***Ethernet protocol*** or in ***wireless LANs.***

Configurations and Transfer Modes

- HDLC provides ***two common transfer modes*** that can be used in different configurations:
  1. Normal Response Mode (NRM) and
  2. Asynchronous Balanced Mode (ABM).
- In ***normal response mode (NRM),*** the station configuration is unbalanced. We have one primary station and multiple secondary stations. A primary station can send commands; a secondary station can only respond. The NRM is used for both point-to-point and multipoint links, as shown in Figure.

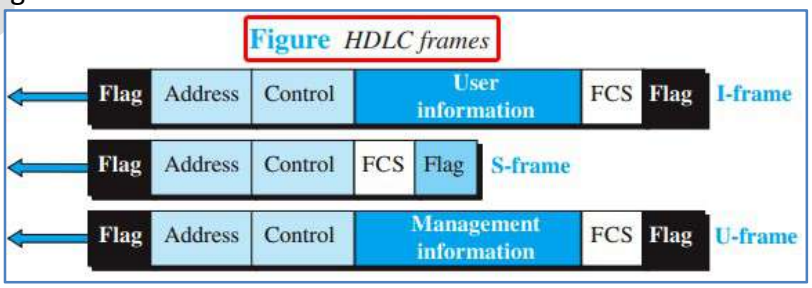


- In ***Asynchronous Balanced Mode (ABM)***, the configuration is balanced. The link is point-to-point, and each station can function as a primary and a secondary (acting as peers), as shown in Figure. This is the common mode today.



Framing

- To provide the flexibility necessary to support all the options possible in the modes and configurations just described, HDLC defines three types of frames:
  - information frames (I-frames)
  - supervisory frames (S-frames), and
  - unnumbered frames (U-frames).
- Each ***type of frame serves as an envelope for the transmission of a different type of message***.
- ***I-frames*** are used to ***data-link user data and control information relating to user data*** (piggybacking).
- ***S-frames*** are ***used only to transport control information***.
- ***U-frames*** are reserved for system management. Information carried by ***U-frames*** is intended for ***managing the link itself***.
- Each frame in HDLC may contain up to six fields, as shown in Figure:
  - a beginning flag field
  - an address field
  - a control field
  - an information field
  - a frame check sequence (FCS) field, and
  - an ending flag field.



- In multiple-frame transmissions, the ending flag of one frame can serve as the beginning flag of the next frame.
- Let us now discuss the fields and their use in different frame types.
  - ***Flag field***. This field contains synchronization pattern 01111110, which identifies both the beginning and the end of a frame.

Prof. Parthasarathy P V

- ***Address field***. This field contains the address of the secondary station. If a primary station created the frame, it contains a to address. If a secondary station creates the frame, it contains a from address. The address field can be one byte or several bytes long, depending on the needs of the network.
- ***Control field***. The control field is one or two bytes used for flow and error control.
- ***Information field***. The information field contains the user's data from the network layer or management information. Its length can vary from one network to another.
- ***FCS field***. The frame check sequence (FCS) is the HDLC error detection field. It can contain either a 2- or 4-byte CRC.

- The ***control field determines the type of frame and defines its functionality***. So let us discuss the format of this field in detail. The format is specific for the type of frame, as shown in Figure.

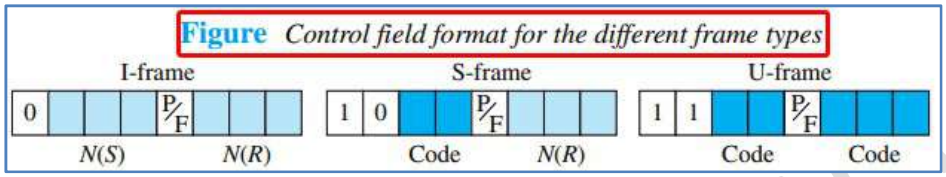
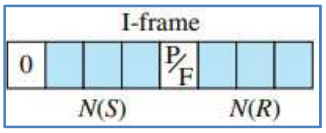


Figure *Control field format for the different frame types*

Control Field for I-Frames

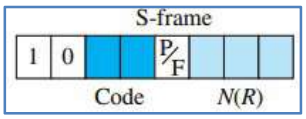
- ***I-frames are designed to carry user data from the network layer***. In addition, they can ***include flow- and error-control information (piggybacking)***. The subfields in the control field are used to define these functions.
- The first bit defines the type. If the first bit of the control field is 0, this means the frame is an I-frame. The next 3 bits, called N(S), define the sequence number of the frame. Note that with 3 bits, we can define a sequence number between 0 and 7. The last 3 bits, called N(R), correspond to the acknowledgment number when piggybacking is used.



- The single bit between N(S) and N(R) is called the P/F bit. The P/F field is a single bit with a dual purpose. It has ***meaning only when it is set (bit = 1) and can mean poll or final***.
- It means ***poll*** when the frame is sent by a primary station to a secondary (when the address field contains the address of the receiver).
- It means ***final*** when the frame is sent by a secondary to a primary (when the address field contains the address of the sender).

Control Field for S-Frames

- ***Supervisory frames are used for flow and error control whenever piggybacking is either impossible or inappropriate***. S-frames do not have information fields. If the first 2 bits of the control field are 10, this means the frame is an S-frame. The last 3 bits, called N(R), correspond to the acknowledgment number (ACK) or negative acknowledgment number (NAK), depending on the type of S-frame. The 2 bits called code are used to define the type of S-frame itself. With 2 bits, we can have ***four types of S-frames***, as described below:



1. ***Receive ready (RR).*** If the value of the code subfield is 00, it is an RR S-frame. This kind of frame acknowledges the receipt of a safe and sound frame or group of frames. In this case, the value of the N(R) field defines the acknowledgment number.
2. ***Receive not ready (RNR).*** If the value of the code subfield is 10, it is an RNR Sframe. This kind of frame is an RR frame with additional functions. It acknowledges the receipt of a frame or group of frames, and it announces that the receiver is busy and cannot receive more frames. It acts as a kind of congestion-control mechanism by asking the sender to slow down. The value of N(R) is the acknowledgment number.
3. ***Reject (REJ).*** If the value of the code subfield is 01, it is an REJ S-frame. This is a NAK frame, but not like the one used for Selective Repeat ARQ. It is a NAK that can be used in Go-Back-

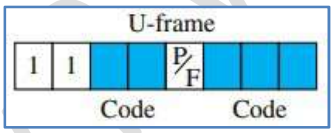
Prof. Parthasarathy V

N ARQ to improve the efficiency of the process by informing the sender, before the sender timer expires, that the last frame is lost or damaged. The value of N(R) is the negative acknowledgment number.

4. ***Selective reject (SREJ).*** If the value of the code subfield is 11, it is an SREJ Sframe. This is a NAK frame used in Selective Repeat ARQ. Note that the HDLC Protocol uses the term selective reject instead of selective repeat. The value of N(R) is the negative acknowledgment number.

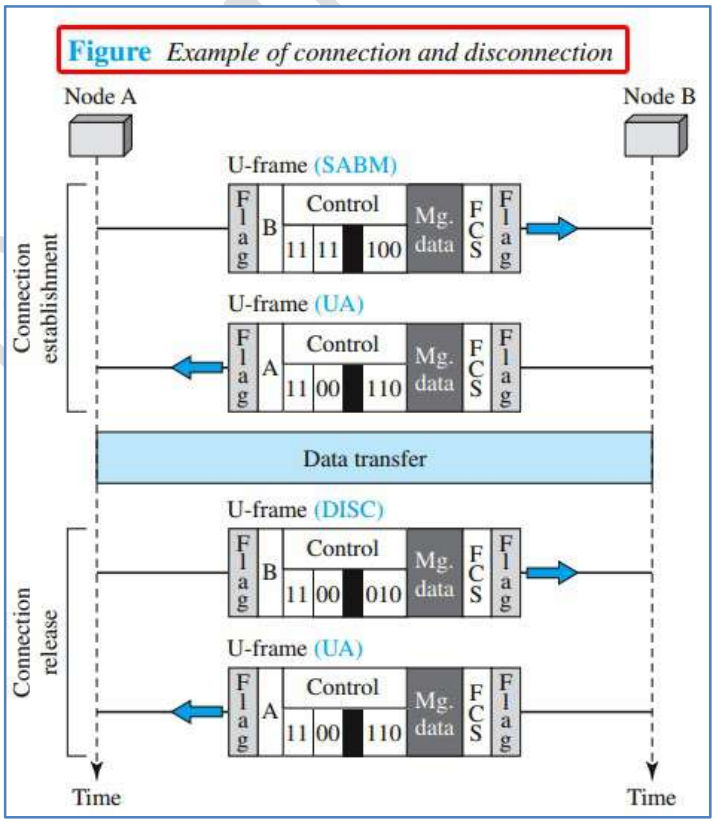
Control Field for U-Frames

- Unnumbered frames are used to exchange session management and control information between connected devices.
- Unlike S-frames, U-frames contain an information field, but one used for system management information, not user data.
- As with S-frames, however, much of the information carried by U-frames is contained in codes included in the control field.
- U-frame codes are divided into two sections:
  - a 2-bit prefix before the P/F bit and
  - a 3-bit suffix after the P/F bit. Together, these two segments (5 bits) can be used to create up to 32 different types of U-frames.



Example 11.5

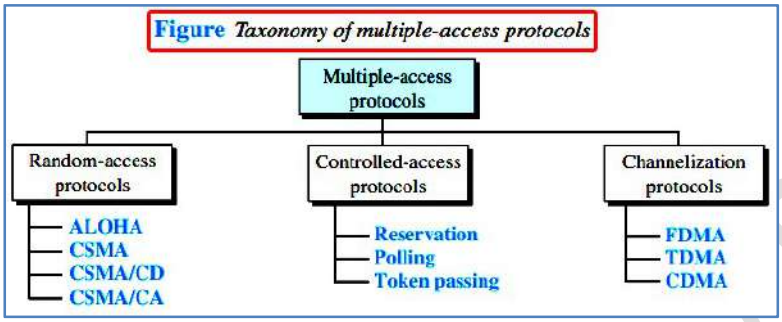
- Figure shows how U-frames can be used for connection establishment and connection release. Node A asks for a connection with a set asynchronous balanced mode (SABM) frame; node B gives a positive response with an unnumbered acknowledgment (UA) frame. After these two exchanges, data can be transferred between the two nodes (not shown in the figure). After data transfer, node A sends a DISC (disconnect) frame to release the connection; it is confirmed by node B responding with a UA (unnumbered acknowledgment).



Chapter 12: Media Access Control (Pg. 325[364] to 344)

Random Access, Controlled Access. Check Sum and Point to Point Protocol.

- When nodes or stations are connected and use a common link, called a multipoint or broadcast link, we need a multiple-access protocol to coordinate access to the link. Many protocols have been devised to handle access to a shared link. All of these protocols belong to a sublayer in the data-link layer called media access control (MAC). We categorize them into three groups, as shown in Figure.



Random Access

- In ***random-access*** or ***contention methods***, no station is superior to another station and none is assigned control over another.
- Each station can transmit when it desires on the condition that it follows the ***predefined procedure***, including ***testing the state of the medium***.
- Two features
  1. First, there is no scheduled time for a station to transmit. Transmission is random among the stations. That is why these methods are called ***random access***.
  2. Second, no rules specify which station should send next. Stations compete with one another to access the medium. That is why these methods are also called ***contention methods***.
- In a random-access method, each station has the right to the medium without being controlled by any other station. However, if more than one station tries to send, there is an access conflict—collision—and the frames will be either destroyed or modified.
- To avoid access conflict or to resolve it when it happens, each station follows a procedure that answers the following questions:
  1. When can the station access the medium?
  2. What can the station do if the medium is busy?
  3. How can the station determine the success or failure of the transmission?
  4. What can the station do if there is an access conflict?
- The random-access methods have evolved from a very interesting protocol known as ***ALOHA***, which used a very simple procedure called ***multiple access (MA).***
- The method was improved with the addition of a procedure that forces the station to sense the medium before transmitting. This was called ***carrier sense multiple access (CSMA).*** This method later evolved into two parallel methods:
- ***Carrier sense multiple access with collision detection (CSMA/CD),*** which tells the station what to do when a collision is detected, and ***carrier sense multiple access with collision avoidance (CSMA/CA)***, which tries to avoid the collision.

Topics Covered: ALOHA – CSMA - CSMA/CD - CSMA/CA

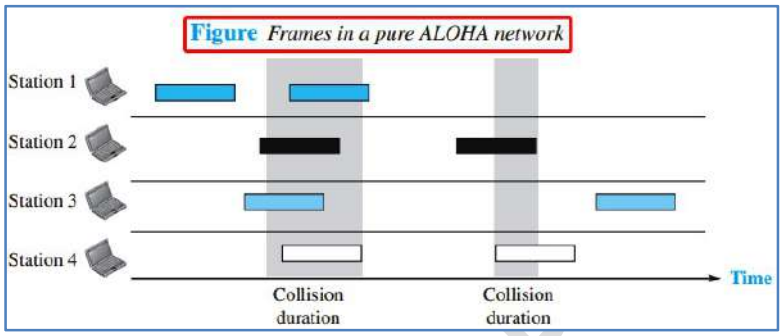
Prof Parthasarathy P V

ALOHA

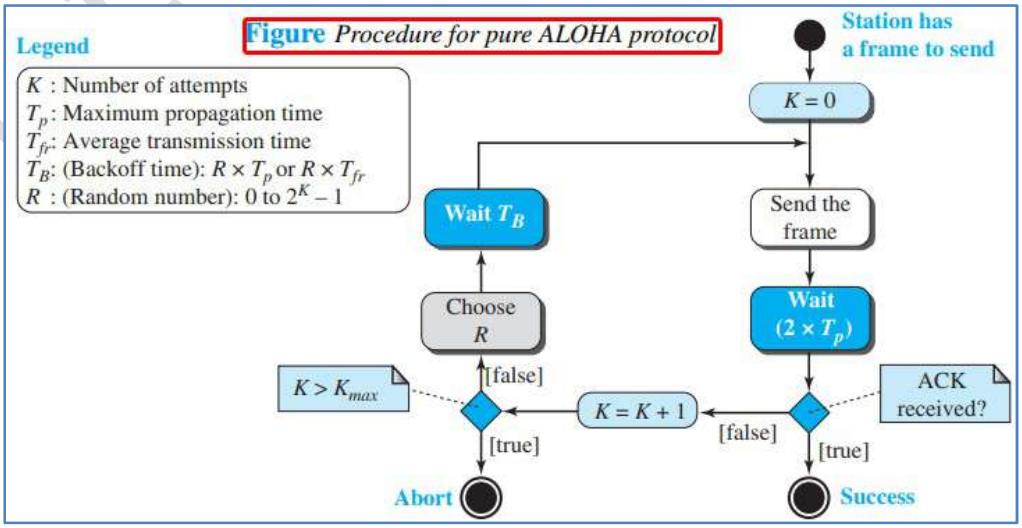
- ***ALOHA***, the earliest random access method, was developed at the University of Hawaii in early 1970. It was ***designed for a radio (wireless) LAN***, but it can be ***used on any shared medium***.
- Two types - ***Pure ALOHA*** and ***Slotted ALOHA***.

Pure ALOHA

- The original ALOHA protocol is called pure ALOHA. This is a simple but elegant protocol. The idea is that each station sends a frame whenever it has a frame to send (multiple access). However, since there is only one channel to share, there is the possibility of collision between frames from different stations. Figure shows an example of frame collisions in pure ALOHA.



- There are four stations (unrealistic assumption) that contend with one another for access to the shared channel. The figure shows that each station sends two frames; there are a total of eight frames on the shared medium. Some of these frames collide because multiple frames are in contention for the shared channel.
- The above figure shows that only two frames survive: one frame from station 1 and one frame from station 3.
- We need to mention that even if one bit of a frame coexists on the channel with one bit from another frame, there is a collision and both will be destroyed. It is obvious that we need to resend the frames that have been destroyed during transmission.
- ***The pure ALOHA protocol relies on acknowledgments from the receiver***. When a station sends a frame, it expects the receiver to send an acknowledgment. If the acknowledgment does not arrive after a time-out period, the station assumes that the frame (or the acknowledgment) has been destroyed and resends the frame.
- ***A collision involves two or more stations. If all these stations try to resend their frames after the time-out, the frames will collide again***. Pure ALOHA dictates that when the ***time-out period passes, each station waits a random amount of time before resending its frame***. The randomness will help avoid more collisions. We call this time the ***backoff time T_B.***
- ***Pure ALOHA has a second method to prevent congesting the channel with retransmitted frames***. After a maximum number of retransmission attempts K_{max}, a station must give up and try later. The ***following Figure shows the procedure for pure ALOHA based on the above strategy***.



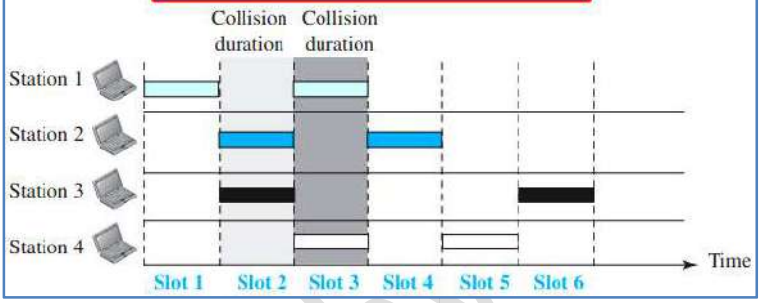
- The time-out period = 2 x maximum possible round-trip propagation delay ***T_p.***

- The backoff time $T_B = K$ (the number of attempted unsuccessful transmissions).
- The formula for T_B depends on the implementation.
- One common formula is the ***binary exponential backoff***.
- In this method, for each retransmission, a multiplier $R = 0$ to $2^K - 1$ is randomly chosen and multiplied by T_p (maximum propagation time) or T_{fr} (the average time required to send out a frame) to find T_B. Note that in this procedure, the range of the random numbers increases after each collision. The value of K_{max} is usually chosen as 15.

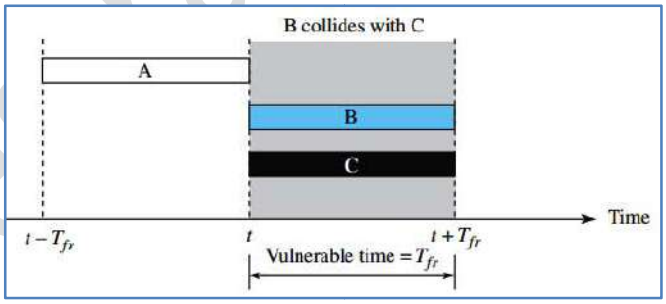
Slotted ALOHA

- Pure ALOHA has a vulnerable time of $2 \times T_{fr}$. This is so because there is no rule that defines when the station can send. A station may send soon after another station has started or just before another station has finished. Slotted ALOHA was invented to improve the efficiency of pure ALOHA.

Figure *Frames in a slotted ALOHA network*



- In **slotted ALOHA we divide the time into slots of T_{fr} seconds** and force the station to send only at the beginning of the time slot. Figure shows an example of frame collisions in slotted ALOHA.
- ***Because a station is allowed to send only at the beginning of the synchronized time slot, if a station misses this moment, it must wait until the beginning of the next time slot***. This means that the station which started at the beginning of this slot has already finished sending its frame.
- Of course, there is still the possibility of collision if two stations try to send at the beginning of the same time slot. However, the vulnerable time is now reduced to one-half, equal to T_{fr}. Figure shows the situation.



Throughput

- The average number of successful transmissions for slotted ALOHA (S) = $G \times e^{-G}$.
- The maximum throughput Smax is 0.368, when G = 1.

Example

A slotted ALOHA network transmits 200-bit frames using a shared channel with a 200-kbps bandwidth. Find the throughput if the system (all stations together) produces

a. 1000 frames per second.
b. 500 frames per second.
c. 250 frames per second.

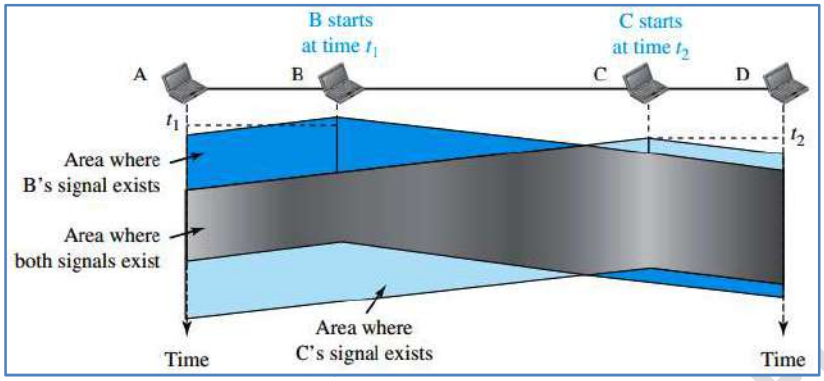
Solution

This situation is similar to the previous exercise except that the network is using slotted ALOHA instead of pure ALOHA. The frame transmission time is 200/200 kbps or 1 ms.

a. In this case G is 1. So $S = G \times e^{-G} = 0.368$ (36.8 percent). This means that the throughput is $1000 \times 0.0368 = 368$ frames. Only 368 out of 1000 frames will probably survive. Note that this is the maximum throughput case, percentagewise.
b. Here G is 1/2. In this case $S = G \times e^{-G} = 0.303$ (30.3 percent). This means that the throughput is $500 \times 0.0303 = 151$. Only 151 frames out of 500 will probably survive.
c. Now G is 1/4. In this case $S = G \times e^{-G} = 0.195$ (19.5 percent). This means that the throughput is $250 \times 0.195 = 49$. Only 49 frames out of 250 will probably survive.

CSMA (Carrier sense multiple access)

- To minimize the chance of collision and increase the performance, the CSMA method was developed. The chance of collision can be reduced if a station senses the medium before trying to use it. ***Carrier sense multiple access (CSMA) requires that each station first listen to the medium (or check the state of the medium) before sending***.
- CSMA can reduce the possibility of collision, but it cannot eliminate it. The reason for this is shown in Figure, a ***space and time model of a CSMA network***. Stations are connected to a shared channel (usually a dedicated medium).



- The possibility of collision still exists because of propagation delay; when a station sends a frame, it still takes time (although very short) for the first bit to reach every station and for every station to sense it. In other words, a station may sense the medium and find it idle, only because the first bit sent by another station has not yet been received.
- At time t_1, station B senses the medium and finds it idle, so it sends a frame. At time t_2 ($t_2 > t_1$), station C senses the medium and finds it idle because, at this time, the first bits from station B have not reached station C. Station C also sends a frame. The two signals collide and both frames are destroyed.

Vulnerable Time

- ***The vulnerable time for CSMA is the propagation time T_p. This is the time needed for a signal to propagate from one end of the medium to the other.***
- When a station sends a frame and any other station tries to send a frame during this time, a collision will result. But if the first bit of the frame reaches the end of the medium, every station will already have heard the bit and will refrain from sending.
- Figure shows the worst case.
- The leftmost station, A, sends a frame at time t_1, which reaches the rightmost station, D, at time $t_1 + T_p$. The gray area shows the vulnerable area in time and space.

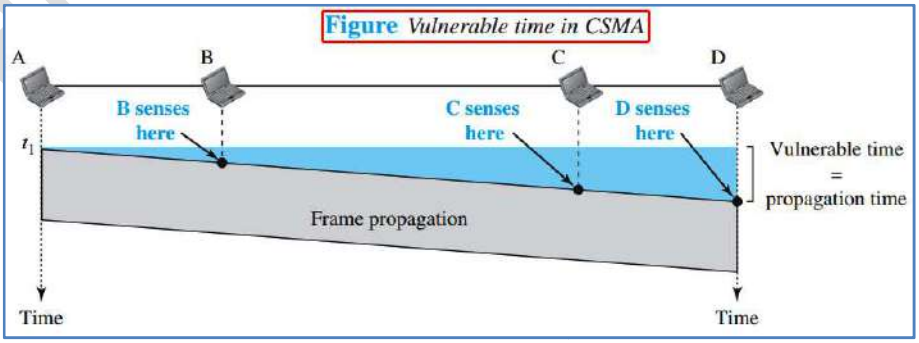


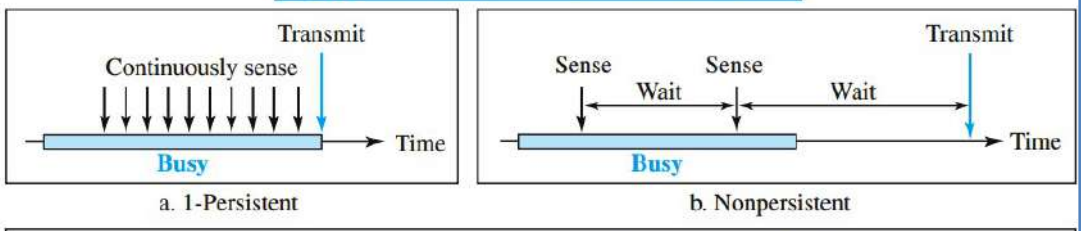
Figure *Vulnerable time in CSMA*

Persistence Methods

- What should a station do if the ***channel is busy***? What should a station do if the ***channel is idle***?
- ***Three methods*** have been devised to answer these questions:
  1. 1-persistent method
  2. nonpersistent method, and
  3. p-persistent method.
- Figure shows the behavior of three persistence methods when a station finds a channel busy.

Prof. Parthasarathy P V

Figure *Behavior of three persistence methods*

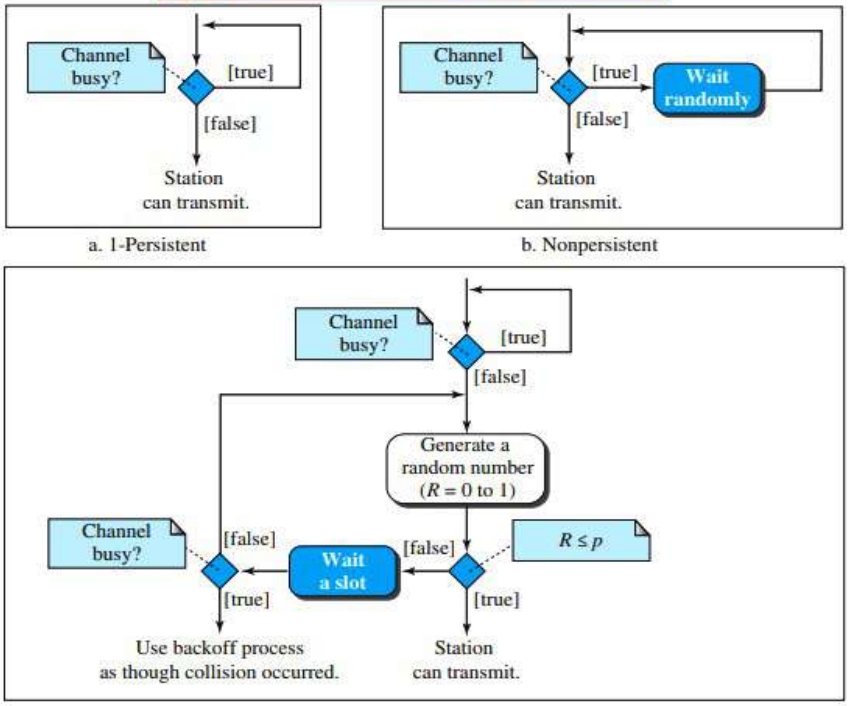


a. 1-Persistent b. Nonpersistent

c. *p*-Persistent

- The following figure shows the flow diagrams for these methods.

Figure *Flow diagram for three persistence methods*



a. 1-Persistent b. Nonpersistent

c. *p*-Persistent

1-Persistent

- The ***1-persistent method is simple and straightforward***. In this method, after the station finds the line idle, it sends its frame immediately (with probability 1).
- ***This method has the highest chance of collision because two or more stations may find the line idle and send their frames immediately***.

Non-persistent

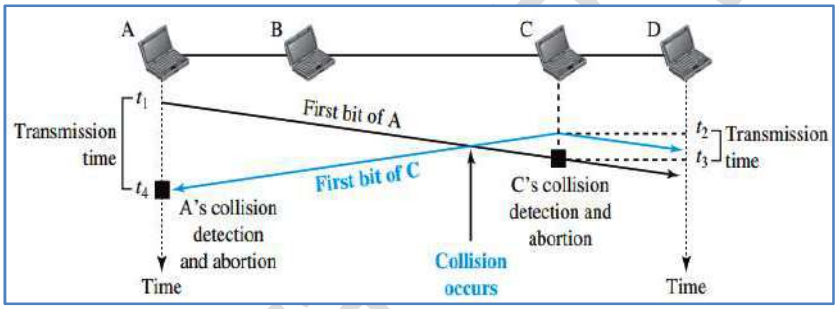
- In the nonpersistent method, a station that has a frame to send senses the line. If the line is idle, it sends immediately. If the line is not idle, it waits a random amount of time and then senses the line again.
- The ***nonpersistent approach reduces the chance of collision*** because it is unlikely that two or more stations will wait the same amount of time and retry to send simultaneously.
- However, ***this method reduces the efficiency of the network*** because the medium remains idle when there may be stations with frames to send.

p-Persistent

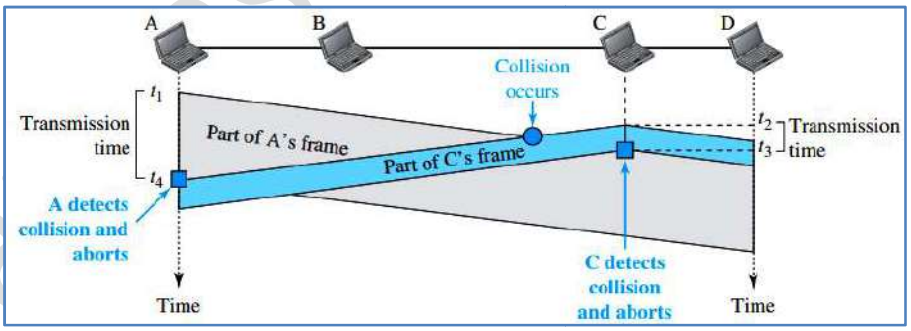
- The ***p-persistent method is used if the channel has time slots with a slot duration equal to or greater than the maximum propagation time.*** The ***p-persistent approach combines the advantages of the other two strategies***.
- ***It reduces the chance of collision and improves efficiency***. In this method, after the station finds the line idle it follows these steps:
  1. With probability p, the station sends its frame.
  2. With probability q = 1 − p, the station waits for the beginning of the next time slot and checks the line again.
     a. If the line is idle, it goes to step 1.
     b. If the line is busy, it acts as though a collision has occurred and uses the backoff procedure.

CSMA/CD

- The CSMA method does not specify the procedure following a collision. Carrier sense multiple access with collision detection (CSMA/CD) augments the algorithm to handle the collision.
- In this method, a ***station monitors the medium after it sends a frame to see if the transmission was successful***. If so, the station is finished. If, however***, there is a collision, the frame is sent again***.
- To better understand CSMA/CD, let us look at the first bits transmitted by the two stations involved in the collision. Although each station continues to send bits in the frame until it detects the collision, we show what happens as the first bits collide. In Figure, stations A and C are involved in the collision.
- At time t_1, station A has executed its persistence procedure and starts sending the bits of its frame. At time t_2, station C has not yet sensed the first bit sent by A.
- Station C executes its persistence procedure and starts sending the bits in its frame, which propagate both to the left and to the right.
- The collision occurs sometime after time t_2. Station C detects a collision at time t_3 when it receives the first bit of A's frame.



- Station C immediately (or after a short time, but we assume immediately) aborts transmission. Station A detects collision at time t_4 when it receives the first bit of C's frame; it also immediately aborts transmission. Looking at the figure, we see that A transmits for the duration $t_4 - t_1$; C transmits for the duration $t_3 - t_2$.
- Now that we know the time durations for the two transmissions, we can show a more complete graph in Figure.



Minimum Frame Size

- For CSMA/CD to work, we ***need a restriction on the frame size***. Before sending the last bit of the frame, the sending station must detect a collision, if any, and abort the transmission.
- Therefore, the frame transmission time T_{fr} must be at least two times the maximum propagation time T_p. To understand the reason, let us think about the worst-case scenario. If the two stations involved in a collision are the maximum distance apart, the signal from the first takes time T_p to reach the second, and the effect of the collision takes another time T_p to reach the first. So the requirement is that the first station must still be transmitting after $2T_p$.

Example

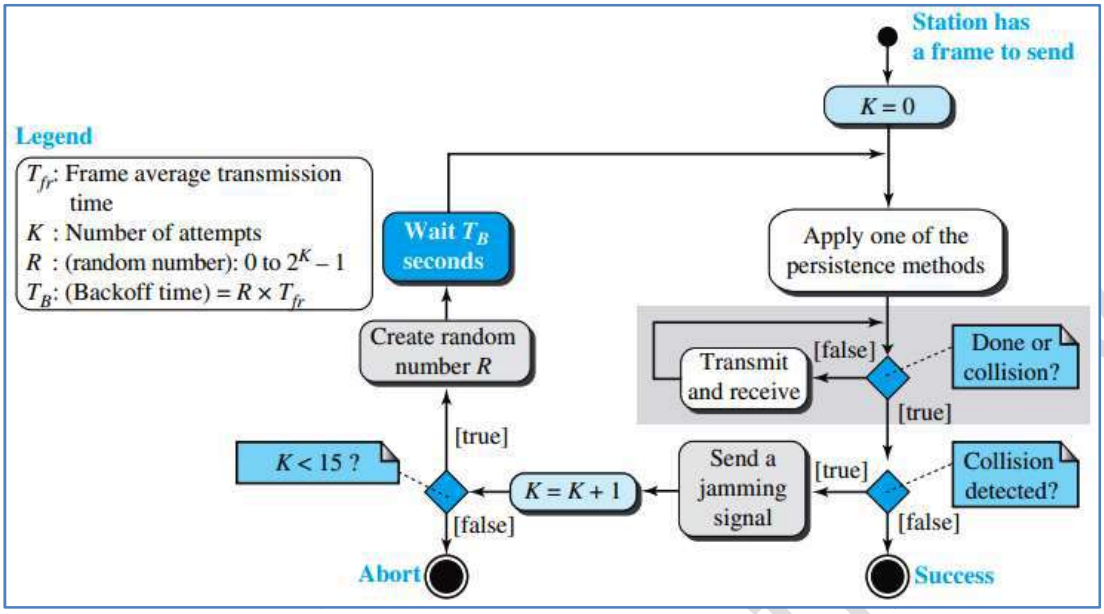
A network using CSMA/CD has a bandwidth of 10 Mbps. If the maximum propagation time (including the delays in the devices and ignoring the time needed to send a jamming signal, as we see later) is 25.6 μs, what is the minimum size of the frame?

Solution

The minimum frame transmission time is $T_{fr} = 2 \times T_p = 51.2$ μs. This means, in the worst case, a station needs to transmit for a period of 51.2 μs to detect the collision. The minimum size of the frame is 10 Mbps × 51.2 μs = 512 bits or 64 bytes. This is actually the minimum size of the frame for Standard Ethernet.

Procedure

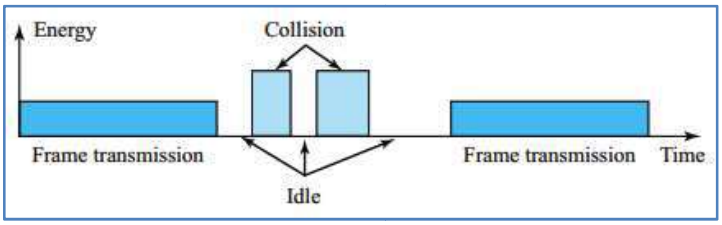
- Now let us look at the flow diagram for CSMA/CD in Figure. It is similar to the one for the ALOHA protocol, but there are differences.



- The ***first difference is the addition of the persistence process***. We need to sense the channel before we start sending the frame by using one of the persistence processes (nonpersistent, 1-persistent, or p-persistent). The corresponding box can be replaced by one of the persistence processes shown in Figure.
- The ***second difference is the frame transmission***. In ALOHA, we first transmit the entire frame and then wait for an acknowledgment. In CSMA/CD, transmission and collision detection are continuous processes.
- The ***third difference is the sending of a short jamming signal to make sure that all other stations become aware of the collision***.

Energy Level

- We can say that the ***level of energy in a channel can have three values***: ***zero, normal, and abnormal***.
- At the ***zero level, the channel is idle***.
- At the ***normal level, a station has successfully captured the channel and is sending its frame***.
- At the ***abnormal level, there is a collision and the level of the energy is twice the normal level***.



- A station that has a frame to send or is sending a frame needs to monitor the energy level to determine if the channel is idle, busy, or in collision mode. Figure shows the situation.

Throughput

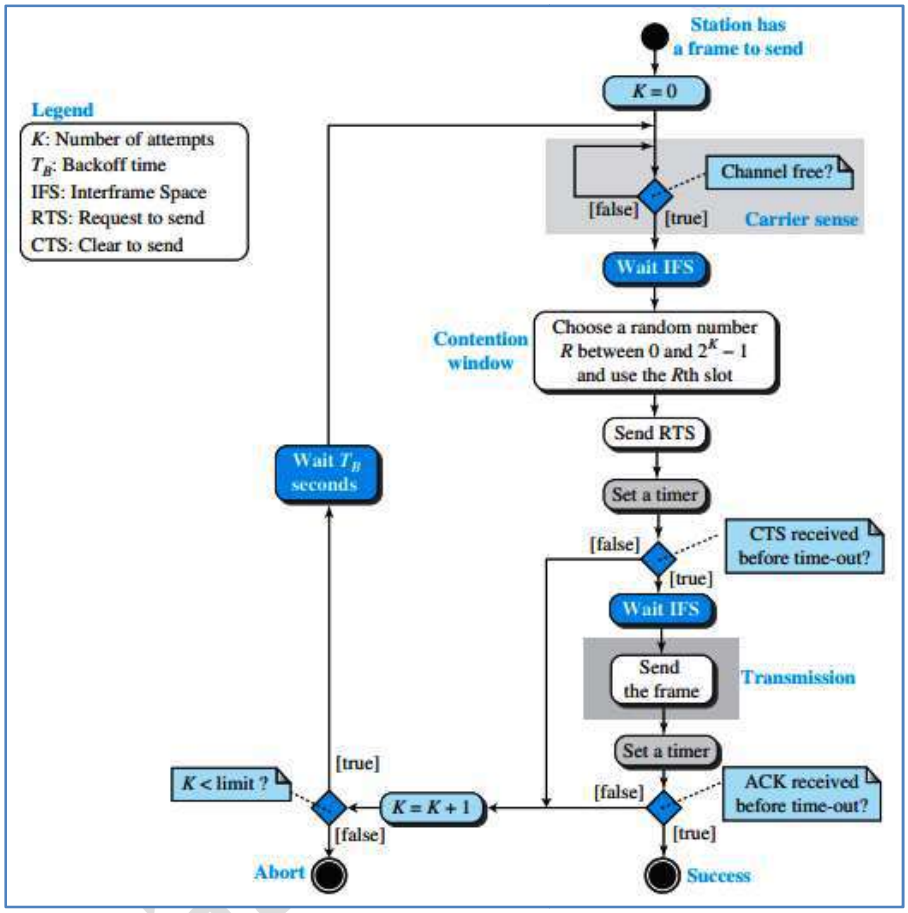
- The throughput ***of CSMA/CD is greater than that of pure or slotted ALOHA***. The maximum throughput occurs at a different value of G and is based on the persistence method and the value of p in the p-persistent approach.
- For the **1-persistent method, the maximum throughput = 50% when G = 1**.
- For the **nonpersistent method, the maximum throughput = up to 90% when G is between 3 and 8**.

Traditional Ethernet

- One of the LAN protocols that used CSMA/CD is the traditional Ethernet with the data rate of 10 Mbps. ***The traditional Ethernet was a broadcast LAN that used the 1-persistence method to control access to the common media***.

CSMA/CA

- ***Carrier sense multiple access with collision avoidance (CSMA/CA) was invented for wireless networks***.
- ***Collisions are avoided through the use of CSMA/CA's three strategies***:
  1. the interframe space
  2. the contention window, and
  3. acknowledgments, as shown in Figure.

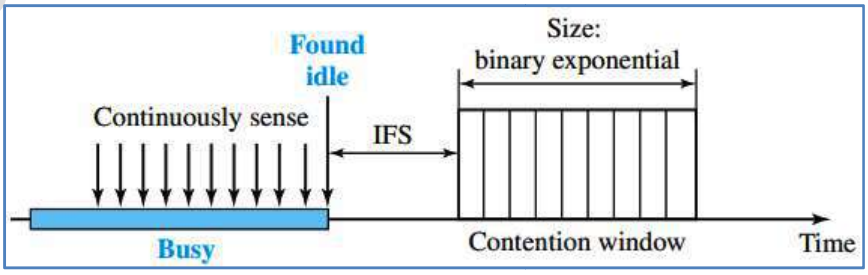


Interframe Space (IFS)

- First, collisions are avoided by deferring transmission even if the channel is found idle.
- When an idle channel is found, the station does not send immediately. It waits for a period of time called the ***interframe space or IFS***.
- The IFS time allows the front of the transmitted signal by the distant station to reach this station.
- After waiting an IFS time, if the channel is still idle, the station can send, but it still needs to wait a time equal to the contention window.
- The IFS variable can also be used to prioritize stations or frame types. For example, a station that is assigned a shorter IFS has a higher priority.

Contention Window

- The ***contention window is an amount of time divided into slots***. A station that is ready to send chooses a random number of slots as its wait time. The number of slots in the window changes according to the binary exponential backoff strategy.



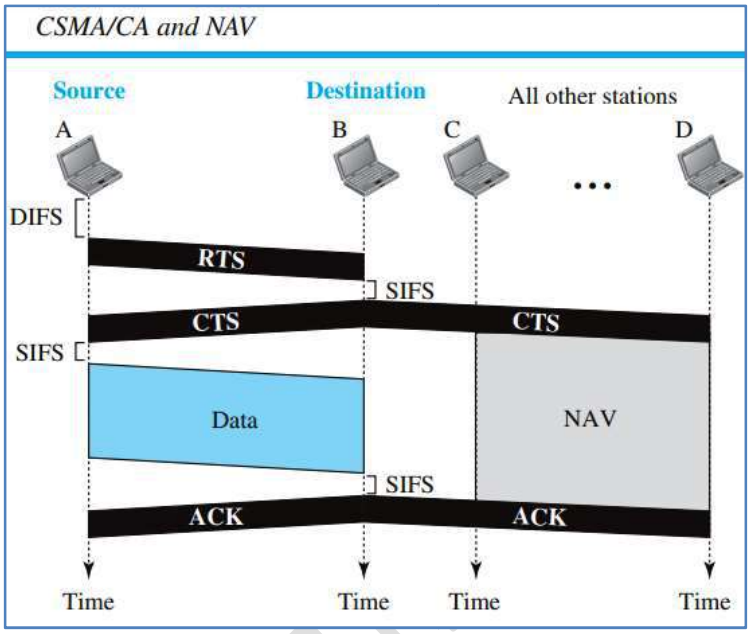
- This means that it is set to one slot the first time and then doubles each time the station cannot detect an idle channel after the IFS time. This is very similar to the ***p-persistent*** method except that a random outcome defines the number of slots taken by the waiting station.

Acknowledgment

- With all these precautions, there ***still may be a collision resulting in destroyed data***. In addition, the data may be corrupted during the transmission. The positive acknowledgment and the time-out timer can help guarantee that the receiver has received the frame.

Frame Exchange Time Line

- Figure shows the exchange of data and control frames in time.



1. Before sending a frame, the source station senses the medium by checking the energy level at the carrier frequency.
   - The ***channel uses a persistence strategy with backoff until the channel is idle***.
   - After the station is found to be idle, the station ***waits for a period of time*** called the ***DCF interframe space (DIFS)***; then the station sends a control frame called the request to send (RTS).
2. After receiving the RTS and waiting a period of time called the short interframe space (SIFS), the destination station sends a control frame, called the clear to send (CTS), to the source station. This control frame indicates that the destination station is ready to receive data.
3. The source station sends data after waiting an amount of time equal to SIFS.
4. The destination station, after waiting an amount of time equal to SIFS, sends an acknowledgment to show that the frame has been received.

Network Allocation Vector (NAV)

How is the collision avoidance aspect of this protocol accomplished? The key - a feature called NAV.

- When a ***station sends an RTS frame, it includes the duration of time that it needs to occupy the channel***. The stations that are affected by this transmission ***create a timer called a network allocation vector (NAV)*** that shows how much time must pass before these stations are allowed to check the channel for idleness.
- Each time a station accesses the system and sends an RTS frame, other stations start their NAV. In other words, each station, before sensing the physical medium to see if it is idle, first checks its NAV to see if it has expired.

Collision During Handshaking

What happens if there is a collision during the time when RTS or CTS control frames are in transition, often called the handshaking period?

- Two or more stations may try to send RTS frames at the same time. These control frames may collide. However, because there is no mechanism for collision detection, the sender assumes there has been a collision if it has not received a CTS frame from the receiver. ***The backoff strategy is employed, and the sender tries again.***

Hidden-Station Problem

- The solution to the hidden station problem is the use of the handshake frames (RTS and CTS). The above figure shows that the RTS message from B reaches A, but not C.
- However, because both B and C are within the range of A, the CTS message, which contains the duration of data transmission from B to A, reaches C. Station C knows that some hidden station is using the channel and refrains from transmitting until that duration is over.

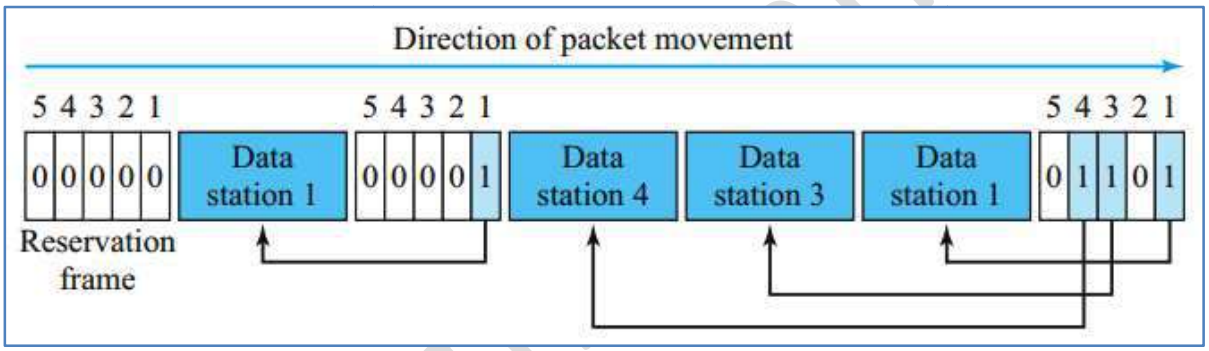
Prof. Parthasarathy P V

Controlled Access

- ***In controlled access, the stations consult one another to find which station has the right to send***.
- A station cannot send unless it has been authorized by other stations. We discuss ***three controlled-access methods***.
  1. **Reservation**
  2. **Polling**
  3. **Token Passing**

1. Reservation

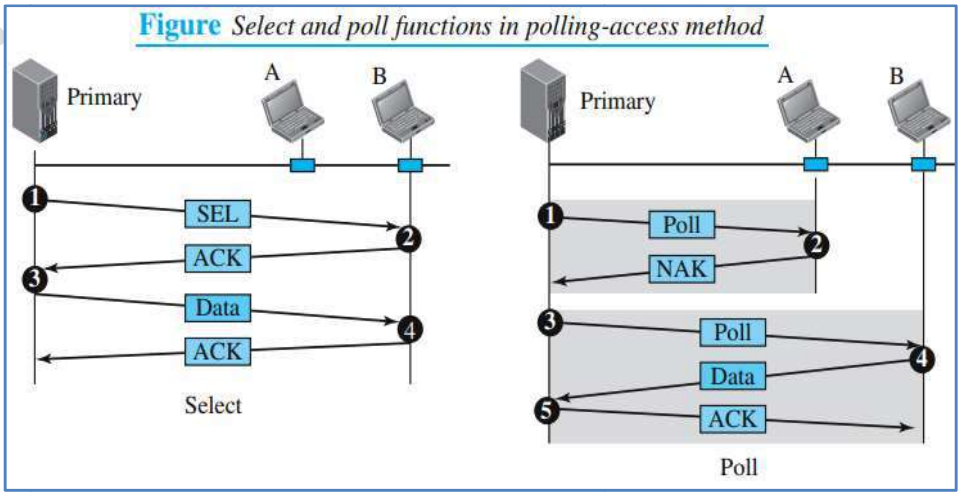
- In the reservation method, a station needs to make a reservation before sending data. Time is divided into intervals. In each interval, a reservation frame precedes the data frames sent in that interval.
- If there are N stations in the system, there are exactly N reservation minislots in the reservation frame. Each minislot belongs to a station. When a station needs to send a data frame, it makes a reservation in its own minislot. The stations that have made reservations can send their data frames after the reservation frame.
- Figure shows a situation with five stations and a five-minislot reservation frame. In the first interval, only stations 1, 3, and 4 have made reservations. In the second interval, only station 1 has made a reservation.



2. Polling

- ***Polling works with topologies*** in which one device is designated as a ***primary station*** and the other devices are ***secondary stations***.
- All data exchanges must be made through the primary device even when the ultimate destination is a secondary device.
- The primary device controls the link; the secondary devices follow its instructions.
- It is up to the primary device to determine which device is allowed to use the channel at a given time.
- The primary device, therefore, is always the initiator of a session (see Figure). ***This method uses poll*** and ***select functions to prevent collisions***.

Figure *Select and poll functions in polling-access method*



- However, the drawback is if the primary station fails, the system goes down.

Select

- The ***select function is used whenever the primary device has something to send***.
- Remember that the primary controls the link. If the primary is neither sending nor receiving data, it knows the link is available. If it has something to send, the primary device sends it.
- What it does not know, however, is whether the target device is prepared to receive. So the primary must alert the secondary to the upcoming transmission and wait for an acknowledgment of the secondary's ready status.
- Before sending data, the primary creates and transmits a select (SEL) frame, one field of which includes the address of the intended secondary.

Poll

- The ***poll function is used by the primary device to ask for transmissions from the secondary devices***. When the primary is ready to receive data, it must ask (poll) each device in turn if it has anything to send.
- When the first secondary is approached, it responds either with a NAK frame if it has nothing to send or with data (in the form of a data frame) if it does.
    1. If the ***response is negative (a NAK frame),*** then the primary polls the next secondary in the same manner until it finds one with data to send.
    2. When ***the response is positive (a data frame),*** the primary reads the frame and returns an acknowledgment (ACK frame), verifying its receipt.

3. Token Passing

- In the ***token-passing method, the stations in a network are organized in a logical ring***.
- In other words, for each station, there is a ***predecessor*** and a ***successor***. The predecessor is the station which is logically before the station in the ring; the successor is the station which is after the station in the ring.
- The ***current station*** is the one that is ***accessing the channel now***. The right to this access has been passed from the predecessor to the current station. The right will be passed to the successor when the current station has no more data to send.

But how is the right to access the channel passed from one station to another?

- In this method, ***a special packet called a token circulates through the ring***. The possession of the token gives the station the right to access the channel and send its data. When a station has some data to send, it waits until it receives the token from its predecessor.
- It then holds the token and sends its data. When the station has no more data to send, it releases the token, passing it to the next logical station in the ring.
- The station cannot send data until it receives the token again in the next round. In this process, when a station receives the token and has no data to send, it just passes the data to the next station.

Token management is needed for this access method.

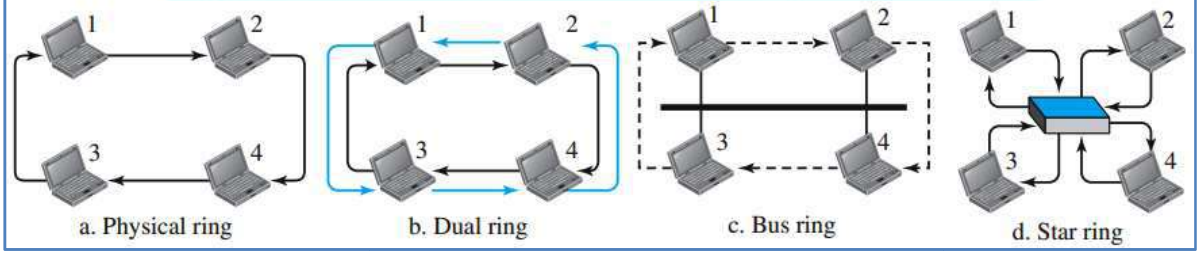
- Stations must be limited in the time they can have possession of the token. The token must be monitored to ensure it has not been lost or destroyed. For example, if a station that is holding the token fails, the token will disappear from the network.
- Another function of token management is to assign priorities to the stations and to the types of data being transmitted.
- And finally, token management is needed to make low-priority stations release the token to high-priority stations.

Logical Ring

- In a token-passing network, stations do not have to be physically connected in a ring; ***the ring can be a logical one***. Figure shows ***four different physical topologies that can create a logical ring***.

Prof. Parthasarathy P V

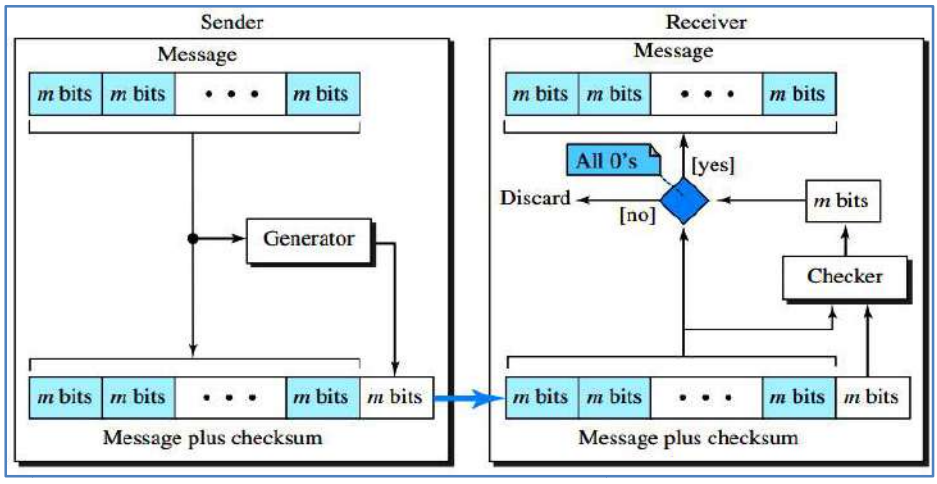
Figure *Logical ring and physical topology in token-passing access method*



- In the ***physical ring topology***, when a station sends the token to its successor, the token cannot be seen by other stations; the successor is the next one in line. This means that the token does not have to have the address of the next successor. The ***problem*** with this topology is that if one of the links—***the medium between two adjacent stations—fails, the whole system fails***.
- The ***dual ring topology uses a second (auxiliary) ring*** which ***operates in the reverse direction*** compared with the main ring. ***The second ring is for emergencies only*** (such as a spare tire for a car). If one of the links in the ***main ring fails, the system automatically combines the two rings to form a temporary ring***. After the failed link is restored, the auxiliary ring becomes idle again. Note that for this topology to work, each station needs to have two transmitter ports and two receiver ports. ***The high-speed Token Ring networks called FDDI (Fiber Distributed Data Interface) and CDDI (Copper Distributed Data Interface) use this topology.***
- In the ***bus ring topology, also called a token bus***, the stations are connected to a single cable called a bus. They, however, ***make a logical ring, because each station knows the address of its successor (and also predecessor for token management purposes).*** When a station has finished sending its data, it releases the token and inserts the address of its successor in the token. Only the station with the address matching the destination address of the token gets the token to access the shared media. ***The Token Bus LAN, standardized by IEEE, uses this topology***.
- In a ***star-ring topology***, the ***physical topology is a star***. There is a hub, however, that acts as the connector. The wiring inside the hub makes the ring; the stations are connected to this ring through the two wire connections. This topology makes the network less prone to failure because if a link goes down, it will be bypassed by the hub and the rest of the stations can operate. Also adding and removing stations from the ring is easier. ***This topology is still used in the Token Ring LAN designed by IBM.***

CHECKSUM (Chapter 10 – P-277)

- ***CHECKSUM*** is an ***error-detecting technique that can be applied to a message of any length***.
- In the Internet, the checksum technique is mostly used at the network and transport layer rather than the data-link layer.
- At the source, the message is first divided into ***m-bit*** units. The generator then creates an extra ***m-bit*** unit called the checksum, which is sent with the message. At the destination, the checker creates a new checksum from the combination of the message and sent checksum.



- If the new checksum is all 0s, the message is accepted; otherwise, the message is discarded (Figure). Note that in the real implementation, the checksum unit is not necessarily added at the end of the message; it can be inserted in the middle of the message.

Concept

- The idea of the traditional checksum is simple. We show this using a simple example.

Example

Suppose the message is a list of five 4-bit numbers that we want to send to a destination. In addition to sending these numbers, we send the sum of the numbers. For example, if the set of numbers is (7, 11, 12, 0, 6), we send (7, 11, 12, 0, 6, 36), where 36 is the sum of the original numbers. The receiver adds the five numbers and compares the result with the sum. If the two are the same, the receiver assumes no error, accepts the five numbers, and discards the sum. Otherwise, there is an error somewhere and the message is not accepted.

One's Complement Addition

- The previous example has one major drawback. Each number can be written as a 4-bit word (each is less than 15) except for the sum. One solution is to use one's complement arithmetic. In this arithmetic, we can represent unsigned numbers between 0 and ***2m − 1*** using only ***m*** bits. If the number has more than ***m*** bits, the extra leftmost bits need to be added to the m rightmost bits (wrapping).

Example

In the previous example, the decimal number 36 in binary is $(100100)_2$. To change it to a 4-bit number we add the extra leftmost bit to the right four bits as shown below.

$$(10)_2 + (0100)_2 = (0110)_2 \rightarrow (6)_{10}$$

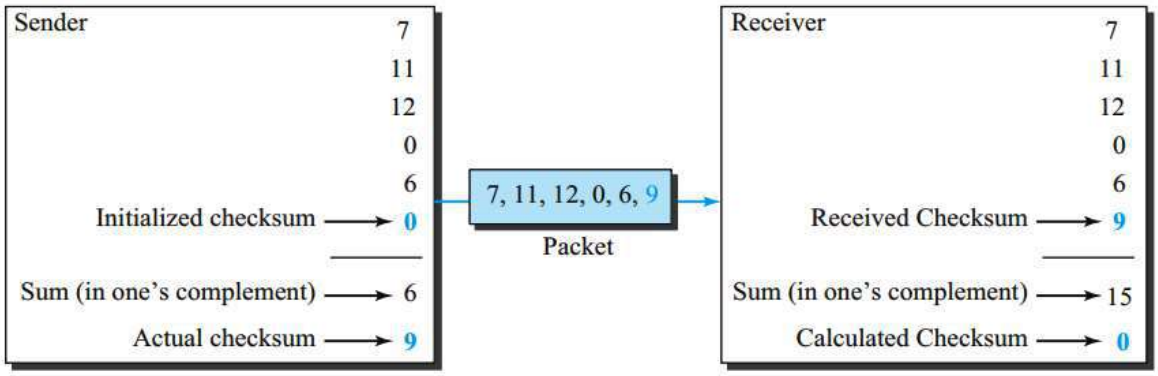
Instead of sending 36 as the sum, we can send 6 as the sum (7, 11, 12, 0, 6, 6). The receiver can add the first five numbers in one's complement arithmetic. If the result is 6, the numbers are accepted; otherwise, they are rejected.

Checksum

- We can make the job of the receiver easier if we send the complement of the sum, the checksum.
- In one's complement arithmetic, the complement of a number is found by completing all bits (changing all 1s to 0s and all 0s to 1s). This is the same as subtracting the number from 2m − 1.
- In one's complement arithmetic, we have two 0s: one positive and one negative, which are complements of each other.
- The positive zero has all m bits set to 0; the negative zero has all bits set to 1 (it is 2m − 1).
- If we add a number with its complement, we get a negative zero (a number with all bits set to 1). When the receiver adds all five numbers (including the checksum), it gets a negative zero. The receiver can complement the result again to get a positive zero.

Example

Let us use the idea of the checksum in Example 10.12. The sender adds all five numbers in one's complement to get the sum = 6. The sender then complements the result to get the checksum = 9, which is 15 − 6. Note that $6 = (0110)_2$ and $9 = (1001)_2$; they are complements of each other. The sender sends the five data numbers and the checksum (7, 11, 12, 0, 6, 9). If there is no corruption in transmission, the receiver receives (7, 11, 12, 0, 6, 9) and adds them in one's complement to get 15. The sender complements 15 to get 0. This shows that data have not been corrupted. Figure shows the process.



Internet Checksum

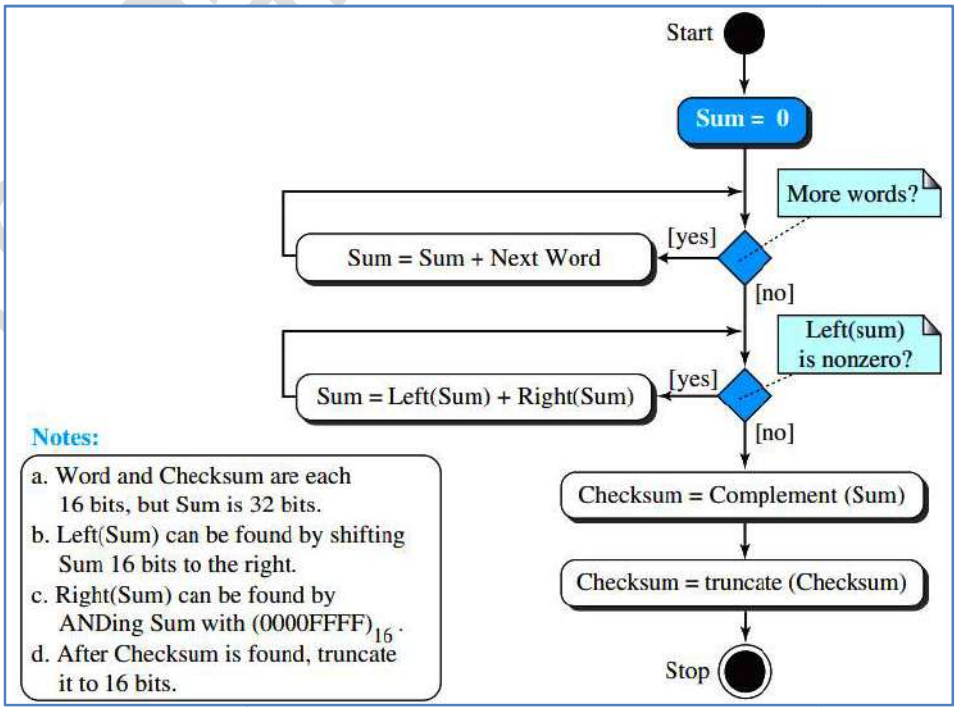
- Traditionally, the Internet has used a 16-bit checksum. The sender and the receiver follow the steps depicted in Table. The sender or the receiver uses five steps.

Table *Procedure to calculate the traditional checksum*

Sender	*Receiver*
1. The message is divided into 16-bit words.	1. The message and the checksum are received.
2. The value of the checksum word is initially set to zero.	2. The message is divided into 16-bit words.
3. All words including the checksum are added using one's complement addition.	3. All words are added using one's complement addition.
4. The sum is complemented and becomes the checksum.	4. The sum is complemented and becomes the new checksum.
5. The checksum is sent with the data.	5. If the value of the checksum is 0, the message is accepted; otherwise, it is rejected.

Algorithm

- We can use the flow diagram of Figure to show the algorithm for calculation of the checksum.
- A program in any language can easily be written based on the algorithm. Note that the first loop just calculates the sum of the data units in two's complement; the second loop wraps the extra bits created from the two's complement calculation to simulate the calculations in one's complement.
- This is needed because almost all computers today do calculation in two's complement.



Performance

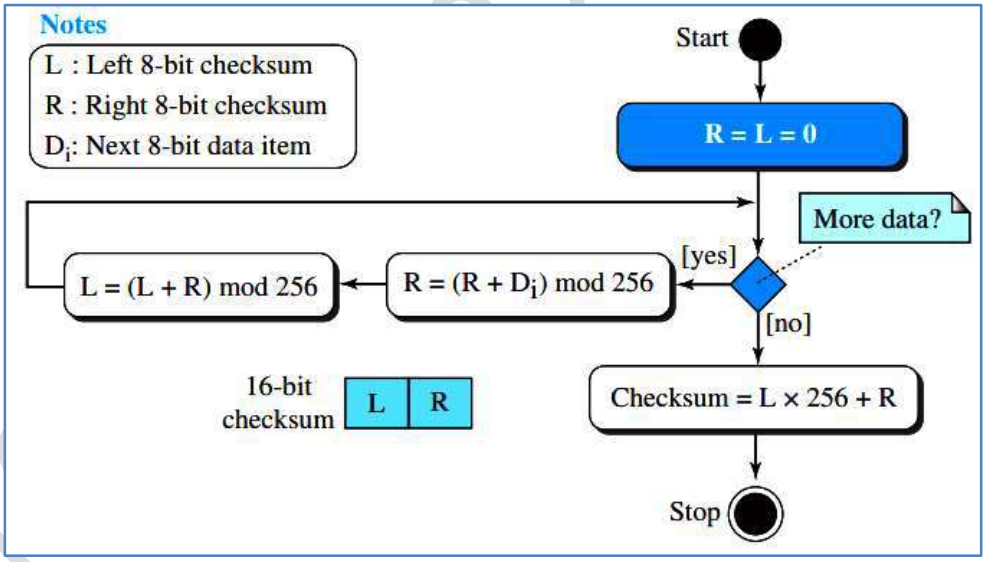
- The traditional checksum uses a small number of bits (16) to detect errors in a message of any size (sometimes thousands of bits). However, it is not as strong as the CRC in error-checking capability.
- For example, if the value of one word is incremented and the value of another word is decremented by the same amount, the two errors cannot be detected because the sum and checksum remain the same. Also, if the values of several words are incremented but the sum and the checksum do not change, the errors are not detected.

Other Approaches to the Checksum

- ***There is one major problem with the traditional checksum calculation***. If two 16-bit items are transposed in transmission, the checksum cannot catch this error. The reason is that the traditional checksum is not weighted: it treats each data item equally.
- Several approaches have been used to prevent this problem. We mention two of them here: Fletcher and Adler.

Fletcher Checksum

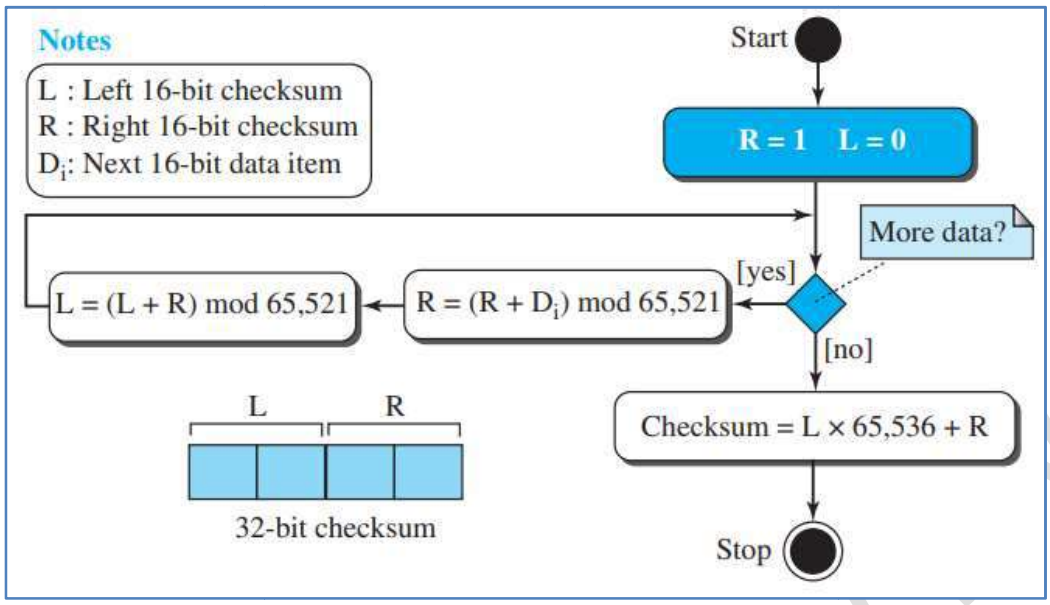
- The Fletcher checksum was devised to weight each data item according to its position. Fletcher has proposed two algorithms: 8-bit and 16-bit. The first, 8-bit Fletcher, calculates on 8-bit data items and creates a 16-bit checksum. The second, 16-bit Fletcher, calculates on 16-bit data items and creates a 32-bit checksum.
- The 8-bit Fletcher is calculated over data octets (bytes) and creates a 16-bit checksum. The calculation is done modulo 256 (2^8), which means the intermediate results are divided by 256 and the remainder is kept. The algorithm uses two accumulators, L and R.
- The first simply adds data items together; the second adds a weight to the calculation.
- There are many variations of the 8-bit Fletcher algorithm; we show a simple one in Figure.



- The 16-bit Fletcher checksum is similar to the 8-bit Fletcher checksum, but it is calculated over 16-bit data items and creates a 32-bit checksum.
- The calculation is done modulo 65,536.

Adler Checksum

- The Adler checksum is a 32-bit checksum. Figure shows a simple algorithm in flowchart form. It is similar to the 16-bit Fletcher with three differences.
  1. Calculation is done on single bytes instead of 2 bytes at a time.
  2. The modulus is a prime number (65,521) instead of 65,536.
  3. L is initialized to 1 instead of 0. It has been proved that a prime modulo has a better detecting capability in some combinations of data.



Point to Point Protocol (PPP) – Chapter 11 (P-309)

- ***One of the most common protocols for point-to-point access is the Point-to-Point Protocol (PPP)***. Today, millions of Internet users who need to connect their home computers to the server of an Internet service provider use PPP.
- To control and manage the transfer of data, there is a need for a point-to-point protocol at the data-link layer. PPP is by far the most common.

Topics Covered: Services – Framing - Transition Phases – Multiplexing

---

*Services (*Services Provided by PPP - Services Not Provided by PPP*)*

Services Provided by PPP

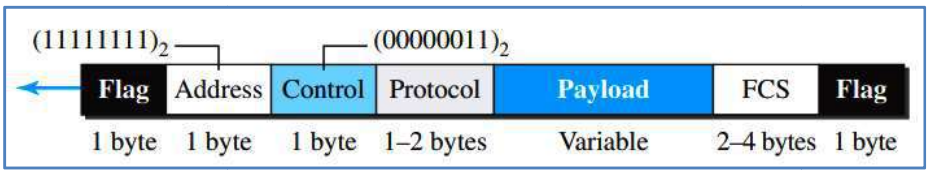
- PPP ***defines the format of the frame to be exchanged between devices***.
- It also ***defines how two devices can negotiate the establishment of the link and the exchange of data***.
- PPP is ***designed to accept payloads from several network layers (not only IP)***.
- ***Authentication is also provided in the protocol***, but it is optional.
- The new version of PPP, called ***Multilink PPP, provides connections over multiple links***. One ***interesting feature of PPP is that it provides network address configuration***. This is particularly ***useful when a home user needs a temporary network address to connect to the Internet***.

Services Not Provided by PPP

- PPP ***does not provide flow control***. A sender can send several frames one after another with no concern about overwhelming the receiver.
- PPP has a very ***simple mechanism for error control. A CRC field is used to detect errors***. If the frame is corrupted, it is silently discarded; the upper-layer protocol needs to take care of the problem.
- Lack of error control and sequence numbering may cause a packet to be received out of order. PPP does not provide a sophisticated addressing mechanism to handle frames in a multipoint configuration.

Framing

- PPP uses a ***character-oriented (or byte-oriented) frame***. Figure shows the format of a PPP frame.



The description of each field follows:

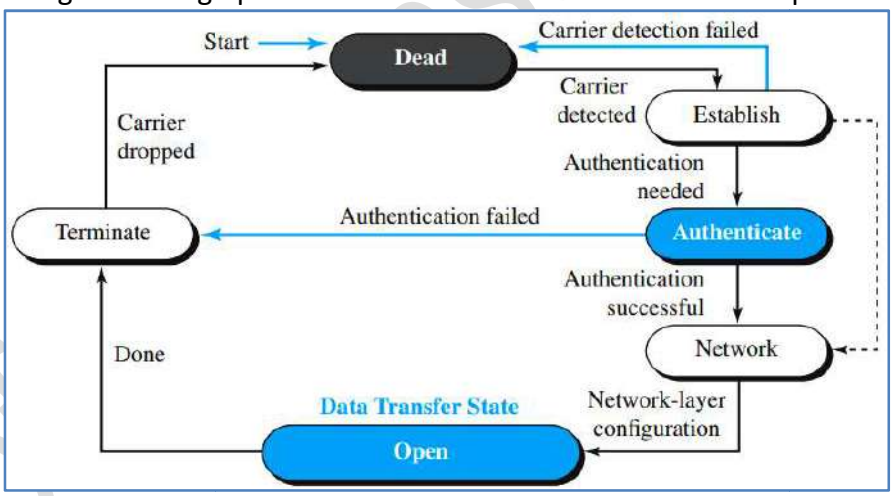
- ***Flag***. A PPP frame starts and ends with a 1-byte flag with the bit pattern 01111110.
- ***Address***. The address field in this protocol is a constant value and set to 11111111 (broadcast address).
- ***Control***. This field is set to the constant value 00000011 (imitating unnumbered frames in HDLC). PPP does not provide any flow control. Error control is also limited to error detection.
- ***Protocol***. The protocol field defines what is being carried in the data field: either user data or other information. This field is by default 2 bytes long, but the two parties can agree to use only 1 byte.
- ***Payload field***. This field carries either the user data or other information. The ***data field*** is a sequence of bytes with the default of a maximum of 1500 bytes; but this can be changed during negotiation. The ***data field is byte-stuffed*** if the flag byte pattern appears in this field.
- ***FCS***. The frame check sequence (FCS) is simply a 2-byte or 4-byte standard CRC.

Byte Stuffing

- Since PPP is a byte-oriented protocol, the flag in PPP is a byte that needs to be escaped whenever it appears in the data section of the frame.
- The escape byte is 01111101, which means that every time the flaglike pattern appears in the data, this extra byte is stuffed to tell the receiver that the next byte is not a flag.

Transition Phases

- A PPP connection goes through phases which can be shown in a transition phase diagram.



- The transition diagram, which is an FSM, ***starts with the dead state***. In this state, there is no active carrier (at the physical layer) and the line is quiet. When one of the two nodes starts the communication, the connection goes into the ***establish state***. ***In this state, options are negotiated between the two parties***.
- If the ***two parties agree that they need authentication, then the system needs to do authentication (an extra step);*** otherwise, the parties can simply start communication.
- The ***link-control protocol packets*** are used for this purpose. Several packets may be exchanged here. ***Data transfer*** takes place in the ***open state***.
- When a connection reaches this state, the exchange of data packets can be started.
- The connection remains in this state until one of the end points wants to terminate the connection. In this case, the system goes to the ***terminate state***.

- The system remains in this state until the carrier (physical-layer signal) is dropped, which moves the system to the ***dead state*** again.

Multiplexing

- Although PPP is a link-layer protocol, it uses another set of protocols to establish the link, authenticate the parties involved, and carry the network-layer data.
- ***Three sets of protocols are defined to make PPP powerful***:
  1. the Link Control Protocol (LCP),
  2. two Authentication Protocols (APs), and
  3. several Network Control Protocols (NCPs).
- At any moment, a PPP packet can carry data from one of these protocols in its data field, as shown in Figure.

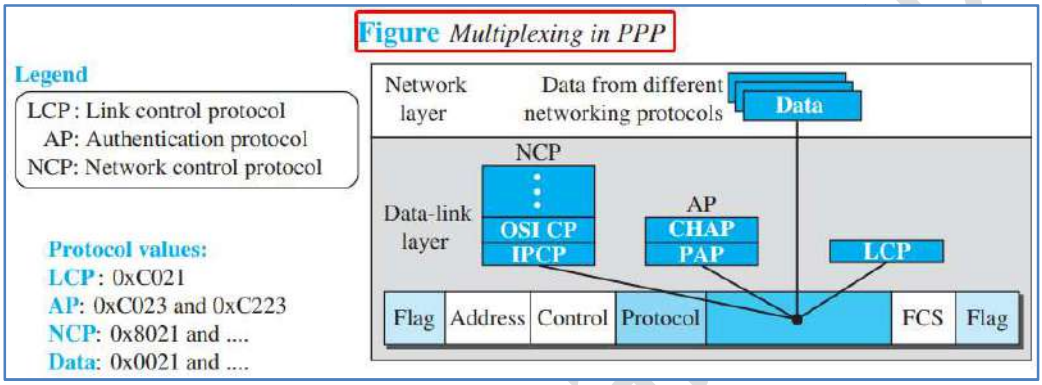
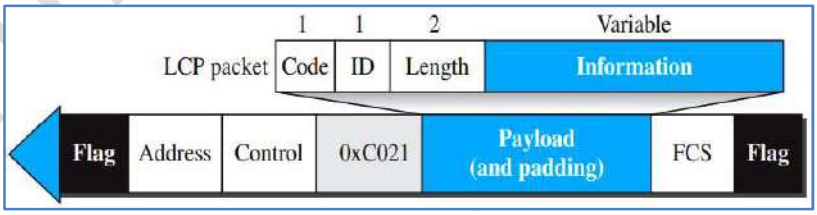


Figure *Multiplexing in PPP*

Link Control Protocol

- The Link Control Protocol (LCP) is ***responsible for establishing, maintaining, configuring, and terminating links***.
- It also provides negotiation mechanisms to set options between the two endpoints.
- Both endpoints of the link must reach an agreement about the options before the link can be established.
- ***All LCP packets are carried in the payload field of the PPP frame with the protocol field set to C021 in hexadecimal*** (see Figure).



- The code field defines the type of LCP packet. There are ***11 types of packets***, as shown in Table.
- There are ***three categories of packets***. The first category, comprising the ***first four packet types***, is used for link configuration during the ***establish phase***. The second category, comprising ***packet types 5 and 6***, is used for link termination during the ***termination phase***. The ***last five packets*** are used for ***link monitoring and debugging***.

Code	Packet Type	Description	
0x01	Configure-request	Contains the list of proposed options and their values	Establish phase
0x02	Configure-ack	Accepts all options proposed	
0x03	Configure-nak	Announces that some options are not acceptable	
0x04	Configure-reject	Announces that some options are not recognized	
0x05	Terminate-request	Request to shut down the line	Termination phase
0x06	Terminate-ack	Accept the shutdown request	
0x07	Code-reject	Announces an unknown code	Monitoring and debugging
0x08	Protocol-reject	Announces an unknown protocol	
0x09	Echo-request	A type of hello message to check if the other end is alive	
0x0A	Echo-reply	The response to the echo-request message	
0x0B	Discard-request	A request to discard the packet	

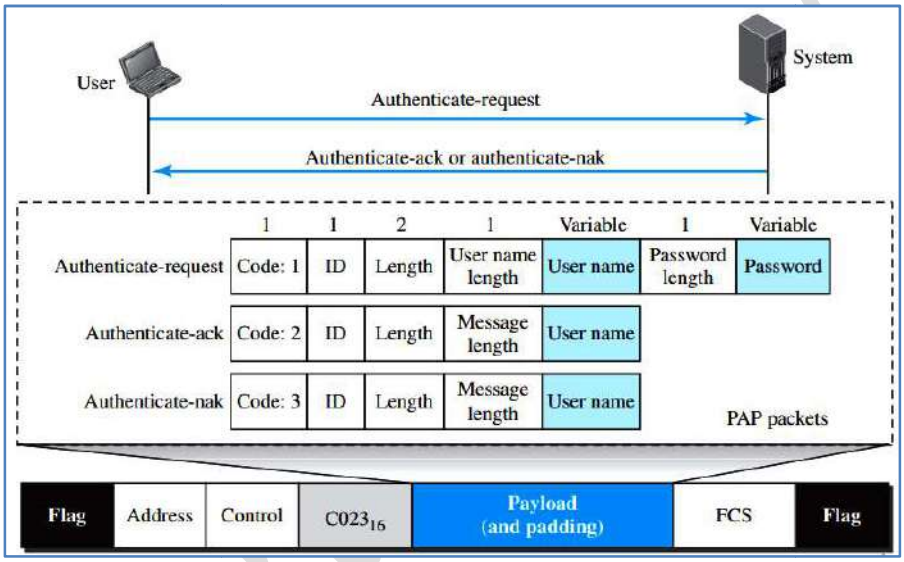
Authentication Protocols

- Authentication plays a very important role in PPP because ***PPP is designed for use over dial-up links where verification of user identity is necessary***.
- Authentication means validating the identity of a user who needs to access a set of resources.
- PPP has created ***two protocols*** for authentication:
  1. Password Authentication Protocol and
  2. Challenge Handshake Authentication Protocol.

  Note that these protocols are used during the authentication phase.

1. PAP (Password Authentication Protocol)

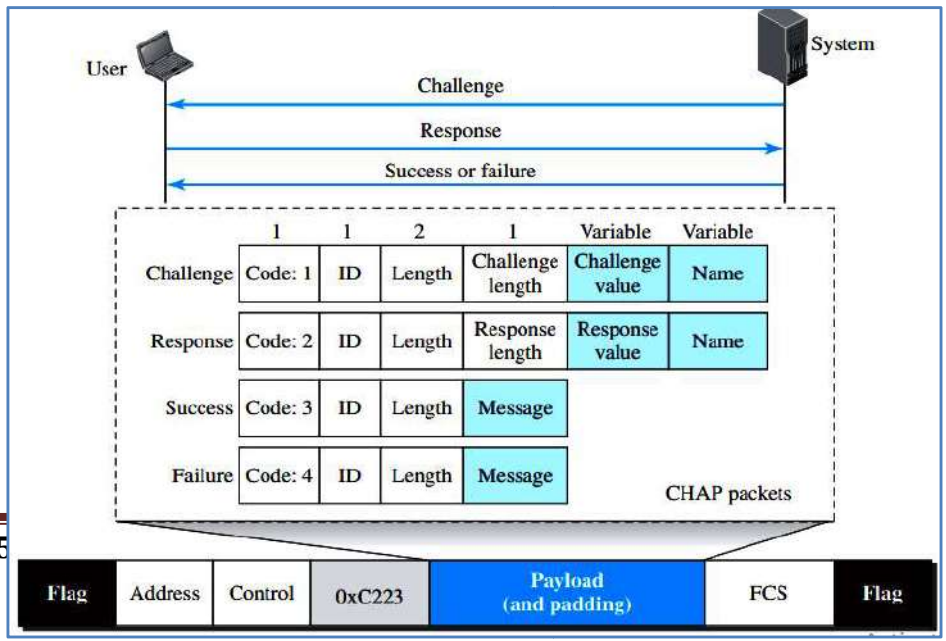
- PAP The Password Authentication Protocol (PAP) is a simple authentication procedure with a two-step process:
  1. The user who wants to access a system sends an authentication identification (usually the user name) and a password.
  2. The system checks the validity of the identification and password and either accepts or denies connection.



- Figure shows the ***three types of packets used by PAP and how they are actually exchanged***.
- When a PPP frame is carrying any PAP packets, the value of the protocol field is 0xC023. The three PAP packets are
  1. authenticate-request – used by ***the user to send the user name and password***
  2. authenticate-ack - used by the ***system to allow access***
  3. authenticate-nak - used by the ***system to deny access***

CHAP (Challenge Handshake Authentication Protocol)

- The ***Challenge Handshake Authentication Protocol (CHAP)*** is a ***three-way handshaking authentication protocol*** that provides greater security than PAP. In this method, the password is kept secret; it is never sent online.
  1. The system sends the user a challenge packet containing a challenge value, usually a few bytes.
  2. The user applies a predefined function that



takes the challenge value and the user's own password and creates a result. The user sends the result in the response packet to the system.

3. The system does the same. It applies the same function to the password of the user (known to the system) and the challenge value to create a result. If the result created is the same as the result sent in the response packet, access is granted; otherwise, it is denied.

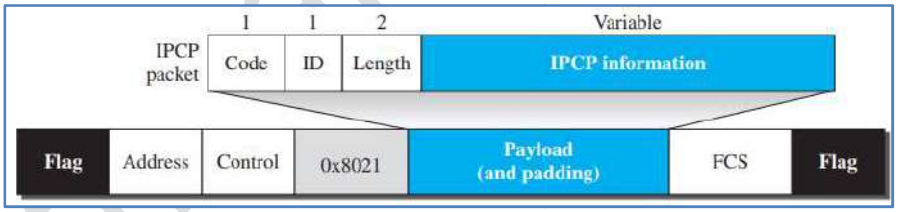
- ***CHAP is more secure than PAP, especially if the system continuously changes the challenge value***. Even if the intruder learns the challenge value and the result, the password is still secret. Figure shows the packets and how they are used.
- ***CHAP packets are encapsulated in the PPP frame with the protocol value C223 in hexadecimal***.
- There are ***four CHAP packets***
  1. ***Challenge -*** used by the system to send the challenge value
  2. ***Response -*** used by the user to return the result of the calculation
  3. ***Success -*** used by the system to allow access to the system
  4. ***Failure*** - used by the system to deny access to the system.

Network Control Protocols

- PPP is a multiple-network-layer protocol. It can carry a network-layer data packet from protocols defined by the Internet, OSI, Xerox, DECnet, AppleTalk, Novel, and so on. To do this, PPP has defined a specific Network Control Protocol for each network protocol. For example, IPCP (Internet Protocol Control Protocol) configures the link for carrying IP data packets. Xerox CP does the same for the Xerox protocol data packets, and so on. Note that none of the NCP packets carry network-layer data; they just configure the link at the network layer for the incoming data.

IPCP (Internet Protocol Control Protocol)

- This ***protocol configures the link used to carry IP packets in the Internet***. IPCP is especially of interest to us. The format of an IPCP packet is shown in Figure. Note that the value of the protocol field in hexadecimal is 8021.



- IPCP defines seven packets, distinguished by their code values, as shown in Table.

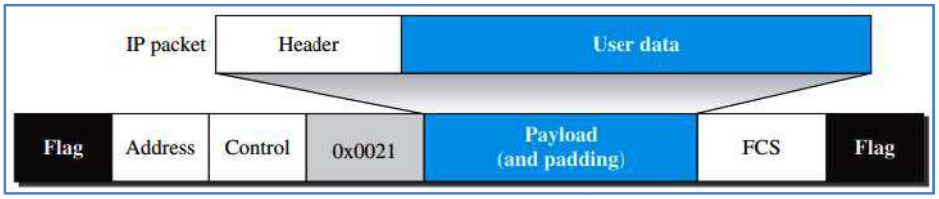
Code	*IPCP Packet*
0x01	Configure-request
0x02	Configure-ack
0x03	Configure-nak
0x04	Configure-reject
0x05	Terminate-request
0x06	Terminate-ack
0x07	Code-reject

Other Protocols

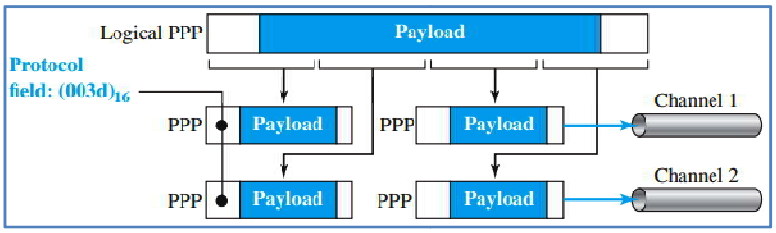
- There are other NCP protocols for other network-layer protocols. The OSI **Network Layer Control Protocol** has a protocol field value of **8023**; the Xerox **NS IDP Control Protocol** has a protocol field value of **8025.**

Data from the Network Layer

- After the network-layer configuration is completed by one of the NCP protocols, the users can exchange data packets from the network layer. Here again, there are different protocol fields for different network layers.
- For example, if PPP is carrying data from the IP network layer, the field value is 0021 (note that the three rightmost digits are the same as for IPCP). If PPP is carrying data from the OSI network layer, the value of the protocol field is 0023, and so on. Figure shows the frame for IP.



Multilink PPP



- The availability of multiple channels in a single point-to-point link motivated the development of Multilink PPP.
- In this case, a logical PPP frame is divided into several actual PPP frames. A segment of the logical frame is carried in the payload of an actual PPP frame, as shown in Figure. To show that the actual PPP frame is carrying a fragment of a logical PPP frame, the protocol field is set to (003d)16. This new development adds complexity. For example, a sequence number needs to be added to the actual PPP frame to show a fragment's position in the logical frame.

Prof. Parthasarathy P V

BCS502 COMPUTER NETWORKS

Module-3: Network Layer

Chapter 18: Introduction to Network Layer: Network layer Services, Packet Switching, IPv4 Address, IPv4 Datagram.
Chapter 22: Next Generation IP: IPv6 Datagram.
Chapter 20: Unicast Routing: Introduction to Routing Algorithms, Unicast Routing Protocols: DVR, LSR, PVR, Unicast Routing protocols: RIP, OSPF, BGP.
Ch. 21. Multicast Routing: Multicasting Routing-MOSPF

Textbook: Ch. 10.1-10.4, 11.1 -11.4, 12.1 - 12.2

Chapter 18: Introduction to Network Layer (Pg. 511[550])

Network layer Services, Packet Switching, IPv4 Address, IPv4 Datagram

---

Introduction to Network Layer

- ***The network layer in the TCP/IP protocol suite is responsible for the host-to-host delivery of datagrams***. It ***provides services to the transport layer and receives services from the data-link layer***.
- In this chapter, we introduce the general concepts and issues in the network layer. This chapter also discusses the addressing mechanism used in the network layer. This chapter prepares the way for discussion of other network-layer issues, which follows in the next four chapters.
- The ***first section introduces the network layer by defining the services provided by this layer***. It first discusses ***packetizing***. It then ***describes forwarding and routing and compares the two***. The section then briefly explains the ***other services such as flow, error, and congestion control***.
- The ***second section discusses packet switching, which occurs at the network layer***. The datagram approach and the virtual-circuit approach of packet switching are described in some detail in this section.
- The ***third section discusses network-layer performance***. It describes different delays that occur in network-layer communication. It also mentions the issue of packet loss. Finally, it discusses the issue of congestion control at the network layer.
- The ***fourth section discusses IPv4 addressing***, probably the most important issue in the network layer. It first describes the address space. It then briefly discusses classful addressing, which belongs to the past but is useful in understanding classless addressing. The section then moves to classless addressing and explains several issues related to this topic. It then discusses DHCP, which can be used to dynamically assign addresses in an organization. Finally, the section discusses NAT, which can be used to relieve the shortage of addresses to some extent.

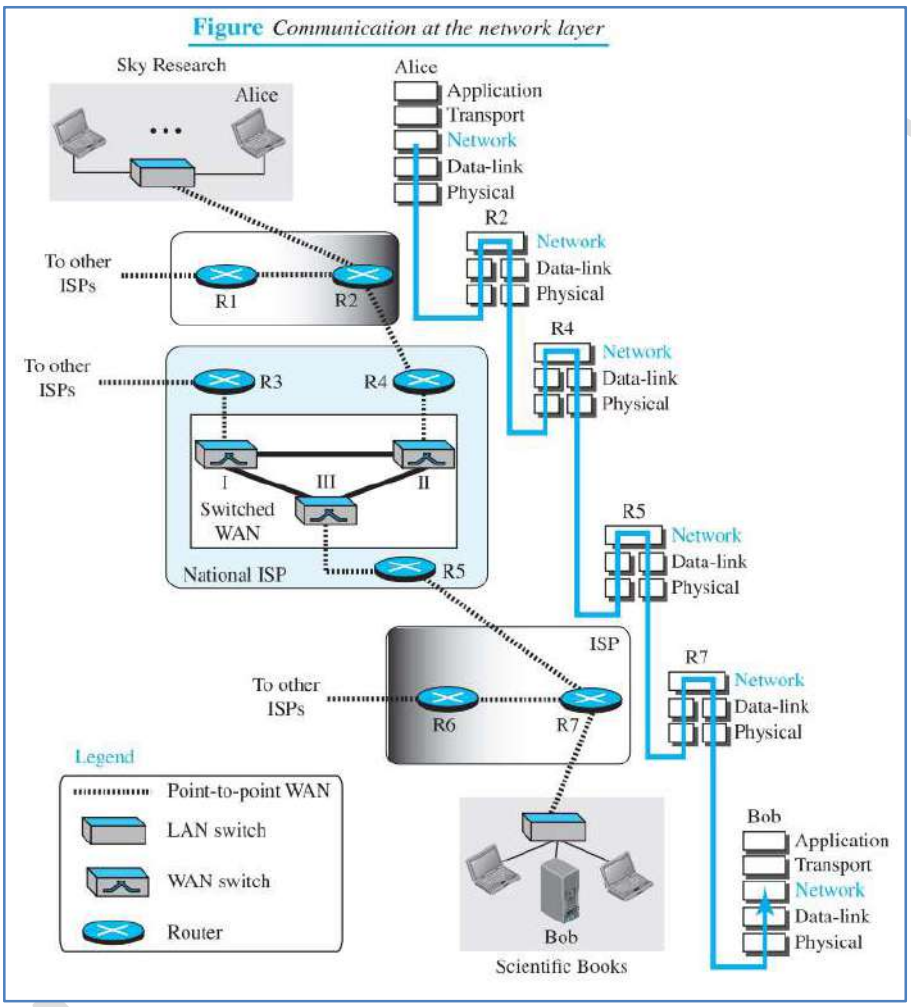
Network layer Services

Packetizing - Routing and Forwarding - Other Services

- The ***figure shows that the Internet is made of many networks (or links) connected through the connecting devices***. In other words, the Internet is an internetwork, a combination of LANs and WANs. To better understand the role of the network layer (or the internetwork layer), we need to think about the connecting devices (routers or switches) that connect the LANs and WANs.
- As the figure shows, the network layer is involved at the source host, destination host, and all routers in the path (R2, R4, R5, and R7). At the source host (Alice), the network layer accepts a packet from a transport layer, encapsulates the packet in a datagram, and delivers the packet to the data-link layer.

Prof. Parthasarathy P V

- At the destination host (Bob), the datagram is decapsulated, and the packet is extracted and delivered to the corresponding transport layer.
- Although the ***source and destination hosts are involved in all five layers of the TCP/IP suite, the routers use three layers if they are routing packets only***; however, they may need the transport and application layers for control purposes.
- A router in the path is normally shown with two data-link layers and two physical layers, because it receives a packet from one network and delivers it to another network.



Packetizing

- The first ***duty of the network layer is definitely packetizing***: ***encapsulating the payload (data received from upper layer) in a network-layer packet at the source and decapsulating the payload from the network-layer packet at the destination***.
- In other words, one duty of the network layer is to carry a payload from the source to the destination without changing it or using it. The network layer is doing the service of a carrier such as the postal office, which is responsible for delivery of packages from a sender to a receiver without changing or using the contents.
- ***The source host receives the payload from an upper-layer protocol, adds a header that contains the source and destination addresses and some other information that is required by the network-layer protocol and delivers the packet to the data-link layer***. The ***source is not allowed to change the content of the payload unless it is too large for delivery and needs to be fragmented***.
- The destination host receives the network-layer packet from its data-link layer, decapsulates the packet, and delivers the payload to the corresponding upper-layer protocol.

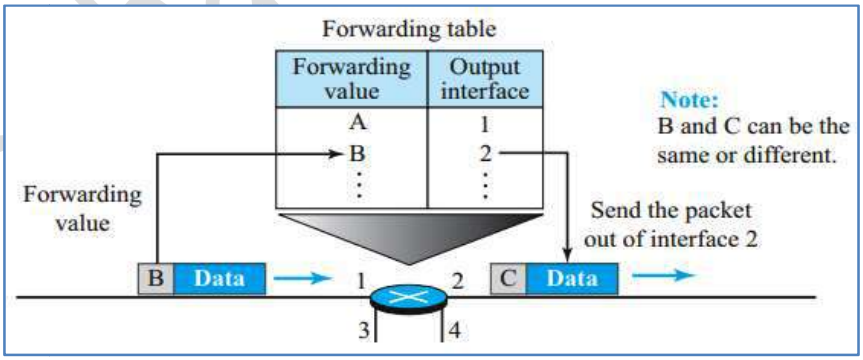
- If the packet is fragmented at the source or at routers along the path, the network layer is responsible for waiting until all fragments arrive, reassembling them, and delivering them to the upper-layer protocol.
- The ***routers in the path are not allowed to decapsulate the packets they received unless the packets need to be fragmented***. The ***routers are not allowed to change source and destination addresses*** either. They just inspect the addresses for the purpose of forwarding the packet to the next network on the path. However, if a packet is fragmented, the header needs to be copied to all fragments and some changes are needed.

Routing

- The network layer is responsible for routing the packet from its source to the destination.
- A physical network is a combination of networks (LANs and WANs) and routers that connect them.
- The ***network layer is responsible for finding the best one among these possible routes***. The network layer ***needs to have some specific strategies for defining the best route***.
- In the Internet today, this is done by running some routing protocols to help the routers coordinate their knowledge about the neighborhood and to come up with consistent tables to be used when a packet arrives.

Forwarding

- If routing is applying strategies and running some routing protocols to create the decision-making tables for each router, ***forwarding can be defined as the action applied by each router when a packet arrives at one of its interfaces***.
- The decision-making table - a router normally uses for ***applying this action*** is called the ***forwarding table (routing table)***.
- When a router receives a packet from one of its attached networks, it needs to forward the packet to another attached network (in unicast routing) or to some attached networks (in multicast routing).
- ***To make this decision, the router uses a piece of information in the packet header, which can be the destination address or a label, to find the corresponding output interface number in the forwarding table***.



- Figure shows the idea of the forwarding process in a router.

Other Services (Error Control, Flow Control, Congestion Control, Quality of Service, Security)

Error Control

- We discussed error detection and correction earlier in Data link layer. Although ***error control also can be implemented in the network layer***, the designers of the network layer in the Internet ignored this issue for the data being carried by the network layer. One reason for this decision is the fact that the packet in the network layer may be fragmented at each router, which makes error checking at this layer inefficient.
- ***The designers of the network layer have added a checksum field to the datagram to control any corruption in the header***, but ***not in the whole datagram***. This ***checksum may prevent any changes or corruptions in the header of the datagram***.

Prof. Parthasarathy P V

- The ***network layer in the Internet does not directly provide error control***, the ***Internet uses an auxiliary protocol, ICMP***, that provides some kind of error control if the datagram is discarded or has some unknown information in the header.

Flow Control

- ***Flow control regulates the amount of data a source can send without overwhelming the receiver***. If the upper layer at the source computer produces data faster than the upper layer at the destination computer can consume it, the receiver will be overwhelmed with data. ***To control the flow of data, the receiver needs to send some feedback to the sender to inform the latter that it is overwhelmed with data.***
- The network layer in the Internet, however, ***does not directly provide any flow control***. The datagrams are sent by the sender when they are ready, without any attention to the readiness of the receiver.

Congestion Control

- Another ***issue in a network-layer protocol is congestion control***. Congestion in the network layer is a ***situation in which too many datagrams are present in an area of the Internet***. Congestion may occur if the number of datagrams sent by source computers is ***beyond the capacity of the network or routers***. In this situation, some routers may drop some of the datagrams.
- However, as more datagrams are dropped, the situation may become worse because, due to the error control mechanism at the upper layers, the ***sender may send duplicates of the lost packets***. If the ***congestion continues, sometimes a situation may reach a point where the system collapses and no datagrams are delivered***.

Quality of Service

- As the Internet has allowed ***new applications such as multimedia communication*** (in particular real-time communication of audio and video), the quality of service (QoS) of the communication has become more and more important.
- The Internet has thrived by providing better quality of service to support these applications. However, to keep the network layer untouched, these provisions are mostly implemented in the upper layer.

Security

- Another issue related to communication at the network layer is security. Security was not a concern when the Internet was originally designed because it was used by a small number of users at universities for research activities; other people had no access to the Internet.
- The network layer was designed with no security provision. Today, however, security is a big concern. ***To provide security for a connectionless network layer, we need to have another virtual level that changes the connectionless service to a connection-oriented service***. This virtual layer, called IPSec.

Prof. Parthasarathy P V

Packet Switching

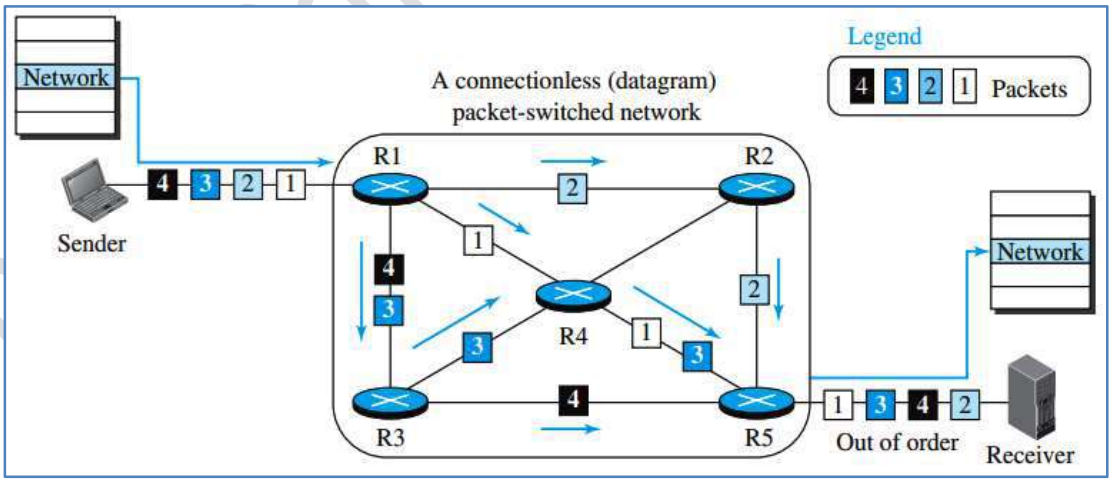
Datagram Approach: Connectionless Service - Virtual-Circuit Approach: Connection-Oriented Service

---

- ***A router is a switch that creates a connection between an input port and an output port*** (or a set of output ports), just as an electrical switch connects the input to the output to let electricity flow.
- Although ***in data communication switching techniques are divided into two broad categories***
  1. circuit switching
  2. packet switching
- ***Only packet switching is used at the network layer because the unit of data at this layer is a packet***. Circuit switching is mostly used at the physical layer; the electrical switch mentioned earlier is a kind of circuit switch.
- ***At the network layer, a message from the upper layer is divided into manageable packets and each packet is sent through the network***. The source of the message sends the packets one by one; the destination of the message receives the packets one by one. The destination waits for all packets belonging to the same message to arrive before delivering the message to the upper layer.
- The connecting devices in a packet-switched network still need to decide how to route the packets to the final destination.
- Today, a packet-switched network can use ***two different approaches to route the packets***:
  1. the datagram approach
  2. the virtual circuit approach.

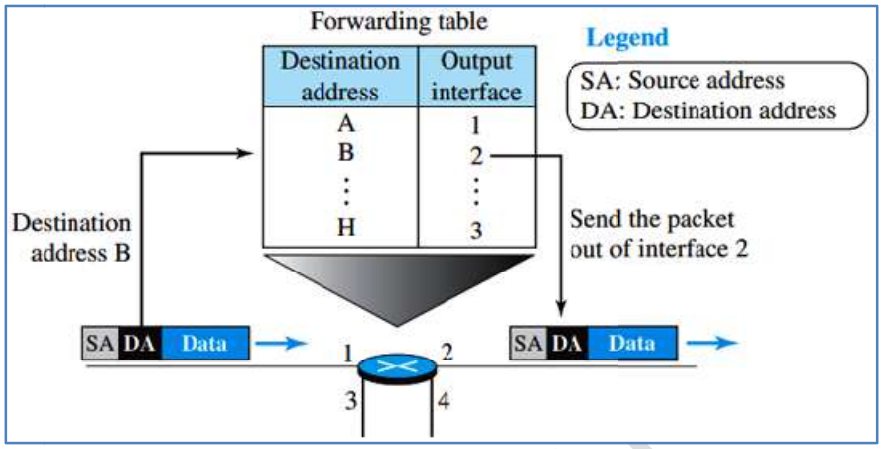
Datagram Approach: Connectionless Service

- When the Internet started, to make it simple, the network layer was designed to provide a connectionless service in which the network-layer protocol treats each packet independently, with each packet having no relationship to any other packet.
- The idea was that the ***network layer is only responsible for delivery of packets from the source to the destination***. In this approach, the packets in a message may or may not travel the same path to their destination. Figure shows the idea.



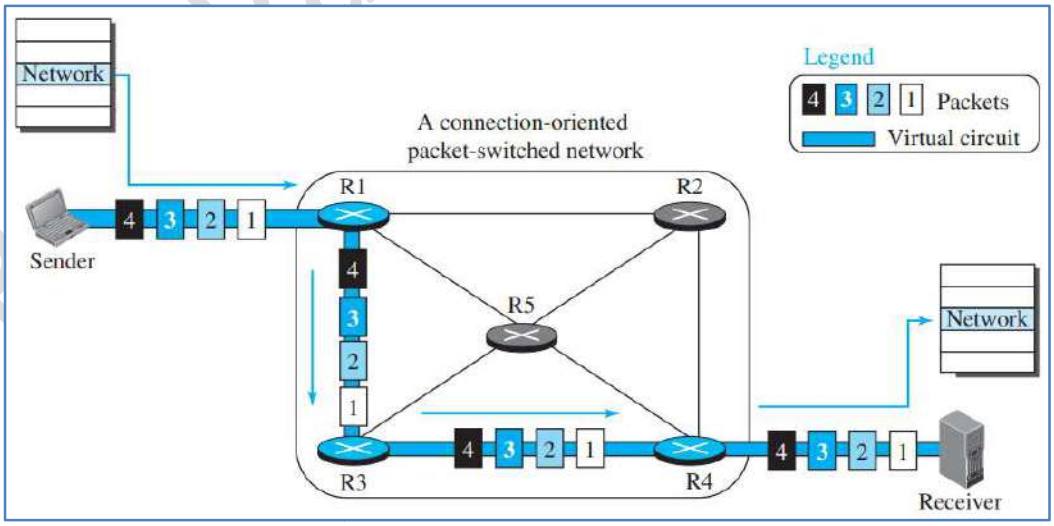
- When the ***network layer provides a connectionless service, each packet traveling in the Internet is an independent entity***; there is no relationship between packets belonging to the same message. The ***switches in this type of network are called routers***.
- A packet belonging to a message may be followed by a packet belonging to the same message or to a different message. A packet may be followed by a packet coming from the same or from a different source.

- ***Each packet is routed based on the information contained in its header: source and destination addresses***. The destination address defines where it should go; the source address defines where it comes from. The router in this case routes the packet based only on the destination address.
- ***The source address may be used to send an error message to the source if the packet is discarded.*** Figure shows the forwarding process in a router in this case. We have used symbolic addresses such as A and B.



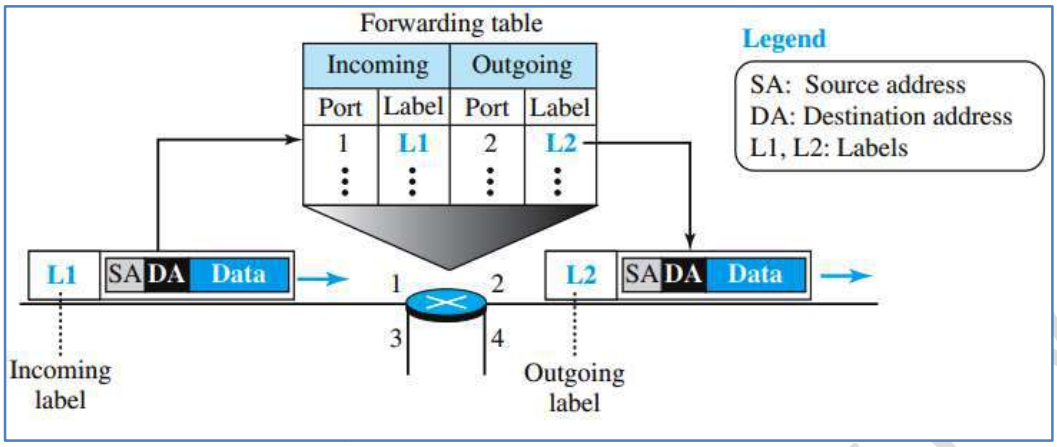
Virtual-Circuit Approach: Connection-Oriented Service

- ***In a connection-oriented service (also called virtual-circuit approach), there is a relationship between all packets belonging to a message***.
- Before all datagrams in a message can be sent, a virtual connection should be set up to define the path for the datagrams. After connection setup, the datagrams can all follow the same path. In this type of service, not only must the packet contain the source and destination addresses, it must also contain a flow label, a virtual circuit identifier that defines the virtual path the packet should follow.
- Shortly, we will show how this flow label is determined, but for the moment, we assume that the packet carries this label. Although it looks as though the use of the label may make the source and destination addresses unnecessary during the data transfer phase, parts of the Internet at the network layer still keep these addresses.
- ***One reason is that part of the packet path may still be using the connectionless service. Another reason is that the protocol at the network layer is designed with these addresses, and it may take a while before they can be changed***. Figure shows the concept of connection-oriented service.



- Each ***packet is forwarded based on the label in the packet***. To follow the idea of connection-oriented design to be used in the Internet, we assume that the packet has a label when it reaches the router.

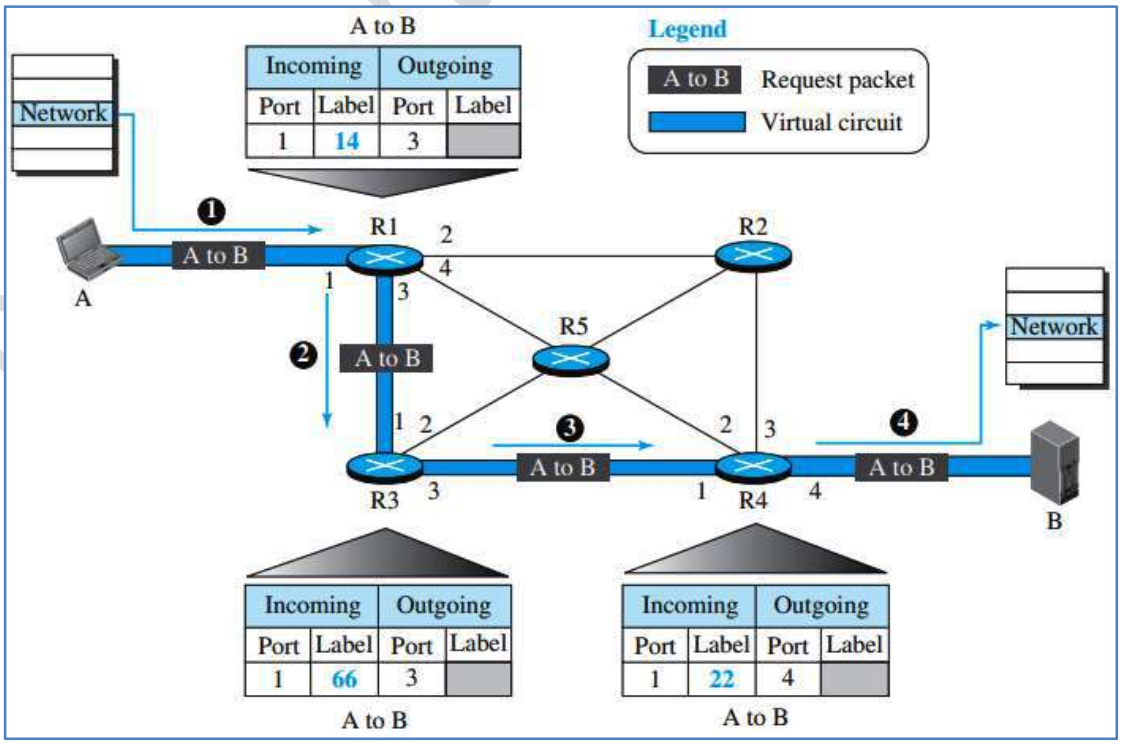
- Figure shows the idea. In this case, the ***forwarding decision is based on the value of the label, or virtual circuit identifier***, as it is sometimes called.



- To create a connection-oriented service, a ***three-phase process is used***: ***setup***, ***data transfer, and teardown***.
- In the ***setup phase***, the source and destination addresses of the sender and receiver are used to make table entries for the connection-oriented service.
- In the ***teardown phase***, the source and destination inform the router to delete the corresponding entries.
- ***Data transfer*** occurs between these two phases.

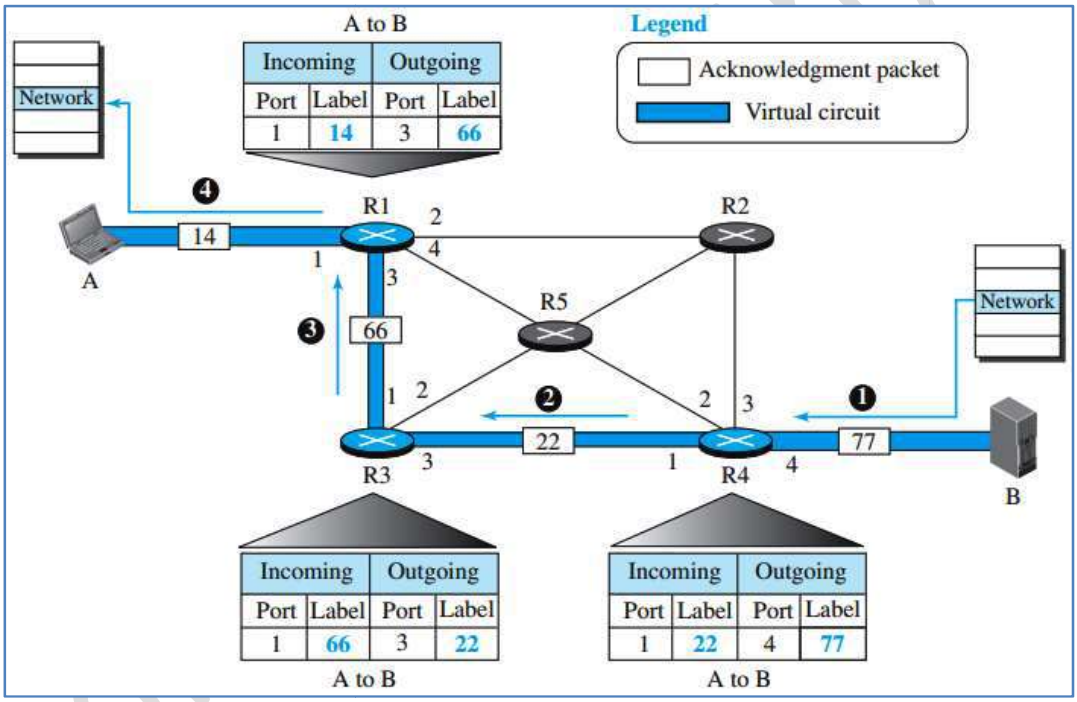
1. Setup Phase

- In the setup phase, a router creates an entry for a virtual circuit. For example, suppose source A needs to create a virtual circuit to destination B. Two auxiliary packets need to be exchanged between the sender and the receiver: the request packet and the acknowledgment packet.
- ***Request packet***: A request packet is sent from the source to the destination. This auxiliary packet carries the source and destination addresses. Figure shows the process.



1. ***Source A sends a request packet to router R1.***
2. ***Router R1 receives the request packet.*** It knows that a packet going from A to B goes out through port 3. Assume that it knows the output port. The router creates an entry in its table for this virtual circuit, but it is only able to fill three of the four columns. The router assigns the incoming port (1) and chooses an available incoming label (14) and the outgoing port (3). It does not yet know the outgoing label, which will be found during the acknowledgment step. The router then forwards the packet through port 3 to router R3.
3. ***Router R3 receives the setup request packet.*** The same events happen here as at router R1; three columns of the table are completed: in this case, incoming port (1), incoming label (66), and outgoing port (3).
4. ***Router R4 receives the setup request packet.*** Again, three columns are completed: incoming port (1), incoming label (22), and outgoing port (4).
5. ***Destination B receives the setup packet, and if it is ready to receive packets from A, it assigns a label to the incoming packets that come from A***, in this case 77, as shown in Figure below. This label lets the destination know that the packets come from A, and not from other sources.

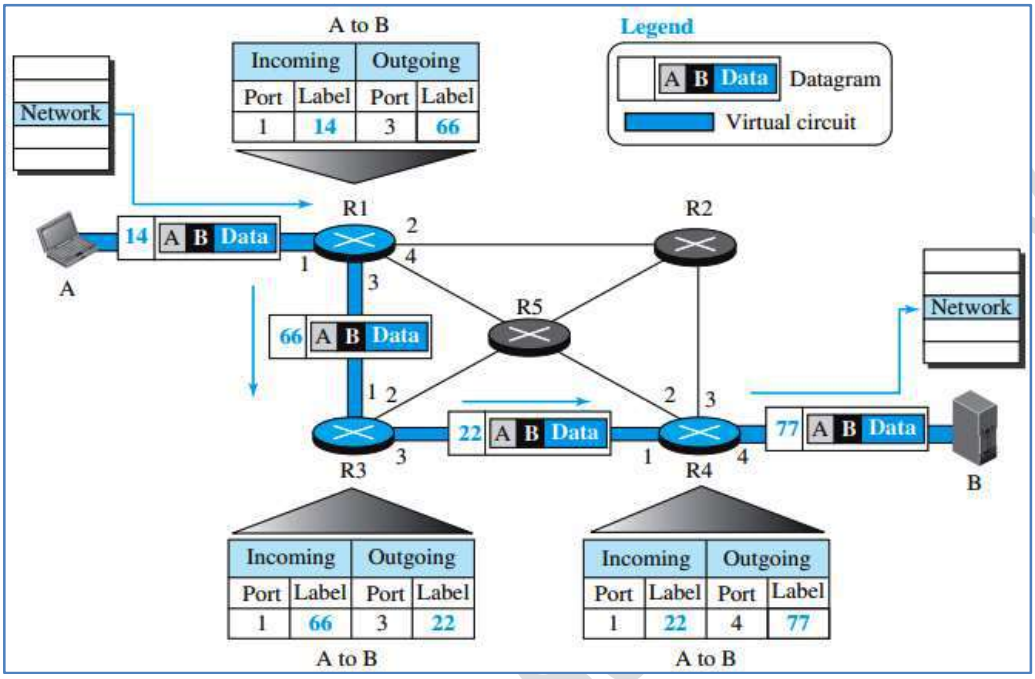
- ***Acknowledgment Packet*** A special packet, called the acknowledgment packet, completes the entries in the switching tables. The following figure shows the process.



1. The ***destination sends an acknowledgment to router R4. The acknowledgment carries the global source and destination addresses*** so the router knows which entry in the table is to be completed. The packet also carries label 77, chosen by the destination as the incoming label for packets from A. Router R4 uses this label to complete the outgoing label column for this entry. Note that 77 is the incoming label for destination B, but the outgoing label for router R4.
2. ***Router R4 sends an acknowledgment to router R3 that contains its incoming label in the table, chosen in the setup phase.*** Router R3 uses this as the outgoing label in the table.
3. ***Router R3 sends an acknowledgment to router R1 that contains its incoming label in the table***, chosen in the setup phase. Router R1 uses this as the outgoing label in the table.
4. Finally ***router R1 sends an acknowledgment to source A that contains its incoming label*** in the table, chosen in the setup phase.
5. The ***source uses this as the outgoing label for the data packets to be sent to destination B***.

2. Data-Transfer Phase

- The second phase is called the data-transfer phase. ***After all routers have created their forwarding table for a specific virtual circuit, then the network-layer packets belonging to one message can be sent one after another.*** In Figure, we show the flow of a single packet, but the process is the same for 1, 2, or 100 packets.



- The source computer uses the label 14, which it has received from router R1 in the setup phase. Router R1 forwards the packet to router R3, but changes the label to 66. Router R3 forwards the packet to router R4, but changes the label to 22. Finally, router R4 delivers the packet to its final destination with the label 77. All the packets in the message follow the same sequence of labels, and the packets arrive in order at the destination.

3. Teardown Phase

- In the teardown phase, source A, after sending all packets to B, sends a special packet called a teardown packet.
- Destination B responds with a confirmation packet.
- All routers delete the corresponding entries from their tables.

IPv4 Addresses (P-428 [P-467])

Address Space - Classful Addressing - Classless Addressing - Dynamic Host Configuration Protocol (DHCP) - Network Address Resolution (NAT)

- The identifier used in the IP layer of the TCP/IP protocol suite to identify the connection of each device to the Internet is called the Internet address or IP address.
- An ***IPv4 address is a 32-bit address that uniquely and universally defines the connection of a host or a router to the Internet***. The IP address is the address of the connection, not the host or the router, because if the device is moved to another network, the IP address may be changed.
- ***IPv4 addresses are unique in the sense that each address defines one, and only one, connection to the Internet***. If a device has ***two connections to the Internet, via two networks, it has two IPv4 addresses***.

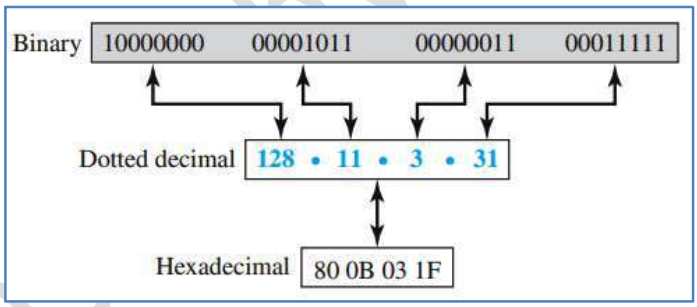
Prof. Parthasarathy

Address Space

- A protocol like ***IPv4 that defines addresses has an address space***. An address space is the total number of addresses used by the protocol. If a protocol uses ***b bits*** to define an address, the address space is 2^b because each bit can have two different values (0 or 1). IPv4 uses ***32-bit*** addresses, which means that the address space is 2^{32} or ***4,294,967,296*** (more than four billion).

Notation

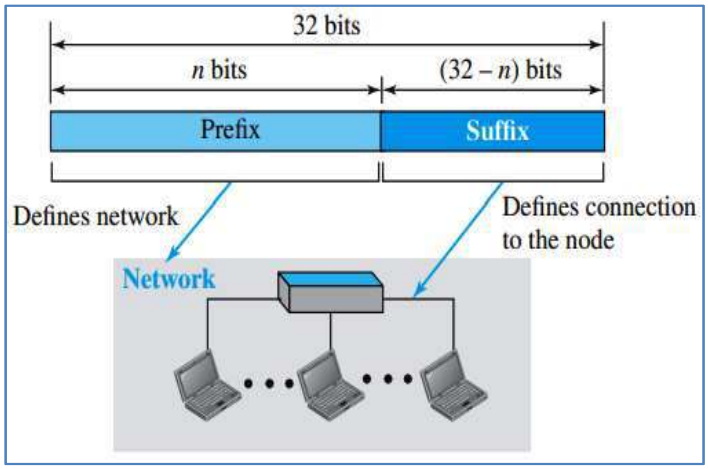
- There are ***three common notations to show an IPv4 address***:
  1. binary notation (base 2)
  2. dotted-decimal notation (base 256), and
  3. hexadecimal notation (base 16).
- ***In binary notation, an IPv4 address is displayed as 32 bits.***
- To make the ***address more readable***, one or more spaces are usually inserted between each ***octet (8 bits). Each octet is often referred to as a byte***.
- To make the IPv4 ***address more compact and easier to read***, it is usually written in ***decimal form with a decimal point (dot) separating the bytes***. This format is referred to as ***dotted-decimal notation***. Note that because each byte (octet) is only 8 bits, each number in the dotted-decimal notation is between 0 and 255.



- We sometimes see an ***IPv4 address in hexadecimal notation***. Each hexadecimal digit is ***equivalent to four bits***. This means that a 32-bit address has 8 hexadecimal digits. ***This notation is often used in network programming***. Figure shows an IP address in the three discussed notations.

Hierarchy in Addressing

- In any communication network that involves delivery, such as a telephone network or a postal network, the addressing system is hierarchical. In a postal network, the postal address (mailing address) includes the country, state, city, street, house number, and the name of the mail recipient. Similarly, a telephone number is divided into the country code, area code, local exchange, and the connection.
- A ***32-bit IPv4 address is also hierarchical***, but ***divided only into two parts***.
  1. The ***first part of the address***, called the prefix, ***defines the network***
  2. the ***second part*** of the address, called the suffix, ***defines the node*** (connection of a device to the Internet).
- Figure shows the prefix and suffix of a 32-bit IPv4 address. The prefix length is n bits and the suffix length is (32 − n) bits.

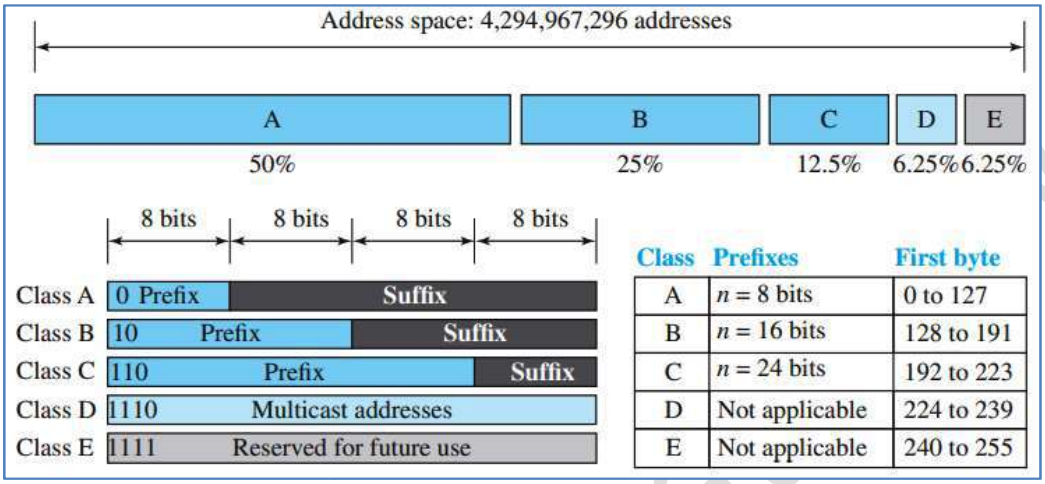


- A ***prefix can be fixed length or variable length***. The network identifier in the ***IPv4 was first designed as a fixed-length prefix***. This scheme, which is ***now obsolete***, is referred to as ***classful addressing***. The new scheme, which is referred to as ***classless addressing, uses a variable-length network prefix***.

Prof. Partha S V

Classful Addressing (fixed-length network prefix)

- When the Internet started, an ***IPv4 address was designed with a fixed-length prefix***, but to ***accommodate both small and large networks, three fixed-length prefixes were designed*** instead of one (n = 8, n = 16, and n = 24).
- The ***whole address space was divided into five classes (class A, B, C, D, and E),*** as shown in Figure. This scheme is referred to as classful addressing.



Class	Prefixes	First byte
A	$n = 8$ bits	0 to 127
B	$n = 16$ bits	128 to 191
C	$n = 24$ bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

- In ***class A, the network length is 8 bits***, but since the first bit, which is 0, defines the class, we can have only seven bits as the network identifier. This means there are only $2^7 = 128$ networks in the world that can have a class A address.
- In ***class B, the network length is 16 bits***, but since the first two bits, which are $(10)_2$, define the class, we can have only ***14 bits*** as the network identifier. This means there are only $2^{14} = 16{,}384$ networks in the world that can have a class B address.
- All addresses that start with $(110)_2$ belong to ***class C***. In class C, the network length is ***24 bits***, but since ***three bits define the class***, we can have only ***21 bits as the network identifier***. This means there are $2^{21} = 2{,}097{,}152$ networks in the world that can have a class C address.
- ***Class D is not divided into prefix and suffix. It is used for multicast addresses***.
- All addresses that start with ***1111 in binary belong to class E***. As in Class D, Class E is not divided into prefix and suffix and ***is used as reserve***.

Address Depletion

- The reason that classful addressing has become obsolete is address exhaustion.
- To understand the problem, let us think about **class A**. This class can be assigned to only 128 organizations in the world, but each organization needs to have a single network (seen by the rest of the world) with 16,777,216 nodes (computers in this single network). Since ***there may be only a few organizations that are this large, most of the addresses in this class were wasted*** (unused).
- ***Class B addresses*** were designed for midsize organizations, but many of the addresses in this class also remained unused.
- ***Class C addresses*** have a completely different flaw in design. The number of addresses that can be used in each network (256) was so small that most companies were not comfortable using a block in this address class.
- ***Class E addresses*** were almost never used, wasting the whole class.

Subnetting and Supernetting

- To improve address depletion, two strategies were proposed and implemented: ***subnetting*** and ***supernetting***.

Prof. Pathasarathy

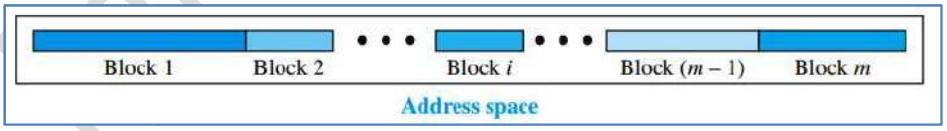
- In ***subnetting***, a class A or class B block is divided into several subnets. Each subnet has a larger prefix length than the original network. For example, if a network in ***class A is divided into four subnets***, each subnet has a prefix of $n_{sub} = 10$. At the same time, if all of the addresses in a network are not used, subnetting allows the addresses to be divided among several organizations.
- While subnetting was devised to divide a large block into smaller ones, ***supernetting*** was devised to combine ***several class C blocks*** into a larger block to be attractive to organizations that need more than the 256 addresses available in a class C block. This idea did not work either because it makes the routing of packets more difficult.

Advantage of Classful Addressing

- ***Given an address, we can easily find the class of the address and, since the prefix length for each class is fixed, we can find the prefix length immediately.***
- In other words, the prefix length in classful addressing is inherent in the address; no extra information is needed to extract the prefix and the suffix.

Classless Addressing (variable-length network prefix - IPv4 addresses)

- ***Subnetting and supernetting in classful addressing did not really solve the address depletion problem***. With the growth of the Internet, it was clear that a ***larger address space was needed as a long-term solution***. The ***larger address space requires that the length of IP addresses also be increased, which means the format of the IP packets needs to be changed***.
- Although the ***long-range solution has already been devised and is called IPv6.***
- The ***short-term solution still uses IPv4 addresses***, but it is ***called classless addressing***.
- ***In 1996, the Internet authorities announced a new architecture called classless addressing***. In classless addressing, variable-length blocks are used that belong to no classes. We can have a ***block of 1 address, 2 addresses, 4 addresses, 128 addresses, and so on***.
- In classless addressing, ***the whole address space is divided into variable length blocks***. The prefix in an address defines the block (network); the suffix defines the node (device). Theoretically, we can have a block of 2^0 , 2^1 , 2^2 , ..., 2^{32} addresses.
- ***One of the restrictions is that the number of addresses in a block needs to be a power of 2***. An organization can be granted one block of addresses. Figure shows the division of the whole address space into nonoverlapping blocks.

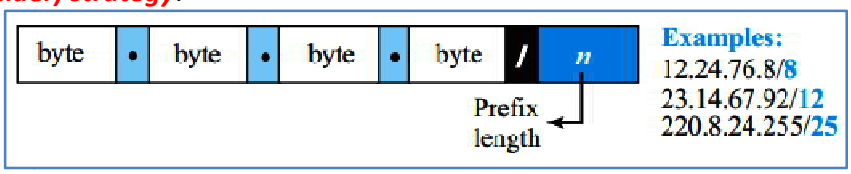


- The ***prefix length in classless addressing is variable (***prefix length that ranges from 0 to 32). The size of the network is inversely proportional to the length of the prefix. A small prefix means a larger network; a large prefix means a smaller network.

Prefix Length: Slash Notation

How to find the prefix length if an address is given?

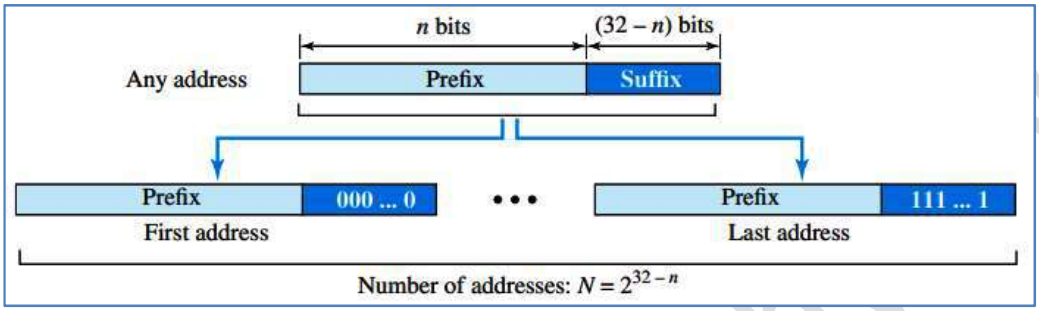
- Since the prefix length is not inherent in the address, we need to separately give the length of the prefix. In this case, the prefix length, n, is added to the address, separated by a slash.
- The notation is informally referred to as slash notation and formally as ***classless interdomain routing or CIDR (pronounced cider) strategy***.
- An address in classless addressing can then be represented as shown in Figure.



Prof. Sunagar P V

Extracting Information from an Address

- Given any address in the block, we normally like to know **three pieces of information about the block to which the address belongs**:
  1. the number of addresses
  2. the first address in the block
  3. the last address.
- Since the value of prefix length, n, is given, we can easily find these three pieces of information, as shown in Figure.



1. The number of addresses in the block is found as $N = 2^{32-n}$.
2. To find the first address, we keep the n leftmost bits and set the *(32 – n)* rightmost bits all to *0s*.
3. To find the last address, we keep the n leftmost bits and set the *(32 – n)* rightmost bits all to *1s*.

Example: A classless address is given as 167.199.170.82/27. We can find the above three pieces of information as follows. The number of addresses in the network is $2^{32-n} = 2^5 = 32$ addresses.

The first address can be found by keeping the first 27 bits and changing the rest of the bits to 0s.

Address: 167.199.170.82/27	10100111	11000111	10101010	01010010
First address: 167.199.170.64/27	10100111	11000111	10101010	01000000

The last address can be found by keeping the first 27 bits and changing the rest of the bits to 1s.

Address: 167.199.170.82/27	10100111	11000111	10101010	01011111
Last address: 167.199.170.95/27	10100111	11000111	10101010	01011111

Address Mask

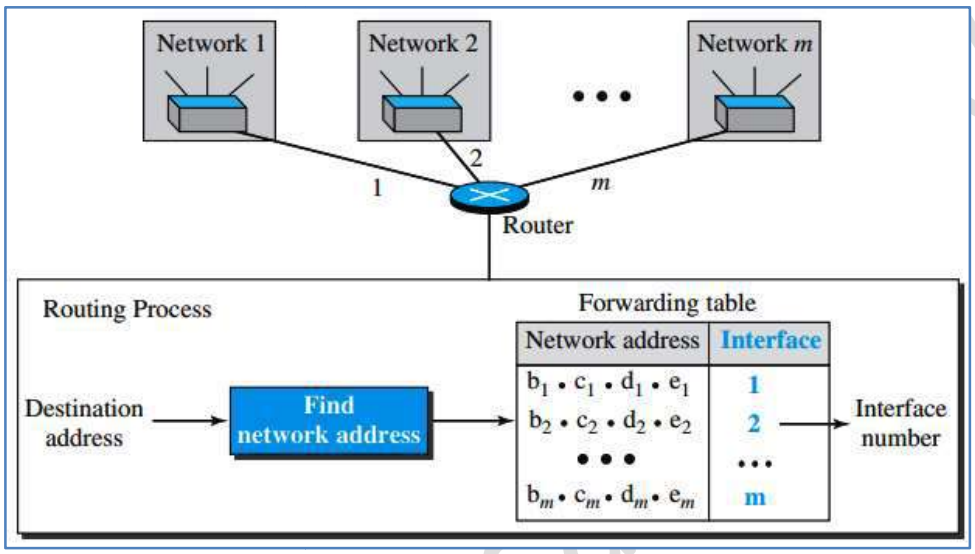
- Another way to find the first and last addresses in the block is to use the ***address mask***.
- The ***address mask is a 32-bit number in which the n leftmost bits are set to 1s and the rest of the bits (32 – n) are set to 0s***. A computer can easily find the address mask because it is the complement of $(2^{32-n} - 1)$.
- The reason for defining a mask in this way is that it can be used by a computer program to extract the information in a block, using the three bit-wise operations NOT, AND, and OR.
  1. The number of addresses in the **block N = NOT (mask) + 1.**
  2. The **first address** in the block = **(Any address in the block) AND (mask).**
  3. The last address in the block = **(Any address in the block) OR [(NOT (mask)].**

Example: We repeat previous example using the mask. The mask in dotted-decimal notation is 256.256.256.224. The AND, OR, and NOT operations can be applied to individual bytes.

Number of addresses in the block:	N = NOT (mask) + 1 = 0.0.0.31 + 1 = 32 addresses
First address:	First = (address) AND (mask) = 167.199.170.82
Last address:	Last = (address) OR (NOT mask) = 167.199.170.255

Network Address

- The above examples show that, given any address, we can find all information about the block.
- The ***first address, the network address, is particularly important because it is used in routing a packet to its destination network***. For the moment, let us assume that an internet is made of m networks and a router with m interfaces. When a packet arrives at the router from any source host, the router needs to know to which network the packet should be sent: from which interface the packet should be sent out. When the packet arrives at the network, it reaches its destination host using another strategy that we discuss later. Figure shows the idea.



- After the network address has been found, the router consults its forwarding table to find the corresponding interface from which the packet should be sent out. The network address is actually the identifier of the network; each network is identified by its network address.

Block Allocation

- The next issue in classless addressing is ***block allocation***. How are the blocks allocated?
- The ***ultimate responsibility of block allocation is given to a global authority called the Internet Corporation for Assigned Names and Numbers (ICANN).*** However, ICANN does not normally allocate addresses to individual Internet users. It assigns a large block of addresses to an ISP (or a larger organization that is considered an ISP in this case). For the proper operation of the CIDR, ***two restrictions need to be applied to the allocated block***.
  1. The number of requested addresses, N, needs to be a power of 2. The reason is that $N = 2^{32-n}$ or $n = 32 - \log_2 N$. If N is not a power of 2, we cannot have an integer value for n.
  2. The requested block needs to be allocated where there is an adequate number of contiguous addresses available in the address space. However, there is a restriction on choosing the first address in the block. The first address needs to be divisible by the number of addresses in the block. The reason is that the first address needs to be the prefix followed by (32 − n) number of 0s. The decimal value of the first address is then

$$\text{first address} = (\text{prefix in decimal}) \times 2^{32-n} = (\text{prefix in decimal}) \times N.$$

Example: An ISP has requested a block of 1000 addresses. Since 1000 is not a power of 2, 1024 addresses are granted. The prefix length is calculated as $n = 32 - \log_2 1024 = 22$. An available block, 18.14.12.0/22, is granted to the ISP. It can be seen that the first address in decimal is 302,910,464, which is divisible by 1024.

Prof. Pratheesh V

Subnetting

- More levels of hierarchy can be created using subnetting. An organization (or an ISP) that is granted a range of addresses may divide the range into several subranges and assign each subrange to a subnetwork (or subnet). Note that nothing stops the organization from creating more levels. A subnetwork can be divided into several sub-subnetworks. A sub-subnetwork can be divided into several sub-sub-subnetworks, and so on.

Designing Subnets

- The ***subnetworks in a network should be carefully designed to enable the routing of packets.*** We assume the total number of addresses granted to the organization is ***N***, the prefix length is ***n***, the assigned number of addresses to each subnetwork is N_{sub}, and the prefix length for each subnetwork is N_{sub}. Then the following steps need to be carefully followed to guarantee the proper operation of the subnetworks.
  1. The number of addresses in each subnetwork should be a power of 2.
  2. The prefix length for each subnetwork should be found using the following formula:

     $$n_{sub} = 32 - \log_2 N_{sub}$$

  3. The starting address in each subnetwork should be divisible by the number of addresses in that subnetwork. This can be achieved if we first assign addresses to larger subnetworks.

Finding Information about Each Subnetwork

- After designing the subnetworks, the information about each subnetwork, such as ***first and last address***, can be found using the process we described to find the information about each network in the Internet.

Address Aggregation

- One of the ***advantages of the CIDR strategy is address aggregation*** (sometimes called address summarization or route summarization).
- When blocks of addresses are combined to create a larger block, routing can be done based on the prefix of the larger block. ***ICANN (Internet Corporation for Assigned Names and Numbers)*** assigns a large block of addresses to an ISP. Each ISP in turn divides its assigned block into smaller subblocks and grants the subblocks to its customers.

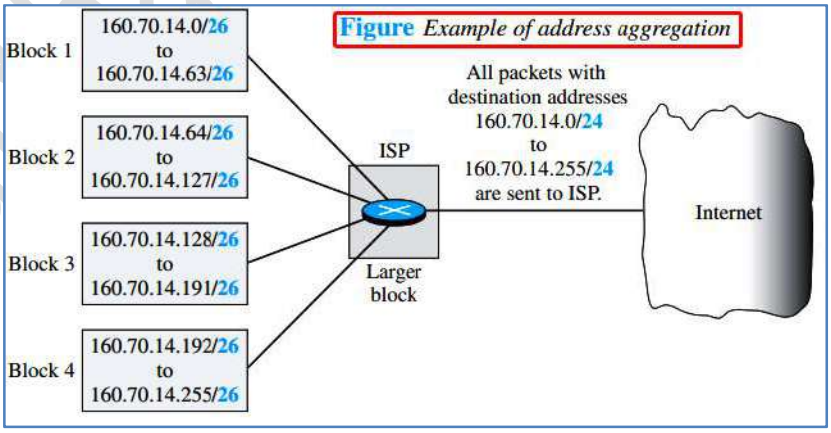


Figure *Example of address aggregation*

Special Addresses

- Five special addresses that are used for special purposes:
  1. this-host address
  2. limited-broadcast address
  3. loopback address
  4. private addresses
  5. multicast addresses.

1. This-host Address

- The only address in the block ***0.0.0.0/32*** is called the this-host address. It is used whenever a host needs to send an IP datagram but ***it does not know its own address to use as the source address***.

Prof. Parthasarathy P V

2. limited-broadcast address

- The only address in the block ***255.255.255.255/32*** is called the ***limited-broadcast address***. It is ***used whenever a router or a host needs to send a datagram to all devices in a network***. The routers in the network, however, block the packet having this address as the destination; ***the packet cannot travel outside the network.***

3. loopback address

- The block ***127.0.0.0/8*** is called the ***loopback address***. A packet with one of the addresses in this block as the ***destination address never leaves the host; it will remain in the host***. ***Any address in the block is used to test a piece of software in the machine.***
- For example, we can write a client and a server program in which one of the addresses in the block is used as the server address. We can test the programs using the same host to see if they work before running them on different computers.

4. private addresses

- Four blocks are assigned as private addresses: ***10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16***, and ***169.254.0.0/16***.

5. multicast addresses

- The block 224.0.0.0/4 is reserved for multicast addresses.

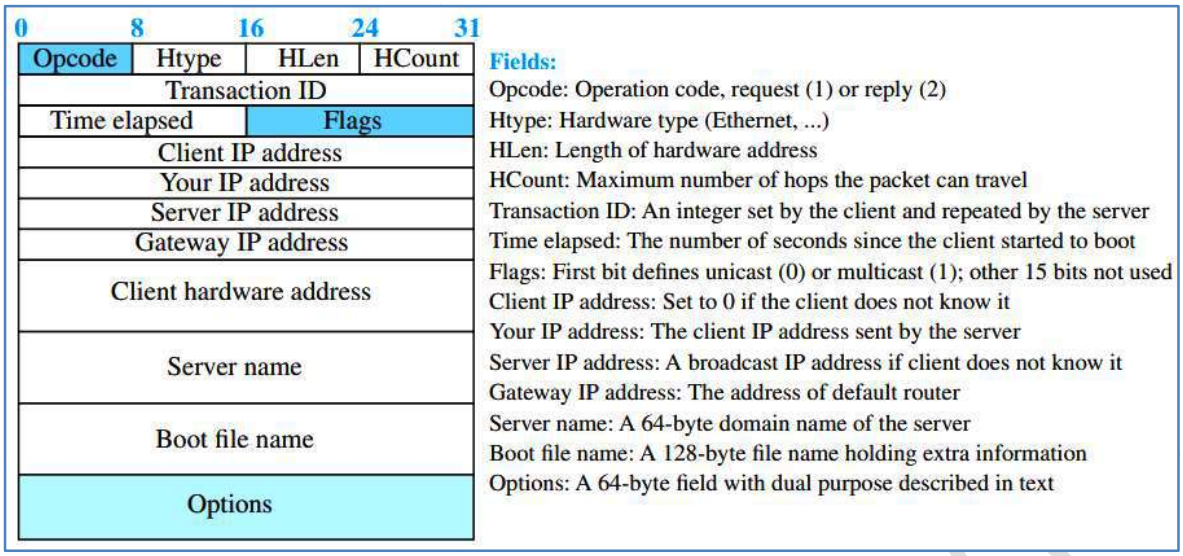
Dynamic Host Configuration Protocol (DHCP)

- We have seen that a large organization or an ISP can receive a block of addresses directly from ICANN and a small organization can receive a block of addresses from an ISP. After a block of addresses are assigned to an organization, the network administration can manually assign addresses to the individual hosts or routers.
- However, ***address assignment in an organization can be done automatically using the Dynamic Host Configuration Protocol (DHCP).*** DHCP is an application-layer program, using the client-server paradigm, that actually helps TCP/IP at the network layer.
- DHCP has found such widespread use in the Internet that it is often called ***a plug-and-play protocol***. In can be used in many situations.
  - A network manager can configure DHCP to assign permanent IP addresses to the host and routers. DHCP can also be configured to provide temporary, on demand, IP addresses to hosts.
  - The second capability can provide a temporary IP address to a traveller to connect her laptop to the Internet while she is staying in the hotel. It also allows an ISP with 1000 granted addresses to provide services to 4000 households, assuming not more than one-forth of customers use the Internet at the same time.
- In addition to its IP address, a computer also needs to know the network prefix (or address mask). Four pieces of information are normally needed: ***the computer address, the prefix, the address of a router, and the IP address of a name server***. DHCP can be used to provide these pieces of information to the host.

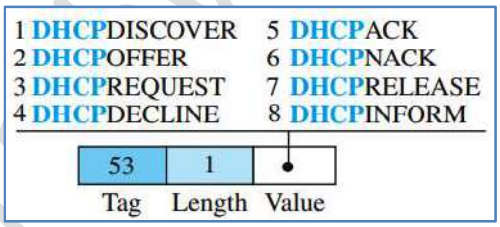
DHCP Message Format

- DHCP is a client-server protocol in which the client sends a request message and the server returns a response message.
- The general format of the DHCP message in shown in Figure.
- Most of the fields are explained in the figure which plays a very important role in DHCP.
- The 64-byte option field has a dual purpose. It can carry either additional information or some specific vendor information. The server uses a number, called a ***magic cookie***, in the format of an IP address with the value of ***99.130.83.99***.

Prof. Parthasarathy P V

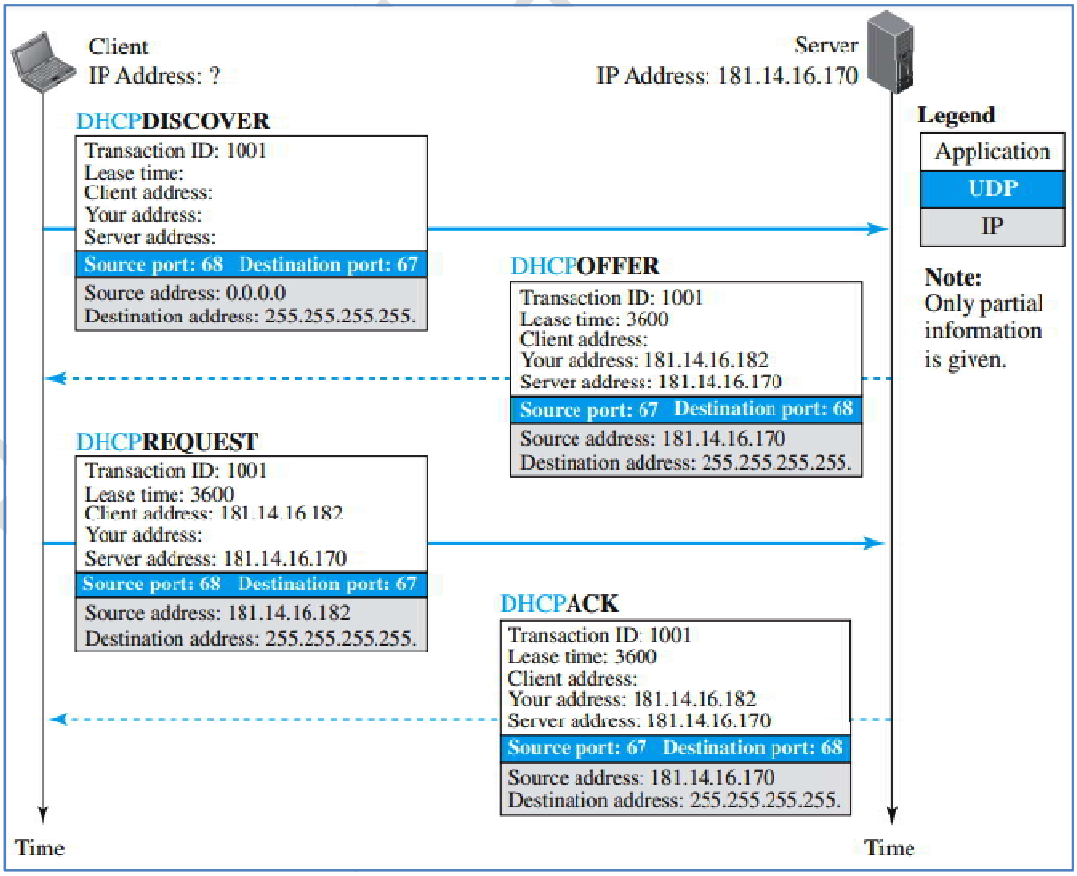


- When the client finishes reading the message, it looks for this magic cookie. If present, the next 60 bytes are options.
- An ***option*** is composed of ***three fields: a 1-byte tag field, a 1-byte length field, and a variable-length value field***. There are several tag fields that are mostly used by vendors. If the tag field is 53, the value field defines one of the 8 message types shown in Figure. We show how these message types are used by DHCP.



DHCP Operation

- Figure shows a simple scenario.



- The ***joining host creates*** a ***DHCPDISCOVER*** message in which only the ***transactionID*** field is set to a random number. This message is encapsulated in a ***UDP user datagram with the source port set to 68 and the destination port set to 67***. The user datagram is encapsulated in an IP datagram with the source address set to 0.0.0.0 ("this host") and the destination address set to 255.255.255.255 (broadcast address). ***The reason is that the joining host knows neither its own address nor the server address.***
- The DHCP server or servers (if more than one) responds with a ***DHCPOFFER*** message in which the ***your address field defines the offered IP address for the joining host and the server address field*** includes the IP address of the server. The user datagram in turn is encapsulated in a datagram with the server address as the source IP address, but the destination address is a broadcast address, in which the server allows other DHCP servers to receive the offer and give a better offer if they can.
- The ***joining host receives one or more offers and selects the best of them***. The joining host then sends a ***DHCPREQUEST*** message to the server that has given the best offer. The fields with known value are set. The message is encapsulated in a user datagram with port numbers as the first message.
- Finally, the ***selected server responds with a DHCPACK message to the client if the offered IP address is valid***. If the server cannot keep its offer, the server sends a DHCPNACK message and the client needs to repeat the process. This message is also broadcast to let other servers know that the request is accepted or rejected.

Two Well-Known Ports

- We said that the DHCP uses two well-known ports (68 and 67) instead of ***one well-known*** and ***one ephemeral (short-lived)***. The reason for ***choosing the well-known port 68 instead of an ephemeral port*** for the client is that the response from the server to the client is broadcast.
- The curious reader may ask what happens if two DHCP clients are running at the same time. This can happen after a power failure and power restoration. In this case the messages can be distinguished by the value of the transaction ID, which separates each response from the other.

Using FTP

- The server does not send all of the information that a client may need for joining the network.
- In the ***DHCPACK message, the server defines the pathname of a file in which the client can find complete information such as the address of the DNS server***.
- The ***client can then use a file transfer protocol to obtain the rest of the needed information***.

Error Control

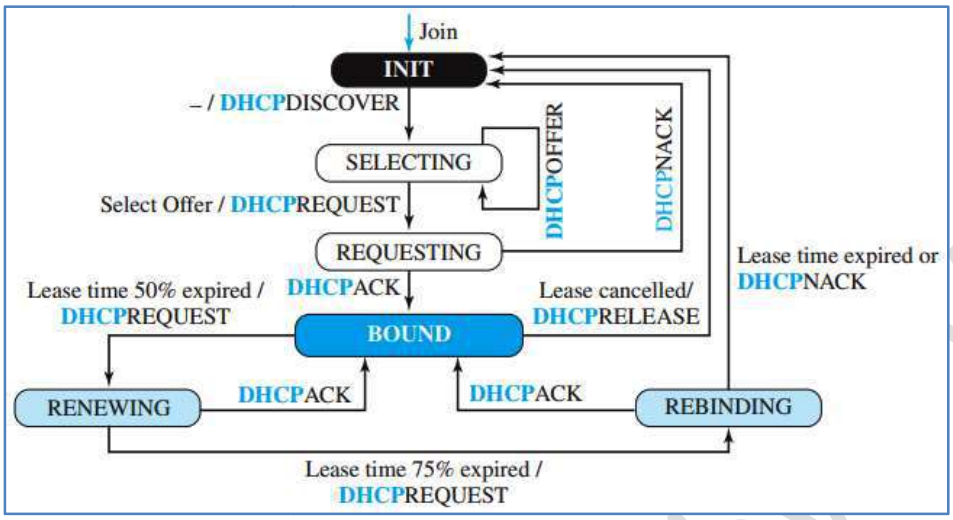
- DHCP uses the service of UDP, which is not reliable. To provide error control, DHCP uses two strategies.
  - ***First***, DHCP requires that UDP use the checksum. The use of the checksum in UDP is optional.
  - ***Second***, the DHCP client ***uses timers and a retransmission policy*** if it does not receive the DHCP reply to a request.

Transition States

- To provide dynamic address allocation, the DHCP client acts as a state machine that performs transitions from one state to another depending on the messages it receives or sends. Figure shows the transition diagram with the main states.
- When the DHCP client ***first starts, it is in the INIT state (initializing state).*** The client broadcasts a discover message.
- When ***it receives an offer, the client goes to the SELECTING state***. While it is there, it may receive more offers.
- After ***it selects an offer, it sends a request message and goes to the REQUESTING state***. If an ***ACK arrives while the client is in this state, it goes to the BOUND state and uses the IP address***.

Prof. Parthasarathy P V

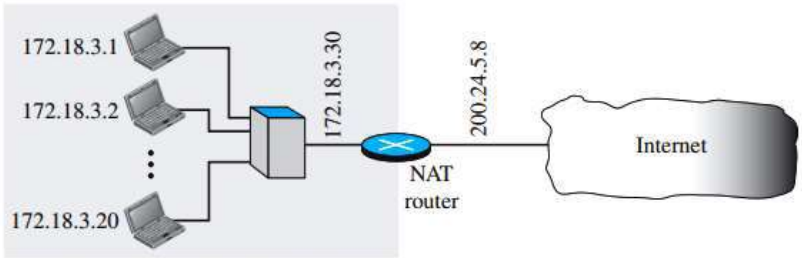
- When the ***lease is 50 percent expired, the client tries to renew it by moving to the RENEWING state***. If the server renews the lease, the client moves to the BOUND state again.



- If the ***lease is not renewed and the lease time is 75 percent expired, the client moves to the REBINDING state***.
- If the ***server agrees with the lease*** (ACK message arrives), ***the client moves to the BOUND state and continues using the IP address***; otherwise, the client moves to the INIT state and requests another IP address. Note that the client can use the IP address only when it is in the BOUND, RENEWING, or REBINDING state.
- The above procedure requires that the client uses ***three timers***:
  1. Renewal timer (set to 50 percent of the lease time)
  2. Rebinding timer (set to 75 percent of the lease time)
  3. Expiration timer (set to the lease time).

Network Address Resolution (NAT)

- A ***technology that can provide the mapping between the private and universal addresses***, and ***at the same time support virtual private networks is Network Address Translation (NAT)***.
- The technology allows a site to use a set of private addresses for internal communication and a set of global Internet addresses (at least one) for communication with the rest of the world. The site must have only one connection to the global Internet through a NAT-capable router that runs NAT software. ***Figure shows a simple implementation of NAT***.



Site using private addresses

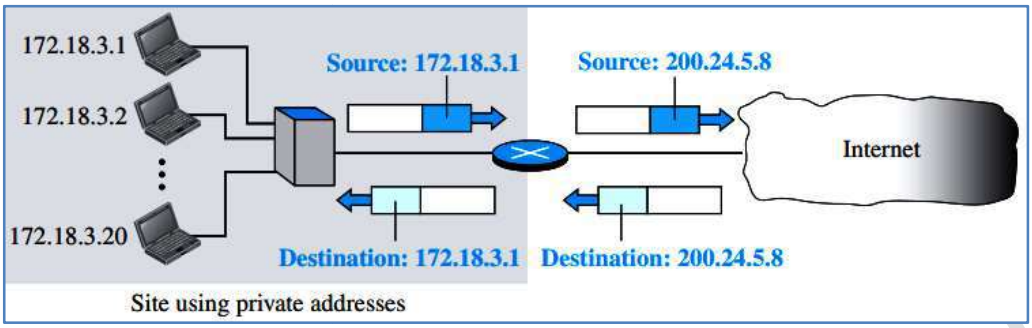
- As the figure shows, the private network uses private addresses. The router that connects the network to the global address uses one private address and one global address. ***The private network is invisible to the rest of the Internet***; ***the rest of the Internet sees only the NAT router with the address 200.24.5.8.***

Address Translation

- All of the outgoing packets go through the NAT router, which replaces the source address in the packet with the global NAT address.
- All incoming packets also pass through the NAT router, which replaces the destination address in the packet (the NAT router global address) with the appropriate private address.

Prof. Parthasarathy V

- Figure shows an example of address translation.



Translation Table

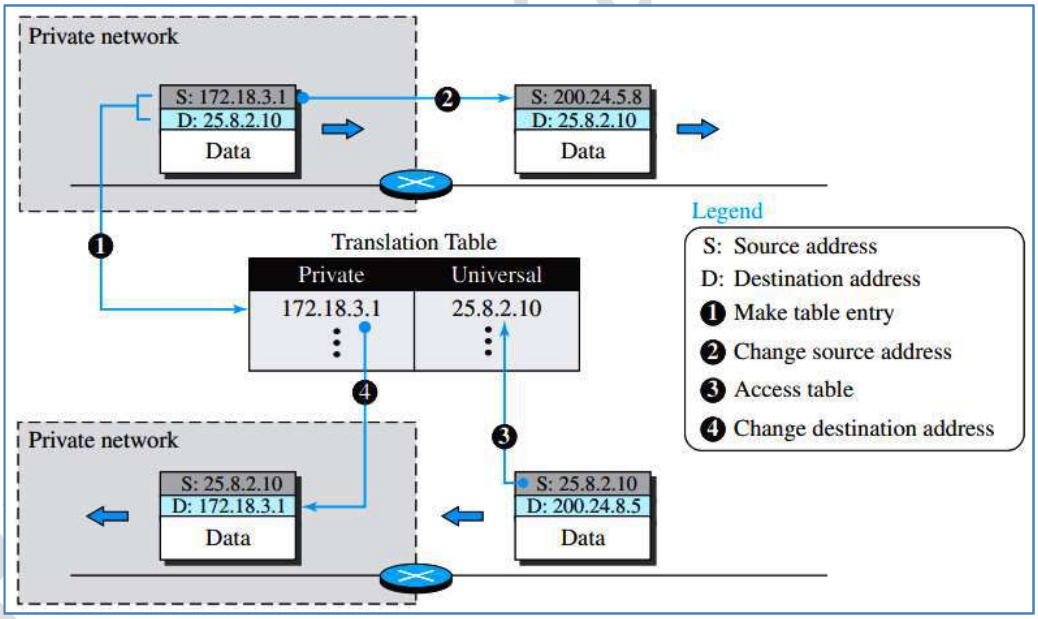
- The reader may have noticed that translating the source addresses for an outgoing packet is straightforward.

But how does the NAT router know the destination address for a packet coming from the Internet?

- There may be tens or hundreds of private IP addresses, each belonging to one specific host. The problem is solved if the NAT router has a translation table.

Using One IP Address

- In its simplest form, a translation table has only two columns: the private address and the external address (destination address of the packet). When the router translates the source address of the outgoing packet, it also makes note of the destination address— where the packet is going.
- When the response comes back from the destination, the router uses the source address of the packet (as the external address) to find the private address of the packet. Figure shows the idea.



- In this strategy, communication must always be initiated by the private network. The NAT mechanism described requires that the private network start the communication.

Using a Pool of IP Addresses

- The use of only one global address by the NAT router allows only one private-network host to access a given external host. To remove this restriction, the ***NAT router can use a pool of global addresses***. For example***, instead of using only one global address*** (200.24.5.8), the ***NAT router can use four addresses*** (200.24.5.8, 200.24.5.9, 200.24.5.10, and 200.24.5.11).

- In this case, four private-network hosts can communicate with the same external host at the same time because each pair of addresses defines a separate connection.

Using Both IP Addresses and Port Addresses

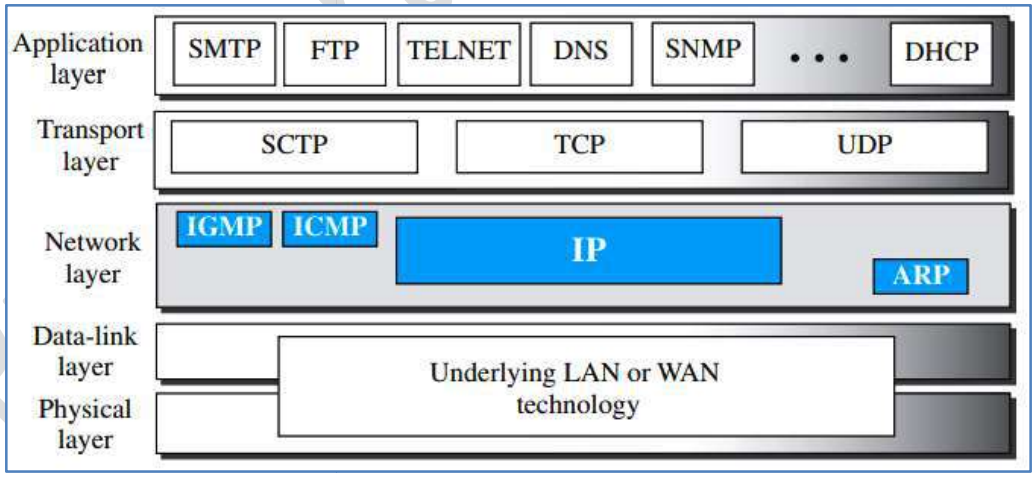
- To ***allow a many-to-many relationship between private-network hosts and external server programs***, we ***need more information in the translation table***.
- For example, suppose two hosts inside a private network with addresses 172.18.3.1 and 172.18.3.2 need to access the HTTP server on external host 25.8.3.2. ***If the translation table has five columns, instead of two, that include the source and destination port addresses and the transport-layer protocol, the uncertainty is eliminated***. Table shows an example of such a table.

Private address	*Private port*	*External address*	*External port*	*Transport protocol*
172.18.3.1	1400	25.8.3.2	80	TCP
172.18.3.2	1401	25.8.3.2	80	TCP
⋮	⋮	⋮	⋮	⋮

IPv4 Datagram

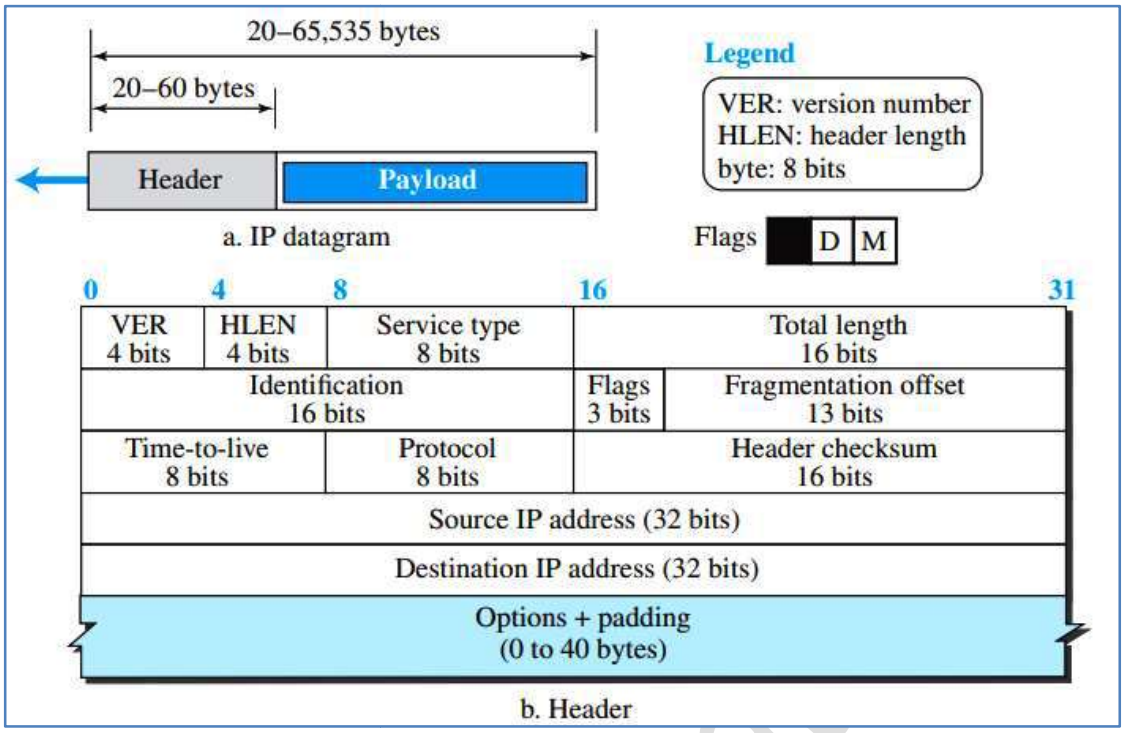
INTERNET PROTOCOL (IP) - Introduction

- The network layer in version 4 can be thought of as one main protocol and three auxiliary ones.
- The main protocol, ***Internet Protocol version 4 (IPv4),*** is ***responsible for packetizing, forwarding, and delivery of a packet*** at the network layer.
- The ***Internet Control Message Protocol version 4 (ICMPv4) helps IPv4 to handle some errors*** that may occur in the network-layer delivery.
- The ***Internet Group Management Protocol (IGMP) is used to help IPv4 in multicasting***.
- The ***Address Resolution Protocol (ARP) is used to glue the network and data-link layers in mapping network-layer addresses to link-layer addresses***. Figure shows the positions of these four protocols in the TCP/IP protocol suite.



Datagram Format

- The following Figure shows the IPv4 datagram format.
- A ***datagram is a variable-length packet consisting of two parts***: ***header and payload (data).***
- The ***header is 20 to 60 bytes in length*** and ***contains information essential to routing and delivery***. It is customary in TCP/IP to show the header in 4-byte sections.



- ***Version Number***: The 4-bit version number (VER) field defines the version of the IPv4 protocol, which, obviously, has the value of 4.
- ***Header Length***: The 4-bit header length (HLEN) field defines the total length of the datagram header in 4-byte words. The IPv4 datagram has a variable-length header. When a device receives a datagram, it needs to know when the header stops and the data, which is encapsulated in the packet, starts. However, to make the value of the header length (number of bytes) fit in a 4-bit header length, the total length of the header is calculated as 4-byte words. The total length is divided by 4 and the value is inserted in the field. The receiver needs to multiply the value of this field by 4 to find the total length.
- ***Service Type***: In the original design of the IP header, this field was referred to as type of service (TOS), which defined how the datagram should be handled. In the late 1990s, IETF redefined the field to provide differentiated services (DiffServ).
- ***Total Length:*** This 16-bit field defines the total length (header plus data) of the IP datagram in bytes. A 16-bit number can define a total length of up to 65,535 (when all bits are 1s). ***This field helps the receiving device to know when the packet has completely arrived***. To find the length of the data coming from the upper layer, subtract the header length from the total length. The header length can be found by multiplying the value in the HLEN field by 4.

Length of data = total length – (HLEN) × 4

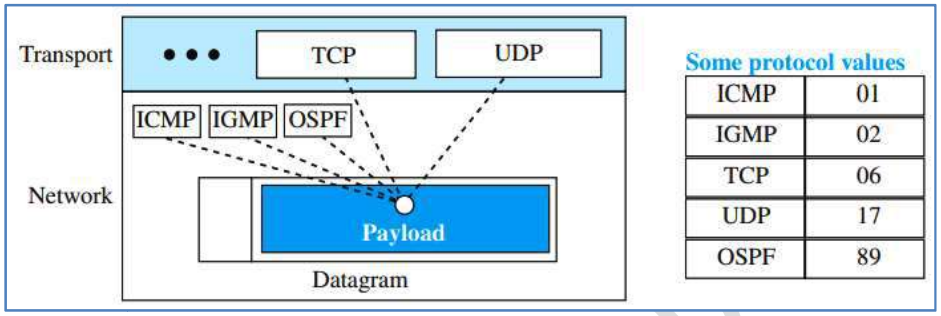
Though a size of 65,535 bytes might seem large, the size of the IPv4 datagram may increase in the near future as the underlying technologies allow even more throughput (greater bandwidth).

- ***Identification, Flags, and Fragmentation Offset:*** These three fields are related to the fragmentation of the IP datagram when the size of the datagram is larger than the underlying network can carry. We discuss the contents and importance of these fields when we talk about fragmentation in the next section.
- ***Time-to-live:*** Due to some malfunctioning of routing protocols a datagram may be circulating in the Internet, visiting some networks over and over without reaching the destination. This may create extra traffic in the Internet. ***The time-to-live (TTL) field is used to control the maximum number of hops (routers) visited by the datagram***. When a source host sends the datagram, it stores a number in this field. This value is approximately two times the maximum number of

Prof. Parimala

routers between any two hosts. Each router that processes the datagram decrements this number by one. If this value, after being decremented, is zero, the router discards the datagram.

- ***Protocol:*** A datagram, for example, can carry a packet belonging to any transport-layer protocol such as UDP or TCP. ***A datagram can also carry a packet from other protocols that directly use the service of the IP***, such as some routing protocols or some auxiliary protocols. The Internet authority has given any protocol that uses the service of IP a unique 8-bit number which is inserted in the protocol field. When the payload is encapsulated in a datagram at the source IP, the corresponding protocol number is inserted in this field; when the datagram arrives at the destination, the value of this field helps to define to which protocol the payload should be delivered.



- ***Header checksum***. IP is not a reliable protocol; it does not check whether the payload carried by a datagram is corrupted during the transmission. IP puts the burden of error checking of the payload on the protocol that owns the payload, such as UDP or TCP.
  - ***The datagram header, however, is added by IP, and its error-checking is the responsibility of IP. Errors in the IP header can be a disaster***.
  - For example, if the destination IP address is corrupted, the packet can be delivered to the wrong host. If the protocol field is corrupted, the payload may be delivered to the wrong protocol. If the fields related to the fragmentation are corrupted, the datagram cannot be reassembled correctly at the destination, and so on. For these reasons, IP adds a header checksum field to check the header, but not the payload.
  - ***Checksum in the Internet normally uses a 16-bit field, which is the complement of the sum of other fields calculated using 1s complement arithmetic***.
- ***Source and Destination Addresses:*** These 32-bit source and destination address fields define the IP address of the source and destination respectively. The source host should know its IP address.
  - ***The destination IP address is either known by the protocol that uses the service of IP or is provided by the DNS***. Note that the value of these fields must remain unchanged during the time the IP datagram travels from the source host to the destination host.
- ***Options***: A datagram header can have up to 40 bytes of options. Options can be used for network testing and debugging. Although options are not a required part of the IP header, option processing is required of the IP software.
  - This ***means that all implementations must be able to handle options if they are present in the header***. The existence of options in a header creates some burden on the datagram handling; some options can be changed by routers, which forces each router to recalculate the header checksum.
- ***Payload:*** Payload, or data, is the main reason for creating a datagram. Payload is the packet coming from other protocols that use the service of IP. Comparing a datagram to a postal package, payload is the content of the package; the header is only the information written on the package.

Prof Parthasarathy P

Chapter 22: Next Generation IP: IPv6 Datagram.

IPv6 ADDRESSING

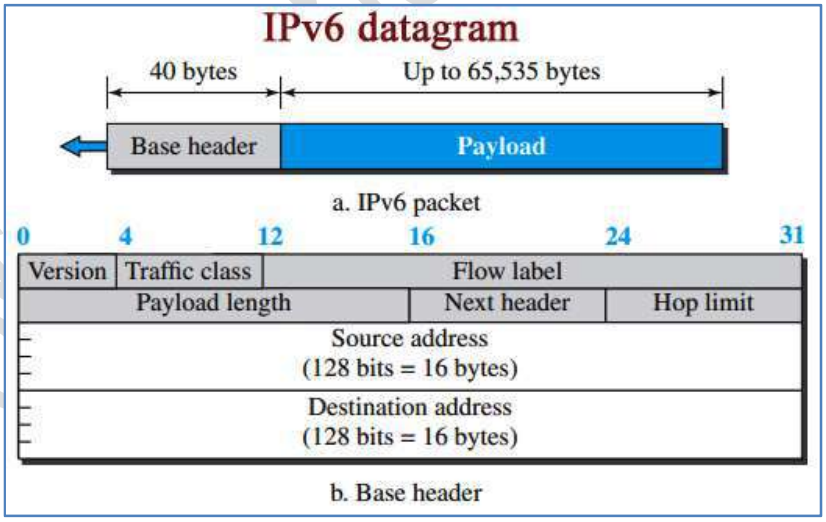
- The main reason for migration from IPv4 to IPv6 is the small size of the address space in IPv4.
- An ***IPv6 address is 128 bits or 16 bytes (octets) long***, four times the address length in IPv4.

THE IPv6 PROTOCOL

- The change of the IPv6 address size requires the change in the IPv4 packet format. The designer of IPv6 shows other changes implemented in the protocol in addition to changing address size and format.
  - ***Better header format***. IPv6 uses a new header format in which options are separated from the base header and inserted, when needed, between the base header and the data. This simplifies and speeds up the routing process because most of the options do not need to be checked by routers.
  - ***New options***. IPv6 has new options to allow for additional functionalities.
  - ***Allowance for extension***. IPv6 is designed to allow the extension of the protocol if required by new technologies or applications.
  - ***Support for resource allocation***. In IPv6, the type-of-service field has been removed, but two new fields, traffic class and flow label, have been added to enable the source to request special handling of the packet. This mechanism can be used to support traffic such as real-time audio and video.
  - ***Support for more security***. The encryption and authentication options in IPv6 provide confidentiality and integrity of the packet.

Packet Format

- The IPv6 packet is shown in Figure. Each packet is composed of a base header followed by the payload. The base header occupies 40 bytes, whereas payload can be up to 65,535 bytes of information.

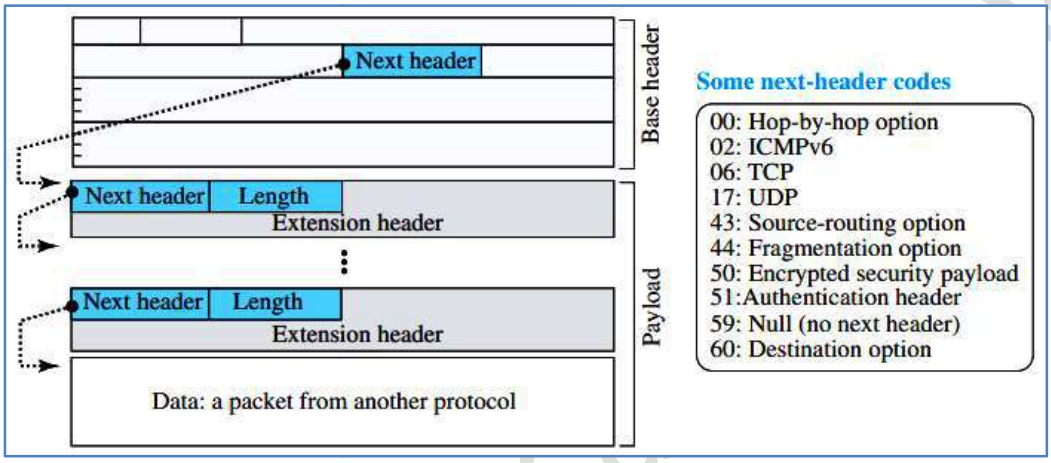


  - ***Version:*** The 4-bit version field defines the version number of the IP. For IPv6, the value is 6.
  - ***Traffic class:*** The 8-bit traffic class field is used to distinguish different payloads with different delivery requirements. It replaces the type-of-service field in IPv4.
  - ***Flow label:*** The flow label is a 20-bit field that is designed to provide special handling for a particular flow of data. We will discuss this field later.
  - ***Payload length***: The 2-byte payload length field defines the length of the IP datagram excluding the header. Note that IPv4 defines two fields related to the length: header length and total

Prof. Basavaraj P V

length. In IPv6, the length of the base header is fixed (40 bytes); only the length of the payload needs to be defined.

- ***Next header***: The next header is an 8-bit field defining the type of the first extension header (if present) or the type of the data that follows the base header in the datagram. This field is similar to the protocol field in IPv4, but we talk more about it when we discuss the payload.
- ***Hop limit:*** The 8-bit hop limit field serves the same purpose as the TTL field in IPv4.
- ***Source and destination addresses:*** The source address field is a 16-byte (128-bit) Internet address that identifies the original source of the datagram. The destination address field is a 16-byte (128-bit) Internet address that identifies the destination of the datagram.
- ***Payload:*** Compared to IPv4, the payload field in IPv6 has a different format and meaning, as shown in Figure.



Concept of Flow and Priority in IPv6

- In version 6, the flow label has been directly added to the format of the IPv6 datagram to allow us to use IPv6 as a connection-oriented protocol.
- To a router, a flow is a sequence of packets that share the same characteristics, such as traveling the same path, using the same resources, having the same kind of security.
- A router that supports the handling of flow labels has a flow label table. The table has an entry for each active flow label; each entry defines the services required by the corresponding flow label.
- When the router receives a packet, it consults its flow label table to find the corresponding entry for the flow label value defined in the packet. It then provides the packet with the services mentioned in the entry.
- A flow label can be used to speed up the processing of a packet by a router. When a router receives a packet, instead of consulting the forwarding table and going through a routing algorithm to define the address of the next hop, it can easily look in a flow label table for the next hop.
- A flow label can be used to support the transmission of real-time audio and video - in digital form, requires resources such as high bandwidth, large buffers, long processing time.
- A process can make a reservation for these resources beforehand to guarantee that real-time data will not be delayed due to a lack of resources. The use of real-time data and the reservation of these resources require other protocols such as ***Real-Time Transport Protocol (RTP)*** and ***Resource Reservation Protocol (RSVP)*** in addition to IPv6.

Fragmentation and Reassembly

- IPv6 ***datagrams can be fragmented only by the source, not by the routers; the reassembly takes place at the destination***. The fragmentation of packets at routers is not allowed to speed up the processing of packets in the router. In IPv6, **the source can check the size of the packet and make the decision to fragment the packet or not.**

Prof. Pallavi V

- ***When a router receives the packet, it can check the size of the packet and drop it if the size is larger than allowed by the MTU of the network ahead.***
- The router then sends a packet-too-big ***ICMPv6 error message to inform the source***.

THE IPv6 PROTOCOL - Extension Header

- An IPv6 packet is made of a base header and some extension headers. The length of the base header is fixed at 40 bytes.
- To give more functionality to the IP datagram, the base header can be followed by up to six extension headers.
  1. ***Hop-by-hop option*** - is used when the source needs to pass information to all routers visited by the datagram.
  2. ***Source routing -*** combines the concepts of the strict source route and the loose source route options of IPv4.
  3. ***Fragmentation -*** only the original source can fragment. A source must use a ***Path MTU Discovery technique*** to find the smallest MTU supported by any network on the path. The source then fragments using this knowledge.
  4. ***Authentication -*** it ***validates the message sender*** and ***ensures the integrity of data***.
  5. ***Encrypted Security Payload*** - that provides confidentiality and guards against eavesdropping.
  6. ***Destination option -*** is used when the source needs to pass information to the destination only. Intermediate routers are not permitted access to this information.

Difference Between IPv4 and IPv6

#	IPv4	IPv6
1.	32-bit address length	128-bit address length
2.	Supports Manual and DHCP address configuration	Supports Auto and renumbering address configuration
3.	End to end, connection integrity is Unachievable	End-to-end, connection integrity is Achievable
4.	It can generate 4.29×10^{9} address space	The address space is quite large it can produce 3.4×10^{38} address space
5.	The Security feature is dependent on the application	IPSEC is an inbuilt security feature in the IPv6 protocol
6.	Address representation is in decimal	Address representation is in hexadecimal
7.	Fragmentation performed by Sender and forwarding routers	Fragmentation is performed only by the sender
8.	Packet flow identification is not available	Available, uses the flow label field in the header
9.	Checksum field is available	Checksum field is not available
10.	It has a broadcast Message Transmission Scheme	Multicast and Anycast message transmission scheme is available
11.	Encryption and Authentication facility not provided	Encryption and Authentication are provided
12.	Has a header of 20-60 bytes.	Has a header of 40 bytes fixed
13.	IPv4 can be converted to IPv6	Not all IPv6 can be converted to IPv4
14.	Consists 4 fields separated by addresses dot (.)	Consists of 8 fields, separated by a colon (:)
15.	IP addresses: five different classes. Class A, Class B, Class C, Class D, Class E.	Does not have any classes of the IP address.
16.	Supports VLSM(Variable Length subnet mask).	Does not support VLSM.
17.	Example: 66.94.29.13	Example: 2001:0000:3238:DFE1:0063:0000:0000:FEFB

Prof. Parthasarathy P V

Chapter 20: Unicast Routing

Introduction to Routing Algorithms, Unicast Routing Protocols: DVR, LSR, PVR, Unicast Routing protocols: RIP, OSPF, BGP

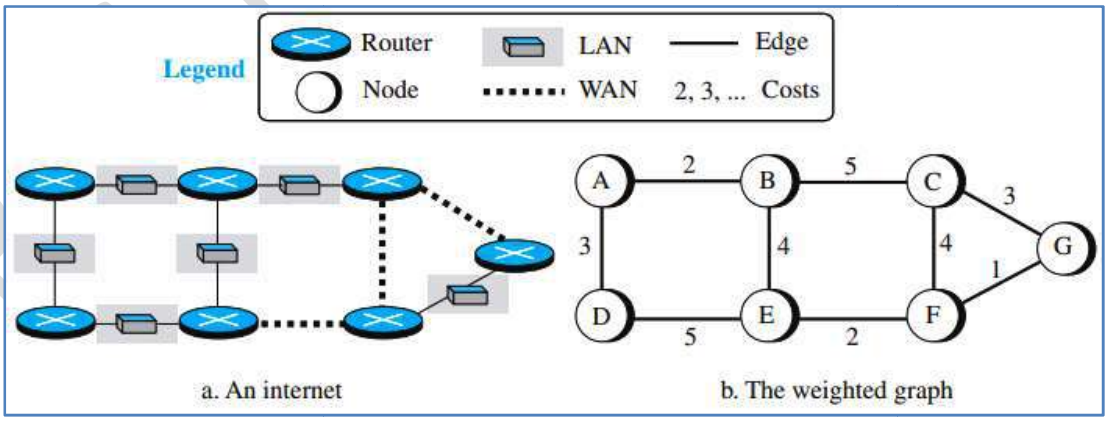
- In an internet, the goal of the network layer is to deliver a datagram from its source to its destination or destinations.
- If a ***datagram is destined for only one destination*** (one-to-one delivery), we have ***unicast routing***.
- If the ***datagram is destined for several destinations*** (one-to-many delivery), we have ***multicast routing***.

INTRODUCTION

- In unicast routing, a packet is routed, hop by hop, from its source to its destination by the help of forwarding tables.
- The ***source host needs no forwarding table*** because it delivers its packet to the default router in its local network. The ***destination host needs no forwarding table*** either because it receives the packet from its default router in its local network.
- This means that ***only the routers that fasten together the networks in the internet need forwarding tables***. With the above explanation, routing a packet from its source to its destination means routing the packet from a ***source router*** (the default router of the source host) to a ***destination router*** (the router connected to the destination network).
- There are several routes that a packet can travel from the source to the destination; what must be determined is which route the packet should take.

An Internet as a Graph

- A graph in computer science is a set of nodes and edges (lines) that connect the nodes. To model an internet as a graph, we can think of each router as a node and each network between a pair of routers as an edge.
- An internet is modeled as a weighted graph, in which each edge is associated with a cost. If a weighted graph is used to represent a geographical area, the nodes can be cities and the edges can be roads connecting the cities; the weights, in this case, are distances between cities.
- In routing, however, the cost of an edge has a different interpretation in different routing protocols. If there is no edge between the nodes, the cost is infinity. Figure shows how an internet can be modeled as a graph.

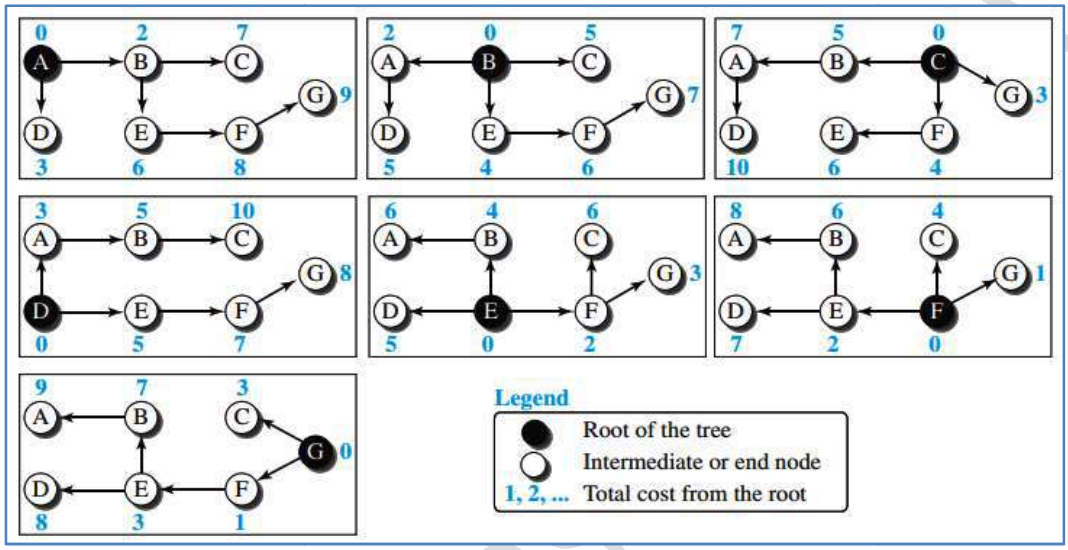


Least-Cost Routing

- The source router chooses a route to the destination router in such a way that the total cost for the route is the least cost among all possible routes.
- In Figure, the best route between A and E is A-B-E, with the cost of 6. Each router needs to find the least-cost route between itself, all the other routers to be able to route a packet using this criteria.

Least-Cost Trees

- If there are N routers in an internet, there are (N − 1) least-cost paths from each router to any other router. This means we need N × (N − 1) least-cost paths for the whole internet. If we have only 10 routers in an internet, we need 90 least-cost paths. ***A better way to see all of these paths is to combine them in a least-cost tree***.
- ***A least-cost tree is a tree with the source router as the root that spans the whole graph (visits all other nodes) and in which the path between the root and any other node is the shortest.*** In this way, we ***can have only one shortest-path tree for each node***; we have N least-cost trees for the whole internet. The following Figure shows the seven least-cost trees for the internet in Figure shown above.



- The least-cost trees for a weighted graph can have several properties if they are created using consistent criteria.
  1. The least-cost route from X to Y in X's tree is the inverse of the least-cost route from Y to X in Y's tree; the cost in both directions is the same. For example, in Figure, the route from A to F in A's tree is (A → B → E → F), but the route from F to A in F's tree is (F → E → B → A), which is the inverse of the first route. The cost is 8 in each case.
  2. Instead of travelling from X to Z using X's tree, we can travel from X to Y using X's tree and continue from Y to Z using Y's tree. For example, in Figure, we can go from A to G in A's tree using the route (A → B → E → F → G). We can also go from A to E in A's tree (A → B → E) and then continue in E's tree using the route (E → F → G). The combination of the two routes in the second case is the same route as in the first case. The cost in the first case is 9; the cost in the second case is also 9 (6 + 3).

ROUTING ALGORITHMS

Distance-Vector Routing (DVR) - Link-State Routing (LSR) - Path-Vector Routing (PVR)

Distance-Vector Routing (DVR)

- The ***distance-vector (DV) routing uses to find the best route***.
- In distance-vector routing, the first thing ***each node creates is its own least-cost tree with the rudimentary information it has about its immediate neighbors***. The ***incomplete trees are exchanged between immediate neighbors to make the trees more and more complete and to represent the whole internet***.

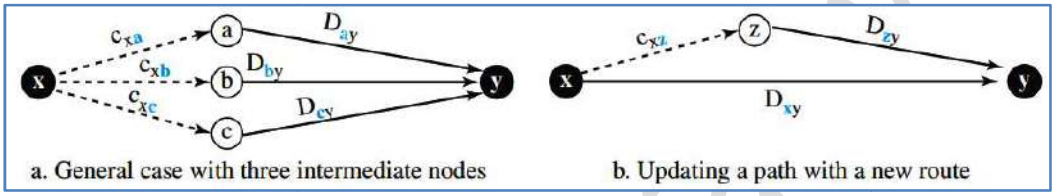
- We can say that in distance-vector routing, a router continuously tells all of its neighbors what it knows about the whole internet (although the knowledge can be incomplete).

Bellman-Ford Equation

- The heart of distance-vector routing is the famous Bellman-Ford equation. This equation is used to find the least cost (shortest distance) between a source node, x, and a destination node, y, through some intermediary nodes (a, b, c, . . .) when the costs between the source and the intermediary nodes and the least costs between the intermediary nodes and the destination are given.
- The following shows the general case in which D_{ij} is the shortest distance and c_{ij} is the cost between nodes i and j.

$$D_{xy} = \min\{(c_{xa} + D_{ay}), (c_{xb} + D_{by}), (c_{xc} + D_{cy}), \ldots\}$$

- In distance-vector routing, normally we want to update an existing least cost with a least cost through an intermediary node, such as z, if the latter is shorter. In this case, the equation becomes simpler, as shown below: $\mathbf{D_{xy} = \min\{D_{xy}, (c_{xz} + D_{zy})\}}$.
- Figure shows the idea graphically for both cases.

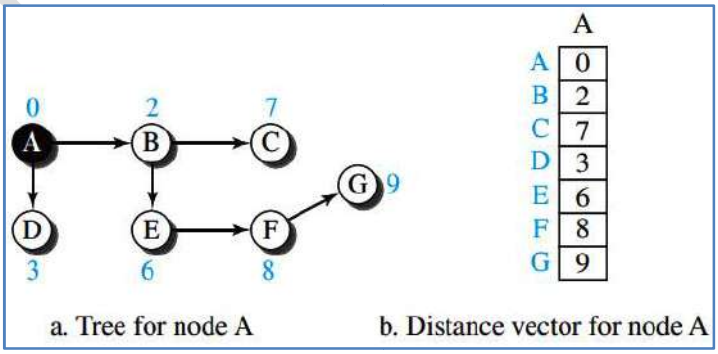


a. General case with three intermediate nodes b. Updating a path with a new route

- We can say that the ***Bellman-Ford equation enables us to build a new least-cost path from previously established least-cost paths***. In Figure, we can think of (a→y), (b→y), and (c→y) as previously established least-cost paths and (x→y) as the new least-cost path.
- We can even think of this equation as the builder of a new least-cost tree from previously established least-cost trees if we use the equation repeatedly.

Distance Vectors

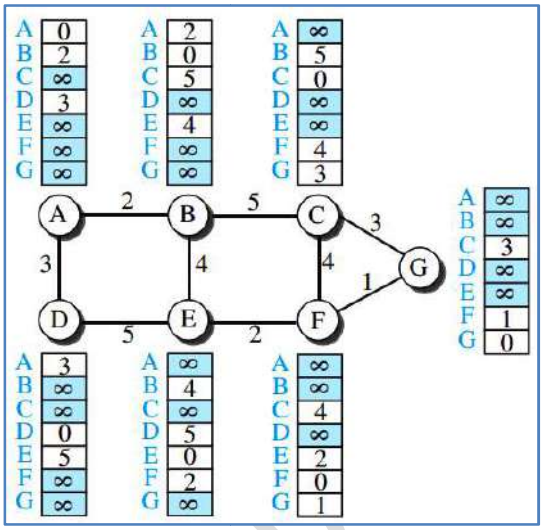
- The concept of a distance vector is the rationale for the name distance-vector routing. A ***least-cost tree is a combination of least-cost paths from the root of the tree to all destinations***. These paths are graphically glued together to form the tree. ***Distance-vector routing unglues these paths and creates a distance vector, a one-dimensional array to represent the tree***. Figure shows the tree for node A in the internet and the corresponding distance vector.



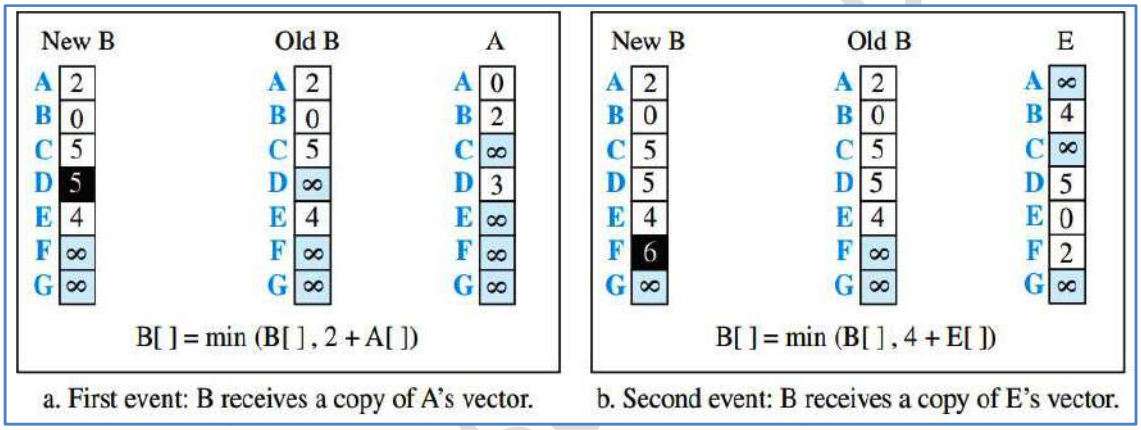
a. Tree for node A b. Distance vector for node A

- Note that the name of the distance vector defines the root, the indexes define the destinations, and the value of each cell defines the least cost from the root to the destination. A ***distance vector does not give the path to the destinations as the least-cost tree does; it gives only the least costs to the destinations***.
- Each node in an internet, when it is booted, creates a very simple distance vector with the minimum information the node can obtain from its neighborhood. The node sends some greeting messages out of its interfaces and discovers the identity of the immediate neighbors and the distance between itself and each neighbor.
- It then makes a simple distance vector by inserting the discovered distances in the corresponding cells and leaves the value of other cells as infinity.
- When we ***know only one distance between two nodes, it is the least cost***. Figure shows all distance vectors for our internet.

Prof. Parthasarathy P V



- These rudimentary vectors cannot help the internet to effectively forward a packet.
- For ***example, node A thinks that it is not connected to node G because the corresponding cell shows the least cost of infinity***. To ***improve these vectors, the nodes in the internet need to help each other by exchanging information***. After each node has created its vector, it sends a copy of the vector to all its immediate neighbors. After a node receives a distance vector from a neighbor, it updates its distance vector using the Bellman-Ford equation (second case).
- The figure shows two asynchronous events, happening one after another with some time in between. In the first event, node A has sent its vector to node B. Node B updates its vector using the cost c_{BA} = 2. In the second event, node E has sent its vector to node B. Node B updates its vector using the cost c_{EA} = 4.



Distance-Vector Routing Algorithm

- Now we can give a simplified pseudocode for the distance-vector routing algorithm for a node is shown here---→
- The algorithm is run by its node independently and asynchronously.
- Lines 4 to 11 initialize the vector for the node.
- Lines 14 to 23 show how the vector can be updated after receiving a vector from the immediate neighbor.
- The for loop in lines 17 to 20 allows all entries (cells) in the vector to be updated after receiving a new vector.
- Note that the node sends its vector in line 12, after

```
Distance_Vector_Routing ( )
{
    D[myself] = 0      // Initialize (create initial vectors for the node)
    for (y = 1 to N)
    {
        if (y is a neighbor)
            D[y] = c[myself][y]
        else
            D[y] = ∞
    }
    send vector {D[1], D[2], …, D[N]} to all neighbors
    // Update (improve the vector with the vector received from a neighbor)
    repeat (forever)
    {
        wait (for a vector Dw from a neighbor w or any change in the link)

        for (y = 1 to N)
        {    // Bellman-Ford equation
            D[y] = min [D[y], (c[myself][w] + Dw[y])]
        }
        if (any change in the vector)
            send vector {D[1], D[2], …, D[N]} to all neighbors
    }
} // End of Distance Vector
```

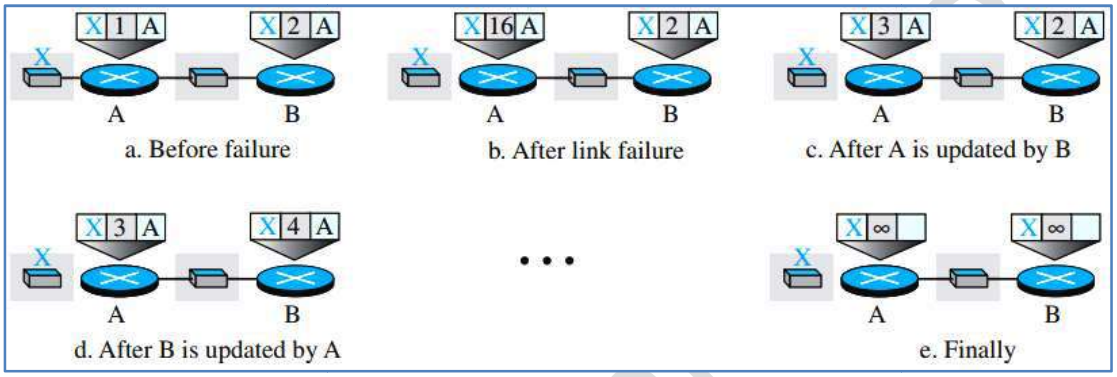
being initialized, and in line 22, after it is updated.

Count to Infinity

- A problem with distance-vector routing is that any decrease in cost (good news) propagates quickly, but any increase in cost (bad news) will propagate slowly.
- ***For a routing protocol to work properly, if a link is broken (cost becomes infinity), every other router should be aware of it immediately, but in distance-vector routing, this takes some time. The problem is referred to as count to infinity.*** It sometimes takes several updates before the cost for a broken link is recorded as infinity by all routers.

Two-Node Loop

- One ***example of count to infinity is the two-node loop problem***. To understand the problem, let us look at the scenario depicted in Figure.



- The figure shows a system with three nodes. We have shown only the portions of the forwarding table needed for our discussion. At the beginning, both nodes A and B know how to reach node X.
- But suddenly, the link between A and X fails. Node A changes its table. If A can send its table to B immediately, everything is fine. However, the system becomes unstable if B sends its forwarding table to A before receiving A's forwarding table.
- Node A receives the update and, assuming that B has found a way to reach X, immediately updates its forwarding table. Now A sends its new update to B. Now B thinks that something has been changed around A and updates its forwarding table. The cost of reaching X increases gradually until it reaches infinity.

Split Horizon

- ***One solution to instability is called split horizon***. In this strategy, instead of flooding the table through each interface, each node sends only part of its table through each interface.
- If, according to its table, node B thinks that the optimum route to reach X is via A, it does not need to advertise this piece of information to A; the information has come from A (A already knows).
- Taking information from node A, modifying it, and sending it back to node A is what creates the confusion. In our scenario, node B eliminates the last line of its forwarding table before it sends it to A.
- In this case, node A keeps the value of infinity as the distance to X. Later, when node A sends its forwarding table to B, node B also corrects its forwarding table. The system becomes stable after the first update: both node A and node B know that X is not reachable.

Poison Reverse

- Using the split-horizon strategy has one drawback. Normally, the corresponding protocol uses a timer, and if there is no news about a route, the node deletes the route from its table. ***When node B in the previous scenario eliminates the route to X from its advertisement to A, node A cannot guess whether this is due to the split-horizon strategy (the source of information was A) or because B has not received any news about X recently***.

- In the ***poison reverse strategy*** B can still advertise the value for X, but if the source of information is A, it can replace the distance with infinity as a warning: "Do not use this value; what I know about this route comes from you."

Three-Node Instability

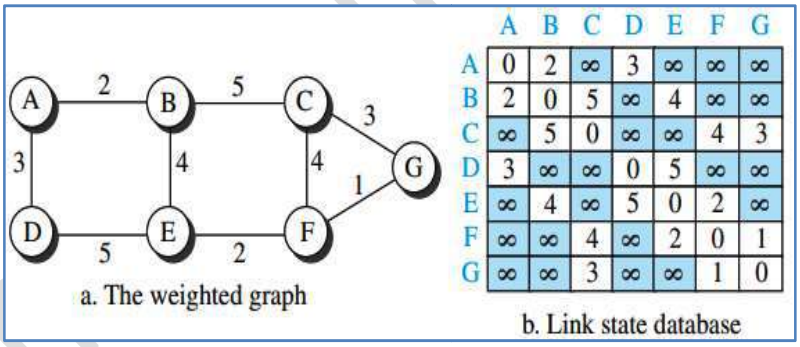
- The two-node instability can be avoided using split horizon combined with poison reverse. However, if the instability is between three nodes, stability cannot be guaranteed.

Link-State Routing (LSR)

- A routing algorithm that directly follows our discussion for ***creating least-cost trees and forwarding tables is link-state (LS) routing***. This method uses the term link-state to define the characteristic of a link (an edge) that represents a network in the internet.
- In this algorithm the cost associated with an edge defines the state of the link. Links with lower costs are preferred to links with higher costs; if the cost of a link is infinity, it means that the link does not exist or has been broken.

Link-State Database (LSDB)

- To create a least-cost tree with this method, each node needs to have a complete map of the network, which means it needs to know the state of each link. The ***collection of states for all links is called the link-state database (LSDB)***.
- ***There is only one LSDB for the whole internet; each node needs to have a duplicate of it to be able to create the least-cost tree***. Figure shows an example of an LSDB for the graph. The LSDB can be represented as a two-dimensional array(matrix) in which the value of each cell defines the cost of the corresponding link.



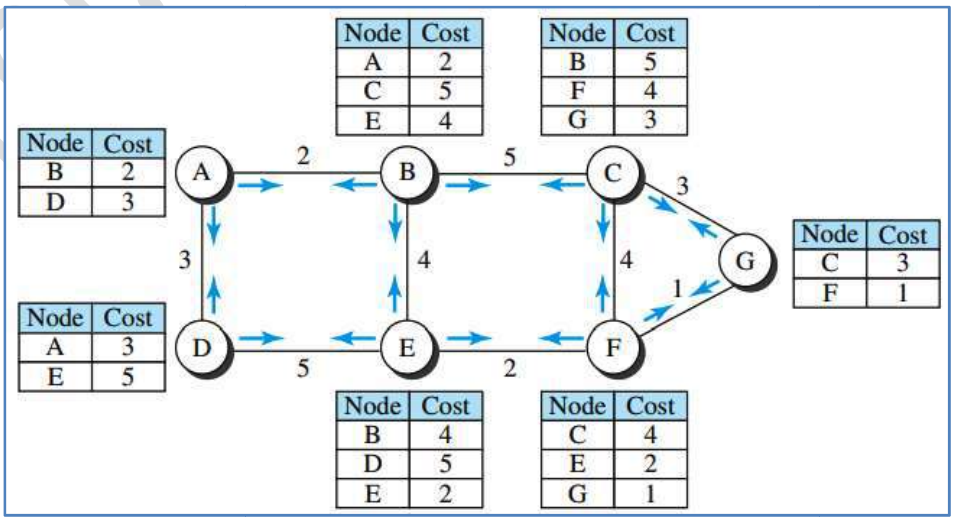
a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

b. Link state database

Now the question is how each node can create this LSDB that contains information about the whole internet? This can be done by a process called ***flooding***.

- Each node can send some greeting messages to all its immediate neighbors (those nodes to which it is connected directly) to collect two pieces of information for each neighboring node: the identity of the node and the cost of the link. ***The combination of these two pieces of information is called the LS packet (LSP)***; the LSP is sent out of each interface, as shown in Figure.



- When a node receives an LSP from one of its interfaces, it compares the LSP with the copy it may already have. If the newly arrived LSP is older than the one it has (found by checking the sequence number), it discards the LSP.
- If it is newer or the first one received, the node discards the old LSP (if there is one) and keeps the received one. It then sends a copy of it out of each interface except the one from which the packet arrived. This guarantees that flooding stops somewhere in the network.

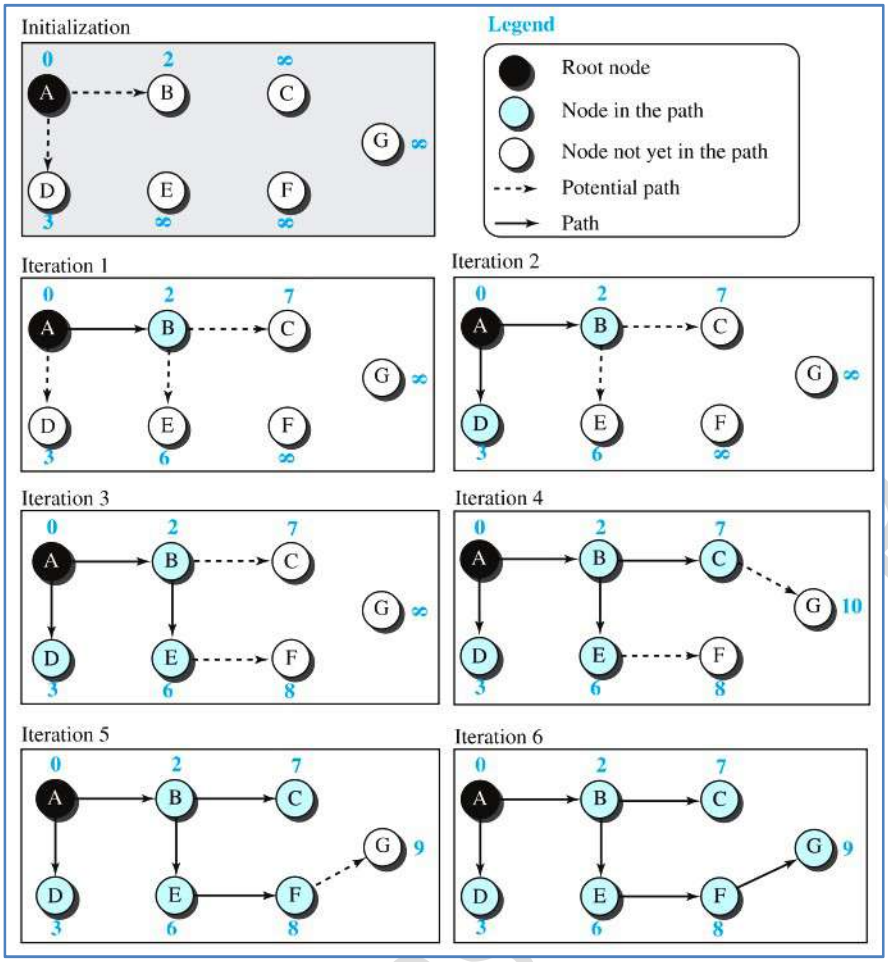
Compare the link-state routing algorithm with the distance-vector routing algorithm	
In the distance-vector routing algorithm, **each router tells its neighbors what it knows about the whole internet**.	In the link-state routing algorithm, **each router tells the whole internet what it knows about its neighbors**.

Formation of Least-Cost Trees

- To create a least-cost tree for itself, using the shared LSDB, each node needs to run the famous Dijkstra Algorithm. This iterative algorithm uses the following steps:
  1. The node chooses itself as the root of the tree, creating a tree with a single node, and sets the total cost of each node based on the information in the LSDB.
  2. The node selects one node, among all nodes not in the tree, which is closest to the root, and adds this to the tree. After this node is added to the tree, the cost of all other nodes not in the tree needs to be updated because the paths may have been changed.
  3. The node repeats step 2 until all nodes are added to the tree.
- We need to convince ourselves that the above three steps finally create the least-cost tree. Table shows a ***simplified version of Dijkstra's algorithm***.

```
Dijkstra's Algorithm ( )
{
  // Initialization
  Tree = {root}                    // Tree is made only of the root
    for (y = 1 to N)               // N is the number of nodes
    {
        if (y is the root)
            D[y] = 0               // D[y] is shortest distance from root to node y
        else if (y is a neighbor)
            D[y] = c[root][y]      // c[x][y] is cost between nodes x and y in LSDB
        else
            D[y] = ∞
    }
    // Calculation
    repeat
    {
        find a node w, with D[w] minimum among all nodes not in the Tree
        Tree = Tree ∪ {w}          // Add w to tree
        // Update distances for all neighbors of w
        for (every node x, which is a neighbor of w and not in the Tree)
        {
            D[x] = min{D[x], (D[w] + c[w][x])}
        }
    } until (all nodes included in the Tree)
} // End of Dijkstra
```

- The following Figure shows the formation of the least-cost tree for the graph using Dijkstra's algorithm. We need to go through an initialization step and six iterations to find the least-cost tree.



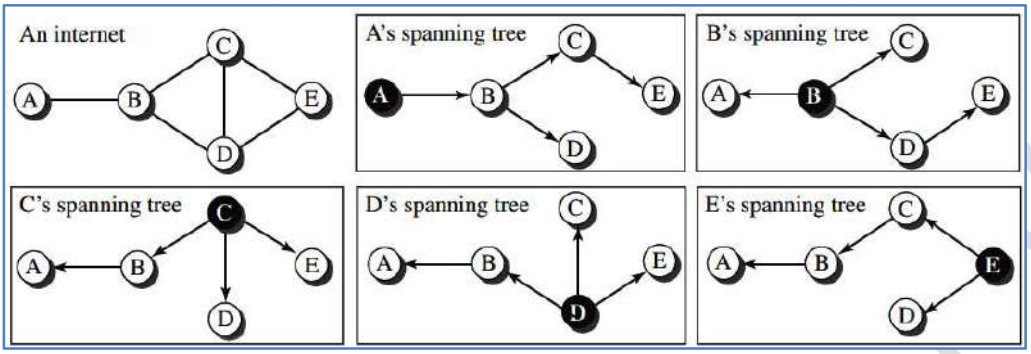
Path-Vector Routing (PVR)

- Both ***link-state and distance-vector routing are based on the least-cost goal***. However, there are instances where this goal is not the priority. For example, assume that there are some routers in the internet that a sender wants to prevent its packets from going through.
- The least-cost goal, applied by LS or DV routing, does not allow a sender to apply specific policies to the route a packet may take. Aside from safety and security, there are occasions in which the goal of routing is merely reachability: ***to allow the packet to reach its destination more efficiently without assigning costs to the route***.
- To ***respond to these demands, a third routing algorithm, called path-vector (PV) routing has been devised***. Path-vector routing does not have the drawbacks of LS or DV routing as described above because it is not based on least-cost routing.
- The ***best route is determined by the source using the policy it imposes on the route***. In other words, the ***source can control the path.***
- Although path-vector routing is ***not actually used in an internet, and is mostly designed to route a packet between ISPs***.

Spanning Trees

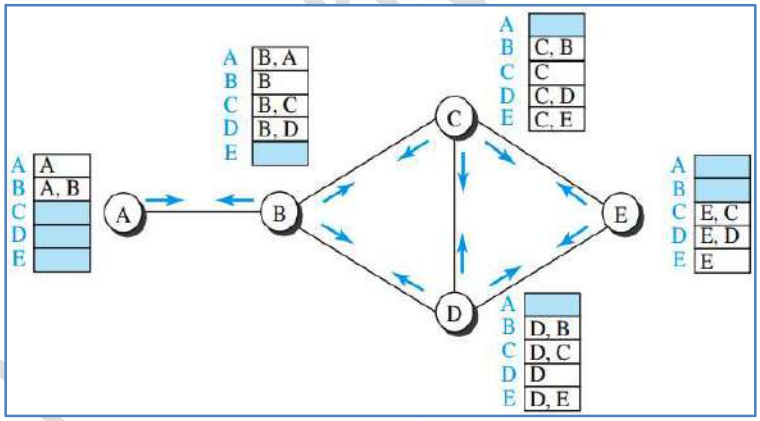
- In path-vector routing, the path from a source to all destinations is also determined by the best spanning tree. The best spanning tree, however, is not the least-cost tree; it is ***the tree determined by the source when it imposes its own policy***. If ***there is more than one route to a destination, the source can choose the route that meets its policy best***.
- A source may apply several policies at the same time.
  - One of the common policies ***uses the minimum number of nodes to be visited***.
  - Another common policy is to ***avoid some nodes as the middle node in a route***.

- Figure shows a small internet with only ***five nodes***. Each source has created its own spanning tree that meets its policy. The policy imposed by all sources is to use the minimum number of nodes to reach a destination. The spanning tree selected by A and E is such that the communication does not pass through D as a middle node. Similarly, the spanning tree selected by B is such that the communication does not pass through C as a middle node.

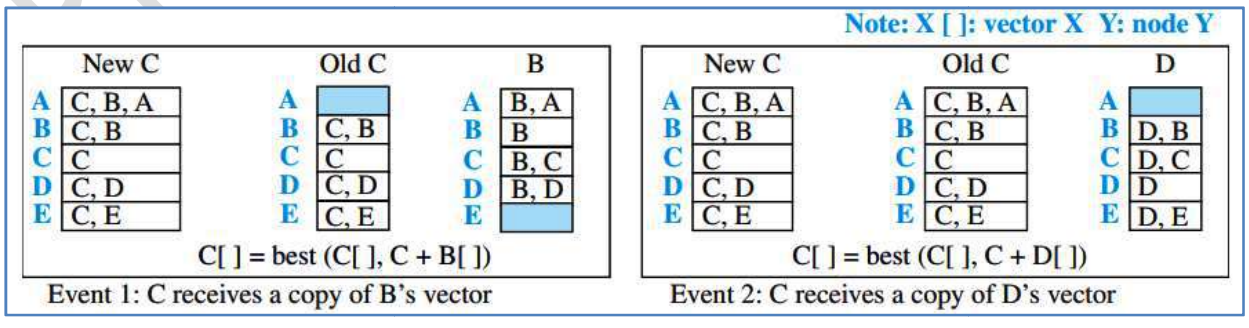


Creation of Spanning Trees

- Path-vector routing, like distance-vector routing, is an asynchronous and distributed routing algorithm. The ***spanning trees are made, gradually and asynchronously, by each node***.
- When a node is ***booted, it creates a path vector based on the information it can obtain about its immediate neighbor***.
- A node sends greeting messages to its immediate neighbors to collect these pieces of information. Figure shows all of these path vectors for our internet.



- Each node, after the creation of the initial path vector, sends it to all its immediate neighbors. Each node, when it receives a path vector from a neighbor, updates its path vector using an equation similar to the Bellman-Ford, but applying its own policy instead of looking for the least cost. We can define this equation as $\text{Path}(x, y) = \text{best}\ \{\text{Path}(x, y), [(x + \text{Path}(v, y)]\}$ for all v's in the internet.
- The policy is defined by selecting the best of multiple paths. Path-vector routing also imposes one more condition on this equation: If Path (v, y) includes x, that path is discarded to avoid a loop in the path. In other words, x does not want to visit itself when it selects a path to y.
- The following figure shows the path vector of node C after two events. In the first event, node C receives a copy of B's vector, which improves its vector: now it knows how to reach node A. In the second event, node C receives a copy of D's vector, which does not change its vector. As a matter of fact the vector for node C after the first event is stabilized and serves as its forwarding table.



Path-Vector Algorithm

- Based on the initialization process and the equation used in updating each forwarding table after receiving path vectors from neighbors, we can write a simplified version of the path vector algorithm as shown in Table.

```
Path_Vector_Routing ( )
{
   // Initialization
   for (y = 1 to N)
   {
        if (y is myself)
             Path[y] = myself
        else if (y is a neighbor)
             Path[y] = myself + neighbor node
        else
             Path[y] = empty
   }
   Send vector {Path[1], Path[2], …, Path[y]} to all neighbors
   repeat (forever)    // Update
   {
        wait (for a vector Path_w from a neighbor w)
        for (y = 1 to N)
        {
             if (Path_w includes myself)
                  discard the path                    // Avoid any loop
             else
                  Path[y] = best {Path[y], (myself + Path_w[y])}
        }
        If (there is a change in the vector)
             Send vector {Path[1], Path[2], …, Path[y]} to all neighbors
   }
} // End of Path Vector
```

- Lines 4 to 12 show the initialization for the node.
- Lines 17 to 24 show how the node updates its vector after receiving a vector from the neighbor.
- The update process is repeated forever. We can see the similarities between this algorithm and the DV algorithm.

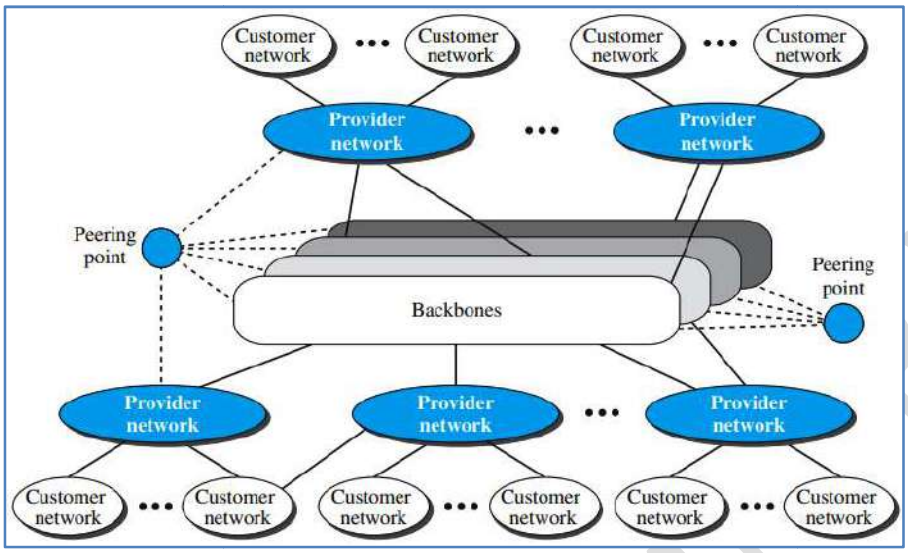
UNICAST ROUTING PROTOCOLS

Internet Structure - Routing Information Protocol (RIP) - Open Shortest Path First (OSPF) - Border Gateway Protocol Version 4 (BGP4)

- In the ***previous section, we discussed unicast routing algorithms***; in ***this section, we discuss unicast routing protocols used in the Internet***.
- A ***protocol needs to define its domain of operation***, ***the messages exchanged, communication between routers, and interaction with protocols in other domains***.
- ***Three common protocols*** used in the Internet:
  1. ***Routing Information Protocol (RIP)***, based on the distance-vector algorithm
  2. ***Open Shortest Path First (OSPF)***, based on the link-state algorithm, and
  3. ***Border Gateway Protocol (BGP)***, based on the path-vector algorithm.

Internet Structure

- The Internet has changed from a tree-like structure, with a single backbone, to a multi-backbone structure run by different private corporations today. Although it is difficult to give a general view of the Internet today, we can say that the Internet has a structure is shown in Figure.



- There are several backbones run by private communication companies that provide global connectivity. These backbones are connected by some peering points that allow connectivity between backbones.
- ***At a lower level***, there are some provider networks that use the backbones for global connectivity but provide services to Internet customers.
- ***Finally***, there are some customer networks that use the services provided by the provider networks.
- Any of these ***three entities (backbone, provider network, or customer network)*** can be called an Internet Service Provider or ISP. They provide services, but at different levels.

Hierarchical Routing

- The Internet today is made of a huge number of networks and routers that connect them. It is obvious that routing in the Internet cannot be done using a single protocol for ***two reasons: a scalability problem and an administrative issue***.
- ***Scalability problem*** means that the ***size of the forwarding tables becomes huge***, searching for a destination in a forwarding table becomes time-consuming, and updating creates a huge amount of traffic.
- The ***administrative issue is related to the Internet structure.*** As the figure shows, each ISP is run by an administrative authority. The administrator needs to have control in its system.
- ***Hierarchical routing means considering each ISP as an autonomous system (AS).***
- Each AS can run a routing protocol that meets its needs, but the global Internet runs a global protocol to glue all ASs together.
- The ***routing protocol run in each AS*** is referred to as
  - intra-AS routing protocol, intradomain routing protocol, ***interior gateway protocol (IGP)***;
  - the global routing protocol is referred to as inter-AS routing protocol, interdomain routing protocol, or ***exterior gateway protocol (EGP).***
  - Presently, the two common ***intradomain*** routing protocols are ***RIP*** and ***OSPF***;
  - the only ***interdomain*** routing protocol is ***BGP***. The situation may change when we move to IPv6.

Prof. Parthasarathy P V

Autonomous Systems

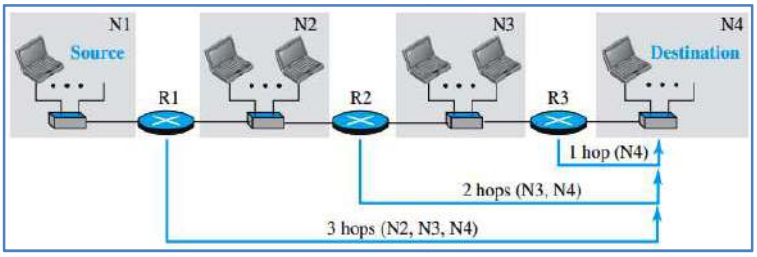
- Each ISP is an autonomous system when it comes to managing networks and routers under its control.
- Although we may have small, medium-size, and large ASs, each AS is given an autonomous number (ASN) by the ICANN. Each ASN is a 16-bit unsigned integer that uniquely defines an AS.
- The autonomous systems are not categorized according to their size; they are categorized according to the way they are connected to other ASs.
- ***Stub AS***. A stub AS has ***only one connection to another AS***. The data traffic can be either initiated or terminated in a stub AS; the data cannot pass through it. A good example of a stub AS is the customer network, which is either the source or the sink of data.
- ***Multihomed AS.*** A multihomed AS can have ***more than one connection to other ASs, but it does not allow data traffic to pass through it***. A good example of such an AS is some of the customer ASs that may use the services of more than one provider network, but their policy does not allow data to be passed through them.
- ***Transient AS.*** A transient AS is ***connected to more than one other AS and also allows the traffic to pass through***. The provider networks and the backbone are good examples of transient ASs.

Routing Information Protocol (RIP)

- The Routing Information Protocol (RIP) is one of the most widely used intra-domain routing protocols based on the distance-vector routing algorithm.

Hop Count

- A router in this protocol basically implements the distance-vector routing algorithm.
- First, since a router in an AS needs to know how to forward a packet to different networks (subnets) in an AS, ***RIP routers advertise the cost of reaching different networks*** instead of reaching other nodes in a theoretical graph.
- Second, to make the implementation of the cost simpler (independent from performance factors of the routers and links, such as delay, bandwidth, and so on), the ***cost is defined as the number of hops, which means the number of networks (subnets) a packet needs to travel through from the source router to the final destination host***.
- Figure shows the concept of hop count advertised by three routers from a source host to a destination host. In RIP, the maximum cost of a path can be 15, which means 16 is considered as infinity (no connection).
- For this reason, ***RIP can be used only in autonomous systems*** in which the diameter of the AS is not more than 15 hops.



Forwarding Tables

- The following figure shows the three forwarding tables for the routers in the Figure shown above.

Forwarding table for R1

Destination network	Next router	Cost in hops
N1	——	1
N2	——	1
N3	R2	2
N4	R2	3

Forwarding table for R2

Destination network	Next router	Cost in hops
N1	R1	2
N2	——	1
N3	——	1
N4	R3	2

Forwarding table for R3

Destination network	Next router	Cost in hops
N1	R2	3
N2	R2	2
N3	——	1
N4	——	1

Prof. Parthasarathy P V

- Note that the first and the third columns together convey the same information as does a distance vector, but the cost shows the number of hops to the destination networks.
- Although a forwarding table in RIP defines only the next router in the second column, it gives the information about the whole least-cost tree based on the second property of these trees, discussed in the previous section. For example, R1 defines that the next router for the path to N4 is R2; R2 defines that the next router to N4 is R3; R3 defines that there is no next router for this path. The tree is then R1 → R2 → R3 → N4.

What the use of the third column of the forwarding table?

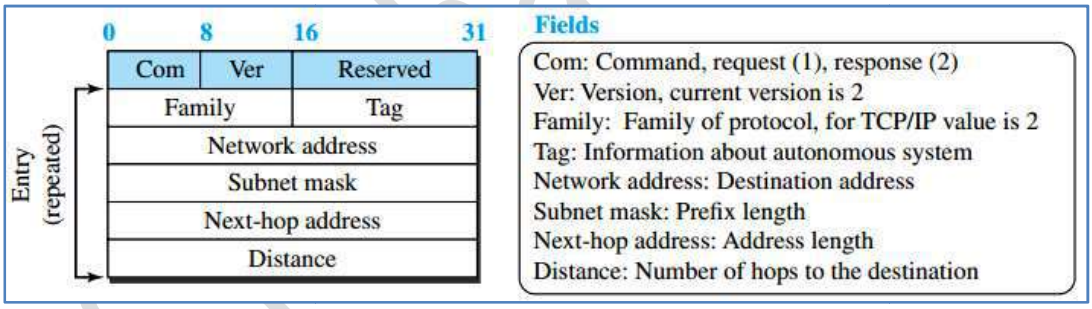
- The third column is ***not needed for forwarding the packet***, but it is ***needed for updating the forwarding table when there is a change in the route.***

RIP Implementation

- RIP is implemented as a process that uses the service of ***UDP on the well-known port number 520***.
- In BSD, RIP is a daemon process (a process running in the background), named routed (abbreviation for route daemon and pronounced route-dee). This means that, although RIP is a routing protocol to help IP route its datagrams through the AS, the RIP messages are encapsulated inside UDP user datagrams, which in turn are encapsulated inside IP datagrams.
- In other words, RIP runs at the application layer, but creates forwarding tables for IP at the network later.
- RIP has gone through two versions: RIP-1 and RIP-2.
- RIP-2 is backward compatible with RIP-1; it allows the use of more information in the RIP messages that were set to 0 in the RIP-1.

RIP Messages

- ***Two RIP processes, a client and a server***, like any other processes, need to exchange messages. RIP-2 defines the format of the message, as shown in Figure.



- Part of the message, which we call entry, can be repeated as needed in a message. Each entry carries the information related to one line in the forwarding table of the router that sends the message.
- RIP has ***two types of messages: request and response***.
- A ***request message*** is sent by a router that has just come up or by a router that has some ***time-out entries***. A request message can ask about specific entries or all entries.
- A ***response (or update) message*** can be ***either solicited or unsolicited***.
  - A ***solicited response*** message is sent only in answer to a request message. It contains information about the destination specified in the corresponding request message.
  - An ***unsolicited response message***, on the other hand, is sent periodically, every 30 seconds or when there is a change in the forwarding table.

RIP Algorithm

- RIP implements the same algorithm as the distance-vector routing algorithm. However, ***some changes need to be made to the algorithm to enable a router to update its forwarding table***:

- Instead of sending only distance vectors, a ***router needs to send the whole contents of its forwarding table in a response message***.
- The ***receiver adds one hop to each cost and changes the next router field to the address of the sending router***. We call each route in the modified forwarding table the ***received route*** and each route in the old forwarding table the ***old route***.
- The received router selects the old routes as the new ones except in the following three cases:
  1. If the received route does not exist in the old forwarding table, it should be added to the route.
  2. If the cost of the received route is lower than the cost of the old one, the received route should be selected as the new one.
  3. If the cost of the received route is higher than the cost of the old one, but the value of the next router is the same in both routes, the received route should be selected as the new one. This is the case where the route was actually advertised.
- The new forwarding table needs to be sorted according to the destination route (mostly using the longest prefix first).

Timers in RIP

- RIP uses three timers to support its operation.
  1. Periodic timer
  2. Expiration timer
  3. Garbage collection timer
- The ***periodic timer controls the advertising of regular update messages***. Each router has one periodic timer that is randomly set to a number ***between 25 and 35 seconds*** (to prevent all routers sending their messages at the same time and creating excess traffic). The timer counts down; when zero is reached, the update message is sent, and the timer is randomly set once again.
- The ***expiration timer governs the validity of a route***. When a router receives update information for a route, the expiration timer is set to 180 sec. for that particular route. Every time a new update for the route is received, the timer is reset. If there is a problem on an internet and no update is received within the allotted 180 seconds, the route is considered expired and the hop count of the route is set to 16, means the destination is unreachable. ***Every route has its own expiration timer***.
- The ***garbage collection timer*** is used to ***purge (clean) a route from the forwarding table***. When the information about a route becomes invalid, the router does not immediately clear that route from its table. Instead, it continues to advertise the route with a metric value of 16. At the same time, a garbage collection timer is set to 120 seconds for that route. ***When the count reaches zero, the route is purged from the table***. This ***timer allows neighbors to become aware of the invalidity of a route prior to purging***.

Performance

- ***Update Messages***. RIP have a very simple format and are sent only to neighbors; they are local. They do not normally create traffic because the routers try to avoid sending them at the same time.
- ***Convergence of Forwarding Tables***. RIP uses the distance-vector algorithm, which can converge slowly if the domain is large, but, since RIP allows only 15 hops in a domain (16 is considered as infinity), there is normally no problem in convergence. The only problems that may ***slow down convergence are count-to-infinity and loops created in the domain***; ***use of poison-reverse and split-horizon strategies*** added to the RIP extension may alleviate the situation.
- ***Robustness***. The ***calculation of the forwarding table depends on information received from immediate neighbors***, which in turn receive their information from their own neighbors. If there is a ***failure or corruption in one router, the problem will be propagated to all routers and the forwarding in each router will be affected***.

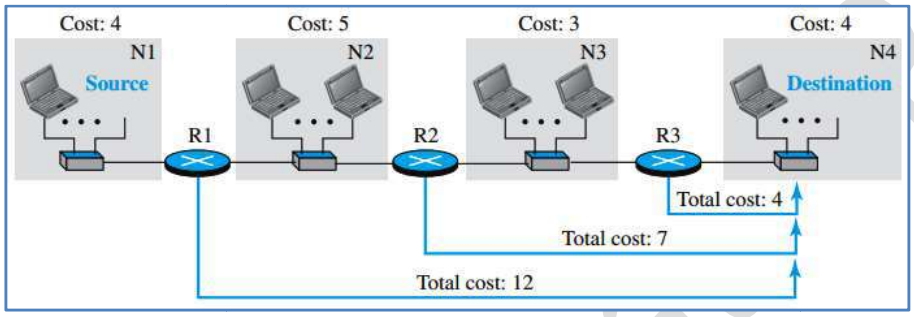
Prof. Parthasarathy P V

Open Shortest Path First (OSPF)

- ***Open Shortest Path First (OSPF) is also an intra-domain routing protocol like RIP***, but it is ***based on the link-state routing protocol***. OSPF is an open protocol, which means that the ***specification is a public document***.

Metric

- In OSPF, the cost of reaching a destination from the host is calculated from the source router to the destination network. However, each link (network) can be ***assigned a weight based on the throughput, round-trip time, reliability***, and so on.
- An administration can also decide to use the hop count as the cost. An interesting point about the cost in OSPF is that different service types (TOSs) can have different weights as the cost. Figure shows the idea of the cost from a router to the destination host network.



Forwarding Tables

- Each ***OSPF router can create a forwarding table after finding the shortest-path tree between itself and the destination using Dijkstra's algorithm.***

Forwarding tables for the simple AS

Forwarding table for R1		
Destination network	Next router	Cost
N1	——	4
N2	——	5
N3	R2	8
N4	R2	12

Forwarding table for R2		
Destination network	Next router	Cost
N1	R1	9
N2	——	5
N3	——	3
N4	R3	7

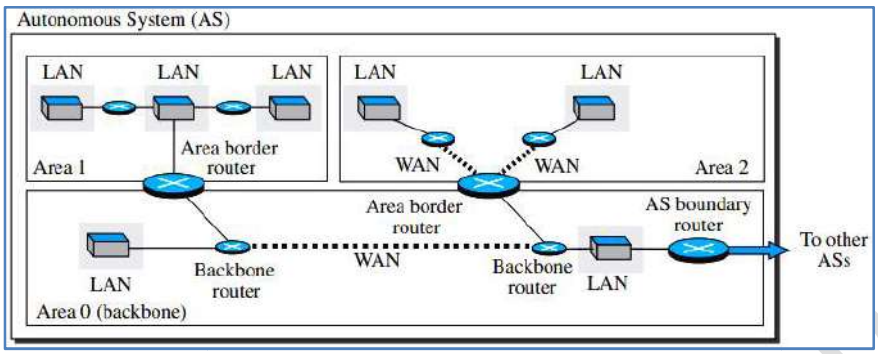
Forwarding table for R3		
Destination network	Next router	Cost
N1	R2	12
N2	R2	8
N3	——	3
N4	——	4

- Comparing the forwarding tables for the OSPF and RIP in the same AS, we find that the only difference is the cost values. In other words, if we use the hop count for OSPF, the tables will be exactly the same.
- The ***reason for this consistency is that both protocols use the shortest-path trees to define the best route from a source to a destination***.

Areas

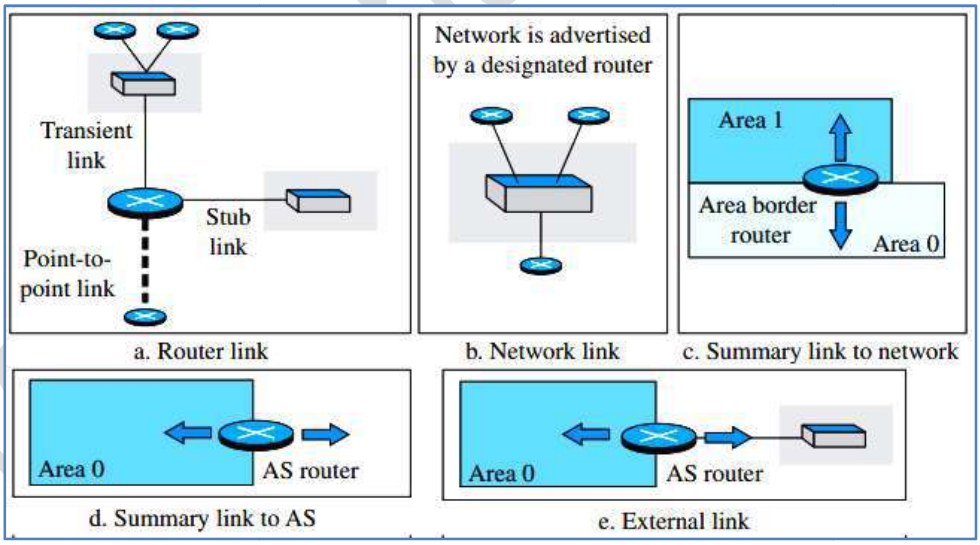
- ***OSPF was designed to be able to handle routing in a small or large autonomous system***. However, the formation of shortest-path trees in OSPF requires that all routers flood the whole AS with their LSPs to create the global LSDB.
- Although this may not create a problem in a small AS, it may have created a huge volume of traffic in a large AS. To prevent this, the ***AS needs to be divided into small sections called areas. Each area acts as a small independent domain for flooding LSPs.*** In other words, OSPF uses another level of hierarchy in routing: the first level is the autonomous system, the second is the area.
- However, ***each router in an area needs to know the information about the link states not only in its area but also in other areas***. For this reason, one of the areas in the AS is designated as the backbone area, responsible for gluing the areas together.

- The routers in the backbone area are responsible for passing the information collected by each area to all other areas. In this way, a router in an area can receive all LSPs generated in other areas. For the purpose of communication, each area has an area identification. The area identification of the backbone is zero. Figure shows an autonomous system and its areas.



Link-State Advertisement

- OSPF is based on the link-state routing algorithm, which requires that a router advertise the state of each link to all neighbors for the formation of the LSDB.
- ***The situation is different in the real world***, in which we need to advertise the existence of different entities as nodes, the different types of links that connect each node to its neighbors, and the different types of cost associated with each link. This means ***we need different types of advertisements, each capable of advertising different situations***.
- There are ***five types*** of link-state advertisements:
  1. **Router link**
  2. **Network link**
  3. **Summary link to network**
  4. **Summary link to AS border router**
  5. **External link.** Figure shows these five advertisements and their uses.



- ***Router link.*** A ***router link advertises the existence of a router as a node***. In addition to ***giving the address of the announcing router*** - define one or more types of links that connect the advertising router to other entities. A ***transient link*** announces a link to a transient network, a network that is connected to the rest of the networks by one or more routers - ***define the address of the transient network and the cost of the link***. A stub link advertises a link to a stub network, a network that is not a through network. Again, the ***advertisement should define the address of the network and the cost***. A ***point-to-point link*** should define the address of the router at the end of the point-to-point line and the cost to get there.

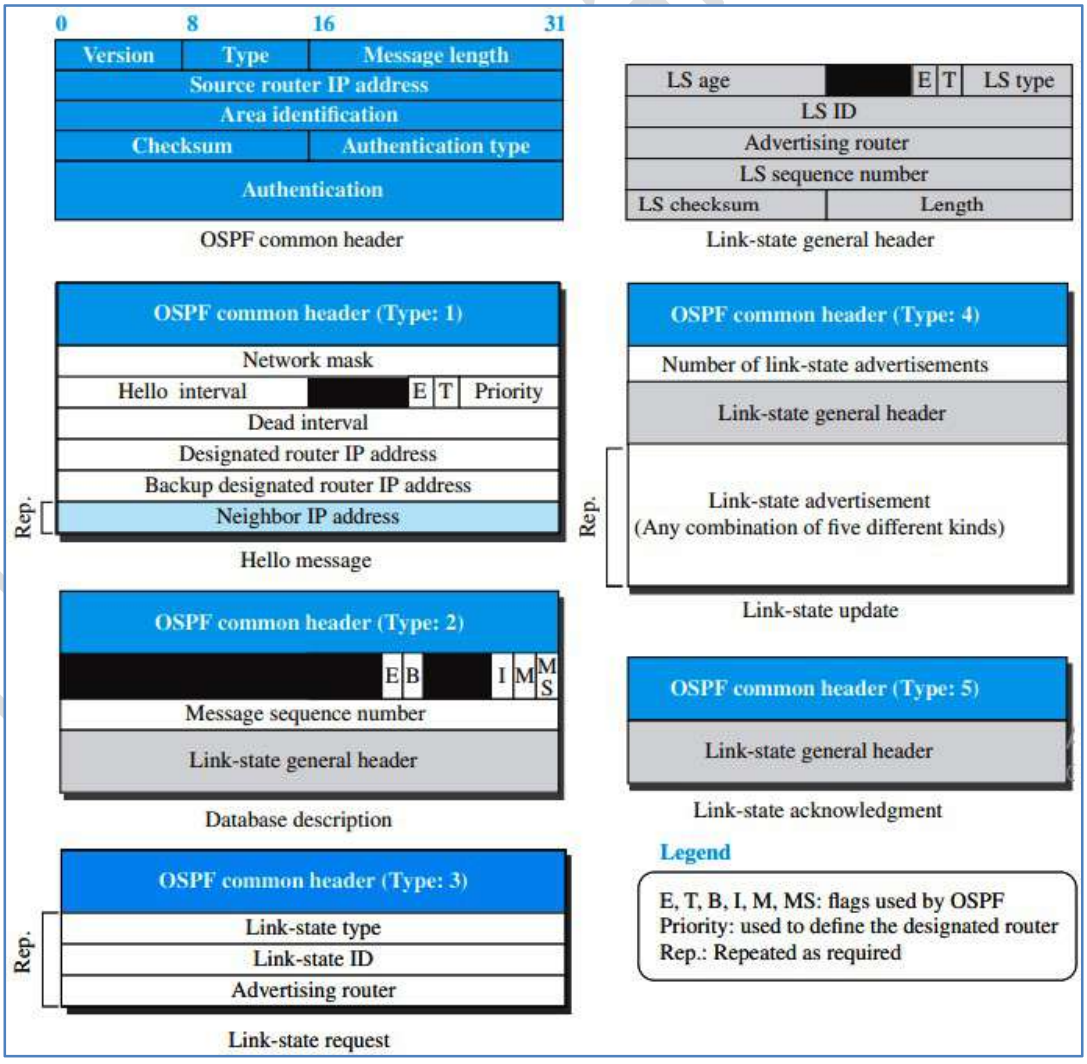
- ***Network link***. A ***network link advertises the network as a node***. Since a network cannot do announcements itself (it is a passive entity), one of the routers is assigned as the designated router and does the advertising. In addition, this type of LSP announces the IP address of all routers (including the designated router as a router and not as speaker of the network), but no cost is advertised because each router announces the cost to the network when it sends a router link advertisement.
- ***Summary link to network***. This is done by an ***area border router; it advertises the summary of links collected by the backbone to an area or the summary of links collected by the area to the backbone***. This type of information exchange is needed to glue the areas together.
- ***Summary link to AS***. This is done by an AS router that advertises the summary links from other ASs to the backbone area of the current AS, information which later can be disseminated to the areas so that they will know about the networks in other ASs. The ***need for this type of information exchange is better understood*** about ***inter-AS routing (BGP)***.
- ***External link.*** This is also done by an AS router ***to announce the existence of a single network outside the AS to the backbone area to be disseminated into the areas***.

OSPF Implementation

- OSPF is implemented as a program in the network layer, using the service of the IP for propagation. An IP datagram that carries a message from ***OSPF sets the value of the protocol field to 89*** - OSPF is a routing protocol to help IP to route its datagrams inside an AS, the OSPF messages are encapsulated inside datagrams. OSPF has gone through ***two versions***: version 1 and version 2. ***Most implementations use version 2.***

OSPF Messages

- OSPF is a very complex protocol; it uses ***five different types of messages***.



- In Figure, we first show the format of the OSPF common header (which is used in all messages) and the link-state general header (which is used in some messages). We then give the outlines of five message types used in OSPF.
  1. ***The hello message (type 1)*** is used by a router to introduce itself to the neighbors and announce all neighbors that it already knows.
  2. The ***database description message (type 2)*** is normally sent in response to the hello message to allow a newly joined router to acquire the full LSDB.
  3. The ***linkstate request message (type 3)*** is sent by a router that needs information about a specific LS.
  4. The ***link-state update message (type 4)*** is the main OSPF message used for building the LSDB. This message, in fact, ***has five different versions (router link, network link, summary link to network, summary link to AS border router, and external link).***
  5. The ***link-state acknowledgment message (type 5)*** is used to create reliability in OSPF; each router that receives a link-state update message needs to acknowledge it.

Authentication

- As Figure shows, the ***OSPF common header has the provision for authentication of the message sender - this prevents a malicious entity from sending OSPF messages to a router and causing the router to become part of the routing system to which it actually does not belong***.

OSPF Algorithm

- OSPF implements the link-state routing algorithm. However, some changes and augmentations ***need to be added to the algorithm***:
  - After each router has created the shortest-path tree, the algorithm needs to use it to create the corresponding routing algorithm.
  - The algorithm needs to be augmented to handle sending and receiving all five types of messages.

Performance

- ***Update Messages***. The link-state messages in OSPF have a somewhat complex format. They also are flooded to the whole area. If the area is large, these messages may create heavy traffic and use a lot of bandwidth.
- ***Convergence of Forwarding Tables***. When the flooding of LSPs is completed, each router can create its own shortest-path tree and forwarding table; convergence is fairly quick. However, each router needs to run Dijkstra's algorithm, which may take some time.
- ***Robustness***. The OSPF protocol is more robust than RIP because, after receiving the completed LSDB, each router is independent and does not depend on other routers in the area. Corruption or failure in one router does not affect other routers as seriously as in RIP.

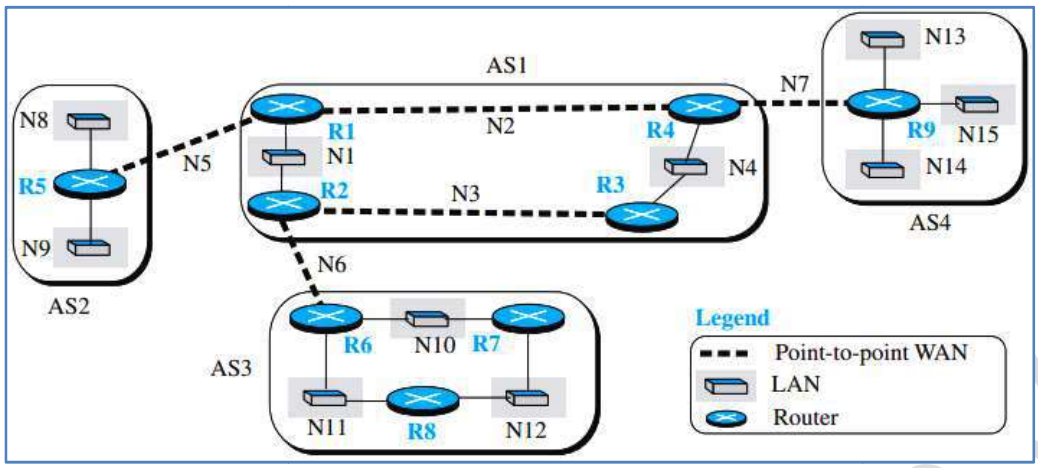
Border Gateway Protocol Version 4 (BGP4)

- The Border Gateway Protocol version 4 (BGP4) is the only inter-domain routing protocol used in the Internet today. ***BGP4 is based on the path-vector algorithm***, but it is tailored to provide information about the reachability of networks in the Internet.

Introduction

- BGP, and in particular BGP4, is a complex protocol. In this section, we introduce the basics of BGP and its relationship with intradomain routing protocols (RIP or OSPF). Figure shows an example of an internet with four autonomous systems. AS2, AS3, and AS4 are stub autonomous systems; AS1 is a transient one. In our example, data exchange between AS2, AS3, and AS4 should pass through AS1.

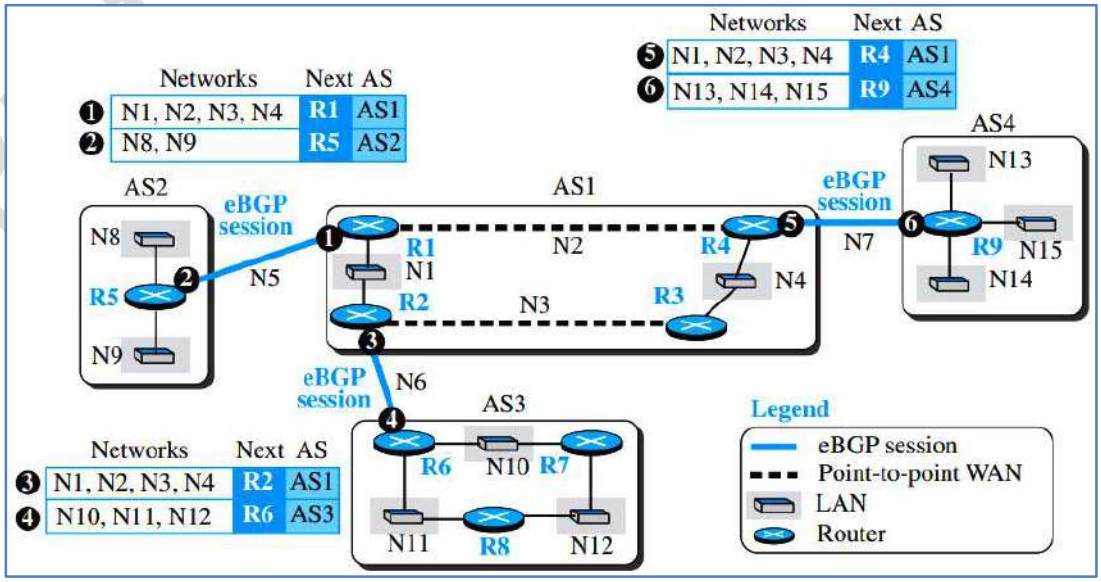
Prof. Parthasarathy P V



- Each autonomous system in this figure uses one of the two common intradomain protocols, RIP or OSPF. Each router in each AS knows how to reach a network that is in its own AS, but it does not know how to reach a network in another AS.
- To enable each router to route a packet to any network in the internet, we ***first install a variation of BGP4, called external BGP (eBGP), on each border router*** (the one at the edge of each AS which is connected to a router at another AS).
- We then ***install the second variation of BGP, called internal BGP (iBGP), on all routers***. This means that the border routers will be running three routing protocols (intradomain, eBGP, and iBGP), but other routers are running two protocols (intradomain and iBGP).

Operation of External BGP (eBGP)

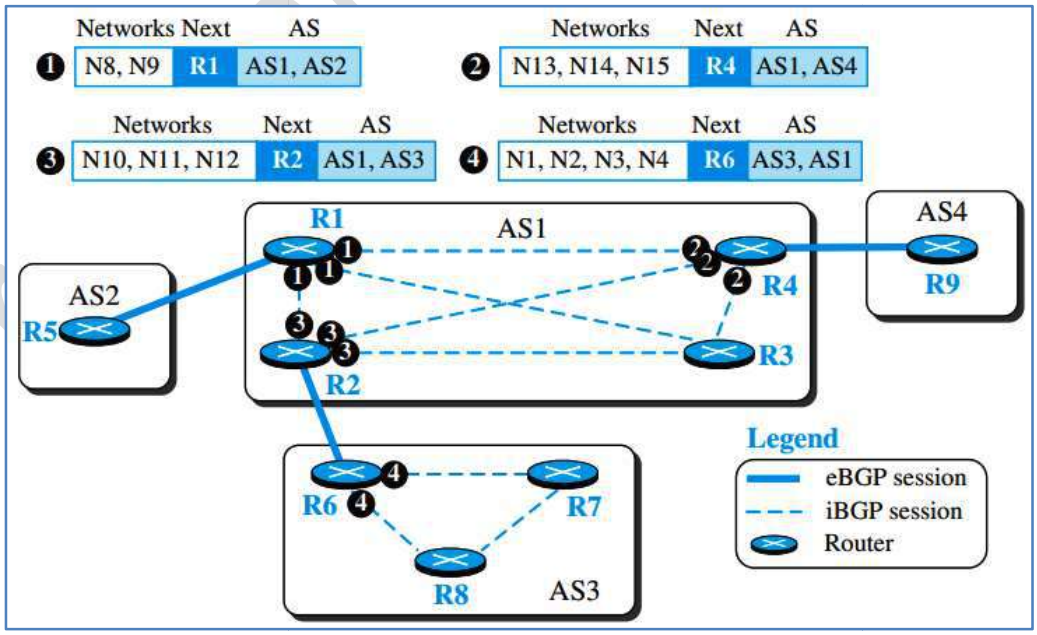
- BGP is a kind of point-to-point protocol. When the software is installed on two routers, they try to ***create a TCP connection using the well-known port 179***. A pair of client and server processes continuously communicates with each other to exchange messages. The ***two routers*** that run the BGP processes are called ***BGP peers*** or ***BGP speakers***.
- The eBGP variation of BGP allows two physically connected border routers in two different ASs to form pairs of eBGP speakers and exchange messages. The routers that are eligible in our example in Figure above form three pairs: R1-R5, R2-R6, and R4- R9.
- The connection between these pairs is established over three physical WANs (N5, N6, and N7). However, there is a need for a logical TCP connection to be created over the physical connection to make the exchange of information possible.
- Each logical connection in BGP parlance is referred to as a session. This means that we need three sessions in our example, as shown in Figure.



- The figure also shows the simplified update messages sent by routers involved in the eBGP sessions. ***The circled number defines the sending router in each case***. For example, ***message number 1 is sent by router R1 and tells router R5 that N1, N2, N3, and N4 can be reached through router R1*** (R1 gets this information from the corresponding intradomain forwarding table).
- Router R5 can now add these pieces of information at the end of its forwarding table. When R5 receives any packet destined for these four networks, it can use its forwarding table and find that the next router is R1.
- The reader may have noticed that the messages exchanged during three eBGP sessions help some routers know how to route packets to some networks in the internet, but the reachability information is not complete. ***There are two problems that need to be addressed***:
  1. ***Some border routers do not know how to route a packet destined for nonneighbor ASs***. For example, R5 does not know how to route packets destined for networks in AS3 and AS4. Routers R6 and R9 are in the same situation as R5: R6 does not know about networks in AS2 and AS4; R9 does not know about networks in AS2 and AS3.
  2. ***None of the nonborder routers know how to route a packet destined for any networks in other ASs.***
- To address the above two problems, we need to allow all pairs of routers (border or nonborder) to run the second variation of the BGP protocol, ***iBGP***.

Operation of Internal BGP (iBGP)

- The iBGP protocol is similar to the eBGP protocol in that it uses the service of TCP on the well-known port 179, ***but it creates a session between any possible pair of routers inside an autonomous system***.
- First, ***if an AS has only one router, there cannot be an iBGP session***.
  - For example, we cannot create an iBGP session inside AS2 or AS4 in our internet.
- Second, ***if there are n routers in an autonomous system, there should be [n × (n – 1) / 2] iBGP sessions*** in that autonomous system (a fully connected mesh) to prevent loops in the system.
- Each router needs to advertise its own reachability to the peer in the session instead of flooding what it receives from another peer in another session. ***Figure shows the combination of eBGP and iBGP sessions in our internet.***



- A session is made on an overlay network (TCP connection), possibly spanning more than one physical network as determined by the route dictated by intradomain routing protocol. To demonstrate, we show the path tables shown below:

Networks	Next	Path
N8, N9	R5	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Path table for R1

Networks	Next	Path
N8, N9	R1	AS1, AS2
N10, N11, N12	R6	AS1, AS3
N13, N14, N15	R1	AS1, AS4

Path table for R2

Networks	Next	Path
N8, N9	R2	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Path table for R3

Networks	Next	Path
N8, N9	R1	AS1, AS2
N10, N11, N12	R1	AS1, AS3
N13, N14, N15	R9	AS1, AS4

Path table for R4

Networks	Next	Path
N1, N2, N3, N4	R1	AS2, AS1
N10, N11, N12	R1	AS2, AS1, AS3
N13, N14, N15	R1	AS2, AS1, AS4

Path table for R5

Networks	Next	Path
N1, N2, N3, N4	R2	AS3, AS1
N8, N9	R2	AS3, AS1, AS2
N13, N14, N15	R2	AS3, AS1, AS4

Path table for R6

Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1
N8, N9	R6	AS3, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4

Path table for R7

Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1
N8, N9	R6	AS3, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4

Path table for R8

Networks	Next	Path
N1, N2, N3, N4	R4	AS4, AS1
N8, N9	R4	AS4, AS1, AS2
N10, N11, N12	R4	AS4, AS1, AS3

Path table for R9

Injection of Information into Intradomain Routing

- The ***role of an interdomain routing protocol such as BGP is to help the routers inside the AS to augment their routing information***.
- The path tables collected and organized by BPG are not used for routing packets; they are injected into intradomain forwarding tables (RIP or OSPF) for routing packets. This can be done in several ways depending on the type of AS.
- In case of a ***stub AS***, the only area border router adds a default entry at the end of its forwarding table, defines the next router to be the speaker router at the end of the eBGP connection.
- In the case of a ***transient AS***, the situation is more complicated. R1 in AS1 needs to inject the whole contents of the path table for R1 in table into its intradomain forwarding table. The situation is the same for R2, R3, and R4.
- ***One issue to be resolved is the cost value***. We know that RIP and OSPF use different metrics. One ***solution is to set the cost to the foreign networks at the same cost value as to reach the first AS in the path***.
- The following table shows the interdomain forwarding tables. For simplicity, we assume that all ASs are using RIP as the intradomain routing protocol. The shaded areas are the augmentation injected by the BGP protocol; the default destinations are indicated as zero.

Des.	Next	Cost
N1	—	1
N4	R4	2
N8	R5	1
N9	R5	1
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R1

Des.	Next	Cost
N1	—	1
N4	R3	2
N8	R1	2
N9	R1	2
N10	R6	1
N11	R6	1
N12	R6	1
N13	R3	3
N14	R3	3
N15	R3	3

Table for R2

Des.	Next	Cost
N1	R2	2
N4	—	1
N8	R2	3
N9	R2	3
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R3

Des.	Next	Cost
N1	R1	2
N4	—	1
N8	R1	2
N9	R1	2
N10	R3	3
N11	R3	3
N12	R3	3
N13	R9	1
N14	R9	1
N15	R9	1

Table for R4

Des.	Next	Cost
N8	—	1
N9	—	1
0	R1	1

Table for R5

Des.	Next	Cost
N10	—	1
N11	—	1
N12	R7	2
0	R2	1

Table for R6

Des.	Next	Cost
N10	—	1
N11	R6	2
N12	—	1
0	R6	2

Table for R7

Des.	Next	Cost
N10	R6	2
N11	—	1
N12	—	1
0	R6	2

Table for R8

Des.	Next	Cost
N13	—	1
N14	—	1
N15	—	1
0	R4	1

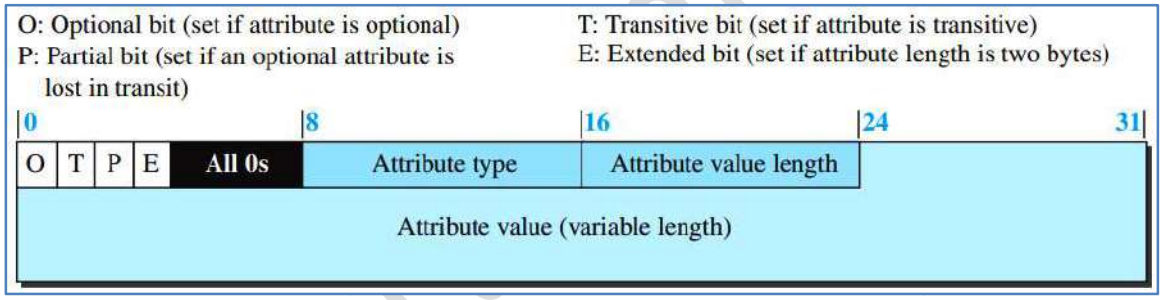
Table for R9

Address Aggregation

- The reader may have realized that intradomain forwarding tables obtained with the help of the BGP4 protocols may become huge in the case of the global Internet because many destination networks may be included in a forwarding table.
- ***Fortunately, BGP4 uses the prefixes as destination identifiers and allows the aggregation of these prefixes*** (discussed in Chapter 18 Introduction to network layer).
- For example, prefixes 14.18.20.0/26, 14.18.20.64/26, 14.18.20.128/26, and 14.18.20.192/26, can be combined into 14.18.20.0/24 if all four subnets can be reached through one path. Even if one or two of the aggregated prefixes need a separate path, the longest prefix principle we discussed earlier allows us to do so.

Path Attributes

- Interdomain routing is more ***involved and naturally needs more information about how to reach the final destination. In BGP these pieces are called path attributes***.
- BGP allows a destination to be associated with up to seven path attributes. Path attributes are divided into ***two broad categories: well-known and optional***.
  - A ***well-known attribute*** must be recognized by all routers; it can be mandatory - it must be present in any BGP update message, or discretionary.
  - An ***optional attribute need not be***. An optional attribute can be either transitive, which means it can pass to the next AS, or intransitive, which means it cannot.
- All attributes are inserted after the corresponding destination prefix in an update message. The format for an attribute is shown in Figure.

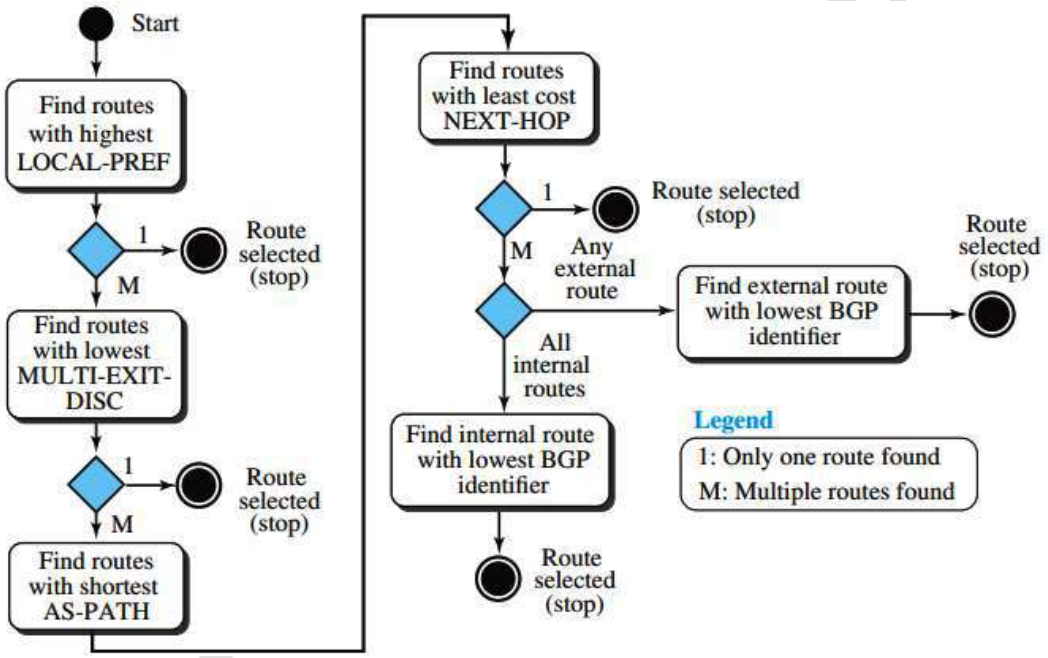


- The ***first byte in each attribute defines the four attribute flags*** (as shown in the figure).
- The ***next byte defines the type of attributes assigned by ICANN*** (only ***seven types*** have been assigned.
- The ***attribute value length defines the length of the attribute value field*** (not the length of the whole attributes section).
- The following gives a brief description of each attribute.
  - ***ORIGIN (type 1).*** This is a well-known mandatory attribute, which defines the source of the routing information. This attribute can be defined by one of the three values: 1, 2, and 3.
    Value 1 - the information about the path has been taken from an intradomain protocol (RIP or OSPF).
    Value 2 - the information comes from BGP.
    Value 3 - it comes from an unknown source.
  - ***AS-PATH (type 2)*** which ***defines the list of autonomous systems through which the destination can be reached***. The AS-PATH attribute (discussed in path-vector routing), helps prevent a loop. Whenever an update message arrives at a router that lists the current AS as the path, the router drops that path. The ***AS-PATH can also be used in route selection***.
  - ***NEXT-HOP (type 3)*** which ***defines the next router to which the data packet should be forwarded***. This attribute helps to inject path information collected through the operations of eBGP and iBGP into the intradomain routing protocols such as RIP or OSPF.
  - ***MULT-EXIT-DISC (type 4).*** The multiple-exit discriminator is an optional intransitive attribute, which discriminates among multiple exit paths to a destination.

- ***LOCAL-PREF (type 5).*** It is ***normally set by the administrator, based on the organization policy***. The routes the administrator prefers are given a higher local preference value (an attribute value of 4-byte unsigned integer).
- ***ATOMIC-AGGREGATE (type 6)*** which ***defines the destination prefix as a single destination network.*** This attribute has no value field, which means the value of the length field is zero.
- ***AGGREGATOR (type 7).*** This is an ***optional transitive attribute***, which emphasizes that the destination prefix is an aggregate. The attribute value gives the number of the last AS that did the aggregation followed by the IP address of the router that did so.

Route Selection

- We have been silent about how a route is selected by a BGP router mostly because our simple example has one route to a destination. In the case ***where multiple routes are received to a destination, BGP needs to select one among them***. The ***route selection process in BGP is not as easy as the ones in the intradomain routing protocol that is based on the shortest-path tree***.
- A route in BGP has some attributes attached to it and it may come from an eBGP session or an iBGP session. Figure shows the flow diagram as used by common implementations.

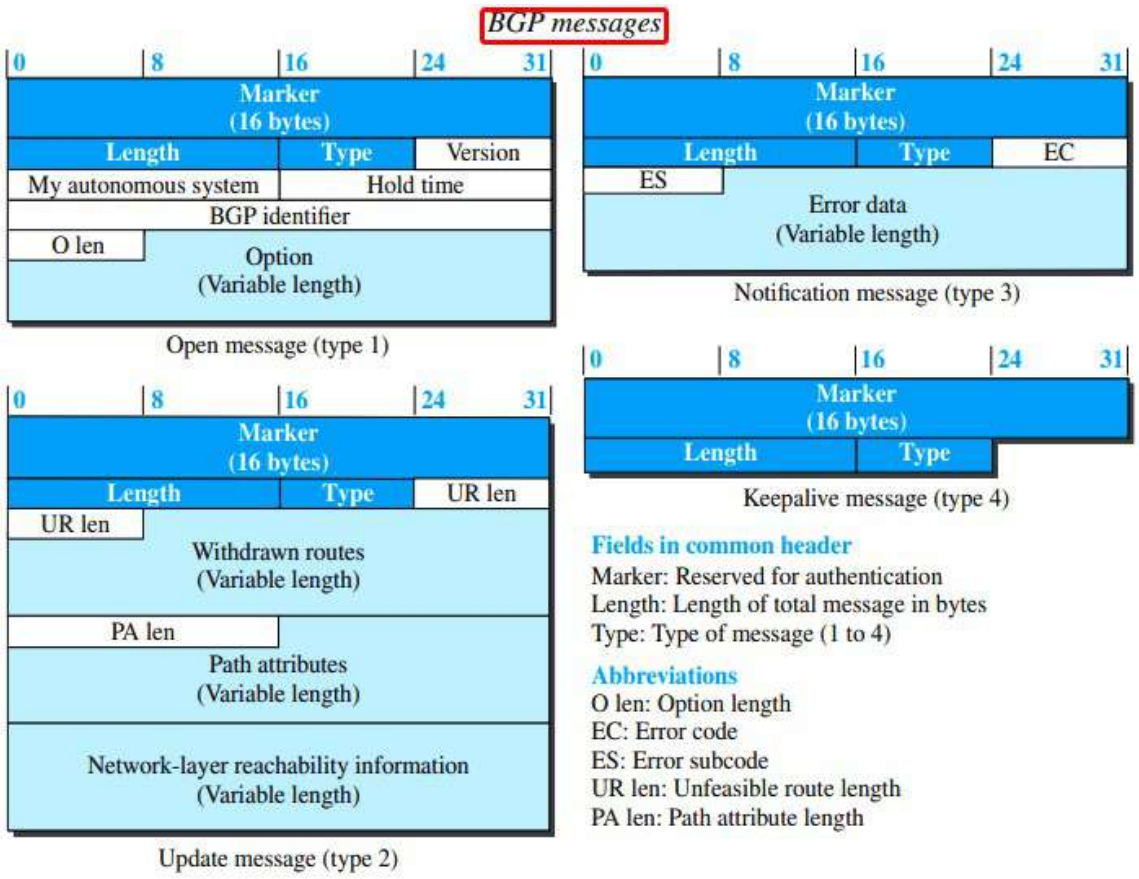


Messages

- BGP uses ***four types of messages for communication between the BGP speakers across the ASs and inside an AS.*** All BGP packets share the same common header.
  1. ***Open Message***. To create a neighborhood relationship, a router running BGP opens a TCP connection with a neighbor and sends an open message.
  2. ***Update Message***. The update message is the heart of the BGP protocol. It is used by a router to withdraw destinations that have been advertised previously, to announce a route to a new destination, or both.
  3. ***Keepalive Message***. The BGP peers that are running exchange keepalive messages regularly (before their hold time expires) to tell each other that they are alive.
  4. ***Notification***. A notification message is sent by a router whenever an error condition is detected or a router wants to close the session.

Performance

- BGP performance can be compared with RIP. BGP speakers exchange a lot of messages to create forwarding tables, but BGP is free from loops and count-to-infinity. The same weakness we mention for RIP about propagation of failure and corruption also exists in BGP.



Ch. 21. Multicast Routing

Multicasting Routing-MOSPF

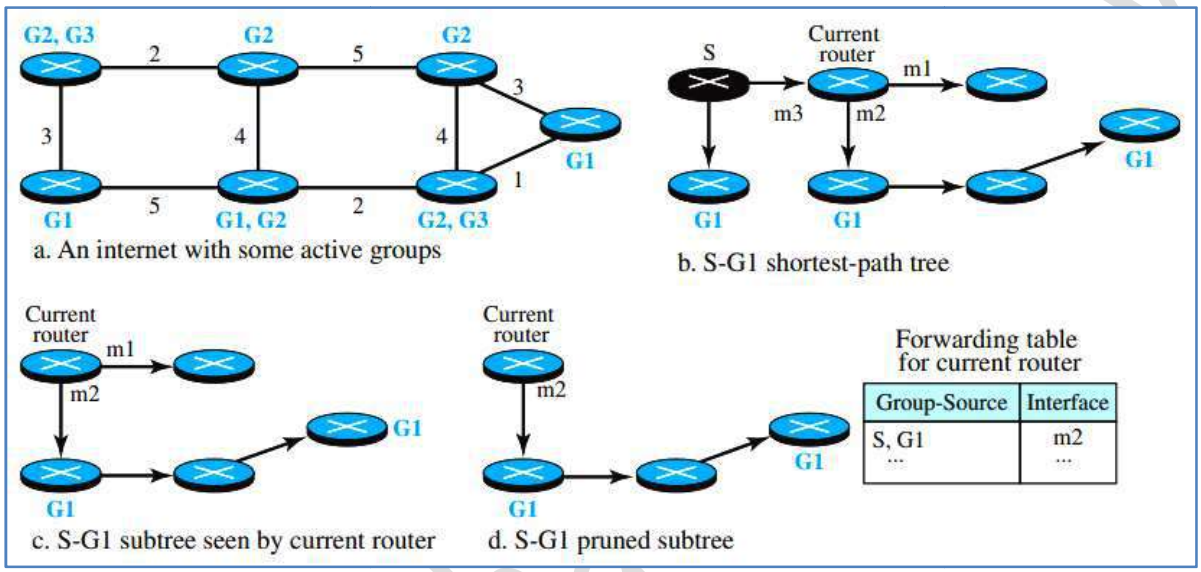
- ***Multicast Open Shortest Path First (MOSPF)*** is the extension of the Open Shortest Path First (OSPF) protocol, which is used in unicast routing.
- It also uses the source-based tree approach to multicasting.
- If the internet is running a unicast link-state routing algorithm, the idea can be extended to provide a multicast link-state routing algorithm.
- Recall that in unicast link-state routing, each router in the internet has a link-state database (LSDB) that can be used to create a shortest-path tree.
- To ***extend unicasting to multicasting, each router needs to have another database***, as with the case of unicast distance-vector routing, to show which interface has an active member in a particular group.
- Now a ***router goes through the following steps to forward a multicast packet*** received from source S and to be sent to destination G (a group of recipients):
  1. ***The router uses the Dijkstra algorithm to create a shortest-path tree with S as the root and all destinations in the internet as the leaves***. In this case, the root of the tree is the source of the packet defined in the source address of the packet. The router is capable of creating this tree because it has the LSDB, the whole topology of the internet; the Dijkstra algorithm can be used to create a tree with any root, no matter which router is using it. The shortest-path tree created this way depends on the specific source. ***For each source we need to create a different tree.***
  2. ***The router finds itself in the shortest-path tree created in the first step***. In other words, the router ***creates a shortest-path subtree with itself*** as the root of the subtree.
  3. The ***shortest-path subtree is actually a broadcast subtree with the router as the root and all networks as the leaves***. The router now uses a strategy similar to the one we describe in the case of DVMRP to prune the broadcast tree and to change it to a multicast tree.

Prof. Parthasarathy P V

The IGMP protocol is used to find the information at the leaf level. MOSPF has added a new type of link state update packet that floods the membership to all routers. The router can use the information it receives in this way and prune the broadcast tree to make the multicast tree.

4. The ***router can now forward the received packet out of only those interfaces that correspond to the branches of the multicast tree***. We need to make certain that a copy of the multicast packet reaches all networks that have active members of the group and that it does not reach those networks that do not.

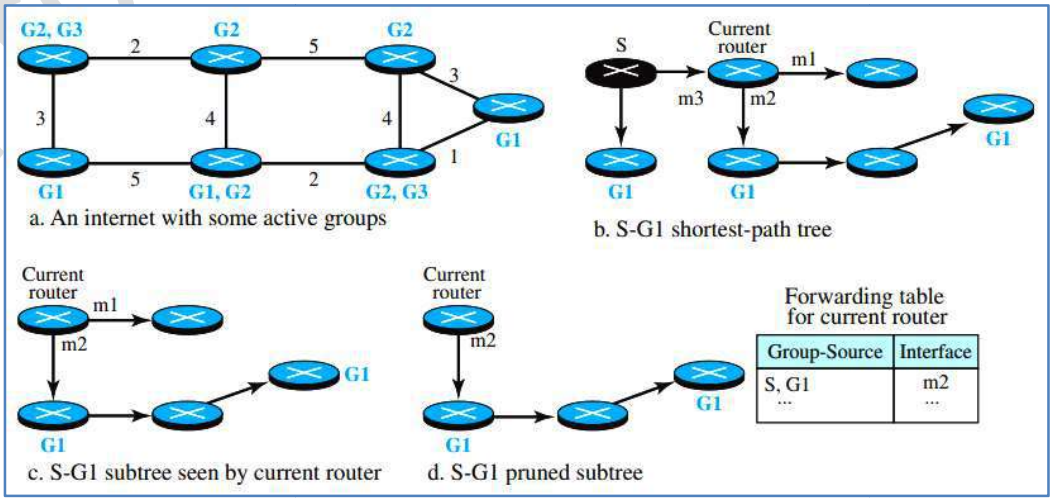
- Figure shows an example of using the steps to change a graph to a multicast tree. For simplicity, we have not shown the network, but we added the groups to each router.
- The figure shows how a source-based tree is made with the source as the root and changed to a multicast subtree with the root at the current router.



Ch. 21. Multicast Routing

Multicasting Routing-MOSPF

- ***Multicast Open Shortest Path First (MOSPF)*** is the extension of the Open Shortest Path First (OSPF) protocol, which is used in unicast routing.
- It also uses the source-based tree approach to multicasting.
- If the internet is running a unicast link-state routing algorithm, the idea can be extended to provide a multicast link-state routing algorithm.
- Recall that in unicast link-state routing, each router in the internet has a link-state database (LSDB) that can be used to create a shortest-path tree.
- To ***extend unicasting to multicasting, each router needs to have another database***, as with the case of unicast distance-vector routing, to show which interface has an active member in a particular group.
- Now a ***router goes through the following steps to forward a multicast packet*** received from source S and to be sent to destination G (a group of recipients):
  1. ***The router uses the Dijkstra algorithm to create a shortest-path tree with S as the root and all destinations in the internet as the leaves***. In this case, the root of the tree is the source of the packet defined in the source address of the packet. The router is capable of creating this tree because it has the LSDB, the whole topology of the internet; the Dijkstra algorithm can be used to create a tree with any root, no matter which router is using it. The shortest-path tree created this way depends on the specific source. ***For each source we need to create a different tree.***
  2. ***The router finds itself in the shortest-path tree created in the first step***. In other words, the router ***creates a shortest-path subtree with itself*** as the root of the subtree.
  3. The ***shortest-path subtree is actually a broadcast subtree with the router as the root and all networks as the leaves***. The router now uses a strategy similar to the one we describe in the case of DVMRP to prune the broadcast tree and to change it to a multicast tree.
     The IGMP protocol is used to find the information at the leaf level. MOSPF has added a new type of link state update packet that floods the membership to all routers. The router can use the information it receives in this way and prune the broadcast tree to make the multicast tree.
  4. The ***router can now forward the received packet out of only those interfaces that correspond to the branches of the multicast tree***. We need to make certain that a copy of the multicast packet reaches all networks that have active members of the group and that it does not reach those networks that do not.
- Figure shows an example of using the steps to change a graph to a multicast tree. For simplicity, we have not shown the network, but we added the groups to each router.
- The figure shows how a source-based tree is made with the source as the root and changed to a multicast subtree with the root at the current router.



BCS502 COMPUTER NETWORKS

Module-4: Transport Layer

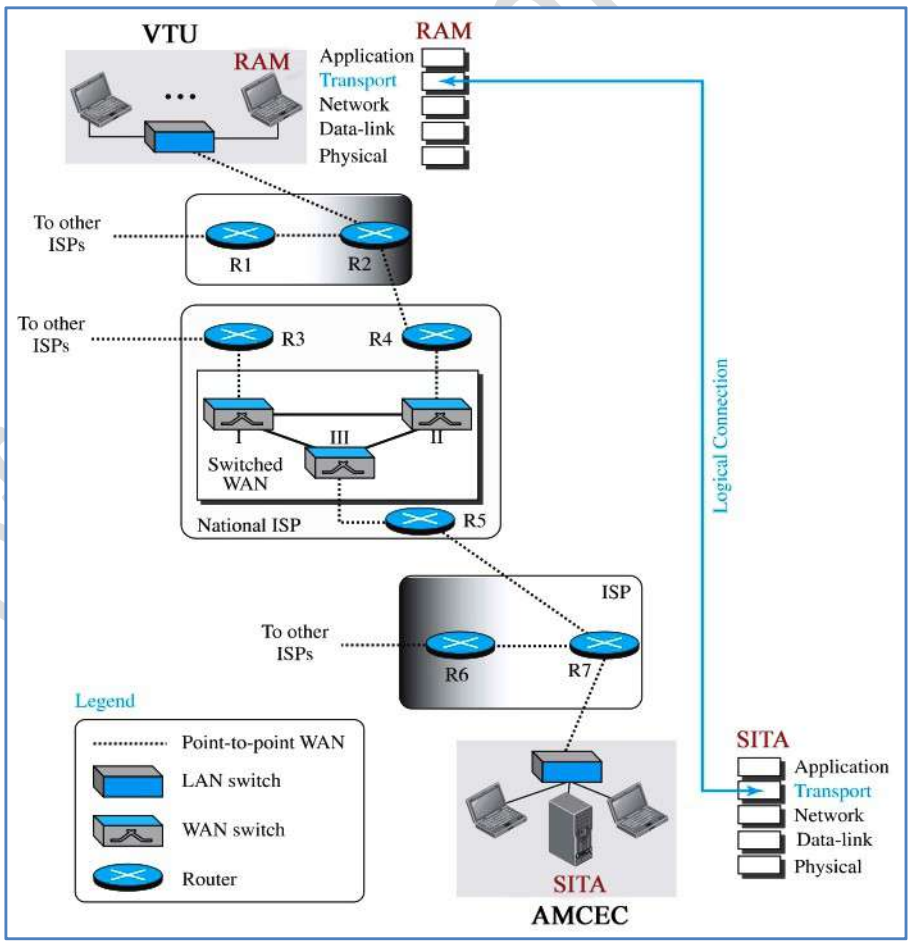
Ch. 23. Introduction to Transport Layer: Introduction, Transport-Layer Protocols
Ch. 24. Transport-Layer Protocols: Services, Features, Segments, TCP connections, Flow control, Error control, Congestion control.
Textbook: Ch. 23.1, 23.2, 24.1 – 24.3.4, 24.3.6 - 24.3.9.

Chapter 23: Introduction to Transport Layer [P-730]

Introduction, Transport-Layer Protocols

Introduction to Transport-Layer

- The transport layer is located between the application layer and the network layer. It provides ***a process-to-process communication between two application layers***, one at the local host and the other at the remote host.
- The transport layer is the heart of the TCP/IP protocol suite; it is the ***end-to-end logical vehicle for transferring data from one point to another in the Internet***.
- Communication is provided using a logical connection, which means that the two application layers, which can be located in different parts of the globe, assume that there is an imaginary direct connection through which they can send and receive messages. Figure shows the idea behind this logical connection.

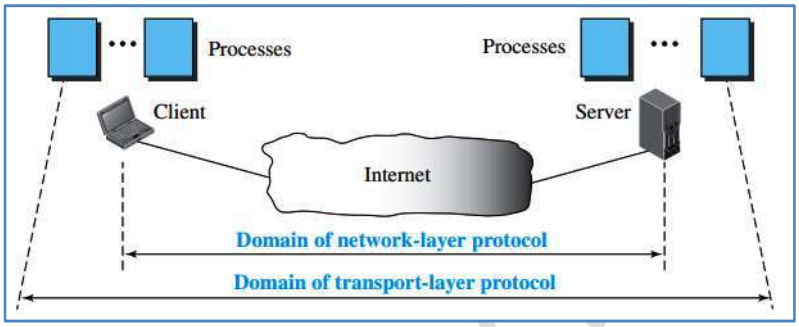


Transport-Layer Services

- The transport layer is responsible for providing services to the application layer; it receives services from the network layer.

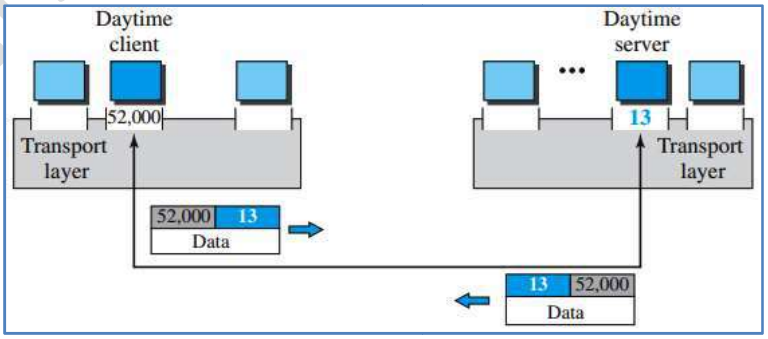
Process-to-Process Communication

- The ***first duty of a transport-layer protocol is to provide process-to-process communication***.
- A ***process is an application-layer entity (running program) that uses the services of the transport layer***. We need to understand the difference between host-to-host communication and process-to-process communication.
- The network layer is responsible for communication at the computer level (host-to-host communication). A ***network-layer protocol can deliver the message*** only to the destination computer. However, ***this is an incomplete delivery***. The message still needs to be handed to the correct process. This is where a transport-layer protocol takes over. A ***transport-layer protocol is responsible for delivery of the message to the appropriate process***.

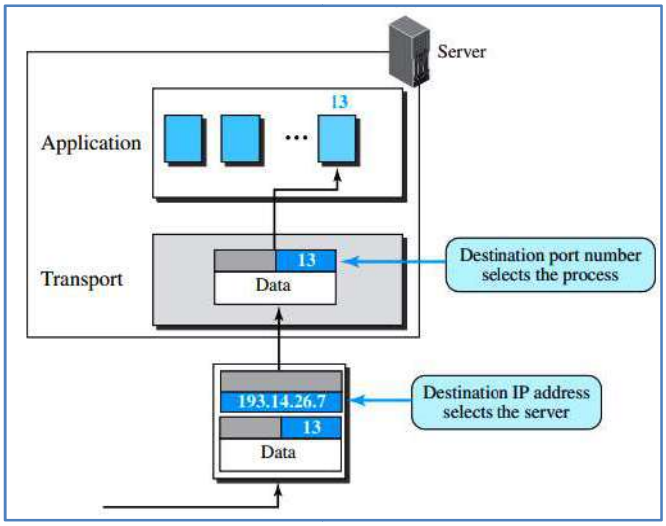


Addressing: Port Numbers

- Although there are a few ways ***to achieve process-to-process communication, the most common is through the client-server paradigm***. A process on the local host, called a ***client***, needs services from a process usually on the remote host, called a ***server***.
- Operating systems today support both multiuser and multiprogramming environments. A remote computer can run several server programs at the same time, just as several local computers can run one or more client programs at the same time.
- For ***communication, we must define the local host, local process, remote host, and remote process***. The local host and the remote host are defined ***using IP addresses***. To define the processes, we need ***second identifiers, called port numbers***. In the TCP/IP protocol suite, ***the port numbers are integers between 0 and 65,535 (16 bits).***
- The ***client program defines itself with a port number, called the ephemeral port number***. The word ephemeral means "short-lived" and is used because the life of a client is normally short. An ephemeral port number is recommended to be greater than 1023 for some client/server programs to work properly.



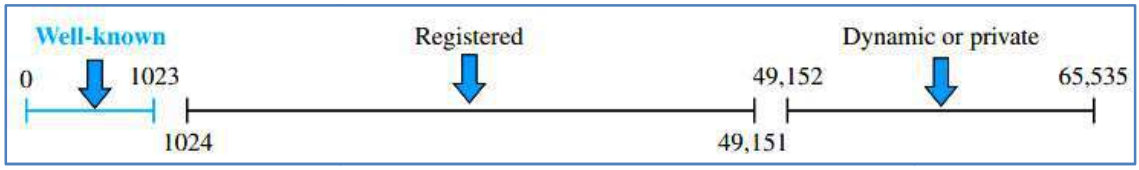
- The server process must also define itself with a port number. ***TCP/IP has decided to use universal port numbers for servers***; these are ***called well-known port numbers***.
- It should be clear by now that the IP addresses and port numbers play different roles in selecting the final destination of data. The destination IP address defines the host among the different hosts in the world. After the ***host has been selected, the port number defines one of the processes on this particular host*** (see Figure).



Prof. Parthasarathy P V

ICANN Ranges

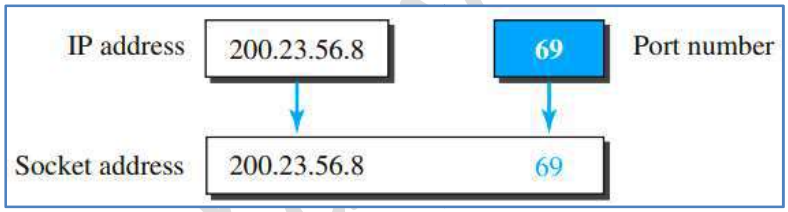
- ICANN has divided the ***port numbers into three ranges: well-known, registered, and dynamic.***



- ***Well-known ports***. The ports ranging from 0 to 1023 are assigned and controlled by ICANN. These are the well-known ports.
- ***Registered ports***. The ports ranging from 1024 to 49,151 are not assigned or controlled by ICANN. They can only be registered with ICANN to prevent duplication.
- ***Dynamic (or private) ports***. The ports ranging from 49,152 to 65,535 are neither controlled nor registered. They can be used as temporary or private port numbers.

Socket Addresses

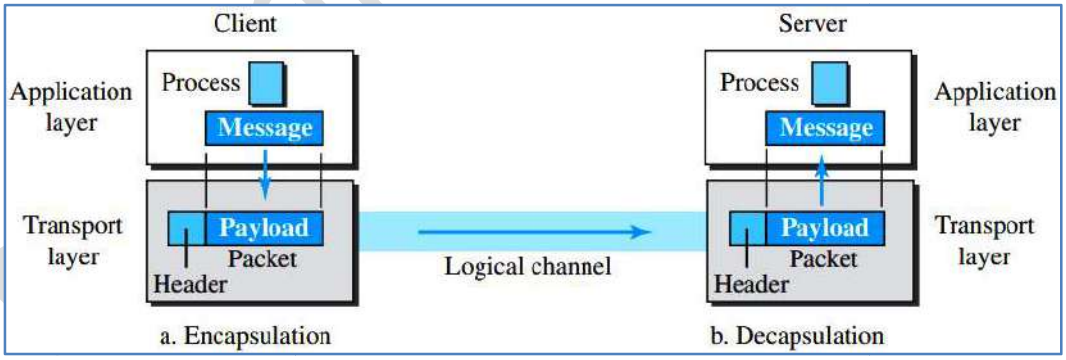
- A transport-layer protocol in the TCP suite needs both the IP address and the port number, at each end, to make a connection. The ***combination of an IP address*** and a ***port number*** is ***called a socket address***. The client socket address defines the client process uniquely just as the server socket address defines the server process uniquely (see Figure).



- ***To use the services of the transport layer in the Internet, we need a pair of socket addresses: the client socket address and the server socket address.*** These four pieces of information are part of the network-layer packet header and the transport-layer packet header. The first header contains the IP addresses; the second header contains the port numbers.

Encapsulation and Decapsulation

- To send a message from one process to another, the transport-layer protocol encapsulates and decapsulates messages (Figure).

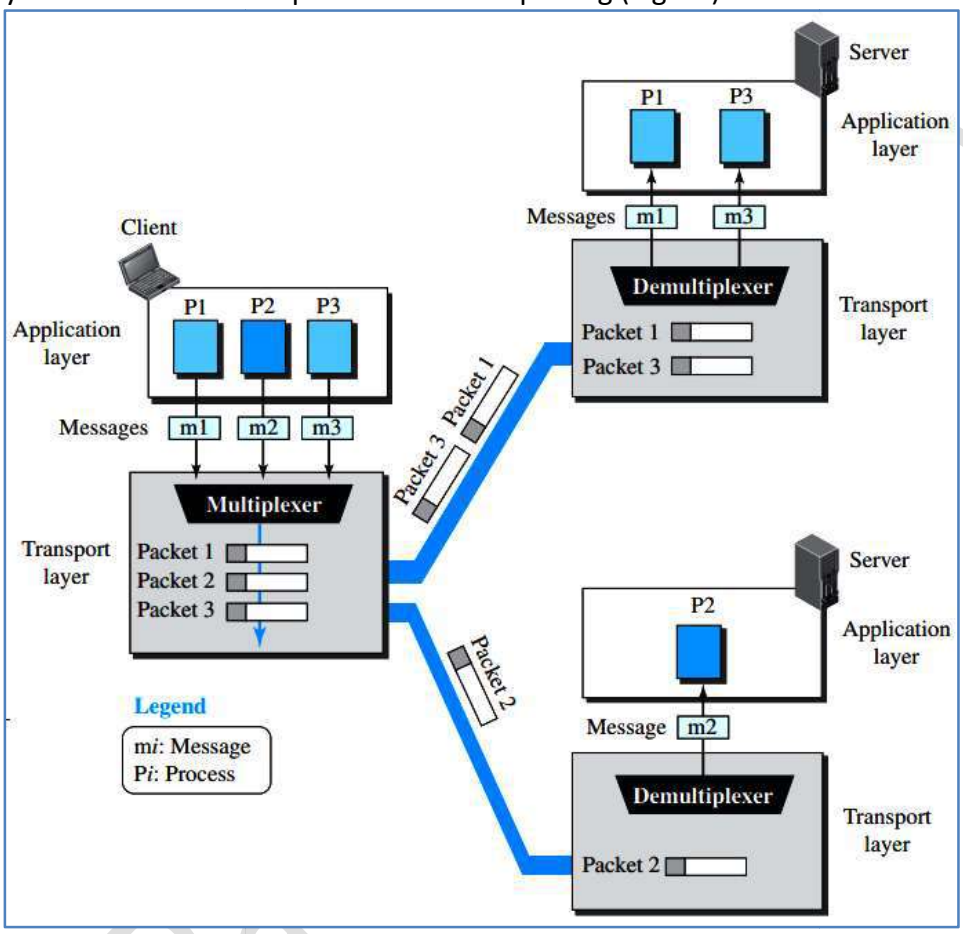


- ***Encapsulation happens at the sender site***. When a process has a message to send, it passes the message to the transport layer along with a pair of socket addresses and some other pieces of information, which depend on the transport-layer protocol.
- The transport layer receives the data and adds the transport-layer header. ***The packets at the transport layer in the Internet are called*** user ***datagrams, segments, or packets, depending on what transport-layer protocol*** we use. We refer to ***transport-layer payloads*** as ***packets***.
- ***Decapsulation happens at the receiver site***. When the message arrives at the destination transport layer, the header is dropped and the transport layer delivers the message to the process running at the application layer.

- The sender socket address is passed to the process in case it needs to respond to the message received.

Multiplexing and Demultiplexing

- Whenever an ***entity accepts items from more than one source, this is referred to as multiplexing*** (many to one); whenever an ***entity delivers items to more than one source, this is referred to as demultiplexing*** (one to many). The transport layer at the source performs multiplexing; the transport layer at the destination performs demultiplexing (Figure).

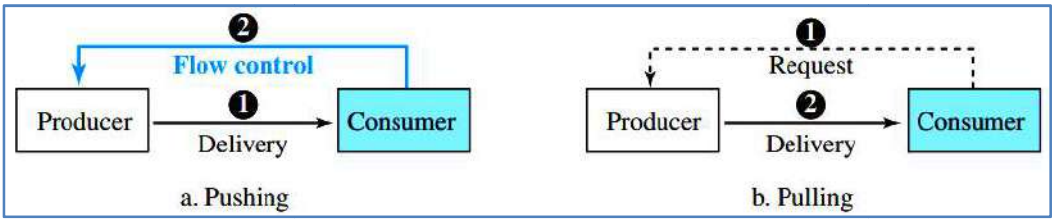


Flow Control

- Whenever an entity produces items and another entity consumes them, there should be a balance between production and consumption rates. If the items are produced faster than they can be consumed, the consumer can be overwhelmed and may need to discard some items. If the items are produced more slowly than they can be consumed, the consumer must wait, and the system becomes less efficient. ***Flow control is related to the first issue. We need to prevent losing the data items at the consumer site.***

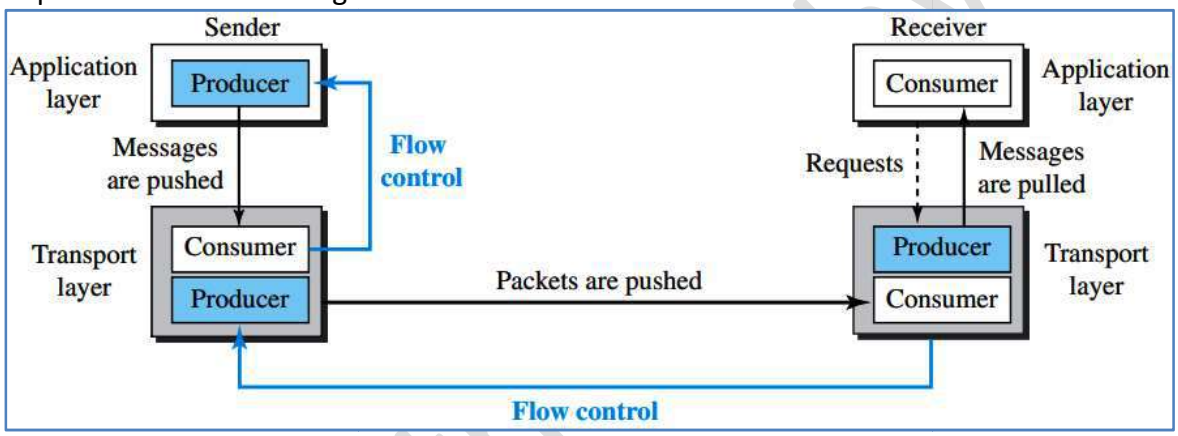
Pushing or Pulling

- Delivery of items from a producer to a consumer can occur in one of ***two ways: pushing or pulling***. If the sender delivers items whenever they are produced - without a prior request from the consumer - the delivery is referred to as pushing. If the producer delivers the items after the consumer has requested them, the delivery is referred to as pulling (Figure).



Flow Control at Transport Layer

- In communication at the transport layer, we are dealing with ***four entities***:
  1. sender process
  2. sender transport layer
  3. receiver transport layer
  4. receiver process.
- The ***sending process*** at the application layer is only a producer. It produces message chunks and pushes them to the transport layer.
- The ***sending transport layer*** has a double role: it is both a consumer and a producer. It consumes the messages pushed by the producer. It encapsulates the messages in packets and pushes them to the receiving transport layer.
- The ***receiving transport layer*** also has a double role: it is the consumer for the packets received from the sender and the producer that decapsulates the messages and delivers them to the application layer.
- The ***receiver process*** - is normally a pulling delivery; the transport layer waits until the application-layer process asks for messages.



- Figure shows that we need at least two cases of flow control: from the sending transport layer to the sending application layer and from the receiving transport layer to the sending transport layer.

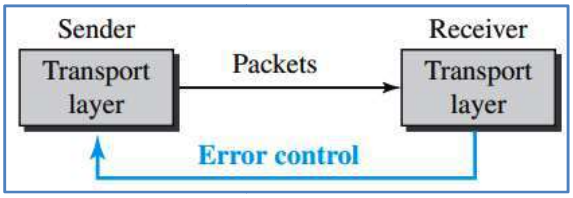
Buffers

- Although flow control can be implemented in several ways, ***one of the solutions is normally to use two buffers: one at the sending transport layer and the other at the receiving transport layer***. A buffer is a set of memory locations that can hold packets at the sender and receiver. The flow control communication can occur by sending signals from the consumer to the producer.
- ***When the buffer of the sending transport layer is full***, it informs the application layer to stop passing chunks of messages; when there are some vacancies, it informs the application layer that it can pass message chunks again.
- ***When the buffer of the receiving transport layer is full***, it informs the sending transport layer to stop sending packets. When there are some vacancies, it informs the sending transport layer that it can send packets again.

Error Control

- In the Internet, since the underlying network layer (IP) is unreliable, we need to make the transport layer reliable if the application requires reliability. Reliability can be achieved to add error control services to the transport layer.
- ***Error control at the transport layer is responsible for***
  1. Detecting and discarding corrupted packets.
  2. Keeping track of lost and discarded packets and resending them.
  3. Recognizing duplicate packets and discarding them.
  4. Buffering out-of-order packets until the missing packets arrive.

- ***Error control***, unlike flow control***, involves only the sending and receiving transport layers***. Figure shows the error control between the sending and receiving transport layers. As with the case of flow control, the receiving transport layer manages error control, most of the time, by informing the sending transport layer about the problems.



Sequence Numbers

- **Error control requires that the sending transport layer knows which packet is to be resent and the receiving transport layer knows which packet is a duplicate, or which packet has arrived out of order. This can be done if the packets are numbered.** We can add a field to the transport-layer packet to hold the sequence number of the packet. When a packet is corrupted or lost, the receiving transport layer can somehow inform the sending transport layer to resend that packet using the sequence number. The receiving transport layer can also detect duplicate packets if two received packets have the same sequence number. The out-of-order packets can be recognized by observing gaps in the sequence numbers.
- If the header of the packet allows m bits for the sequence number, the sequence numbers range from ***0*** to $2^m - 1$. For example, if m is 4, the only sequence numbers are 0 through 15, inclusive. However, we can wrap around the sequence. So the sequence numbers in this case are: ***0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, …***

Acknowledgment

- We can use both positive and negative signals as error control, but we discuss only positive signals, which are more common at the transport layer. The receiver side can send an acknowledgment (ACK) for each of a collection of packets that have arrived safe and sound. The receiver can simply discard the corrupted packets.
- The sender can detect lost packets if it uses a timer. When a packet is sent, the sender starts a timer. If an ACK does not arrive before the timer expires, the sender resends the packet. Duplicate packets can be silently discarded by the receiver. Out-of-order packets can be either discarded (to be treated as lost packets by the sender), or stored until the missing one arrives.

Combination of Flow and Error Control

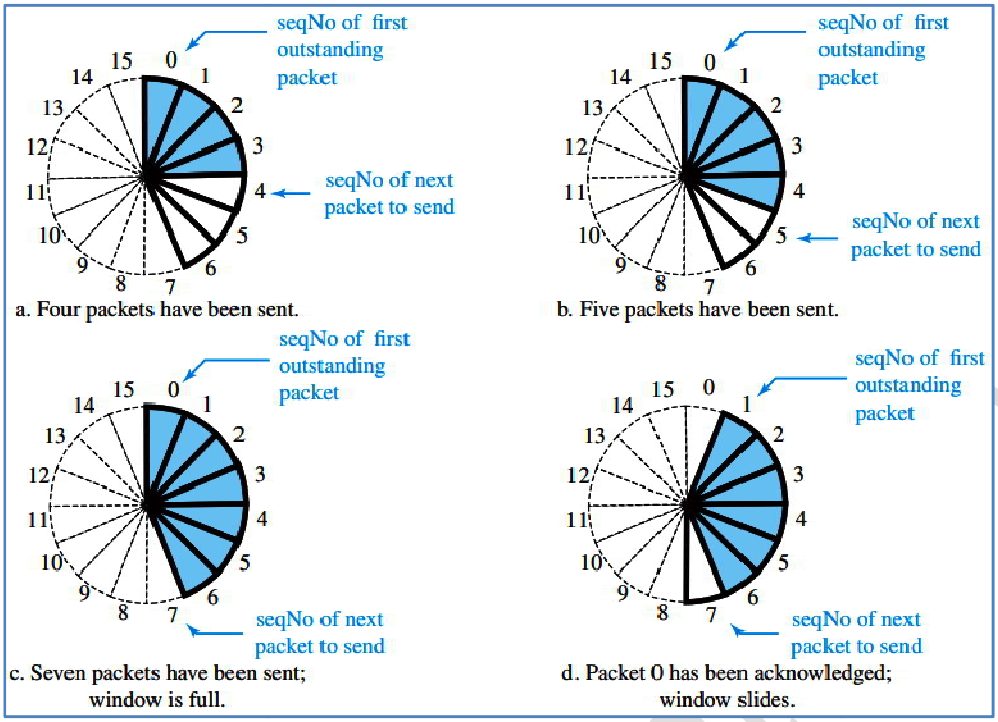
- ***Flow control*** requires the use of two buffers, one at the sender site and the other at the receiver site. ***Error control*** requires the use of sequence and acknowledgment numbers by both sides.
- These two requirements can be combined if we use two numbered buffers, one at the sender, one at the receiver.
- ***At the sender***, when a packet is prepared to be sent, we use the number of the next free location, x, in the buffer as the sequence number of the packet. When the packet is sent, a copy is stored at memory location x, awaiting the acknowledgment from the other end. When an acknowledgment related to a sent packet arrives, the packet is purged and the memory location becomes free.
- ***At the receiver***, when a packet with sequence number y arrives, it is stored at the memory location y until the application layer is ready to receive it. An acknowledgment can be sent to announce the arrival of packet y.

Sliding Window

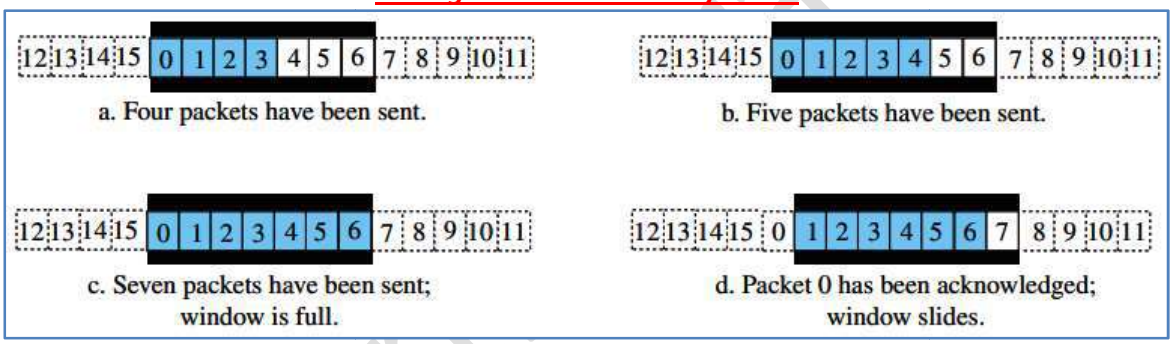
- Since the sequence numbers use modulo 2^m, a circle can represent the sequence numbers from ***0*** to $2^m - 1$ (Figure). The buffer is represented as a set of slices, called the sliding window, that occupies part of the circle at any time.
- At the sender site, when a packet is sent, the corresponding slice is marked. When all the slices are marked, it means that the buffer is full and no further messages can be accepted from the application layer. ***When an acknowledgment arrives, the corresponding slice is unmarked.***

Prof. Parthasarathy P V

Sliding window in circular format



Sliding window in Linear format



Congestion Control

- An important issue in a packet-switched network, such as the Internet, is congestion.
- Congestion in a network may occur if the load on the network—the number of packets sent to the network—is greater than the capacity of the network—the number of packets a network can handle.
- ***Congestion control refers to the mechanisms and techniques that control the congestion and keep the load below the capacity.***
- Congestion in a network or internetwork occurs because routers and switches have queues—buffers that hold the packets before and after processing. Congestion at the transport layer is actually the result of congestion at the network layer, which manifests itself at the transport layer.

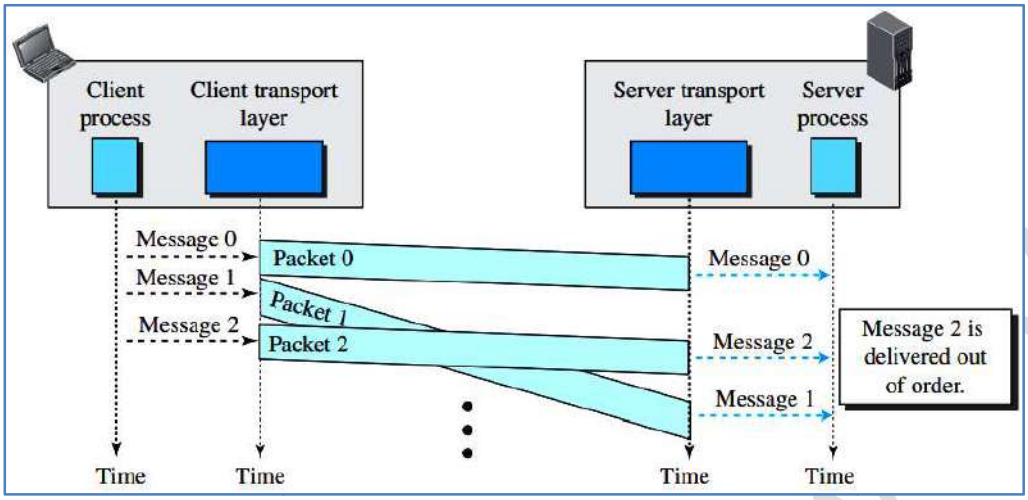
Connectionless and Connection-Oriented Protocols

- A transport-layer protocol provide two types of services: connectionless and connection-oriented.
- ***At the transport layer, we are not concerned about the physical paths of packets*** (we assume a logical connection between two transport layers). Connectionless service at the transport layer means ***independency between packets; connection-oriented means dependency.***

Connectionless Service

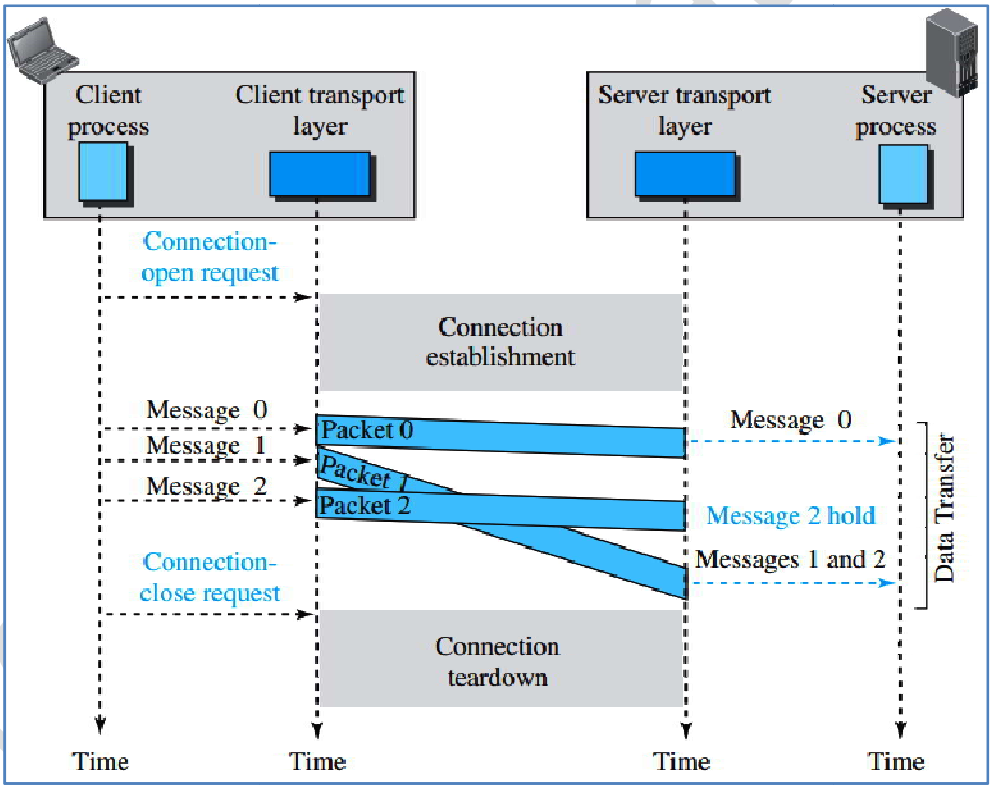
- In a connectionless service, the source process (application program) needs to divide its message into chunks of data of the size acceptable by the transport layer and deliver them to the transport layer one by one.

- The transport layer treats each chunk as a single unit without any relation between the chunks. When a chunk arrives from the application layer, the transport layer encapsulates it in a packet and sends it.



Connection-Oriented Service

- In a connection-oriented service, the client and the server first need to establish a logical connection between themselves. The data exchange can only happen after the connection establishment. After data exchange, the connection needs to be torn down.

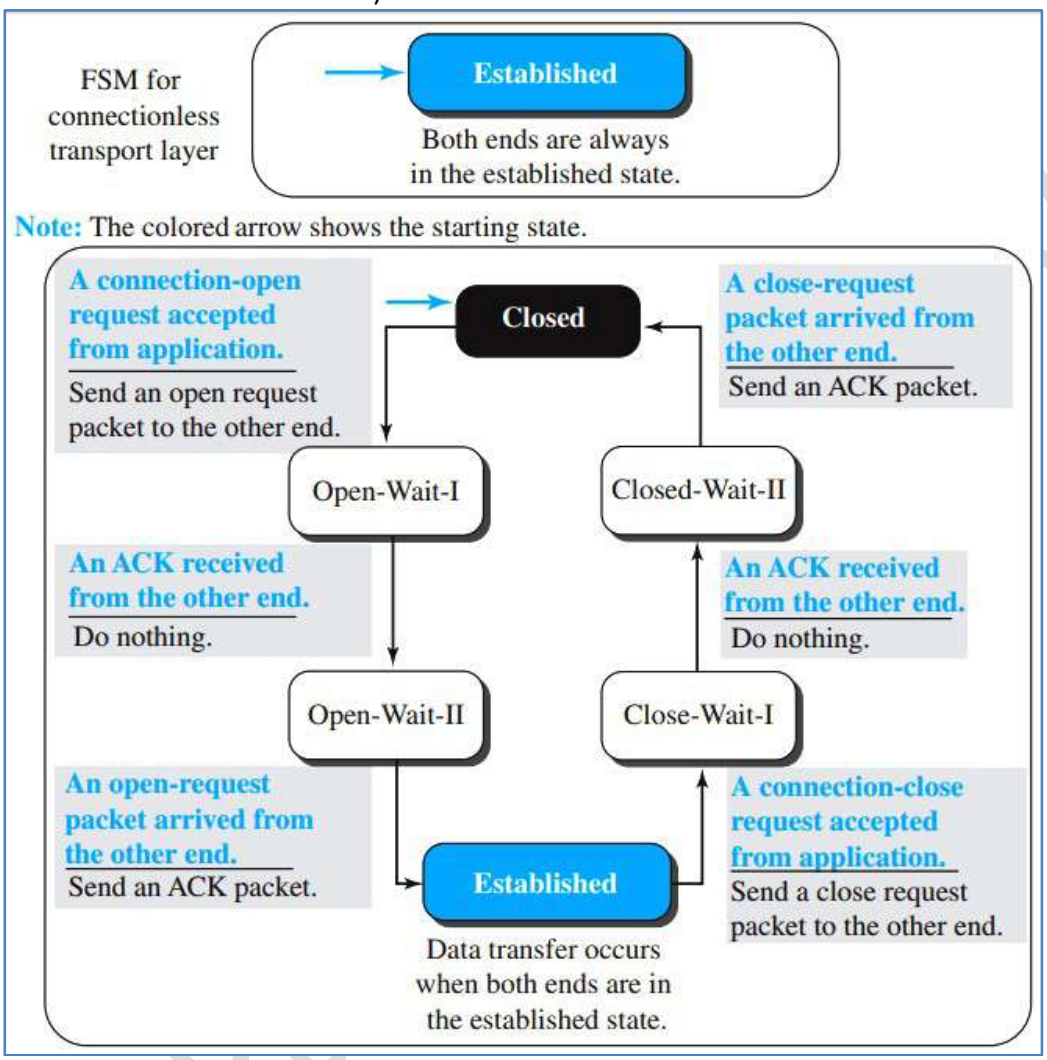


- At the transport layer, connection-oriented service involves only the two hosts; the service is end to end. This means that we should be able to make a connection-oriented protocol at the transport layer over either a connectionless or connection-oriented protocol at the network layer.
- Figure shows the ***connectionestablishment, data-transfer, and tear-down phases in a connection-oriented service at the transport layer***. We can ***implement flow control, error control, and congestion control*** in a connectionoriented protocol.

Finite State Machine

- The behavior of a transport-layer protocol, both when it provides a connectionless and when it provides a connection-oriented protocol, can be better shown as a finite state machine (FSM).

- Figure shows a representation of a transport layer using an FSM. Using this tool, each transport layer (sender or receiver) is taught as a machine with a finite number of states. The machine is always in one of the states until an event occurs. Each event is associated with two reactions: defining the list (possibly empty) of actions to be performed and determining the next state (which can be the same as the current state).



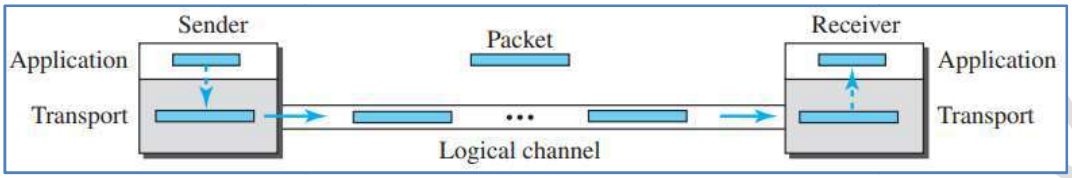
- An FSM in a connection-oriented transport layer, on the other hand, needs to go through ***three states before reaching the established state.***
- The machine also needs to go through three states before closing the connection. The machine is in the closed state when there is no connection. It remains in this state until a request for opening the connection arrives from the local process; the machine sends an open request packet to the remote transport layer and moves to the ***open-wait-I state***.
- When an ***acknowledgment is received*** from the other end, the local FSM moves to the ***open-wait-II*** state. Then a unidirectional connection has been established, but if a bidirectional connection is needed, the machine needs to wait in this state until the other end also requests a connection.
- When ***the request is received***, the machine sends an acknowledgment and moves to the established state. Data and data acknowledgment can be exchanged between the two ends when they are both in the established state.
- To ***tear down a connection, the application layer sends a close request message to its local transport layer.*** The transport layer sends a close-request packet to the other end and moves to close-wait-I state. When an acknowledgment is received from the other end, the machine moves to the close-wait-II state and waits for the close-request packet from the other end. When this packet arrives, the machine sends an acknowledgment and moves to the closed state.

TRANSPORT-LAYER PROTOCOLS

- The TCP/IP protocol uses a transport-layer protocol that is either a modification or a combination of some of these protocols.

Simple Protocol

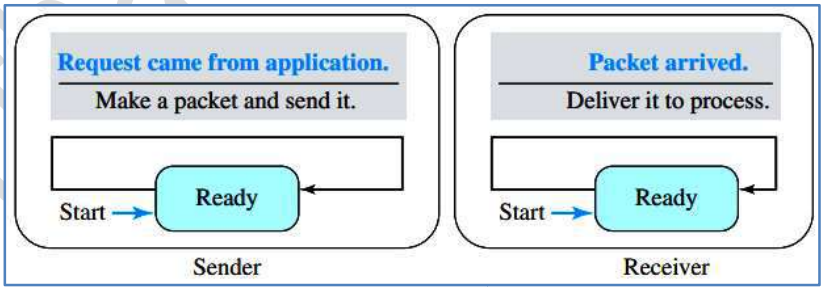
- Our first protocol - simple connectionless protocol with neither flow nor error control. We assume that the receiver can immediately handle any packet it receives. In other words, the receiver can never be overwhelmed with incoming packets. Figure shows the layout for this protocol.



- The ***transport layer at the sender*** gets a message from its application layer, makes a packet out of it, and sends the packet.
- The ***transport layer at the receiver receives a packet*** from its network layer, extracts the message from the packet, and delivers the message to its application layer. The transport layers of the sender and receiver provide transmission services for their application layers.

FSMs

- The sender site should not send a packet until its application layer has a message to send. The receiver site cannot deliver a message to its application layer until a packet arrives. We can show these requirements using two FSMs. Each ***FSM has only one state, the ready state***.
- The ***sending machine remains in the ready state until a request comes from the process in the application layer***. When this event occurs, the sending machine encapsulates the message in a packet and sends it to the receiving machine.
- The ***receiving machine remains in the ready state until a packet arrives from the sending machine***. When this event occurs, the receiving machine decapsulates the message out of the packet and delivers it to the process at the application layer.
- Figure shows the FSMs for the simple protocol. We see later that the UDP protocol is a slight modification of this protocol.



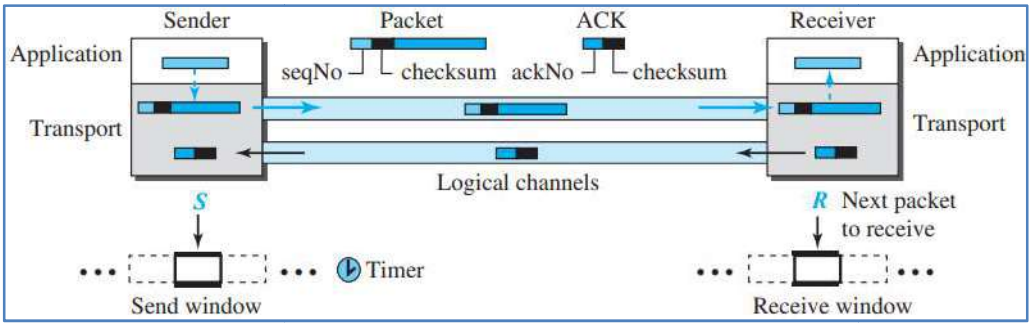
Stop-and-Wait Protocol

- Our second protocol is a connection-oriented protocol called the Stop-and-Wait protocol, which uses both flow and error control. Both the sender and the receiver use a sliding window of size 1.
- The sender sends one packet at a time and waits for an acknowledgment before sending the next one. To detect corrupted packets, we need to add a checksum to each data packet. When a packet arrives at the receiver site, it is checked. If its checksum is incorrect, the packet is corrupted and silently discarded.
- ***The silence of the receiver is a signal for the sender that a packet was either corrupted or lost.***
- ***Every time the sender sends a packet, it starts a timer***. If an acknowledgment arrives before the timer expires, the timer is stopped and the sender sends the next packet (if it has one to send). If the timer expires, the sender resends the previous packet, assuming that the packet was either lost or corrupted. This means that the sender needs to keep a copy of the packet until its acknowledgment arrives.

Sequence Numbers

- To prevent duplicate packets, the protocol uses sequence numbers and acknowledgment numbers. A field is added to the packet header to hold the sequence number of that packet.

Prof. Parthasarathy P V



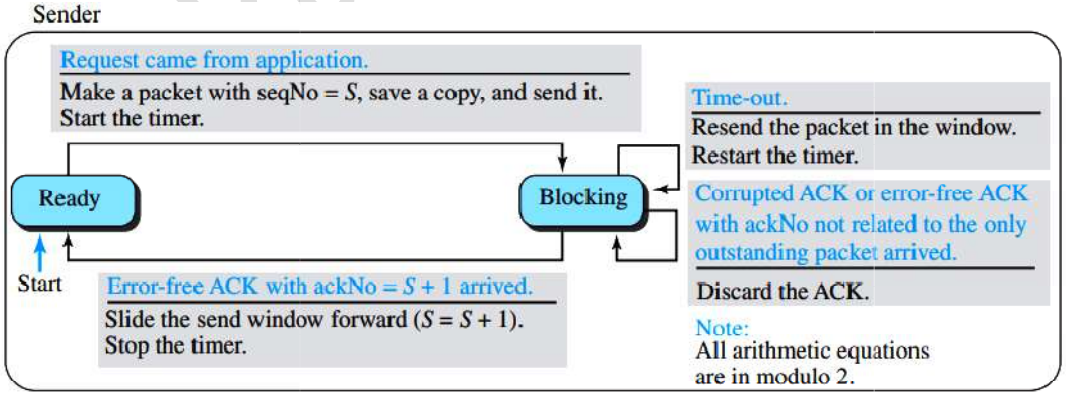
- Let us discuss the range of sequence numbers we need. Assume we have used x as a sequence number; we only need to use x + 1 after that.
- To show this, assume that the sender has sent the packet with sequence number x. Three things can happen.
  1. **The packet arrives safe and sound at the receiver site; the receiver sends an acknowledgment.** The acknowledgment arrives at the sender site, causing the sender to send the next packet numbered x + 1.
  2. **The packet is corrupted or never arrives at the receiver site**; the sender resends the packet (numbered x) after the time-out. The receiver returns an acknowledgment.
  3. **The packet arrives safe and sound at the receiver site; the receiver sends an acknowledgment, but the acknowledgment is corrupted or lost**. The sender resends the packet (numbered x) after the time-out. Note that the packet here is a duplicate. The receiver can recognize this fact because it expects packet x + 1 but packet x was received.

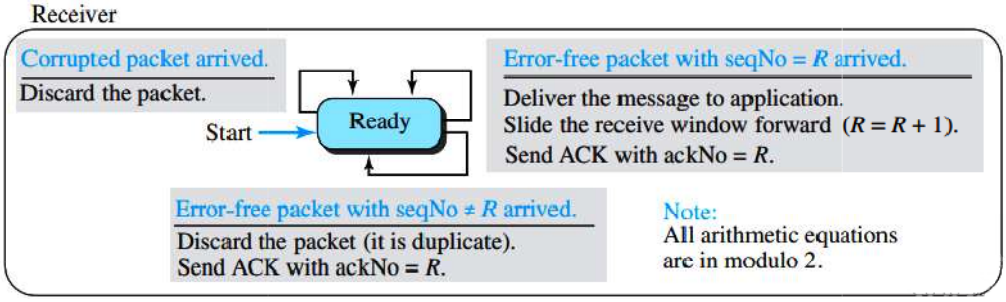
Acknowledgment Numbers

- Since the sequence numbers must be suitable for both data packets and acknowledgments, we use this convention: The acknowledgment numbers always announce, in modulo-2 arithmetic, the sequence number of the next packet expected by the receiver.
- The sender has a control variable, which we call **S (sender),** that points to the only slot in the send window. The receiver has a control variable, which we call **R (receiver),** that points to the only slot in the receive window.

FSMs

- Figure shows the FSMs for the Stop-and-Wait protocol. Since the protocol is a connection-oriented protocol, both ends should be in the established state before exchanging data packets. The states are actually nested in the established state.





Sender

- The sender is initially in the ready state, but it can move between the ready and blocking state. The variable S is initialized to 0.
- ***Ready state.*** When the sender is in this state, it is only waiting for one event to occur. If a request comes from the application layer, the sender creates a packet with the sequence number set to S. A copy of the packet is stored, and the packet is sent. ***The sender then starts the only timer and then moves to the blocking state.***
- ***Blocking state.*** When the sender is in this state, ***three events can occur***:
    1. If an error-free ACK arrives with the ackNo related to the next packet to be sent, which means ackNo = (S + 1) modulo 2, then the timer is stopped. The window slides, S = (S + 1) modulo 2. Finally, the sender moves to the ready state.
    2. If a corrupted ACK or an error-free ACK with the ackNo ≠ (S + 1) modulo 2 arrives, the ACK is discarded.
    3. If a time-out occurs, the sender resends the only outstanding packet and restarts the timer.

Receiver

- The receiver is always in the ready state. ***Three events may occur***:
    1. If an error-free packet with seqNo = R arrives, the message in the packet is delivered to the application layer. The window then slides, R = (R + 1) modulo 2. Finally an ACK with ackNo = R is sent.
    2. If an error-free packet with seqNo ≠ R arrives, the packet is discarded, but an ACK with ackNo = R is sent.
    3. If a corrupted packet arrives, the packet is discarded.

Efficiency

- The Stop-and-Wait protocol is very inefficient if our channel is thick and long. By thick, we mean that our channel has a large bandwidth (high data rate); by long, we mean the round-trip delay is long. The product of these two is called the ***bandwidth-delay*** product.

Pipelining

- In networking and in other areas, a task is often begun before the previous task has ended. This is known as ***pipelining***. There is no pipelining in the Stop-and-Wait protocol because a sender must wait for a packet to reach the destination and be acknowledged before the next packet can be sent.
- However, pipelining does apply to our next two protocols because several packets can be sent before a sender receives feedback about the previous packets.
- Pipelining improves the efficiency of the transmission if the number of bits in transition is large with respect to the ***bandwidth-delay*** product.

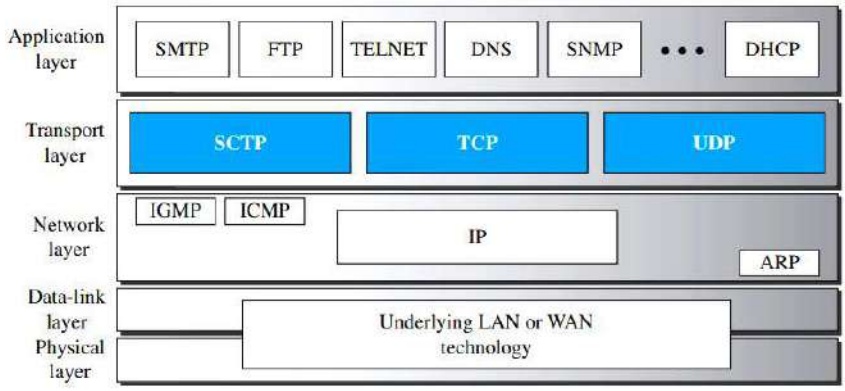
Prof. Parthasarathy P V

Chapter 24: Transport-Layer Protocols [P-775]

Services, Features, Segments, TCP connections, Flow control, Error control, Congestion control

Textbook: Ch. 24.1 – 24.3.4, 24.3.6 - 24.3.9

- The position of these three protocols in the TCP/IP protocol suite



Services

- Each protocol provides a different type of service and should be used appropriately.
- ***UDP***: UDP is an unreliable connectionless transport-layer protocol used for its simplicity and efficiency in applications where error control can be provided by the application-layer process.
- ***TCP:*** TCP is a reliable connection-oriented protocol that can be used in any application where reliability is important.
- ***SCTP:*** SCTP is a new transport-layer protocol that combines the features of UDP and TCP.

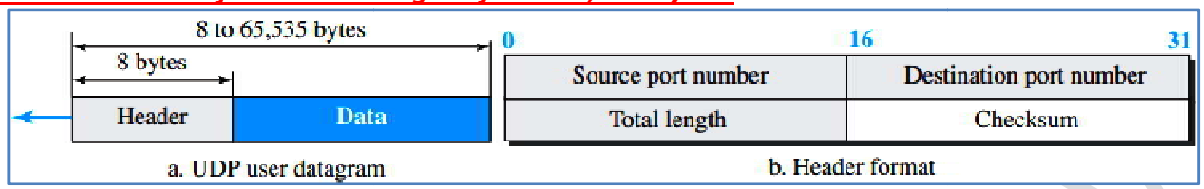
Port Numbers

Port	*Protocol*	*UDP*	*TCP*	*SCTP*	*Description*
7	Echo	√	√	√	Echoes back a received datagram
9	Discard	√	√	√	Discards any datagram that is received
11	Users	√	√	√	Active users
13	Daytime	√	√	√	Returns the date and the time
17	Quote	√	√	√	Returns a quote of the day
19	Chargen	√	√	√	Returns a string of characters
20	FTP-data		√	√	File Transfer Protocol
21	FTP-21		√	√	File Transfer Protocol
23	TELNET		√	√	Terminal Network
25	SMTP		√	√	Simple Mail Transfer Protocol
53	DNS	√	√	√	Domain Name Service
67	DHCP	√	√	√	Dynamic Host Configuration Protocol
69	TFTP	√	√	√	Trivial File Transfer Protocol
80	HTTP		√	√	HyperText Transfer Protocol
111	RPC	√	√	√	Remote Procedure Call
123	NTP	√	√	√	Network Time Protocol
161	SNMP-server	√			Simple Network Management Protocol
162	SNMP-client	√			Simple Network Management Protocol

User Datagram Protocol (UDP)

- The User Datagram Protocol (UDP) is a connectionless, unreliable transport protocol - providing process-to-process communication instead of host-to-host communication.
- ***Advantages***. UDP is a very simple protocol using a minimum of overhead.

- If a process wants to send a small message and does not care much about reliability, it can use UDP. Sending a small message using UDP takes much less interaction between the sender and receiver than using TCP.
- ***UDP packets***, called ***user datagrams***, have a ***fixed-size header of 8 bytes*** made of four fields, each of 2 bytes (16 bits). Figure shows the format of a user datagram.
- ***The 16 bits can define a total length of 0 to 65,535 bytes.***



UDP Services

- Process-to-Process Communication
- Connectionless Services
- Flow Control
- Error Control
- Checksum
- Congestion Control
- Encapsulation and Decapsulation
- Queuing
- Multiplexing and Demultiplexing

Process-to-Process Communication

- UDP provides process-to-process communication using socket addresses, a combination of IP addresses and port numbers.

Connectionless Services

- UDP provides a connectionless service. This means that each user datagram sent by UDP is an independent datagram.
- There is no relationship between the different user datagrams even if they are coming from the same source process and going to the same destination program.
- The user datagrams are not numbered.
- Also, unlike TCP, there is no connection establishment and no connection termination. This means that each user datagram can travel on a different path.
- One of the consequence of being connectionless is that the process that uses UDP cannot send a stream of data to UDP and expect UDP to chop them into different, related user datagrams.

Flow Control

- UDP is a very simple protocol. ***There is no flow control***, and hence no window mechanism. The receiver may overflow with incoming messages. The lack of flow control means that the process using UDP should provide for this service, if needed.

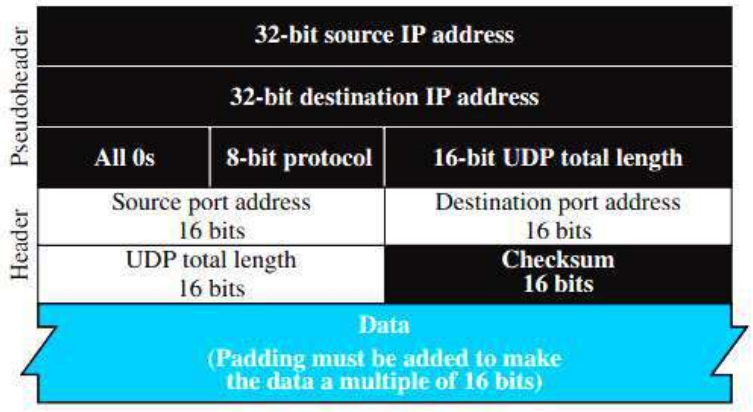
Error Control

- There is no error control mechanism in UDP except for the checksum. This means that the sender does not know if a message has been lost or duplicated. When the receiver detects an error through the checksum, the user datagram is silently discarded. The lack of error control means that the process using UDP should provide for this service, if needed.

Checksum

- UDP checksum calculation includes three sections: a pseudoheader, the UDP header, and the data coming from the application layer. The pseudoheader is the part of the header of the IP packet in which the user datagram is to be encapsulated with some fields filled with 0s (see Figure).

Prof. Parthasarathy P V



Congestion Control

- Since UDP is a connectionless protocol, it does not provide congestion control. UDP assumes that the packets sent are small and sporadic and cannot create congestion in the network. This assumption may or may not be true today, when UDP is used for interactive real-time transfer of audio and video.

Encapsulation and Decapsulation

- To send a message from one process to another, the UDP protocol encapsulates and decapsulates messages.

Queuing

- In UDP, queues are associated with ports. At the client site, when a process starts, it requests a port number from the operating system. Some implementations create both an incoming and an outgoing queue associated with each process. Other implementations create only an incoming queue associated with each process.

Multiplexing and Demultiplexing

- In a host running a TCP/IP protocol suite, there is only one UDP but possibly several processes that may want to use the services of UDP. To handle this situation, UDP multiplexes and demultiplexes.

Typical Applications

The following shows some typical applications that can benefit more from the services of UDP than from those of TCP.

- UDP is suitable for a process that requires simple request-response communication with little concern for flow and error control. It is not usually used for a process such as FTP that needs to send bulk data.
- UDP is suitable for a process with internal flow- and error-control mechanisms. For example, the Trivial File Transfer Protocol (TFTP) process includes flow and error control. It can easily use UDP.
- UDP is a suitable transport protocol for multicasting. Multicasting capability is embedded in the UDP software but not in the TCP software.
- UDP is used for management processes such as SNMP.
- UDP is used for some route updating protocols such as Routing Information Protocol (RIP).
- UDP is normally used for interactive real-time applications that cannot tolerate uneven delay between sections of a received message.

TCP Services (P743 – 785)

Process-to-Process Communication

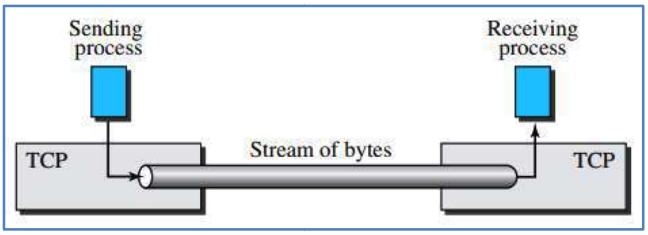
- As with UDP, TCP provides process-to-process communication using port numbers.

Stream Delivery Service

- TCP is a stream-oriented protocol - allows the sending process to deliver data as a stream of bytes and allows the receiving process to obtain data as a stream of bytes.

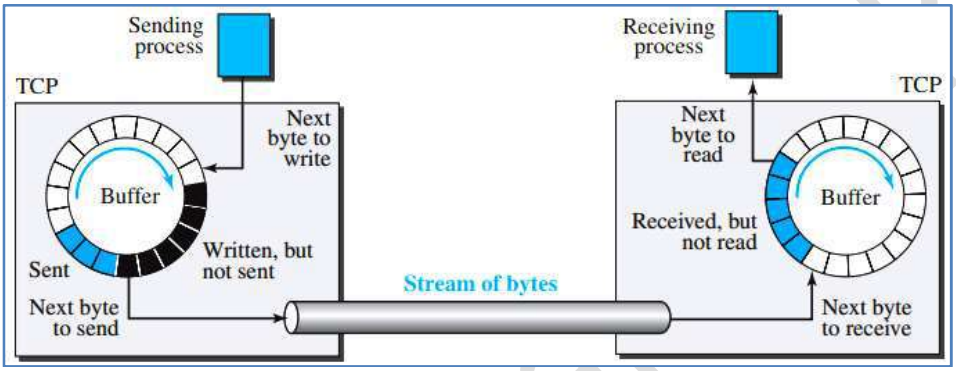
Prof. Parthasarathy P V

- TCP creates an environment in which the two processes seem to be connected by an imaginary "tube" that carries their bytes across the Internet (Figure). The sending process produces (writes to) the stream and the receiving process consumes (reads from) it.



Sending and Receiving Buffers

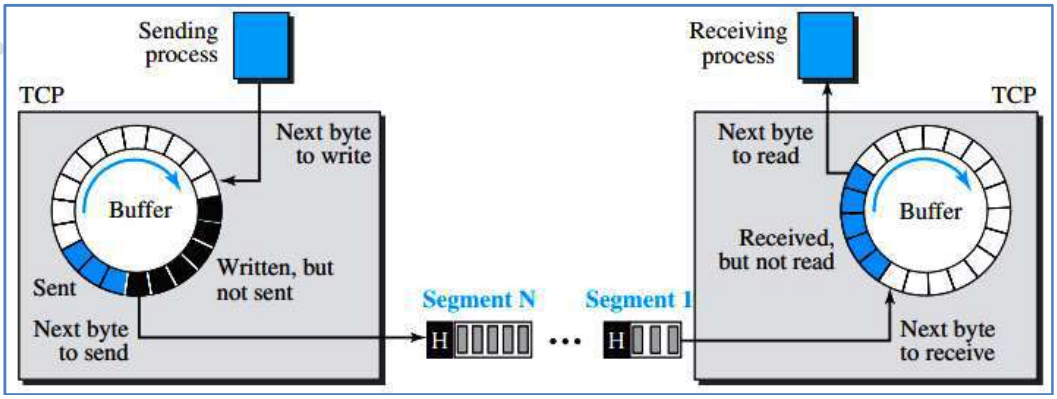
- Because the sending and the receiving processes may not necessarily write or read data at the same rate, TCP needs buffers for storage.
- There are two buffers, the sending buffer and the receiving buffer, one for each direction. One way to implement a buffer is to use a circular array of 1-byte locations as shown in Figure. For simplicity, we have shown two buffers of 20 bytes each; normally the buffers are hundreds or thousands of bytes, depending on the implementation.



- The figure shows the movement of the data in one direction. At the ***sender, the buffer has three types of chambers***. The white section contains empty chambers that can be filled by the sending process (producer). The colored area holds bytes that have been sent but not yet acknowledged. The TCP sender keeps these bytes in the buffer until it receives an acknowledgment.
- The operation of the buffer at the ***receiver*** is simpler. The circular buffer is ***divided into two areas*** (shown as white and colored). The white area contains empty chambers to be filled by bytes received from the network. The colored sections contain received bytes that can be read by the receiving process. ***When a byte is read by the receiving process, the chamber is recycled and added to the pool of empty chambers***.

Segments

- Although buffering handles the disparity between the speed of the producing and consuming processes, we need one more step before we can send data.
- At the transport layer, ***TCP groups a number of bytes together into a packet called a segment***. ***TCP adds a header to each segment*** (for control purposes) and delivers the segment to the network layer for transmission.
- The ***segments are encapsulated in an IP datagram and transmitted***. This entire operation is transparent to the receiving process.



Full-Duplex Communication

- TCP offers full-duplex service, where data can flow in both directions at the same time. Each TCP endpoint then has its own sending and receiving buffer, and segments move in both directions.

Multiplexing and Demultiplexing

- Like UDP, TCP performs multiplexing at the sender and demultiplexing at the receiver. However, since TCP is a connection-oriented protocol, a connection needs to be established for each pair of processes.

Connection-Oriented Service

- TCP Unlike UDP, is a connection-oriented protocol. When a process at site A wants to send to and receive data from another process at site B, the following three phases occur:
  1. The two TCP's establish a logical connection between them.
  2. Data are exchanged in both directions.
  3. The connection is terminated.
- ***Note that this is a logical connection, not a physical connection***. The TCP segment is encapsulated in an IP datagram and can be sent ***out of order, or lost or corrupted, and then resent***.
- ***Each may be routed over a different path to reach the destination***. TCP creates a stream-oriented environment in which it accepts the responsibility of delivering the bytes in order to the other site.

Reliable Service

- TCP is a reliable transport protocol. It uses an acknowledgment mechanism to check the safe and sound arrival of data. We will discuss this feature further in the section on error control.

Features

Numbering System

- Although the TCP software keeps track of the segments being transmitted or received, there is no field for a segment number value in the segment header. Instead, there are ***two fields***, called the ***sequence number*** and the ***acknowledgment number***. These two fields refer to a byte number and not a segment number.

Byte Number

- TCP numbers all data bytes (octets) that are transmitted in a connection. Numbering is independent in each direction. When TCP receives bytes of data from a process, TCP stores them in the sending buffer and numbers them.
- The numbering does not necessarily start from 0. Instead, TCP chooses an arbitrary number between ***0*** and ***$2^{32} - 1$*** for the number of the first byte. For example, if the number happens to be 1057 and the total data to be sent is 6000 bytes, the bytes are numbered from 1057 to 7056. We will see that byte numbering is used for flow and error control.

Sequence Number

- After the bytes have been numbered, TCP assigns a sequence number to each segment that is being sent. The sequence number, in each direction, is defined as follows:
  1. The sequence number of the first segment is the ISN (initial sequence number), which is a random number.
  2. The sequence number of any other segment is the sequence number of the previous segment plus the number of bytes (real or imaginary) carried by the previous segment. Later, we show that some control segments are thought of as carrying one imaginary byte.

Example

Suppose a TCP connection is transferring a file of 5000 bytes. The first byte is numbered 10001. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1000 bytes?

- ***The following shows the sequence number for each segment:***
  Segment 1 → Sequence Number: 10001 Range: 10001 to 11000
  Segment 2 → Sequence Number: 11001 Range: 11001 to 12000

Prof Parthasarathy P V

Segment 3 → Sequence Number: 12001 Range: 12001 to 13000
Segment 4 → Sequence Number: 13001 Range: 13001 to 14000
Segment 5 → Sequence Number: 14001 Range: 14001 to 15000

- When a segment carries a combination of data and control information (piggybacking), it uses a sequence number. If a segment does not carry user data, it does not logically define a sequence number.

Acknowledgment Number

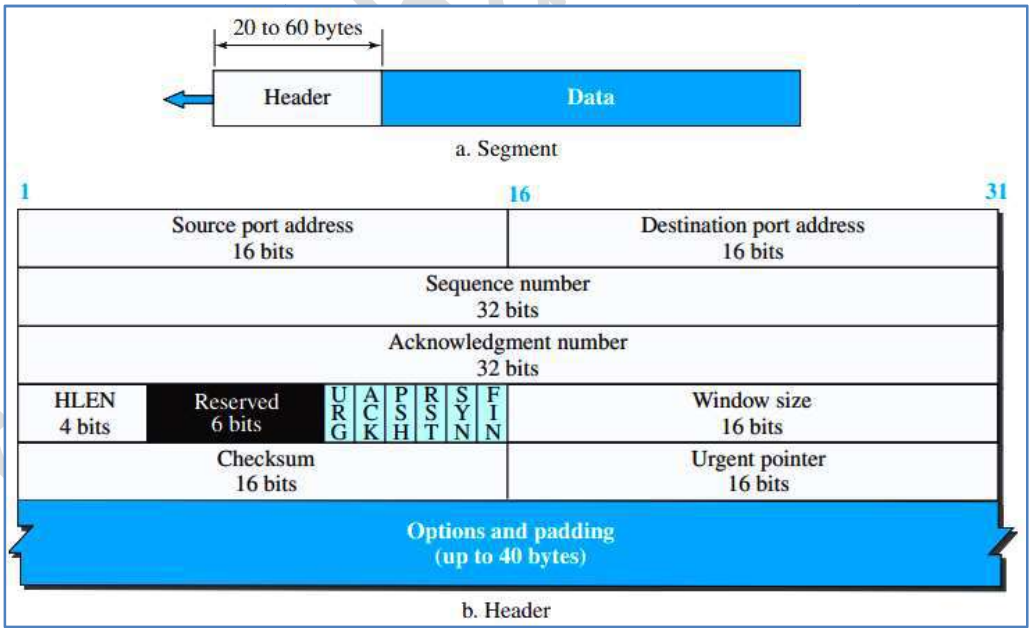
- Communication in TCP is full duplex; when a connection is established, both parties can send and receive data at the same time.
- Each party numbers the bytes, usually with a different starting byte number. The sequence number in each direction shows the number of the first byte carried by the segment. Each party also uses an acknowledgment number to confirm the bytes it has received.
- However, ***the acknowledgment number defines the number of the next byte that the party expects to receive***. In addition, the acknowledgment number is cumulative, which means that the party takes the number of the last byte that it has received, safe and sound, adds 1 to it, and announces this sum as the acknowledgment number.
- The term cumulative here means that if a party uses 5643 as an acknowledgment number, it has received all bytes from the beginning up to 5642. Note that this does not mean that the party has received 5642 bytes, because the first byte number does not have to be 0.

Segments

- A packet in TCP is called a segment.

Format

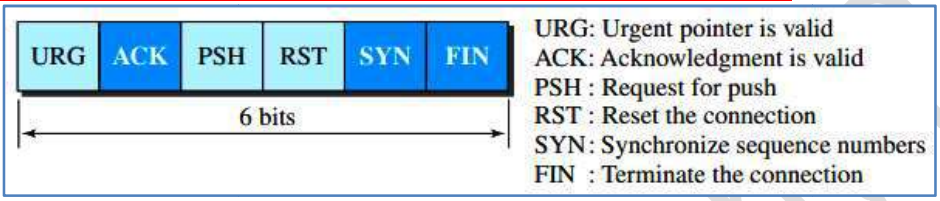
- The format of a segment is shown in Figure. The segment consists of a ***header of 20 to 60 bytes***, ***followed by data from the application program***. The header is 20 bytes if there are no options and up to 60 bytes if it contains options.



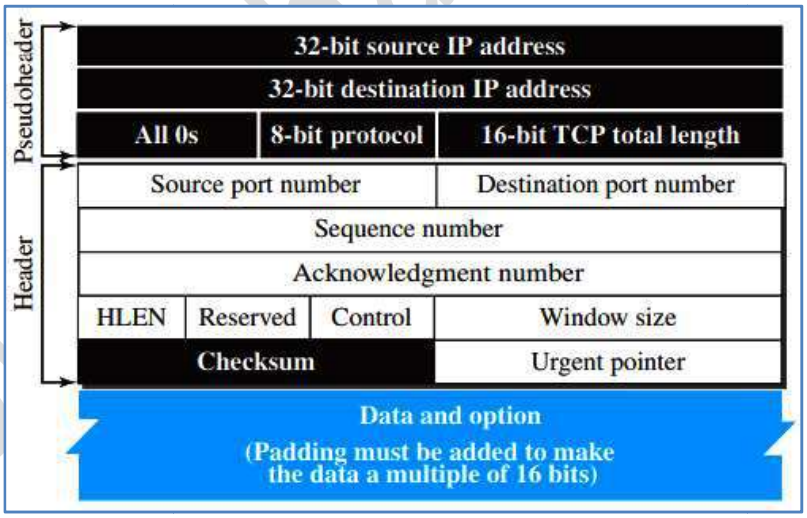
- ***Source port address***. This is a ***16-bit field*** that defines the port number of the application program in the host that is sending the segment.
- ***Destination port address***. This is a ***16-bit field*** that defines the port number of the application program in the host that is receiving the segment.
- ***Sequence number.*** This ***32-bit field*** defines the number assigned to the first byte of data contained in this segment. TCP is a stream transport protocol - to ensure connectivity, each byte to be transmitted is numbered. The sequence number tells the destination which byte in this sequence is

the first byte in the segment. ***During connection establishment*** each party uses a random number generator to ***create an initial sequence number (ISN),*** which is usually different in each direction.

- ***Acknowledgment number***. This ***32-bit field*** defines the byte number that the receiver of the segment is expecting to receive from the other party. If the receiver of the segment has successfully received byte number x from the other party, it returns x + 1 as the acknowledgment number. ***Acknowledgment and data can be piggybacked together.***
- ***Header length***. This ***4-bit field*** indicates the number of 4-byte words in the TCP header. The length of the header can be between 20 and 60 bytes. Therefore, the value of this field is always between 5 (5 × 4 = 20) and 15 (15 × 4 = 60).
- ***Control***. This field defines ***6 different control bits or flags, as shown in Figure***. One or more of these bits can be set at a time. These ***bits enable flow control, connection establishment and termination, connection abortion, and the mode of data transfer in TCP***.



- ***Window size***. This field defines the window size of the sending TCP in bytes. Note that the length of this field is ***16 bits***, which means that the maximum size of the window is 65,535 bytes. ***This value is normally referred to as the receiving window (rwnd) and is determined by the receiver***. The ***sender must obey the dictation of the receiver in this case***.
- ***Checksum***. This ***16-bit field contains the checksum***. The calculation of the checksum for TCP follows the same procedure as the one described for UDP. The use of the checksum in the UDP datagram is optional, whereas the use of the checksum for TCP is mandatory. The same pseudoheader, serving the same purpose, is added to the segment. For the TCP pseudoheader, the value for the protocol field is 6. See Figure.



- ***Urgent pointer***. This ***16-bit field***, which is valid only if the urgent flag is set, is used when the segment contains urgent data. It defines a value that must be added to the sequence number to obtain the number of the last urgent byte in the data section of the segment.
- ***Options***. There can be ***up to 40 bytes of optional information in the TCP header***.

Encapsulation

- A TCP segment encapsulates the data received from the application layer. The TCP segment is encapsulated in an IP datagram, which in turn is encapsulated in a frame at the data-link layer.

TCP connections

Topics covered

- **Connection Establishment (Three-Way Handshaking - SYN Flooding Attack)**
- **Data Transfer (Pushing Data - Urgent Data)**
- **Connection Termination (Three-Way Handshaking - Half-Close)**
- **Connection Reset**

- TCP is connection-oriented. A connection-oriented transport protocol establishes a logical path between the source and destination. All of the segments belonging to a message are then sent over this logical path.
- Using a single logical pathway for the entire message facilitates the acknowledgment process as well as retransmission of damaged or lost frames. You may wonder how TCP, which uses the services of IP, a connectionless protocol, can be connection-oriented.
- The point is that a ***TCP connection is logical, not physical***. TCP operates at a higher level. TCP uses the services of IP to deliver individual segments to the receiver, but it controls the connection itself. If a segment is lost or corrupted, it is retransmitted.
- Unlike TCP, ***IP is unaware of this retransmission***. If a ***segment arrives out of order, TCP holds it until the missing segments arrive; IP is unaware of this reordering***.
- In TCP, connection-oriented transmission requires ***three phases***:
  1. Connection establishment
  2. Data transfer
  3. Connection termination.

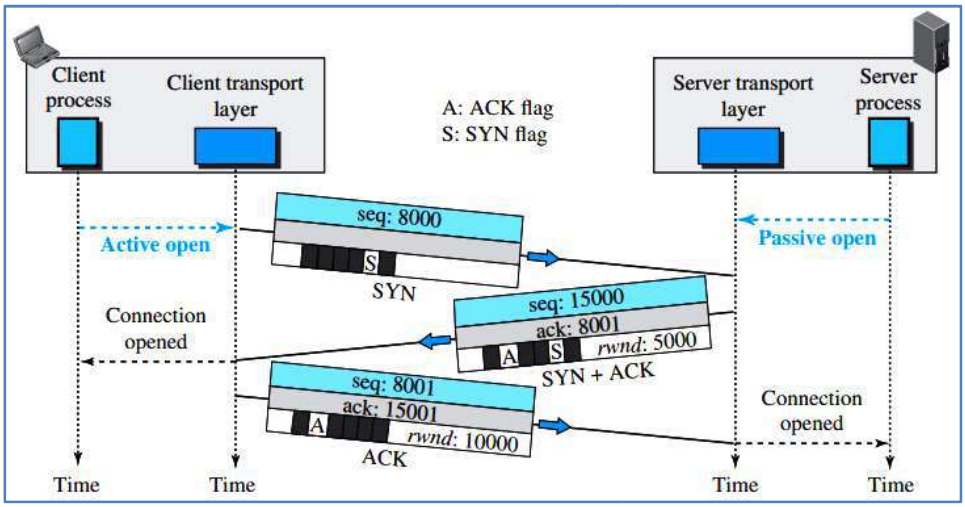
---

1. Connection Establishment (Three-Way Handshaking - SYN Flooding Attack)

- TCP transmits data in full-duplex mode. When two TCPs in two machines are connected, they are able to send segments to each other simultaneously. This implies that each party must initialize communication and get approval from the other party before any data are transferred.

Three-Way Handshaking

- The ***connection establishment in TCP is called three-way handshaking***. In our example, an application program, called the client, wants to make a connection with another application program, called the server, using TCP as the transport-layer protocol.
- The process starts with the server. ***The server program tells its TCP that it is ready to accept a connection. This request is called a passive open***. Although the server TCP is ready to accept a connection from any machine in the world, it cannot make the connection itself.
- The client program issues a request for an active open. A client that wishes to connect to an open server tells its TCP to connect to a particular server. ***TCP can now start the three-way handshaking process, as shown in Figure.***



- To show the process we use time lines. Each segment has values for all its header fields and perhaps for some of its option fields too.

Prof. Parthasarathy P V

- We show the ***sequence number***, the ***acknowledgment number***, the ***control flags*** (only those that are set), and ***window size*** if relevant. The ***three steps*** in this phase are as follows.
  1. The ***client sends the first segment, a SYN segment***, in which only the SYN flag is set. This segment is for synchronization of sequence numbers. The client in our example chooses a random number as the first sequence number and sends this number to the server. This sequence number is called ***the initial sequence number (ISN).***
     - The ***SYN segment is a control segment and carries no data***. However, it consumes one sequence number because it needs to be acknowledged. We can say that the SYN segment carries one imaginary byte.
  2. The ***server sends the second segment, a SYN + ACK segment with two flag bits*** set as: SYN and ACK. This segment has a dual purpose.
     - First, it is a SYN segment for communication in the other direction. The server uses this segment to initialize a sequence number for numbering the bytes sent from the server to the client. The server also acknowledges the receipt of the SYN segment from the client by setting the ACK flag and displaying the next sequence number it expects to receive from the client.
  3. The ***client sends the third segment (just an ACK segment***) It acknowledges the receipt of the second segment with the ACK flag and acknowledgment number field. Note that the ACK segment does not consume any sequence numbers if it does not carry data, but some implementations allow this third segment in the connection phase to carry the first chunk of data from the client. In this case, the segment consumes as many sequence numbers as the number of data bytes.

SYN Flooding Attack - denial of service attack - cookie

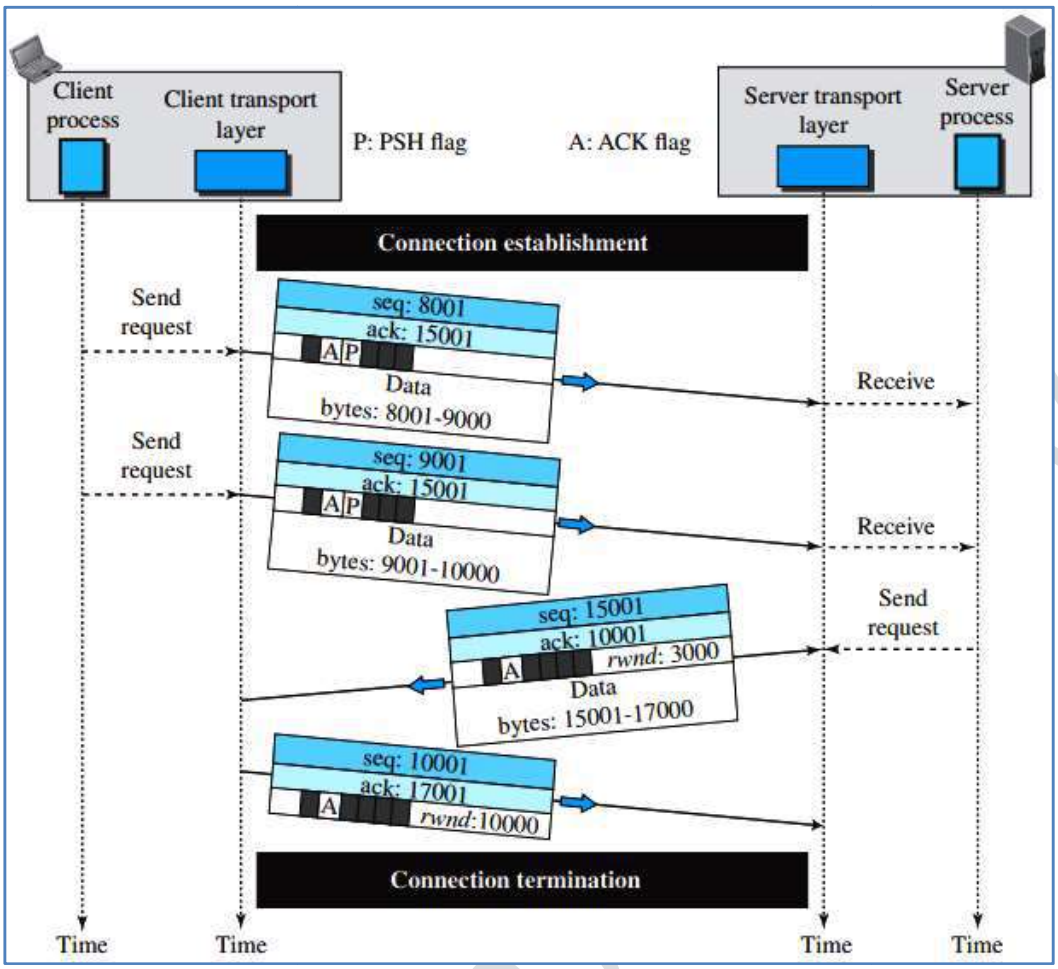
- The connection establishment procedure in TCP is susceptible to a ***serious security problem called SYN flooding attack***. This happens when ***one or more malicious attackers*** send a large number of SYN segments to a server pretending that each of them is coming from a different client by faking the source IP addresses in the datagrams.
- The server, assuming that the clients are issuing an active open, allocates the necessary resources, such as ***creating transfer control block (TCB) tables and setting timers***.
- The ***TCP server then sends the SYN + ACK segments to the fake clients***, ***which are lost***. When the server waits for the third leg of the handshaking process, however, resources are allocated without being used.
- If, during this short period of time, the number of SYN segments is large, the server eventually runs out of resources and may be unable to accept connection requests from valid clients.
- This SYN flooding attack belongs to a group of security attacks known as a ***denial of service attack***, in which an attacker monopolizes a system with so many service requests that the system overloads and denies service to valid requests.
- Some implementations of TCP have strategies to alleviate the effect of a SYN attack. Some have imposed a limit of connection requests during a specified period of time. Others try to filter out datagrams coming from unwanted source addresses. One **recent strategy is to postpone resource allocation until the server can verify that the connection request is coming from a valid IP address, by using what is called a *cookie***. SCTP, the new transport-layer protocol uses this strategy.

---

2. Data Transfer (Pushing Data - Urgent Data)

- ***After connection is established, bidirectional data transfer can take place***. The client and server can send data and acknowledgments in both directions. We will study the rules of acknowledgment later in the chapter; for the moment, it is enough to know that data traveling in the same direction as an acknowledgment are carried on the same segment. The ***acknowledgment is piggybacked with the data***. Figure shows an example.

Prof. Parthasarathy P V



- In this example, after a connection is established, the ***client sends 2,000 bytes of data in two segments***. The server then sends 2,000 bytes in one segment. The client sends one more segment. The first three segments carry both data and acknowledgment, but the last segment carries only an acknowledgment because there is no more data to be sent.
- Note the ***values of the sequence and acknowledgment numbers***. The data segments sent by the client have the ***PSH (push) flag*** set so that the server TCP knows to deliver data to the server process as soon as they are received. The ***segment from the server, on the other hand, does not set the push flag.*** Most TCP implementations have the option to set or not to set this flag.

Pushing Data

- The ***sending TCP uses a buffer to store the stream of data coming from the sending application program***. The sending TCP can select the segment size. The receiving TCP also buffers the data when they arrive and delivers them to the application program when the application program is ready or when it is convenient for the receiving TCP. This type of flexibility increases the efficiency of TCP.
- ***There are occasions in which*** the application program has no need for this flexibility. Delayed transmission and delayed delivery of data may not be acceptable by the application program.
- ***TCP can handle such a situation***. The application program at the sender can request a push operation. This means that the sending TCP must not wait for the window to be filled. It must create a segment and send it immediately. The sending TCP must also set the push bit (PSH) to let the receiving TCP know that the segment includes data that must be delivered to the receiving application program as soon as possible and not to wait for more data to come.

Urgent Data

- There are ***occasions in which an application program needs to send urgent bytes, some bytes that need to be treated in a special way by the application at the other end***. The solution is to send a

segment with the URG bit set. The ***sending application program tells the sending TCP that the piece of data is urgent***. The sending TCP ***creates a segment and inserts the urgent data*** at the beginning of the segment. The ***rest of the segment can contain normal data from the buffer***. The urgent pointer field in the header defines the end of the urgent data (the last byte of urgent data).

- ***For example, if the segment sequence number is 15000 and the value of the urgent pointer is 200***, the first byte of urgent data is the byte 15000 and the last byte is the byte 15200. The rest of the bytes in the segment (if present) are non-urgent.

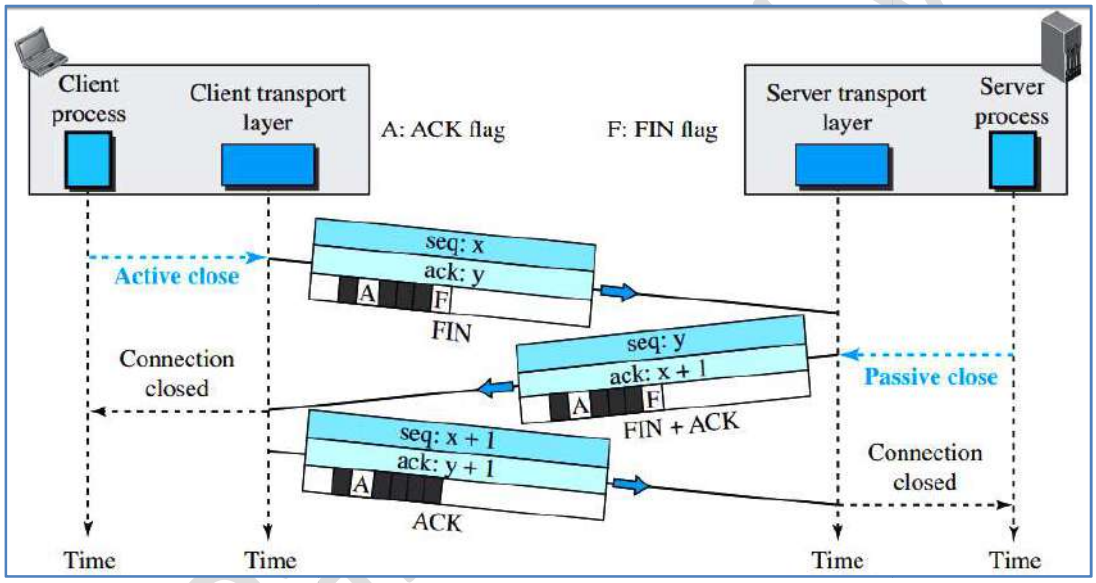
---

3. Connection Termination (Three-Way Handshaking - Half-Close)

- Either of the two parties involved in exchanging data (client or server) can close the connection, although it is usually initiated by the client. Most implementations today allow ***two options*** for connection termination: ***three-way handshaking*** and ***four-way handshaking with a half-close option***.

Three-Way Handshaking

- Most implementations today allow three-way handshaking for connection termination, as shown in Figure.

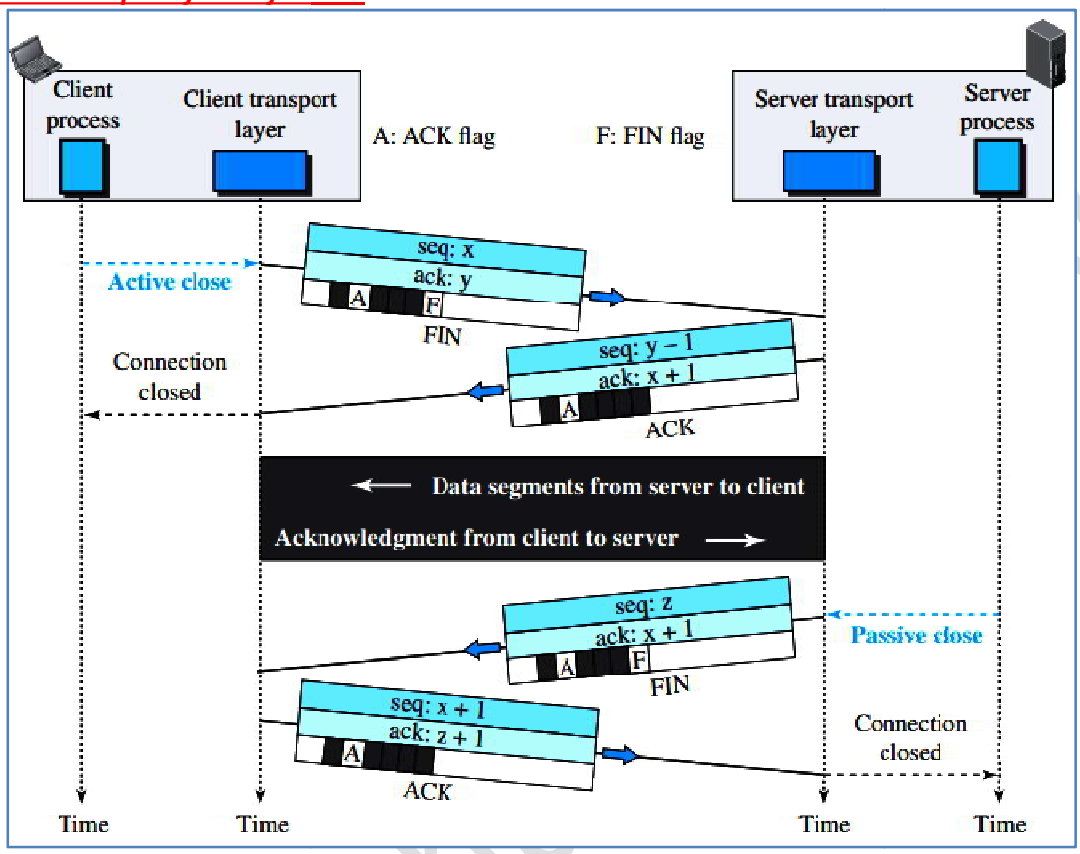


1. In this situation, ***the client TCP, after receiving a close command from the client process, sends the first segment, a FIN segment in which the FIN flag is set***. Note that a FIN segment can include the last chunk of data sent by the client or it can be just a control segment as shown in the figure. If it is only a control segment, it consumes only one sequence number because it needs to be acknowledged.
2. ***The server TCP, after receiving the FIN segment, informs its process of the situation and sends the second segment, a FIN + ACK segment, to confirm the receipt of the FIN segment from the client and at the same time to announce the closing of the connection in the other direction***. This segment can also contain the last chunk of data from the server. If it does not carry data, it consumes only one sequence number because it needs to be acknowledged.
3. ***The client TCP sends the last segment, an ACK segment, to confirm the receipt of the FIN segment from the TCP server.*** This segment contains the acknowledgment number, which is one plus the sequence number received in the FIN segment from the server. This segment cannot carry data and consumes no sequence numbers.

Half-Close

- ***In TCP, one end can stop sending data while still receiving data. This is called a halfclose***. Either the server or the client can issue a half-close request. It can occur when the server needs all the data before processing can begin. A good example is sorting.

- When the client sends data to the server to be sorted, the server needs to receive all the data before sorting can start.
- The ***server-to-client direction must remain open to return the sorted data***. The server, after receiving the data, still needs time for sorting; its outbound direction must remain open. ***Figure shows an example of a half-close.***



- The ***data transfer from the client to the server stops. The client half-closes the connection by sending a FIN segment***. The server accepts the half-close by sending the ACK segment. The server, however, can still send data. When the server has sent all of the processed data, it sends a FIN segment, which is acknowledged by an ACK from the client.
- After ***half-closing the connection, data can travel from the server to the client and acknowledgments can travel from the client to the server***. The client cannot send any more data to the server.

Connection Reset

- TCP at one end may deny a connection request, may abort an existing connection, or may ***terminate an idle connection***. All of these are done with the ***RST (reset) flag***.

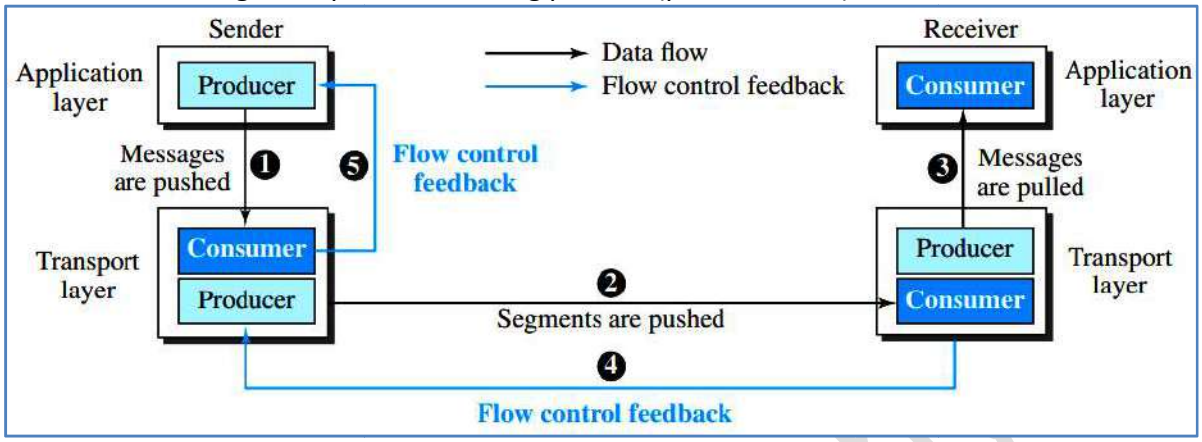
Flow control

Topics covered

- **Opening and Closing Windows**
- **Shrinking of Windows**
- **Silly Window Syndrome**

- ***Flow control balances the rate a producer creates data with the rate a consumer can use the data***. TCP separates flow control from error control. We ***assume that the logical channel between the sending and receiving TCP is error-free***.

- ***Figure shows unidirectional data transfer*** between a sender and a receiver; bidirectional data transfer can be deduced from the unidirectional process.
- The ***figure shows that data travel from the sending process down to the sending TCP***, from the sending TCP to the receiving TCP, and from the receiving TCP up to the receiving process (paths 1, 2, and 3). Flow control feedbacks, however, are traveling from the receiving TCP to the sending TCP and from the sending TCP up to the sending process (paths 4 and 5).



- ***Most implementations of TCP do not provide flow control feedback from the receiving process to the receiving TCP***; they let the receiving process pull data from the receiving TCP whenever it is ready to do so.
- The receiving TCP controls the sending TCP; the sending TCP controls the sending process.
- Flow control feedback from the sending TCP to the sending process (path 5) is achieved through simple rejection of data by the sending TCP when its window is full. Flow control concentrates on the feedback sent from the receiving TCP to the sending TCP (path 4).

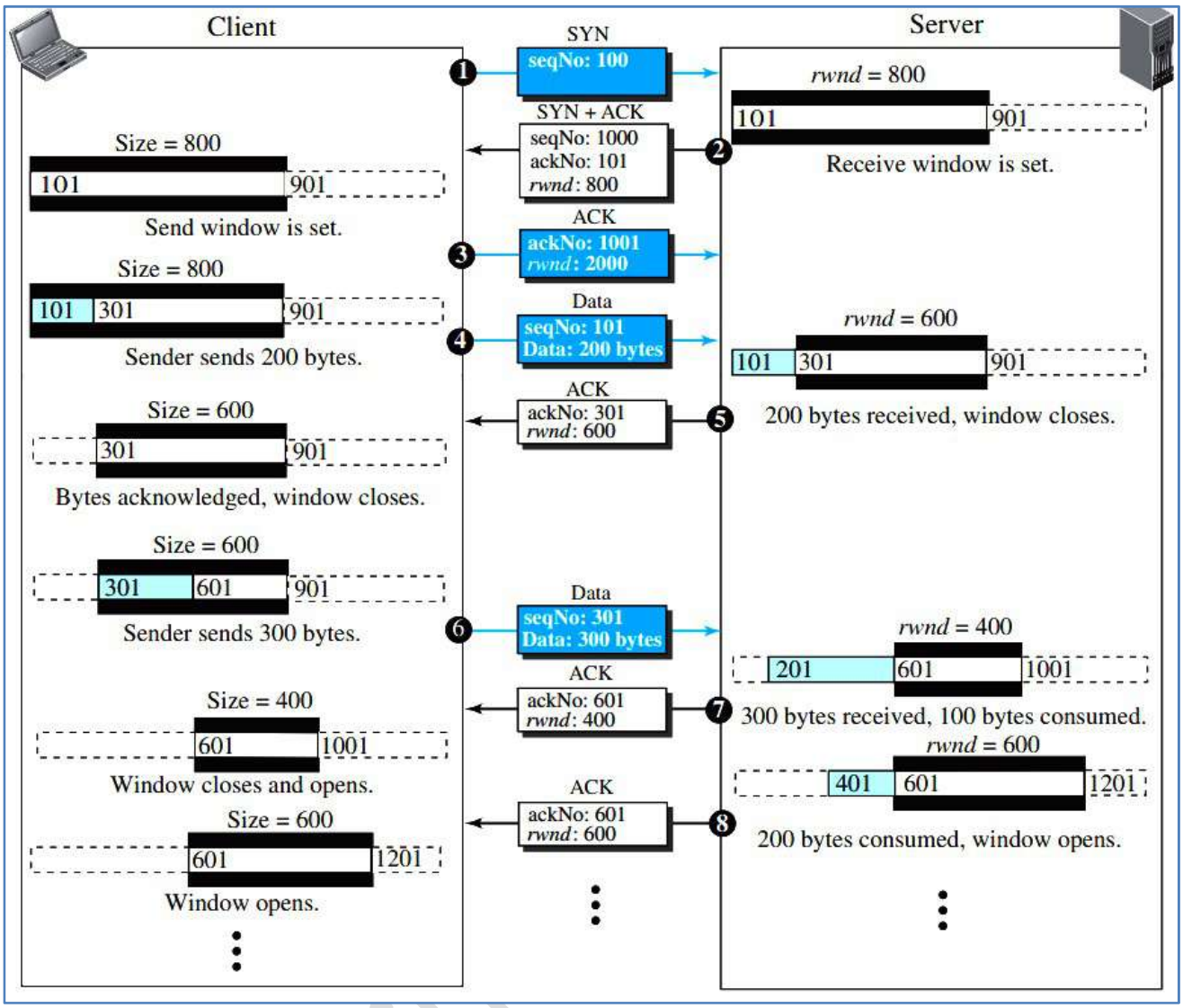
Opening and Closing Windows

- To ***achieve flow control, TCP forces the sender and the receiver to adjust their window sizes, although the size of the buffer for both parties is fixed when the connection is established***.
- The ***receive window closes*** when more bytes arrive from the sender; it ***opens*** when more bytes are pulled by the process. We assume that it does not shrink.
- The ***opening***, ***closing***, and ***shrinking*** of the send window is ***controlled by the receiver***. The send window closes when a new acknowledgment allows it to do so. The send window opens when the ***receive window size (rwnd) advertised by the receiver allows it to do so*** (***new ackNo + new rwnd > last ackNo + last rwnd***). The send window shrinks in the event this situation does not occur.

A Scenario

- We show how the send and receive windows are set during the connection establishment phase, and how their situations will change during data transfer. ***Figure shows a simple example of unidirectional data transfer (from client to server)***. For the time being, we ignore error control, assuming that no segment is corrupted, lost, duplicated, or has arrived out of order. Note that we have shown only two windows for unidirectional data transfer. Although the client defines server's window size of 2000 in the third segment, we have not shown that window because the communication is only unidirectional.
- ***Eight segments are exchanged between the client and server***:
  1. The ***first segment is from the client to the server (a SYN segment) to request connection***. The client announces its initial seqNo = 100. When this segment arrives at the server, it allocates a buffer size of 800 (an assumption) and sets its window to cover the whole buffer (rwnd = 800). Note that the number of the next byte to arrive is 101.
  2. The ***second segment is from the server to the client***. This is an ***ACK + SYN segment***. The segment uses ackNo = 101 to show that it expects to receive bytes starting from 101. It also announces that the client can set a buffer size of 800 bytes.

Prof. Parthasarathy P V



3. The ***third segment is the ACK segment*** from the client to the server. Note that the client has defined a rwnd of size 2000, but we do not use this value in our figure because the communication is only in one direction.
4. After the client has set its window with the ***size (800) dictated by the server, the process pushes 200 bytes of data***. The TCP client numbers these bytes 101 to 300. It then creates a segment and sends it to the server. The segment shows the starting byte number as 101 and the segment carries 200 bytes. The window of the client is then adjusted to show that 200 bytes of data are sent but waiting for acknowledgment. When this segment is received at the server, the bytes are stored, and the receive window closes to show that the next byte expected is byte 301; the stored bytes occupy 200 bytes of buffer.
5. The ***fifth segment is the feedback from the server to the client***. The server acknowledges bytes up to and including 300 (expecting to receive byte 301). The segment also carries the size of the receive window after decrease (600). The client, after receiving this segment, purges the acknowledged bytes from its window and closes its window to show that the next byte to send is byte 301. The window size, however, decreases to 600 bytes. Although the allocated buffer can store 800 bytes, the window cannot open (moving its right wall to the right) because the receiver does not let it.
6. ***Segment 6 is sent by the client after its process pushes 300 more bytes***. The segment defines seqNo as 301 and contains 300 bytes. When this segment arrives at the server, the server stores them, but it has to reduce its window size. After its process has pulled 100 bytes of data, the window closes from the left for the amount of 300 bytes, but opens from the right for the

amount of 100 bytes. The result is that the size is only reduced 200 bytes. The receiver window size is now 400 bytes.

7. ***In segment 7, the server acknowledges the receipt of data, and announces that its window size is 400.*** When this segment arrives at the client, the client has no choice but to reduce its window again and set the window size to the value of rwnd = 400 advertised by the server. The send window closes from the left by 300 bytes, and opens from the right by 100 bytes.
8. ***Segment 8 is also from the server after its process has pulled another 200 bytes.*** Its window size increases. The new rwnd value is now 600. The segment informs the client that the server still expects byte 601, but the server window size has expanded to 600. Some implementations may ***not allow advertisement of the rwnd at this time; the server then needs to receive some data before doing so.*** After this segment arrives at the client, the client opens its window by 200 bytes without closing it. The result is that its window size increases to 600 bytes.

Shrinking of Windows

- The receive window cannot shrink. The send window, on the other hand, can shrink if the receiver defines a value for rwnd that results in shrinking the window. However, some implementations do not allow shrinking of the send window.
- The limitation does not allow the right wall of the send window to move to the left. In other words, the receiver needs to keep the following relationship between the last and new acknowledgment and the last and new rwnd values to prevent shrinking of the send window.

$$\text{new ackNo} + \text{new rwnd} \geq \text{last ackNo} + \text{last rwnd}$$

- The left side of the inequality represents the new position of the right wall with respect to the sequence number space; the right side shows the old position of the right wall.
- The relationship shows that the right wall should not move to the left. The inequality is a mandate for the receiver to check its advertisement. However, note that the inequality is valid only if $S_f < S_n$; we need to remember that all calculations are in modulo 2^{32}.

Window Shutdown

- We said that shrinking the send window by moving its right wall to the left is strongly discouraged. However, ***there is one exception: the receiver can temporarily shut down the window by sending a rwnd of 0. This can happen if for some reason the receiver does not want to receive any data from the sender for a while.***
- In this case, the sender does not actually shrink the size of the window, but stops sending data until a new advertisement has arrived.
- Even when the window is shut down by an order from the receiver, the sender can always send a segment with 1 byte of data. This is ***called probing and is used to prevent a deadlock***.

Silly Window Syndrome

- ***A serious problem can arise in the sliding window operation when either the sending application program creates data slowly or the receiving application program consumes data slowly, or both.*** Any of these situations results in the sending of data in very small segments, which ***reduces the efficiency of the operation***.
- For example, if TCP sends segments containing only 1 byte of data, it means that a 41-byte datagram (20 bytes of TCP header and 20 bytes of IP header) transfers only 1 byte of user data. Here the overhead is 41/1, which indicates that we are using the capacity of the network very inefficiently.
- The inefficiency is even worse after accounting for the data-link layer and physical-layer overhead. This ***problem*** is called the ***silly window syndrome***.

Prof. Parthasarathy PV

Syndrome Created by the Sender

- The sending TCP may create a ***silly window syndrome if it is serving an application program that creates data slowly***, for example, 1 byte at a time. The application program writes 1 byte at a time into the buffer of the sending TCP. If the sending TCP does not have any specific instructions, it may create segments containing 1 byte of data. The result is a lot of 41-byte segments that are traveling through an internet.
- The ***solution is to prevent the sending TCP from sending the data byte by byte***. The sending TCP must be forced to wait and collect data to send in a larger block. How long should the sending TCP wait? If it waits too long, it may delay the process. If it does not wait long enough, it may end up sending small segments. Nagle found an elegant solution. ***Nagle's algorithm is simple***:
  1. The sending TCP sends the first piece of data it receives from the sending application program even if it is only 1 byte.
  2. After sending the first segment, ***the sending TCP accumulates data in the output buffer and waits until either the receiving TCP sends an acknowledgment*** or until enough data have accumulated to fill a maximum-size segment. At this time, the sending TCP can send the segment.
  3. ***Step 2 is repeated for the rest of the transmission***. Segment 3 is sent immediately if an acknowledgment is received for segment 2, or if enough data have accumulated to fill a maximum-size segment.

Syndrome Created by the Receiver

- The receiving TCP may create a silly window syndrome if it is serving an application program that consumes data slowly, for example, 1 byte at a time.
- Suppose that the sending application program creates data in blocks of 1 kilobyte, but the receiving application program consumes data 1 byte at a time. Also suppose that the input buffer of the receiving TCP is 4 kilobytes. The sender sends the first 4 kilobytes of data. The receiver stores it in its buffer. Now its buffer is full.
- It advertises a window size of zero, which means the sender should stop sending data. The receiving application reads the first byte of data from the input buffer of the receiving TCP. Now there is 1 byte of space in the incoming buffer. The receiving TCP announces a window size of 1 byte, which means that the sending TCP, which is eagerly waiting to send data, takes this advertisement as good news and sends a segment carrying only 1 byte of data.
- The procedure will continue. One byte of data is consumed and a segment carrying 1 byte of data is sent. Again we have an efficiency problem and the silly window syndrome.
- ***Two solutions have been proposed to prevent the silly window syndrome*** created by an application program that consumes data more slowly than they arrive.
  1. ***Clark's solution*** is to ***send an acknowledgment as soon as the data arrive, but to announce a window size of zero*** until either there is enough space to accommodate a segment of maximum size or until at least half of the receive buffer is empty.
  2. The ***second solution*** is to ***delay sending the acknowledgment***. This means that when a segment arrives, it is not acknowledged immediately. The receiver waits until there is a decent amount of space in its incoming buffer before acknowledging the arrived segments. The delayed acknowledgment prevents the sending TCP from sliding its window. After the sending TCP has sent the data in the window, it stops. This kills the syndrome.
- ***Delayed acknowledgment also has another advantage***: it ***reduces traffic***. The receiver does not have to acknowledge each segment. However, there also is a disadvantage in that the delayed acknowledgment may result in the sender unnecessarily retransmitting the unacknowledged segments. The ***protocol balances the advantages and disadvantages*** - defines that the acknowledgment should not be delayed by more than 500 ms.

Prof. Parthasarathy P V

Error control

Topics covered

- **Checksum**
- **Acknowledgment**
- **Cumulative Acknowledgment (ACK)**
- **Selective Acknowledgment (SACK)**
- **Retransmission**
- **Out-of-Order Segments**
- **FSMs for Data Transfer in TCP**

- TCP provides reliability using error control. Error control includes mechanisms for detecting and resending corrupted segments, resending lost segments, storing out-of-order segments until missing segments arrive, and detecting and discarding duplicated segments. Error control in TCP is achieved through the use of three simple tools: checksum, acknowledgment, and time-out.

Checksum

- Each segment includes a checksum field, which is used to check for a corrupted segment. If a segment is corrupted, as detected by an invalid checksum, the segment is discarded by the destination TCP and is considered as lost. TCP uses a 16-bit checksum that is mandatory in every segment.

Acknowledgment

- ***TCP uses acknowledgments to confirm the receipt of data segments***. Control segments that carry no data, but consume a sequence number, are also acknowledged. ACK segments are never acknowledged.
- ***In the past, TCP used only one type of acknowledgment: cumulative acknowledgment***. Today, some TCP implementations also use ***selective acknowledgment***.

Cumulative Acknowledgment (ACK)

- TCP was originally designed to acknowledge receipt of segments cumulatively. The receiver advertises the next byte it expects to receive, ignoring all segments received and stored out of order. This is sometimes referred to as ***positive cumulative acknowledgment, or ACK***.
- The ***word positive indicates that no feedback is provided for discarded, lost, or duplicate segments***. The ***32-bit ACK*** field in the TCP header is used for cumulative acknowledgments, and its value is valid only when the ACK flag bit is set to 1.

Selective Acknowledgment (SACK)

- More and more implementations are adding another type of acknowledgment called ***selective acknowledgment, or SACK***. A ***SACK does not replace an ACK, but reports additional information to the sender***. A SACK reports a block of bytes that is out of order, and also a block of bytes that is duplicated, i.e., received more than once. However, since there is no provision in the TCP header for adding this type of information, SACK is implemented as an option at the end of the TCP header.

Generating Acknowledgments

When does a receiver generate acknowledgments?

- During the evolution of TCP, several rules have been defined and used by several implementations. We give the most common rules here. The order of a rule does not necessarily define its importance.
  1. When ***end A sends a data segment to end B, it must include (piggyback) an acknowledgment that gives the next sequence number it expects to receive***. This rule ***decreases*** the number of segments needed and therefore reduces traffic.

Prof. Parthasarathy PV

2. When ***the receiver has no data to send and it receives an in-order segment (with expected sequence number) and the previous segment has already been acknowledged, the receiver delays sending an ACK segment until another segment arrives or until a period of time (normally 500 ms) has passed***.
3. When ***a segment arrives with a sequence number that is expected by the receiver***, and the previous in-order segment has not been acknowledged, the receiver immediately sends an ACK segment. There should not be more than two in-order unacknowledged segments at any time. This prevents the unnecessary retransmission of segments that may create congestion in the network.
4. When a ***segment arrives with an out-of-order sequence number that is higher than expected***, the receiver immediately sends an ACK segment announcing the sequence number of the next expected segment. This leads to the fast retransmission of missing segments.
5. When a ***missing segment arrives, the receiver sends an ACK segment to announce the next sequence number expected***. This informs the receiver that segments reported missing have been received.
6. If a ***duplicate segment arrives, the receiver discards the segment, but immediately sends an acknowledgment indicating the next in-order segment expected***. This solves some problems when an ACK segment itself is lost.

Retransmission

- The ***heart of the error control mechanism is the retransmission of segments***. When a segment is sent, it is stored in a queue until it is acknowledged. When the retransmission timer expires or when the sender receives three duplicate ACKs for the first segment in the queue, that segment is retransmitted.
- ***Retransmission after RTO*** - The sending TCP maintains one retransmission time-out (RTO) for each connection. When the timer matures, i.e. times out, TCP resends the segment in the front of the queue (the segment with the smallest sequence number) and restarts the timer. Note that again we assume $S_f < S_n$. We will see later that the value of RTO is dynamic in TCP and is updated based on the round-trip time (RTT) of segments. RTT is the time needed for a segment to reach a destination and for an acknowledgment to be received.
- ***Retransmission after Three Duplicate ACK Segments*** - The previous rule about retransmission of a segment is sufficient if the value of RTO is not large. To expedite service throughout the Internet by allowing senders to retransmit without waiting for a time out, most implementations today follow the three duplicate ACKs rule and retransmit the missing segment immediately. This feature is called fast retransmission. In this version, if three duplicate acknowledgments (i.e., an original ACK plus three exactly identical copies) arrive for a segment, the next segment is retransmitted without waiting for the time-out.

Out-of-Order Segments

- TCP implementations today ***do not discard out-of-order segments***. They store them temporarily and flag them as out-of-order segments until the missing segments arrive.
- Data may arrive out of order and be temporarily stored by the receiving TCP, but TCP guarantees that no out-of-order data are delivered to the process.

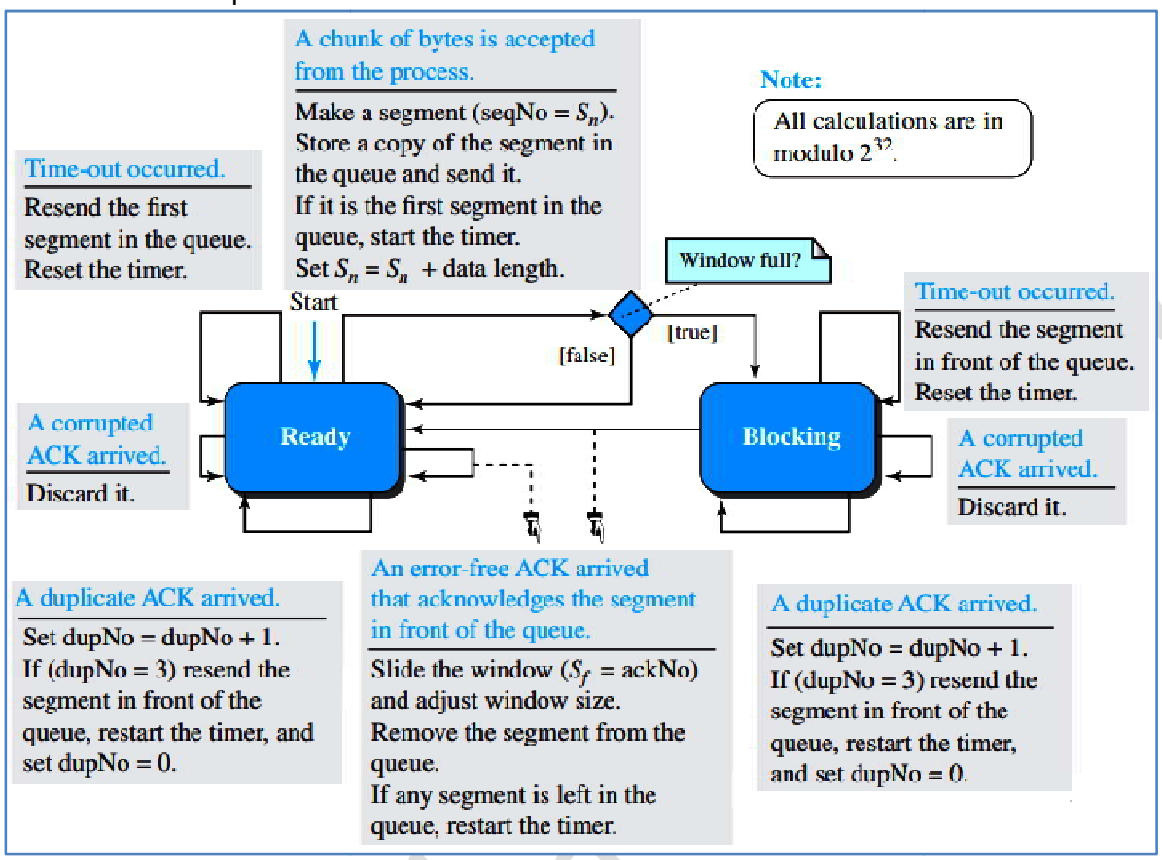
FSMs for Data Transfer in TCP

- Data transfer in TCP is close to the Selective-Repeat protocol with a slight similarity to GBN. Since TCP accepts out-of-order segments, TCP can be thought of as behaving more like the SR protocol, but since the original acknowledgments are cumulative, it looks like GBN. However, if the TCP implementation uses SACKs, then TCP is closest to SR.

Prof. Parthasarathy P V

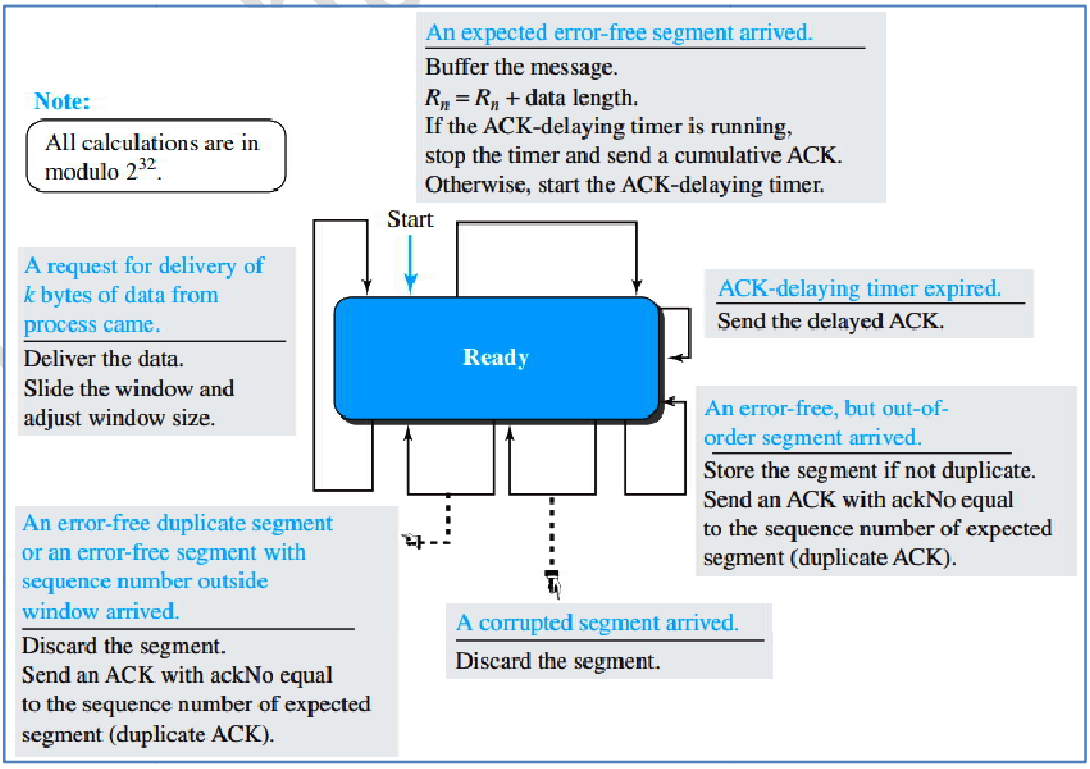
Sender-Side FSM

- We assume that the communication is unidirectional and the segments are acknowledged using ACK segments. We also ignore selective acknowledgments and congestion control for the moment.
- Figure shows the simplified FSM for the sender site.



Receiver-Side FSM

- Now let us show a simplified FSM for the receiver-side TCP protocol similar to the one we discuss for the SR protocol, but with some changes specific to TCP. We assume that the communication is unidirectional and the segments are acknowledged using ACK segments.



- We also ignore the selective acknowledgment and congestion control for the moment. ***Figure shows the simplified FSM for the receiver.***

Congestion control

Topics covered - TCP uses different policies to handle the congestion in the network.

- **Congestion Window**
- **Congestion Detection**
- **Congestion Policies**
- **Fast Recovery**
- **Policy Transition**
- **Additive Increase, Multiplicative Decrease**
- **TCP Throughput**

Congestion Window

- Flow control in TCP, the ***size of the send window is controlled by the receiver*** using the value of ***rwnd***, which is advertised in each segment traveling in the opposite direction. The use of this strategy guarantees that the receive window is never overflowed with the received bytes (no end congestion). This, however, does not mean that the intermediate buffers, buffers in the routers, do not become congested.
- A ***router may receive data from more than one sender***. ***No matter how large the buffers of a router may be***, it may be overwhelmed with data, which results in dropping some segments sent by a specific TCP sender.
- TCP ***needs to worry about congestion in the middle because many segments*** lost may seriously affect the error control. More segment loss means resending the same segments again, resulting in worsening the congestion, and finally the collapse of the communication.
- TCP is an ***end-to-end protocol that uses the service of IP***. The congestion in the router is in the IP territory and should be taken care of by IP. IP is a simple protocol with no congestion control. TCP, itself, needs to be responsible for this problem.
- ***TCP cannot ignore the congestion in the network; it cannot aggressively send segments to the network***. The result of such aggressiveness would hurt the TCP itself. TCP cannot be very conservative, either, sending a small number of segments in each time interval, because this means not utilizing the available bandwidth of the network. TCP needs to define policies that accelerate the data transmission when there is no congestion and decelerate the transmission when congestion is detected.
- ***To control the number of segments to transmit, TCP uses another variable called a congestion window***, ***cwnd***, whose size is controlled by the congestion situation in the network.
- The ***cwnd (related to the congestion in the middle - network)*** variable and the ***rwnd*** variable (***related to the congestion at the end***) together define the size of the send window in TCP.

Actual window size = minimum (rwnd, cwnd)

Congestion Detection

How a TCP sender can detect the possible existence of congestion in the network?

- The TCP sender uses the occurrence of ***two events*** as signs of congestion in the network:
  1. **Time-out**
  2. **Receiving three duplicate ACKs.**
- ***Time-out***. If a TCP sender does not receive an ACK for a segment or a group of segments before the time-out occurs, it assumes that the corresponding segment or segments are lost and the loss is due to congestion.
- ***Receiving of three duplicate ACKs (four ACKs with the same acknowledgment number).*** When a TCP receiver sends a duplicate ACK, it is the sign that a segment has been delayed, but sending three duplicate ACKs is the sign of a missing segment, which can be due to congestion in the

Prof Parthasarathy P V

network. However, the congestion in the case of three duplicate ACKs can be less severe than in the case of time-out.

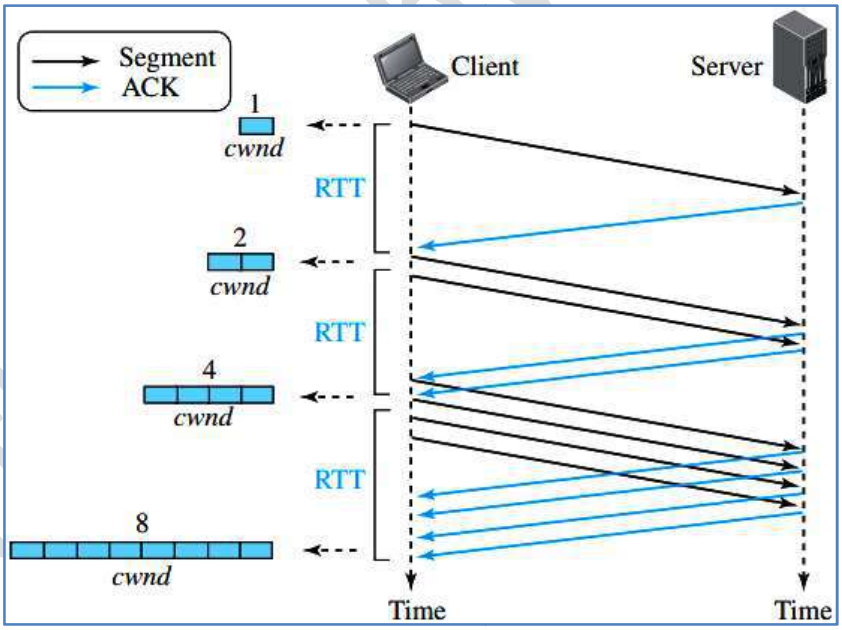
- When ***a receiver sends three duplicate ACKs***, it means that one segment is missing, but three segments have been received. The network is either slightly congested or has recovered from the congestion.
- An ***earlier version of TCP,*** called ***Taho TCP***, ***treated both events*** (time-out and three duplicate ACKs) similarly. ***The later version of TCP,*** called ***Reno TCP***, treats these two signs differently.
- A very interesting point in TCP congestion is that the TCP sender uses only one feedback from the other end to detect congestion: ***ACKs***. The lack of regular, timely receipt of ***ACKs***, which results in a ***time-out***, is the ***sign of a strong congestion***; the ***receiving of three duplicate ACKs is the sign of a weak congestion in the network***.

Congestion Policies

- TCP's general policy for handling congestion is based on ***three algorithms***
  1. **Slow start algorithm**
  2. **Congestion avoidance**
  3. **Fast recovery.**

1. Slow Start: Exponential Increase

- The slow-start algorithm is based on the idea that the size of the congestion window (cwnd) starts with one maximum segment size (MSS), but it increases one MSS each time an acknowledgment arrives. The MSS is a value negotiated during the connection establishment, using an option of the same name.
- The name of this algorithm is misleading; the algorithm starts slowly, but grows exponentially. ***To show the idea, let us look at Figure.***



- We assume that ***rwnd*** is much larger than ***cwnd***, so that the sender window size always equals cwnd. We also assume that each segment is of the same size and carries MSS bytes. For simplicity, we also ignore the delayed-ACK policy and assume that each segment is acknowledged individually.
- The sender starts with cwnd = 1. This means that the sender can send only one segment. After the first ACK arrives, the acknowledged segment is purged from the window, which means there is now one empty segment slot in the window. The size of the congestion window is also increased by 1 because the arrival of the acknowledgment is a good sign that there is no congestion in the network. The size of the window is now 2. After sending two segments and receiving two individual acknowledgments for them, the size of the congestion window now becomes 4, and so on.

If an ACK arrives, cwnd = cwnd + 1.

- If we look at the size of the ***cwnd*** in terms of ***round-trip times (RTTs),*** we find that the ***growth rate*** is exponential in terms of each round trip time, which is a very aggressive approach:

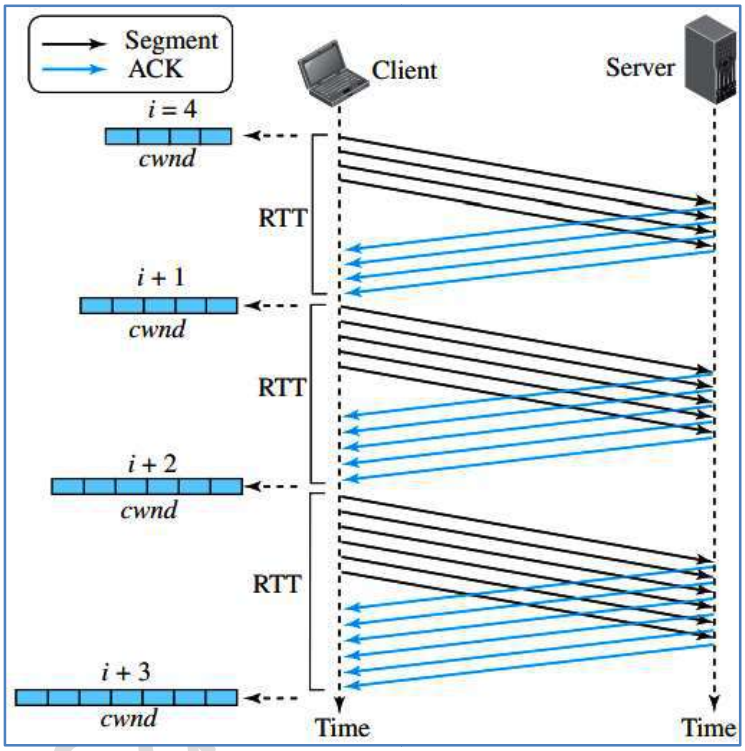
Start	→	$cwnd = 1 \rightarrow 2^0$
After 1 RTT	→	$cwnd = cwnd + 1 = 1 + 1 = 2 \rightarrow 2^1$
After 2 RTT	→	$cwnd = cwnd + 2 = 2 + 2 = 4 \rightarrow 2^2$
After 3 RTT	→	$cwnd = cwnd + 4 = 4 + 4 = 8 \rightarrow 2^3$

- A slow start cannot continue indefinitely.

- There must be a threshold to stop this phase. The sender keeps track of a variable named ***ssthresh*** (slow-start threshold). When the size of the ***window in bytes reaches this threshold***, ***slow start stops and the next phase starts.***
- We need to mention that the slow-start strategy is slower in the case of delayed acknowledgments.

2. Congestion Avoidance: Additive Increase

- If we continue with the slow-start algorithm, the size of the congestion window increases exponentially. To avoid congestion before it happens, we must slow down this exponential growth.
- ***TCP defines another algorithm called congestion avoidance***, which ***increases*** the ***cwnd*** additively instead of exponentially. When the size of the congestion window reaches the slow-start threshold in the case where ***cwnd = i,*** the slow-start phase stops and the additive phase begins. In this algorithm, each time the whole "window" of segments is acknowledged, the size of the congestion window is increased by one. A window is the number of segments transmitted during RTT. Figure shows the idea.



- The sender starts with ***cwnd = 4***. This means that the ***sender can send only four segments***. After four ACKs arrive, the acknowledged segments are purged from the window, which means there is now one extra empty segment slot in the window. The size of the congestion window is also increased by 1.
- The size of window is now 5. After sending five segments and receiving five acknowledgments for them, the size of the congestion window now becomes 6, and so on. The size of the congestion window in this algorithm is also a function of the number of ACKs that have arrived and can be determined as follows: ***If an ACK arrives, cwnd = cwnd 1 (1/cwnd).***
- If we look at the size of the ***cwnd*** in terms of ***round-trip times (RTTs),*** we find that the growth rate is linear in terms of each round-trip time, which is much more conservative than the slow-start approach.

Start	→	$cwnd = i$
After 1 RTT	→	$cwnd = i + 1$
After 2 RTT	→	$cwnd = i + 2$
After 3 RTT	→	$cwnd = i + 3$

Fast Recovery

- The ***fast-recovery algorithm is optional in TCP***. It starts when three duplicate ACKs arrive, which is interpreted as light congestion in the network. This algorithm is also an additive increase, but it increases the size of the congestion window when a duplicate ACK arrives (after the three duplicate ACKs that trigger the use of this algorithm). ***If a duplicate ACK arrives, cwnd = cwnd 1 (1 / cwnd).***

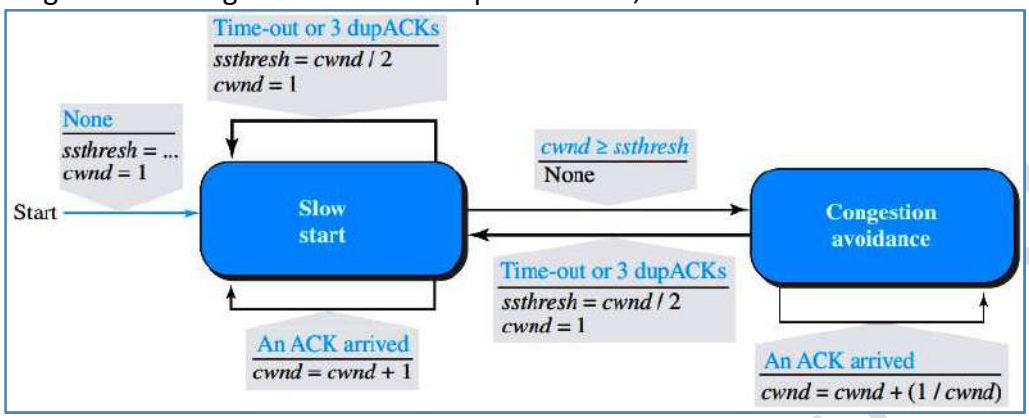
Policy Transition

- We discussed three congestion policies in TCP. Now the question is when each of these policies is used and when TCP moves from one policy to another. To answer these questions, we need to refer to ***three versions*** of TCP:
  1. **Taho TCP**
  2. **Reno TCP**
  3. **New Reno TCP.**

Prof. Parthasa

Taho TCP

- The early TCP, known as Taho TCP, used only two different algorithms in their congestion policy: slow start and congestion avoidance. We ***use Figure to show the FSM*** for this version of TCP. However, we need to mention that we have deleted some small trivial actions, such as incrementing and resetting the number of duplicate ACKs, to make the FSM less crowded, simpler.

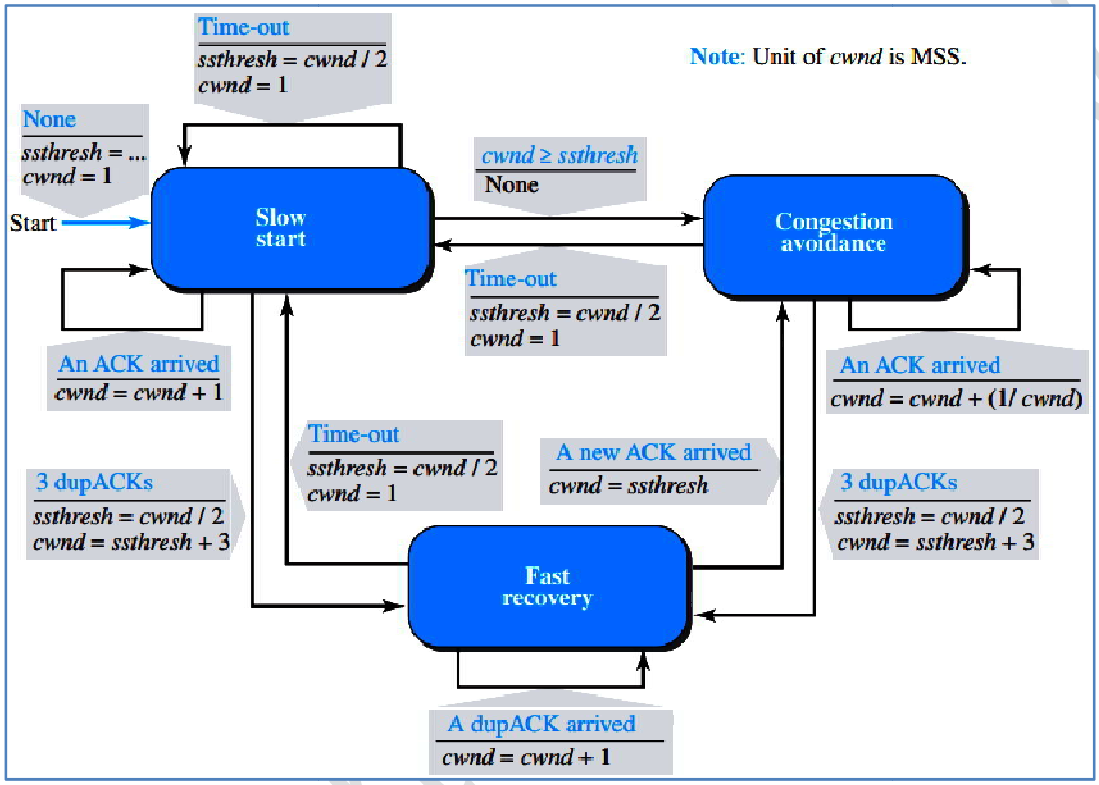


- ***Taho TCP*** treats the ***two signs*** used for ***congestion detection***, ***time-out and three duplicate ACKs***, in the same way. In this version, when the connection is established, TCP starts the slow-start algorithm and sets the ssthresh variable to a pre-agreed value (normally a multiple of MSS) and the cwnd to 1 MSS. In this state, as we said before, each time an ACK arrives, the size of the congestion window is incremented by 1. We know that this policy is very aggressive and exponentially increases the size of the window, which may result in congestion.
- If ***congestion is detected*** (occurrence of time-out or arrival of three duplicate ACKs), TCP immediately interrupts this aggressive growth and restarts a new slow start algorithm by limiting the threshold to half of the current cwnd and resetting the congestion window to 1.
- If ***no congestion is detected*** while reaching the threshold, TCP learns that the ceiling of its ambition is reached; it should not continue at this speed. It moves to the congestion avoidance state and continues in that state.
- In the ***congestion-avoidance state***, the size of the congestion window is increased by 1 each time a number of ACKs equal to the current size of the window has been received. For example, if the window size is now 5 MSS, five more ACKs should be received before the size of the window becomes 6 MSS. If congestion is detected in this state, TCP again resets the value of the ***ssthresh*** to half of the current ***cwnd*** and moves to the ***slow-start state*** again.
- Although in this version of TCP the size of ***ssthresh*** is continuously adjusted in each congestion detection. For example, if the ***original ssthresh = 8 MSS*** and congestion is detected when TCP is in the congestion avoidance state and the value of the ***cwnd = 20***, the new value of the ***ssthresh = 10***, which means it has been increased.

Reno TCP

- A newer version of TCP, called Reno TCP, added a new state to the congestion-control FSM, called the fast-recovery state. This version treated the ***two signals of congestion***, ***time-out*** and the ***arrival of three duplicate ACKs***, differently.
- In this version, if a time-out occurs, TCP moves to the slow-start state (or starts a new round if it is already in this state); on the other hand, ***if three duplicate ACKs arrive, TCP moves to the fast-recovery state and remains there as long as more duplicate ACKs arrive***.
- ***The fast-recovery state is a state somewhere between the slow-start and the congestion-avoidance states.*** It behaves like the slow start, in which the cwnd grows exponentially, but the cwnd starts with the value of ssthresh plus 3 MSS (instead of 1).

- When TCP enters the fast-recovery state, ***three major events may occur***.
  1. If ***duplicate ACKs continue to arrive***, TCP stays in this state, but the cwnd grows exponentially.
  2. If a ***time-out occurs***, TCP assumes that there is real congestion in the network and moves to the slow-start state.
  3. If a ***new (nonduplicate) ACK arrives***, TCP moves to the congestion-avoidance state, but deflates the size of the cwnd to the ssthresh value, as though the three duplicate ACKs have not occurred, and transition is from the slow-start state to the congestion-avoidance state.
- Figure shows the simplified FSM for Reno TCP. Again, we have removed some trivial events to simplify the figure and discussion.



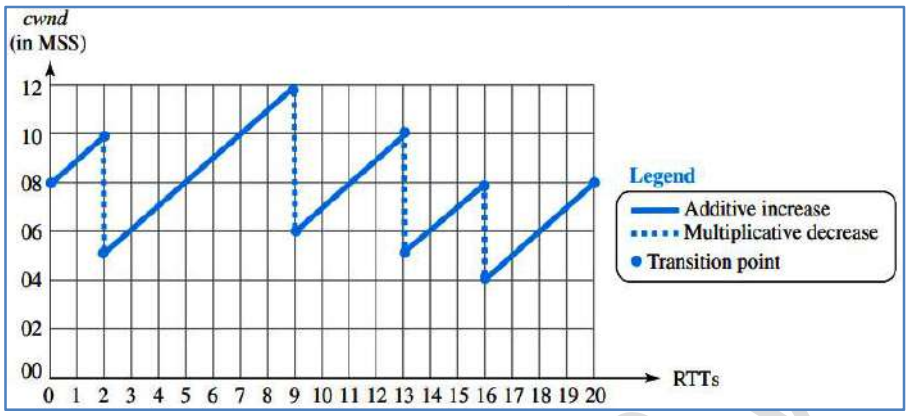
NewReno TCP

- A later version of TCP, called ***NewReno TCP***, made an extra optimization on the Reno TCP. In this version, ***TCP checks to see if more than one segment is lost in the current window when three duplicate ACKs arrive. When TCP receives three duplicate ACKs, it retransmits the lost segment until a new ACK (not duplicate) arrives.***
- If the new ACK defines the end of the window when the congestion was detected, TCP is certain that only one segment was lost. However, if the ACK number defines a position between the retransmitted segment and the end of the window, it is possible that the segment defined by the ACK is also lost.
- ***NewReno TCP retransmits this segment to avoid receiving more and more duplicate ACKs for it***.

Additive Increase, Multiplicative Decrease

- ***Out of the three versions of TCP, the Reno version is most common today***. It has been observed that, in this version, most of the time the congestion is detected and taken care of by observing the three duplicate ACKs.
- Even if there are some time-out events, TCP recovers from them by aggressive exponential growth.

- In a long TCP connection, if we ignore the slow-start states and short exponential growth during fast recovery, the TCP congestion window is ***cwnd = cwnd + (1 / cwnd)*** when an ACK arrives (congestion avoidance), and ***cwnd = cwnd / 2*** when congestion is detected, as though SS does not exist and the length of FR is reduced to zero.



- The ***first*** is called ***additive increase***; the ***second*** is called ***multiplicative decrease***. This means that the congestion window size, after it passes the initial slow-start state, follows a saw tooth pattern called ***additive increase, multiplicative decrease (AIMD),*** as shown in Figure.

TCP Throughput

- The throughput for TCP, which is based on the congestion window behavior, can be easily found if the ***cwnd*** is a constant (flat line) function of RTT. The throughput with this unrealistic assumption is ***throughput = cwnd / RTT***. In this assumption, TCP sends a cwnd bytes of data and receives acknowledgement for them in RTT time.
- The ***behavior of TCP, as shown in Figure above, is not a flat line;*** it is like saw teeth, with many minimum and maximum values. If each tooth were exactly the same, we could say that the ***throughput = [(maximum + minimum) / 2] / RTT***.
- However, we know that the value of the maximum is twice the value of the minimum because in each congestion detection the value of ***cwnd*** is set to half of its previous value. So the throughput can be better calculated as ***throughput = (0.75) W_{max} / RTT*** in which ***W_{max}*** is the average of window sizes when the congestion occurs.

Example

If MSS = 10 KB (kilobytes) and RTT = 100 ms in Figure, we can calculate the throughput as shown:

W_{max} = (10 + 12 + 10 + 8 + 8) / 5 = 9.6 MSS

Throughput = (0.75 W_{max} / RTT) = 0.75 × 960 kbps / 100 ms = 7.2 Mbps

---

Prof. Parthasarathy P V

BCS502 COMPUTER NETWORKS

Module-5: Application Layer

Ch. 25. Introduction to Application Layer: Introduction, Client-Server Programming
Ch. 26. Standard Client-Server Protocols: World Wide Web and HTTP, FTP, Electronic Mail, Domain Name System (DNS), TELNET, Secure Shell (SSH).

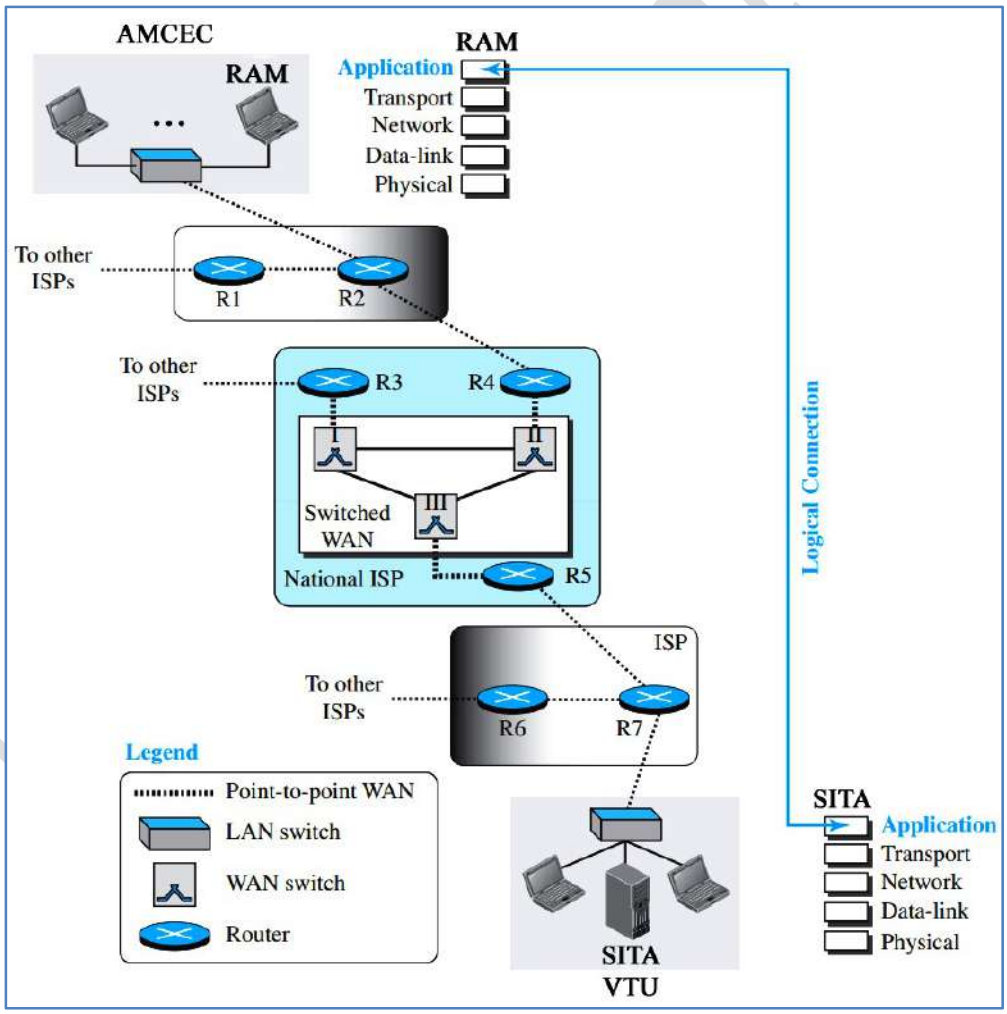
Textbook: Ch. 25.1-25.2, 26.1-26.6

Chapter 25: Introduction to Application Layer [P-854]

Introduction, Client-Server Programming

Introduction to Application Layer

- The whole Internet, hardware and software, was designed and developed to provide services at the application layer. The fifth layer of the TCP/IP protocol suite is where these services are provided for Internet users. The other four layers are there to make these services possible.
- Communication is provided using a logical connection, which means that the two application layers assume that there is an imaginary direct connection through which they can send and receive messages. Figure shows the idea behind this logical connection.



Providing Services

- ***The Internet was originally designed for the purpose: to provide service to users around the world***. The ***layered architecture of the TCP/IP protocol suite***, however, makes the ***Internet more flexible than other communication networks*** such as postal or telephone networks.

- Each layer in the suite was originally made up of one or more protocols, but new protocols can be added or some protocols can be removed or replaced by the Internet authorities. However, if a protocol is added to each layer, it should be designed in such a way that it uses the services provided by one of the protocols at the lower layer. If a protocol is removed from a layer, care should be taken to change the protocol at the next higher layer that supposedly uses the services of the removed protocol.
- The ***application layer*** is different from other layers in that it is the ***highest layer in the suite***. The protocols in this layer do not provide services to any other protocol in the suite; they ***only receive services from the protocols in the transport layer***. This means that protocols can be removed from this layer easily. New protocols can be also added to this layer as long as the new protocols can use the services provided by one of the transport-layer protocols.
- Since the application layer is the only layer that provides services to the Internet user, the flexibility of the application layer allows new application protocols to be easily added to the Internet, which has been occurring during the lifetime of the Internet. When the Internet was created, only a few application protocols were available to the users; today we cannot give a number for these protocols because new ones are being added constantly.

Standard Application-Layer Protocols

- There are several application-layer protocols that have been standardized and documented by the Internet authority, and we are using them in our daily interaction with the Internet. ***Each standard protocol is a pair of computer programs that interact with the user and the transport layer to provide a specific service to the user***.
- In the case of these application protocols, we should know what types of services they provide, how they work, and the options that we can use with these applications, and so on.
- The ***study of these protocols enables a network manager to easily solve the problems that may occur when using these protocols***. The deep understanding of how these protocols work will also give us some ideas about how to create new nonstandard protocols.

Nonstandard Application-Layer Protocols

- A programmer can create a nonstandard application-layer program if you can write two programs that provide service to the user by interacting with the transport layer.
- It is the creation of a nonstandard (proprietary) protocol, which does not even need the approval of the Internet authorities if privately used, that has made the Internet so popular worldwide.
- A private company can create a new customized application protocol to communicate with all of its offices around the world using the services provided by the first four layers of the TCP/IP protocol suite without using any of the standard application programs.
- What is needed is to write programs, in one of the computer languages that ***use the available services provided by the transport-layer protocols***.

Application-Layer Paradigms

- To use the Internet we need ***two application programs*** to interact with each other
  - ***one running on a computer somewhere in the world***
  - ***the other running on another computer somewhere else in the world.***
- The ***two programs need to send messages to each other through the Internet infrastructure***.
- ***Two paradigms*** have been developed during the lifetime of the Internet:
  - ***the client-server paradigm***
  - ***the peer-to-peer paradigm.***

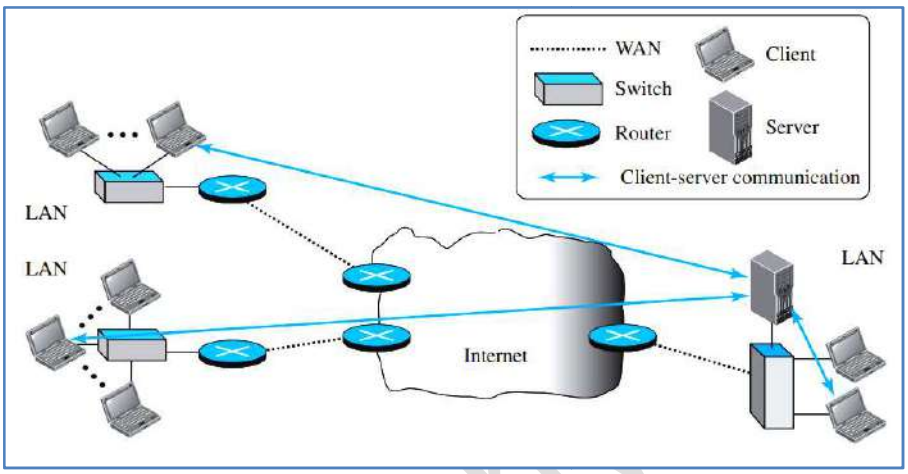
Traditional Paradigm: Client-Server

- The traditional paradigm is called the client-server paradigm. In this paradigm, ***the service provider*** is ***an application program***, ***called the server process***; it ***runs continuously***, ***waiting for another***

Prof. Parthasarathy PV

application program, called the ***client process***, to ***make a connection through the Internet and ask for service***.

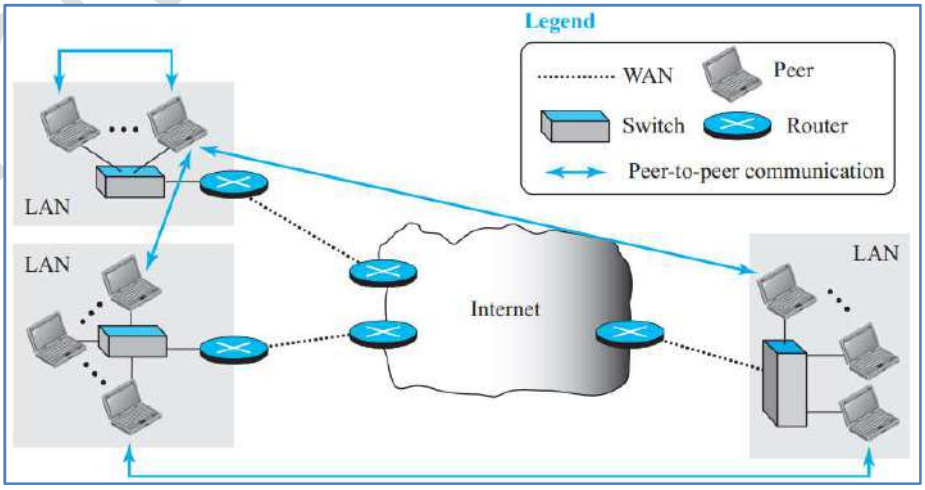
- There are normally some server processes that can provide a specific type of service, but there are many clients that request service from any of these server processes. The ***server process must be running all the time; the client process is started when the client needs to receive service***.
- Although the communication in the ***client-server paradigm*** is between ***two application programs***, the ***role of each program is totally different***. In other words, we cannot run a client program as a server program or vice versa.



- Figure shows an example of a client-server communication in which three clients communicate with one server to receive the services provided by this server.
- ***One problem*** with this paradigm is that the concentration of the ***communication load is on the server***, which means the server should be a powerful computer. Even a powerful computer may become overwhelmed if a large number of clients try to connect to the server at the same time.
- ***Another problem*** is that there should ***be a service provider willing to accept the cost and create a powerful server for a specific service***, which means the service must always return some type of income for the server in order to encourage such an arrangement. Several traditional services are still using this paradigm, including the ***World Wide Web (WWW)*** and its vehicle ***HyperText Transfer Protocol (HTTP), file transfer protocol (FTP), secure shell (SSH), e-mail***, and so on.

New Paradigm: Peer-to-Peer

- A new paradigm, called the ***peer-to-peer paradigm (P2P paradigm)*** has emerged to respond to the needs of some new applications. In this paradigm, there is no need for a server process to be running all the time and waiting for the client processes to connect. The ***responsibility is shared between peers***. A computer connected to the Internet can provide service at one time and receive service at another time. A computer can even provide and receive services at the same time. Figure shows an example of communication in this paradigm.



- One of the areas that really fits in this paradigm is the Internet telephony. Communication by phone is indeed a peer-to-peer activity; no party needs to be running forever waiting for the other party to call. Another area in which the peer-to-peer paradigm can be used is when some computers connected to the Internet have something to share with each other

- Although the peer-to-peer paradigm has been proved to be ***easily scalable and cost-effective in eliminating the need for expensive servers to be running and maintained all the time***, there are also some challenges.
- The main ***challenge has been security***; it is more difficult to create secure communication between distributed services than between those controlled by some dedicated servers.
- The ***other challenge is applicability***; it appears that not all applications can use this new paradigm. For example, not many Internet users are ready to become involved, if one day the Web can be implemented as a peer-to-peer service. There are ***some new applications***, such as ***BitTorrent, Skype, IPTV, and Internet telephony*** that use this paradigm.

Mixed Paradigm

- An application may choose to use a mixture of the two paradigms by combining the advantages of both.
- For example, a light-load client-server communication can be used to find the address of the peer that can offer a service. When the address of the peer is found, the actual service can be received from the peer by using the peer-topeer paradigm.

CLIENT-SERVER PROGRAMMING

Topics covered

- **Application Programming Interface**
- **Using Services of the Transport Layer**
- **Iterative Communication Using UDP**
- **Iterative Communication Using TCP**
- **Concurrent Communication**

- In a client-server paradigm, communication at the ***application layer is between two running application programs called processes: a client and a server***.
- A ***client*** is a running program that initializes the communication by sending a request; a server is another application program that waits for a request from a client.
- The ***server*** handles the request received from a client, prepares a result, and sends the result back to the client. This definition of a server implies that a server must be running when a request from a client arrives, but the client needs to be run only when it is needed.
- We need to be careful that the server program is started before we start running the client program. ***The lifetime of a server is infinite***: it should be started and run forever, waiting for the clients.
- The ***lifetime of a client is finite***: it normally sends a finite number of requests to the corresponding server, receives the responses, and stops.

Application Programming Interface

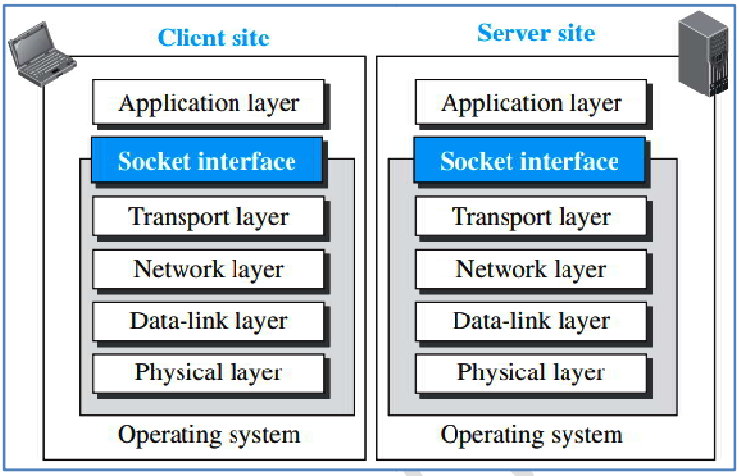
(Sockets - Socket Addresses - Finding Socket Addresses - Server Site - Client Site)

How can a client process communicate with a server process?

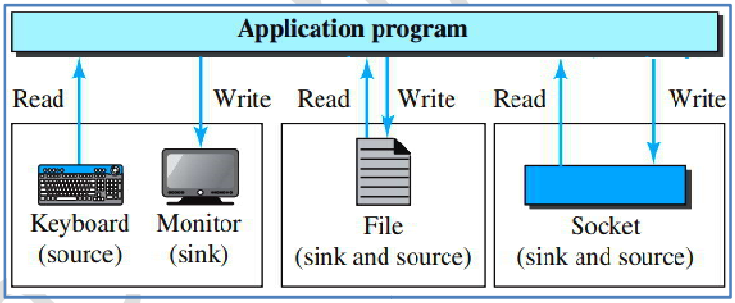
- If we need a process to be able to communicate with another process, we need a new set of instructions to tell the lowest four layers of the TCP/IP suite to open the connection, send and receive data from the other end, and close the connection. A set of instructions of this kind is normally referred to as an ***application programming interface (API).***
- An interface in programming is a set of instructions between two entities. In this case, ***one of the entities is the process at the application layer*** and the ***other is the operating system that encapsulates the first four layers of the TCP/IP protocol suite***.

Prof. Parthasarathy P V

- A ***computer manufacturer needs to build the first four layers of the suite in the operating system and include an API***. In this way, the processes running at the application layer are able to communicate with the operating system when sending and receiving messages through the Internet.
- ***Several APIs*** have been designed for communication. Three among them are common: ***socket interface, Transport Layer Interface (TLI), and STREAM***.
- Socket interface started in the early 1980s at UC Berkeley as part of a UNIX environment. The socket interface is a set of instructions that provide communication between the application layer and the operating system, ***as shown in Figure.***

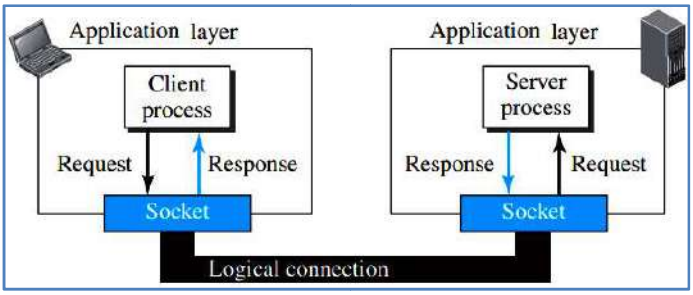


- The idea of sockets allows us to use the set of all instructions already designed in a programming language for other sources and sinks. For example, in most computer languages, like C, C++, or Java, we have several instructions that can read and write data to other sources and sinks such as a keyboard (a source), a monitor (a sink), or a file (source and sink). We can use the same instructions to read from or write to sockets.
- We are ***adding only new sources and sinks to the programming language without changing the way we send data or receive data***. ***Figure shows the idea and compares the sockets with other sources and sinks.***



Sockets

- Socket is supposed to behave like a terminal or a file, it is not a physical entity like them; it is an abstraction. It is an object that is created and used by the application program.
- As far as the application layer is concerned, communication between a client process and a server process is communication between two sockets, created at two ends, ***as shown in Figure***.

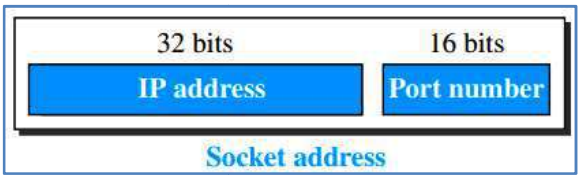


- The client thinks that the socket is the entity that receives the request and gives the response; the server thinks that the socket is the one that has a request and needs the response.
- If we create two sockets, one at each end, and define the source and destination addresses correctly, we can use the available instructions to send and receive data. The rest is the responsibility of the operating system and the embedded TCP/IP protocol.

Socket Addresses

- The ***interaction between a client and a server is two-way communication***. In a ***two-way*** communication, we ***need a pair of addresses***: ***local (sender)*** and ***remote (receiver).*** The local address in one direction is the remote address in the other direction and vice versa.
- Since communication in the client-server paradigm is between two sockets, ***we need a pair of socket addresses for communication: a local socket address and a remote socket address***. We need to define a socket address in terms of identifiers used in the TCP/IP protocol suite.

- A socket address should first define the computer on which a client or a server is running. A computer in the Internet is uniquely defined by its IP address, a 32-bit integer in the current Internet version.
- Several client or server processes may be running at the same time on a computer, which means that we need another identifier to define the specific client or server involved in the communication. ***An application program can be defined by a port number, a 16-bit integer***. This means that a ***socket address should be a combination of an IP address and a port number*** as shown in Figure.



- Since a socket defines the end-point of the communication, we can say that a socket is identified by a pair of socket addresses, a local and a remote.

Finding Socket Addresses - Server Site

- How can a client or a server find a pair of socket addresses for communication? The situation is different for each site.
- The server needs a local (server) and a remote (client) socket address for communication.
- ***Local Socket Address***: The local (server) socket address is ***provided by the operating system***. The ***operating system knows the IP address of the computer on which the server process is running***. The port number of a server process, however, needs to be assigned. If the server process is a standard one defined by the Internet authority, a port number is already assigned to it. - For example, the assigned port number for a Hypertext Transfer Protocol (HTTP) is the integer 80, which cannot be used by any other process. If the server process is not standard, the designer of the server process can choose a port number, in the range defined by the Internet authority, and assign it to the process. ***When a server starts running, it knows the local socket address***.
- ***Remote Socket Address***: The remote socket address ***for a server is the socket address of the client that makes the connection***. Since the server can serve many clients, it does not know beforehand the remote socket address for communication. The server can find this socket address when a client tries to connect to the server. The client socket address, which is contained in the request packet sent to the server, becomes the remote socket address that is used for responding to the client. In other words, although the local socket address for a server is fixed and used during its lifetime, the remote socket address is changed in each interaction with a different client.

Finding Socket Addresses - Client Site

- The ***client also needs a local (client) and a remote (server) socket address for communication***.
- ***Local Socket Address:*** The local (client) socket address is also provided by the operating system. The operating system knows the IP address of the computer on which the client is running. The port number, however, is a 16-bit temporary integer that is assigned to a client process each time the process needs to start the communication. The port number, however, needs to be assigned from a set of integers defined by the Internet authority and called the ephemeral (temporary) port numbers. The operating system, however, ***needs to guarantee that the new port number is not used by any other running client process***. The ***operating system needs to remember the port number to be able to redirect the response received from the server process to the client process that sent the request***.
- ***Remote Socket Address:*** Finding the remote (server) socket address for a client, however, needs more work. When a client process starts, it should know the socket address of the server it wants to connect to. We will have ***two situations*** in this case.
    1. Sometimes, the user who starts the client process knows both the server port number and IP address of the computer on which the server is running. This usually occurs in situations when we have written client and server applications and we want to test them. For example, write some simple client and server programs and we test them using this approach. In this

Prof. Parthasarathy P V

situation, the programmer can provide these two pieces of information when he runs the client program.

2. Although each standard application has a well-known port number, most of the time, we do not know the IP address. This happens in situations such as when we need to contact a web page, send an e-mail to a friend, copy a file from a remote site, and so on. In these situations, the server has a name, an identifier that uniquely defines the server process. Examples of these identifiers are URLs, such as www.xxx.yyy, or e-mail addresses, such as xxxx@yyyy.com. The client process should now change this identifier (name) to the corresponding server socket address. The client process normally knows the port number because it should be a well-known port number, but the IP address can be obtained using another client-server application called the ***Domain Name System (DNS).***

Using Services of the Transport Layer

- A pair of processes provide services to the users of the Internet, human or programs.
- A pair of processes need to use the services provided by the transport layer for communication because there is no physical communication at the application layer.
- There are three common transport-layer protocols in the TCP/IP suite: UDP, TCP, and SCTP. Most standard applications have been designed to use the services of one of these protocols.
- When we write a new application, we can decide which protocol we want to use. The choice of the transport layer protocol seriously affects the capability of the application processes.

UDP Protocol

- UDP provides connectionless, unreliable, datagram service. Connectionless service means that there is no logical connection between the two ends exchanging messages. Each message is an independent entity encapsulated in a datagram. UDP does not see any relation (connection) between consequent datagrams coming from the same source and going to the same destination.
- UDP is not a reliable protocol. Although it may check that the data is not corrupted during the transmission, it does not ask the sender to resend the corrupted or lost datagram.
- For some applications, UDP has an advantage: it is message-oriented. It gives boundaries to the messages exchanged. An application program may be designed to use UDP if it is sending small messages and the simplicity and speed is more important for the application than reliability. For example, some management and multimedia applications fit in this category.

TCP Protocol

- TCP provides connection-oriented, reliable, byte-stream service. TCP requires that two ends first create a logical connection between themselves by exchanging some connection-establishment packets. This phase, which is sometimes called ***handshaking***, establishes some parameters between the two ends, including the size of the data packets to be exchanged, the size of buffers to be used for holding the chunks of data until the whole message arrives, and so on.
- After the handshaking process, the two ends can send chunks of data in segments in each direction. By numbering the bytes exchanged, the continuity of the bytes can be checked.
- For example, if some bytes are lost or corrupted, the receiver can request the resending of those bytes, which makes TCP a reliable protocol. TCP also can ***provide flow control and congestion control***.
- ***One problem with the TCP protocol is that it is not message-oriented; it does not put boundaries on the messages exchanged***. Most of the standard applications that need to send long messages and require reliability may benefit from the service of the TCP.

SCTP Protocol

- SCTP provides a service which is a combination of the two other protocols. Like TCP, SCTP provides a connection-oriented, reliable service, but it is not byte-stream oriented.

Prof. Parthasarathy P V

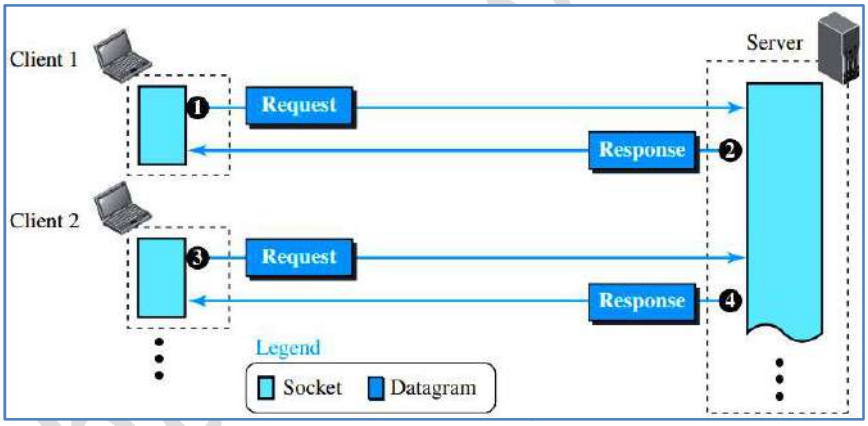
- It is a message-oriented protocol like UDP. In addition, SCTP can provide multi-stream service by providing multiple network-layer connections.
- SCTP is normally suitable for any application that needs reliability and at the same time needs to remain connected, even if a failure occurs in one network-layer connection.

Iterative Communication Using UDP

- Communication between a client program and a server program can occur iteratively or concurrently. Although several client programs can access the same server program at the same time, the server program can be designed to respond iteratively or concurrently.
- An iterative server can process one client request at a time; it receives a request, processes it, and sends the response to the requestor before handling another request. When the server is handling the request from a client, the requests from other clients, and even other requests from the same client, need to be queued at the server site and wait for the server to be freed.
- The ***received and queued requests are handled in the first-in, first-out fashion. In this section, we discuss iterative communication using UDP***.

Sockets Used for UDP

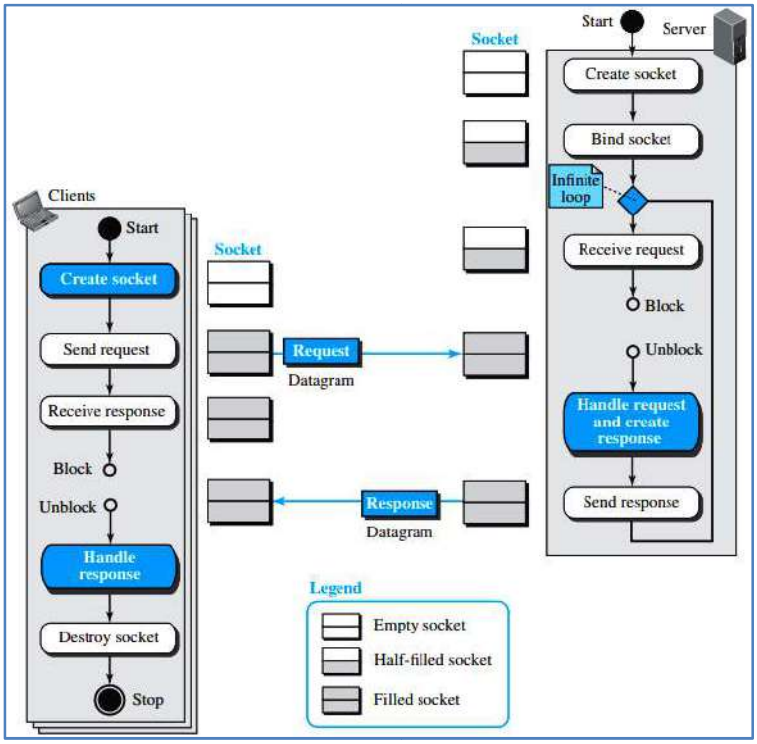
- In UDP communication, the client and server use only one socket each. The socket created at the server site lasts forever; the socket created at the client site is closed (destroyed) when the client process terminates. Figure shows the lifetime of the sockets in the server and client processes.



- ***Different clients use different sockets, but the server creates only one socket and changes only the remote socket address each time a new client makes a connection***. ***This is logical***, because the server does know its own socket address, but does not know the socket addresses of the clients who need its services; it needs to wait for the client to connect before filling this part of the socket address.

Flow Diagram

- UDP provides a connectionless service, in which a client sends a request and the server sends back a response. Figure shows a simplified flow diagram for iterative communication.
- There are multiple clients, but only one server. Each client is served in each iteration of the loop in the server. Note that there is no connection establishment or connection termination.
- Each client sends a single datagram and receives a single datagram.



- In other words, if a client wants to send two datagrams, it is considered as two clients for the server. The second datagram needs to wait for its turn. The diagram also shows the status of the socket after each action.

Server Process

- The server makes a ***passive open***, in which it becomes ready for the communication, but ***it waits until a client process makes the connection***. It ***creates an empty socket***. It then binds the socket to the server and the well-know port, in which only ***part of the socket*** (the server socket address) is filled (binding can happen at the time of creation depending on the underlying language).
- The ***server then issues a receive request command, which blocks until it receives a request from a client***. The server then fills the rest of the socket (the client socket section) from the information obtained in the request. The request is the process and the response is sent back to the client.
- The ***server now starts another iteration waiting for another request to arrive*** (an infinite loop). Note that in each iteration, the socket becomes only half-filled again; the client socket address is erased. It is totally filled only when a request arrives.

Client Process

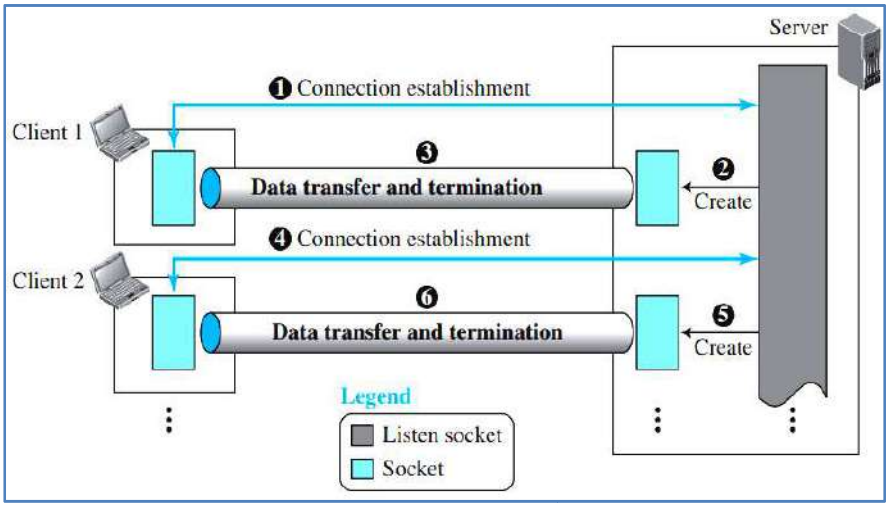
- The ***client process makes an active open***.
- It starts a connection. It creates an empty socket and then issues the send command, which fully fills the socket, and sends the request.
- The client then issues a receive command, which is blocked until a response arrives from the server. The response is then handled and the socket is destroyed.

Iterative Communication Using TCP

- TCP is a connection-oriented protocol.
- Before sending or receiving data, a connection needs to be established between the client and the server.
- After the connection is established, the two parties can send and receive chunks of data as long as they have data to do so.
- Although iterative communication using TCP is not very common, because it is simpler.

Sockets Used in TCP

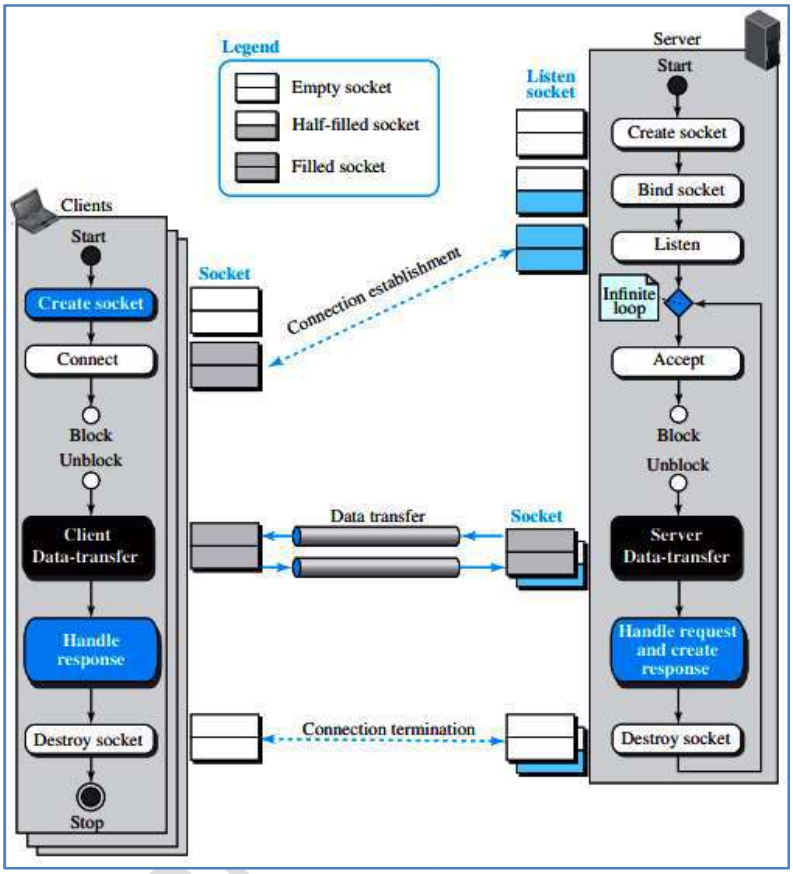
- The TCP server uses ***two different sockets***, ***one for connection establishment (***listen socket***)*** and the ***other for data transfer (***socket***)***. The reason for having two types of sockets is to separate the connection phase from the data exchange phase.
- A server uses a listen socket to listen for a new client trying to establish connection.
- After the connection is established, the server creates a socket to exchange data with the client and finally to terminate the connection. The client uses only one socket for both connection establishment and data exchange (see Figure).



Prof. Parthasarathy P V

Flow Diagram

- Figure shows a simplified flow diagram for iterative communication using TCP. There are multiple clients, but only one server.
- Each client is served in each iteration of the loop. The flow diagram is almost similar to the one for UDP, but there are differences that we explain for each site.



Server Process

- In Figure, the TCP server process, like the UDP server process, creates a socket and binds it, but these two commands create the listen socket to be used only for the connection establishment phase.
- The server process then calls the listen procedure, to allow the operating system to start accepting the clients, completing the connection phase, and putting them in the waiting list to be served.
- The server process now starts a loop and serves the clients one by one. In each iteration, the server process issues the accept procedure that removes one client from the waiting list of the connected clients for serving.
- If the list is empty, the accept procedure blocks until there is a client to be served. When the accept procedure returns, it creates a new socket for data transfer. The server process now uses the client socket address obtained during the connection establishment to fill the remote socket address field in the newly created socket. At this time the client and server can exchange data.

Client Process

- The client flow diagram is almost similar to the UDP version except that the client data-transfer box needs to be defined for each specific case.

Concurrent Communication

- A concurrent server can process several client requests at the same time. This can be done using the available provisions in the underlying programming language.
- In C, a server can create several child processes, in which a child can handle a client. In Java, threading allows several clients to be handled by each thread.

Prof. Parthasarathy